



Information Retrieval



Academic Year 2025/2026

Luigi Santangelo
luigi.santangelo@unipv.it

University of Pavia
**Master Degree in
Computer Engineering**

Information Retrieval

- Architecture of the modern Information Retrieval Systems
- Data Structures for managing data
- Algorithms for retrieving data
- Boolean Retrieval Systems
- Free Text Retrieval System
- Word Embedding, Word2Vec and Doc2Vec
- Solr and MongoDB ^{reduced}
- Augmentation ^{new}
- Text Classification ^{new}
- Content Image Retrieval Systems

The lectures

- From the 30th of September till the end of December
 - Monday (02.00PM - 04.00PM)
 - Tuesday (04.00PM - 06.00PM)
 - Friday (No Lecture)
- 6 CFU
- 60 hours
 - 38 hours: lectures
 - 22 hours: laboratory

The WebPage

<https://elearning.unipv.it/course/view.php?id=9888>

- Notifications and Alerts
- Teaching Materials



Login with username and password for all university services

The Book (1)

Christopher D. Manning, Prabhakar Raghavan and Hinrich Schütze,
Introduction to Information Retrieval, Cambridge University Press. 2008

<https://nlp.stanford.edu/IR-book/>

Chapters 1, 2, 3, 4, 5, 6, 7, 13, 19, 20

The Book (2)

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein

Introduction to algorithms

The Book (3)

Online resources

The Exam

Written

- 10 open questions
- One hour

Project discussion

The Project

Implementing one or more topics described during lectures

for example: similarities, error detection, vector space model, ...

Some hints are available on the web page

Writing a paper describing the project

One student per project

The Survey

- Proposals, criticisms and suggestions will be welcome
- You will be given instructions on how to answer the questionnaire
- Not mandatory but strongly recommended
- Anonymous

Contacts

For any information and appointments: **luigi.santangelo@unipv.it**

Data Structures for Digital Information Retrieval Systems

Academic Year 2025/2026

Luigi Santangelo
luigi.santangelo@unipv.it

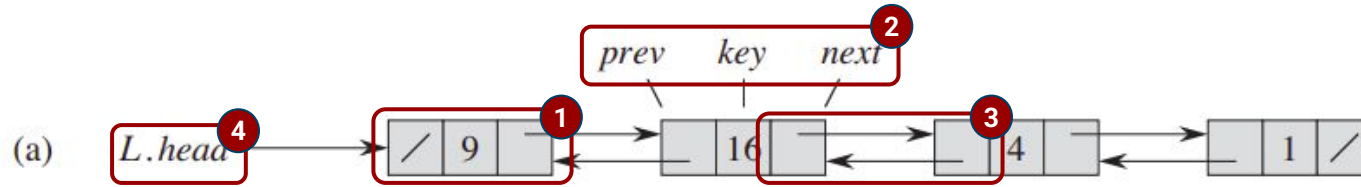
University of Pavia
**Master Degree in
Computer Engineering**

Data Structures

- Linked List
- Hash Table
- Binary Tree
- B-Tree
- Binary Heap

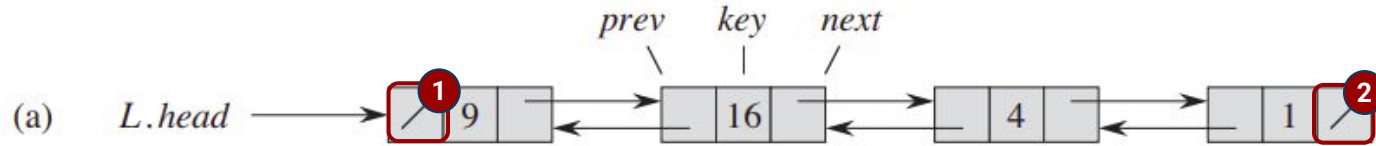
Linked List

Linked List



1. Data structure for managing objects in a **linear order**
2. Each item contains the **key** (can be anything, integer, double, string, complex data) and **two pointers** to move back and forth
3. Given an element x in the list, we can move to its **successor and predecessor**
4. A pointer points to the **head of the list**

Linked List

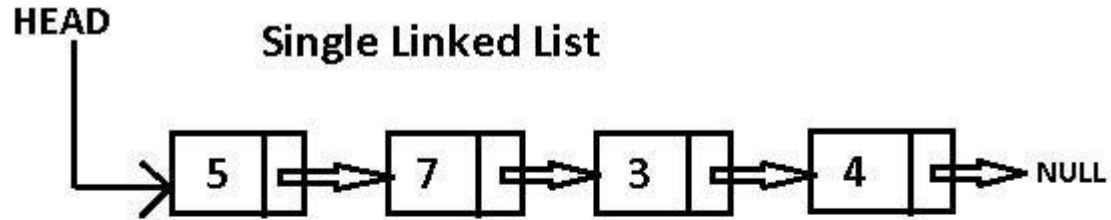


1. The predecessor of the first item is NULL
2. The successor of the last item is NULL

Linked List

A list may have several forms:

- **Singly Linked List:** no predecessor is stored in each item



Linked List

A list may have several forms:

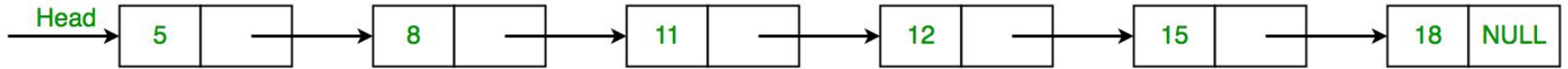
- **Circular Linked List:** the successor of the last item is linked to the the first item and the predecessor of the first item is linked to the last item



Linked List

A list may have several forms:

- **Sorted Linked List:** all the elements are sorted in non decreasing order



Sorted Linked List: searching operation

```
search(L, k)
{
    x = L.head;
    while (x != NULL && x.key < k)
        x = x.next;
    if (x != NULL && x.key == k) return x;
    return NULL;
}
```

L: the list

k: the key to look for

Complexity: **$O(n)$**

Sorted Linked List: inserting operation

```
insert(L, x)
{
    tmp = L.head
    // insert at the first position
    if (tmp == null || tmp.key >= x.key) {
        x.next = tmp
        L.head = x
    } else {
        while (tmp.next != NULL && tmp.next.key < x.key)
            tmp = tmp.next
        x.next = tmp.next
        tmp.next = x
    }
}
```

L: the list

x: the item to be inserted

Complexity: **$O(n)$** (worst case)

Sorted Linked List: deleting operation

```
delete(L, k)
{
    if (L.head == null) return;
    if (L.head.key == k)
    {
        L.head = L.head.next;
        return;
    }
    c = L.head;
```

```
        while (c.next != null &&
               c.next.key <= k)
        {
            if (c.next.key == k)
            {
                c.next = c.next.next;
                return;
            }
            c = c.next;
        }
        return;
    }
```

L: the list

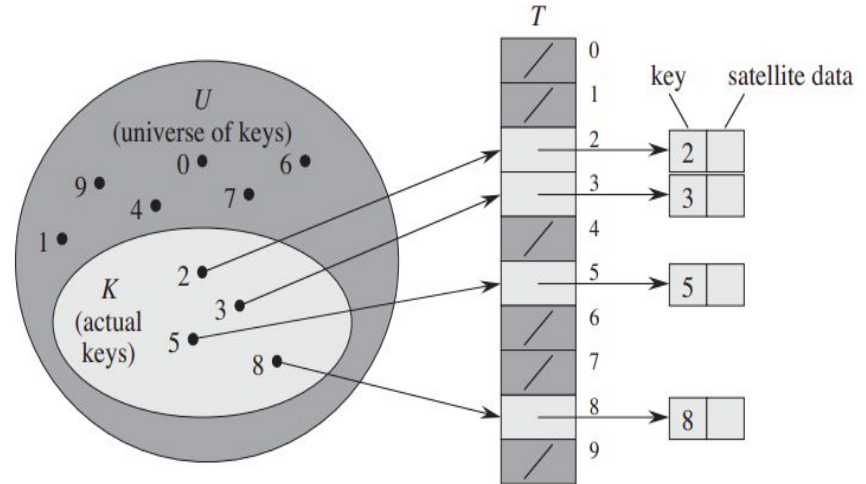
k: key to be deleted

Complexity: **$O(n)$** (worst case)

Hash Table

Hash Table

- Data structure for retrieving data in $O(1)$ time
- A key is used to access the array directly
- An Hash Table can be easily implemented using an Array



Hash Table: operations

```
insert(T, x)
{
    T[x.key] = x;
}
```

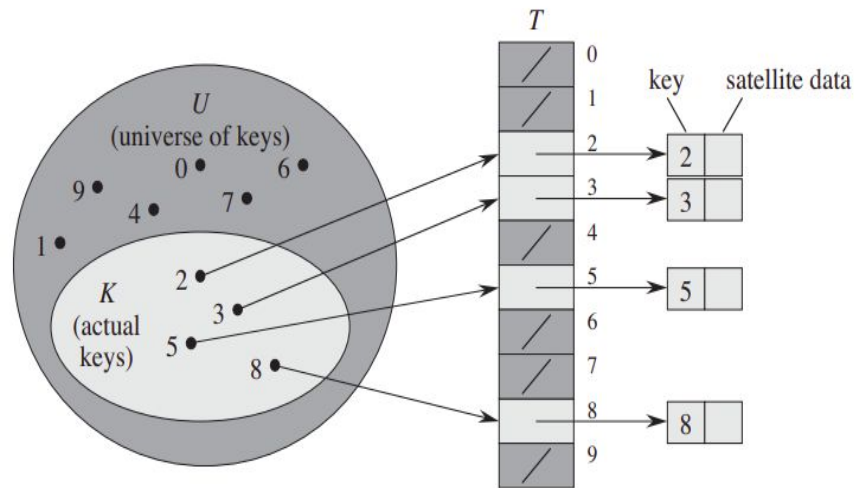
```
search(T, k)
{
    return T[k];
}
```

```
delete(T, x)
{
    T[x.key] = null;
}
```

Complexity: $O(1)$

Hash Functions

- If we use the key for accessing the Hash Table, **the size of the Hash Table** is $|U|$ where U is the universe of keys
- If $|U|$ is too large, we cannot store such **amount of data** in our memory
- The idea is to use an **hash function h** , which maps each key in the Universe with another key belonging to the set H where $|H| \ll |U|$



Hash Table: operations with hash function

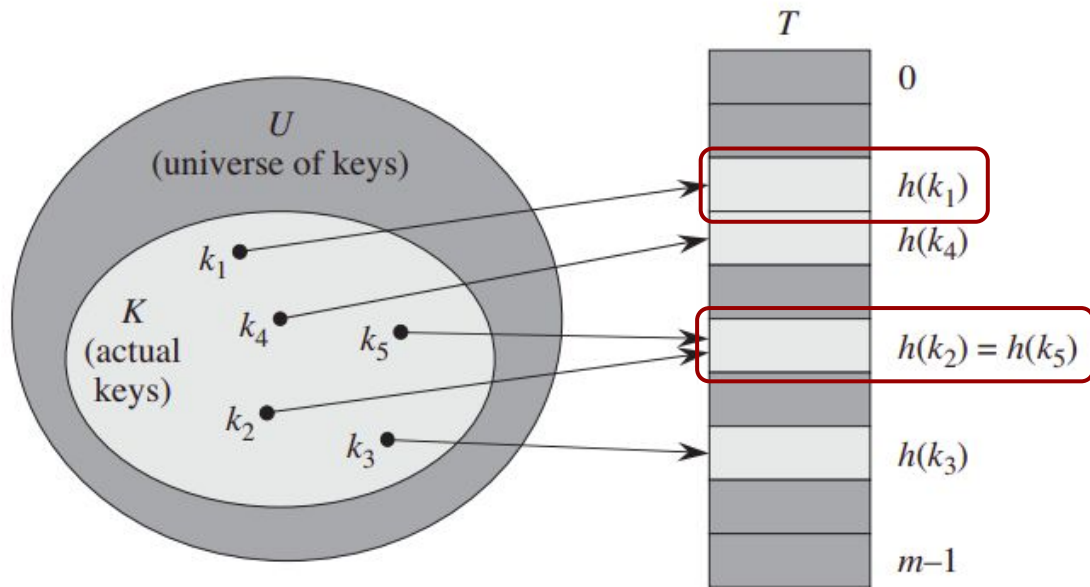
```
insert(T, x)
{
    T[h(x.key)] = x;
}
```

Complexity: $O(1)$

```
search(T, k)
{
    return T[h(k)];
}
```

```
delete(T, k)
{
    T[h(k)] = null;
}
```

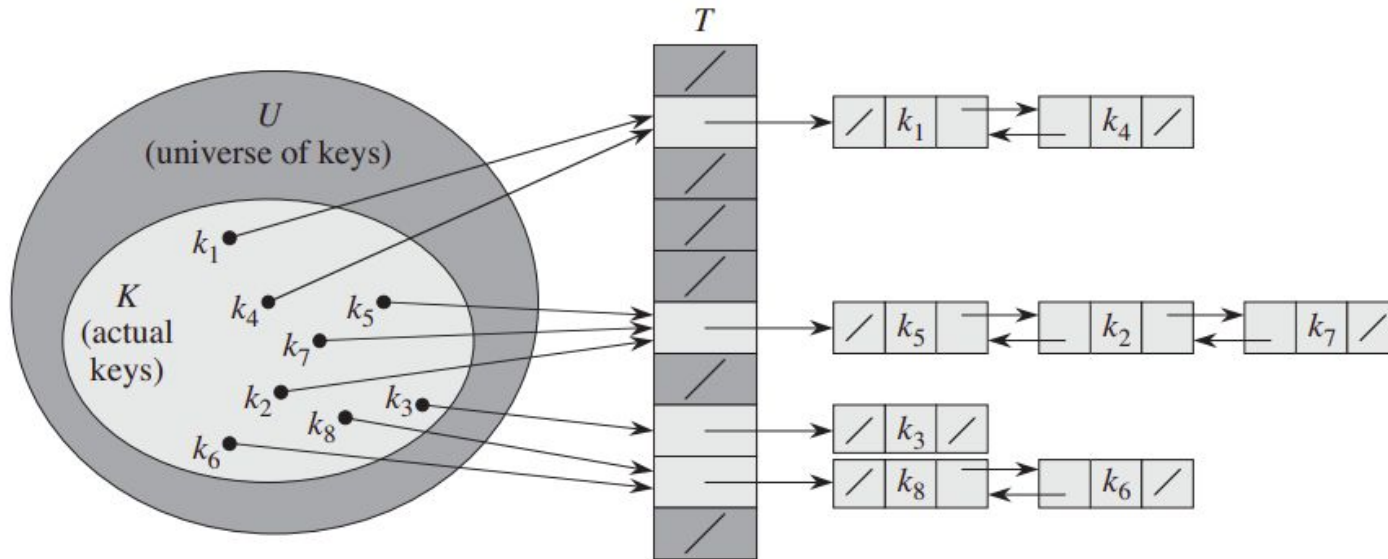
Collision



How to avoid collision

1. Be aware: **avoiding collision** is impossible!!!!
2. Using suitable hash functions to **minimizing collisions**
3. **Chaining**: place all the elements that hash to the same slot into the same linked list (see next picture)

Chaining



Hash Table: operations with chaining

```
insert(T, x)
{
    insert x at the head of the list T[h(x.key)];
}
```

Complexity: $O(1)$

```
search(T, k)
{
    search for an element with key k in list T[h(k)];
}
```

Complexity: it depends on the list length (see later)

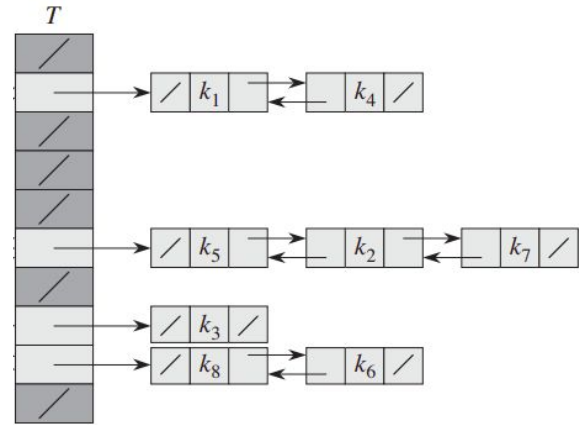
```
delete(T, k)
{
    delete x from the list T[h(k)];
}
```

Complexity: it depends on the list length (see later)

Complexity of the searching operation

- Let T be the Hash Table
- Let m the number of slots available in T
- Let n the number of elements currently stored in T
- Let $\alpha = n/m$ the **load factor** (representing the average number of element stored in a chain)

$$m = 10, n = 8, \alpha = 8/10 = 0.8$$



Complexity of the searching operation

- Worst case: all n keys end up to the same slot. The time for the searching operation is $O(n)$
- Average case: it depends on how well the hash function h is **able to distribute the set of keys** to be stored among the m slots.
- It is well known that the total time required for a successful search is

$$O(1 + \alpha)$$

Hash function: the division method

Let m be the size of the Hash Table, an hash function can be

$$h(k) = k \bmod m$$

Usually, m should be a **prime value** (to distribute evenly all the keys).

For example, we have an universe of 2000 elements, and we agree to have 3 elements on each chain. So, we need to use a prime value not too close to $2000/3$.

Then, we use $m = 701$

$$h(k) = k \bmod 701$$

Hash function: the multiplication method

- Let A be a constant in the range $0 < A < 1$
- Multiply k by A
- Take the decimal part of kA
- Multiply kA by m
- Take the floor

$$h(k) = \text{floor}(m(kA \bmod 1))$$

Hash function: the multiplication method

The advantage of this method is that **m is not critical**.

Usually we say **$m = 2^p$** , for some p .

For example $A = (\text{sqrt}(5) - 1) / 2 = 0.618033989$

$p = 14, m = 2^{14} = 16384$

$h(123456) = \text{floor} (16384 * (123456 * 0.618033989 \bmod 1)) =$

$\text{floor} (16384 * 0,004145984) = 67$

A little remind about Recursion

What is a recursive function?

```
private static int factRecursive(int num){
```

```
    if(num < 0){  
        return -1;  
    }
```

```
    if(num == 1 || num == 0){  
        return 1;  
    }
```

```
    num = num * factRecursive(num-1);  
    return num;  
}
```

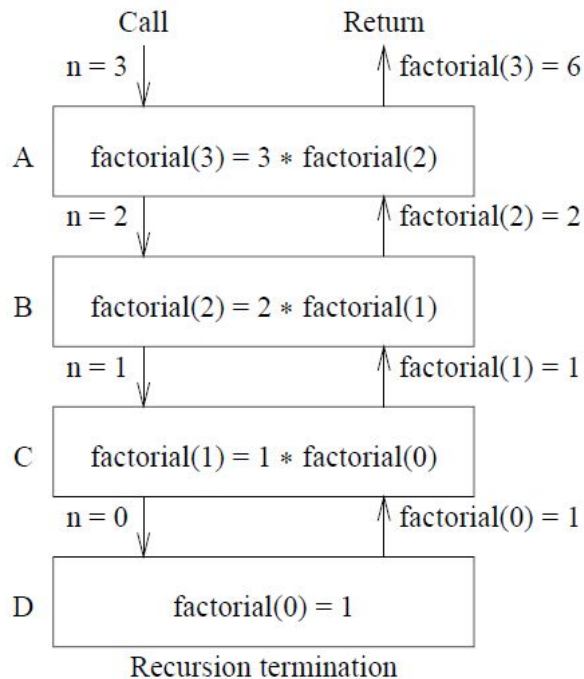
Recursive Method

Base Case (necessary condition to break recursive call)

Recursive call to same method

Recursive Function: Factorial

```
int factorial(int num)
{
    # base case
    if (num == 1)
        return 1;
    # recursive step
    else
        return num * factorial(n-1);
}
```



(a)

Recursive Function: Fibonacci sequence

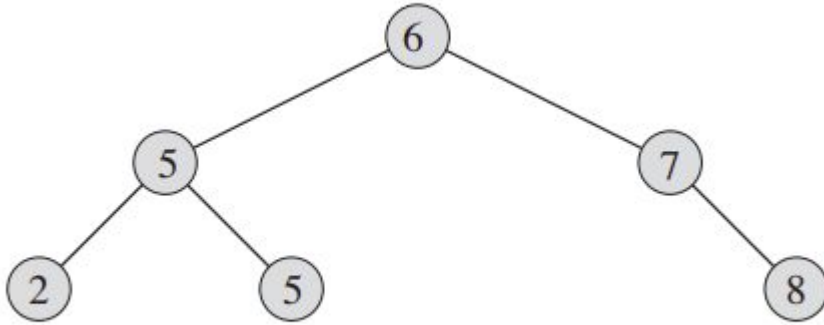
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, ...

```
int fibonacci(int n) {  
    if(n == 0) {          # base case 1  
        return 0;  
    } else if(n == 1) {   # base case 2  
        return 1;  
    } else {              # recursive step  
        return (fibonacci(n-1) + fibonacci(n-2));  
    }  
}
```

Binary Tree

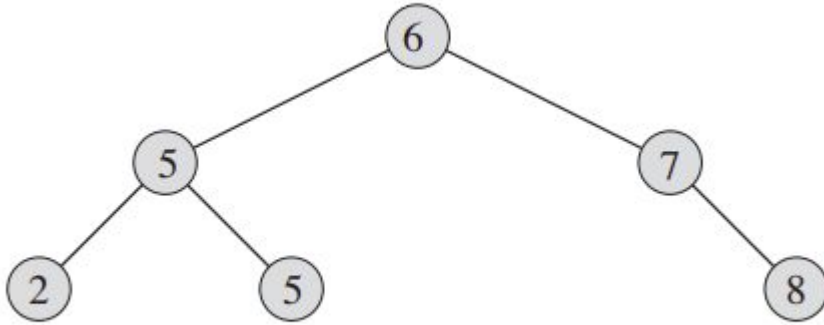


Binary Tree: properties



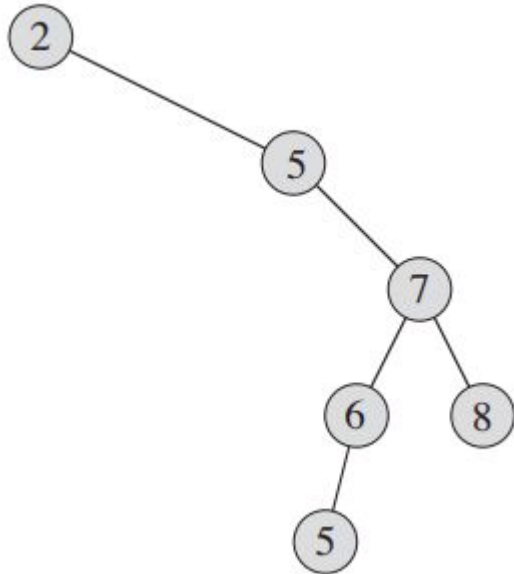
1. Each node can have 0, 1 or 2 children
2. All the children on the left subtree have a key $\leq$ than the key of the root
3. All the children on the right subtree have a key $\geq$ than the key of the root

Binary Tree: properties



- If the tree is balanced, the worst time for searching a key is $O(\log n)$

Unbalanced Binary Tree



- If the tree is completely unbalanced, the worst time for searching a key is $O(n)$, it works like a list

Inorder Tree Walk

```
inorderTreeWalk(x)
{
    if (x != NULL)
    {
        inorderTreeWalk(x.left);
        print x.key;
        inorderTreeWalk(x.right);
    }
}
```

- Recursive procedure
- x is the root of the Binary Tree

Complexity: $O(n)$

Search

```
treeSearch(x, k)
{
    if (x == NULL || k == x.key)
        return x;
    if (k < x.key)
        return treeSearch(x.left, k)
    else
        return treeSearch(x.right, k)
}
```

- Recursive procedure
- x is the root of the Binary Tree
- k is the key we are looking for

Complexity: $O(\log n)$

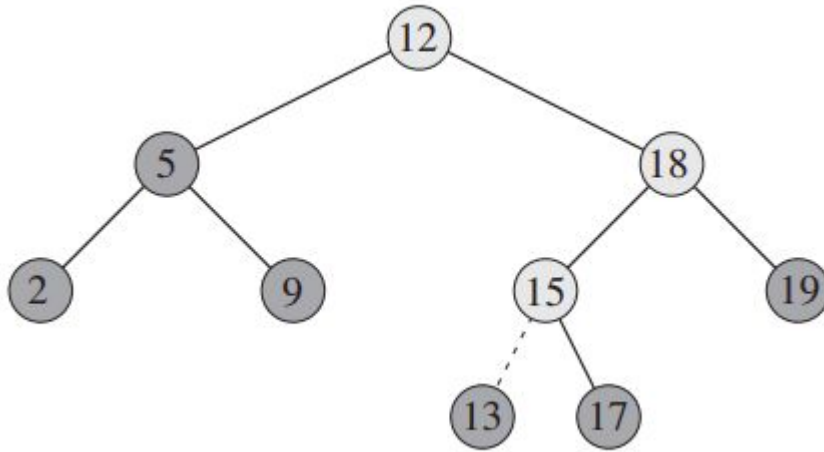
Search

```
treeSearch(x, k)
{
    while (x != null && x.key != k)
    {
        if (k < x.key)
            x = x.left;
        else
            x = x.right;
    }
    return x;
}
```

- Iterative procedure
- x is the root of the Binary Tree
- k is the key we are looking for

Complexity: $O(\log n)$

Insert



- We want to insert 13
- We walk down the tree, till reaching 15, whose left child is null
- We add 13 as left child

Insert

```
treeInsert(T, z)
{
    x = T.root
    y = NULL

    while (x != NULL)
    {
        y = x
        if (z.key < x.key)
            x = x.left
        else
            x = x.right
    }
    z.parent = y
```

```
    if (y == null)
        T.root = z
    else if (z.key < y.key)
        y.left = z;
    else
        y.right = z
```

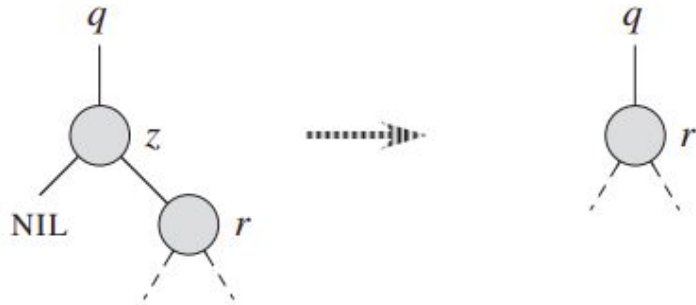
Complexity: $O(\log n)$

Deletion

Deleting a node in a binary tree is not easy as Inserting

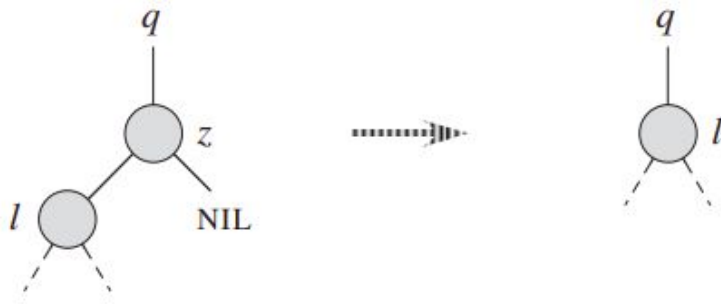
Four different cases

Deletion: case 1



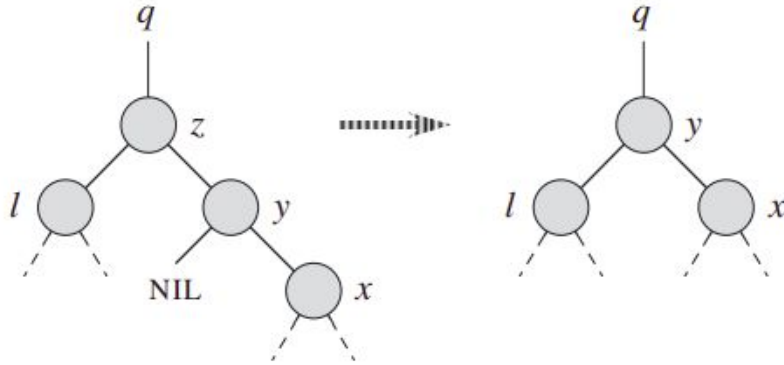
- We want to delete z
- If z has not left child, we replace z with the right child (this works even when z has no right child)

Deletion: case 2



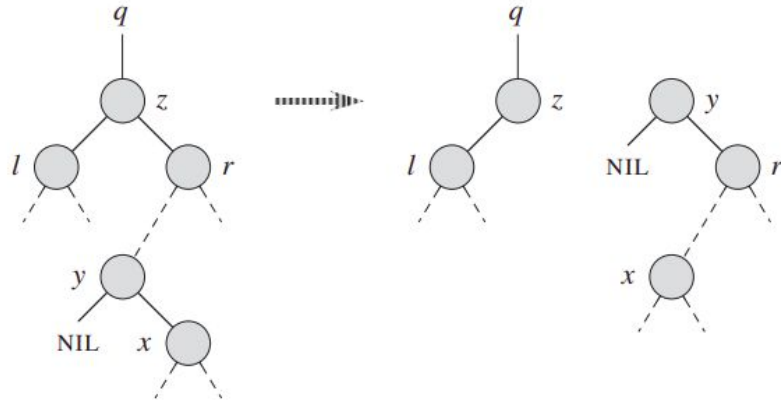
- We want to delete z
- If z has not right child, we replace z with the left child (this works even when z has no right child)

Deletion: case 3



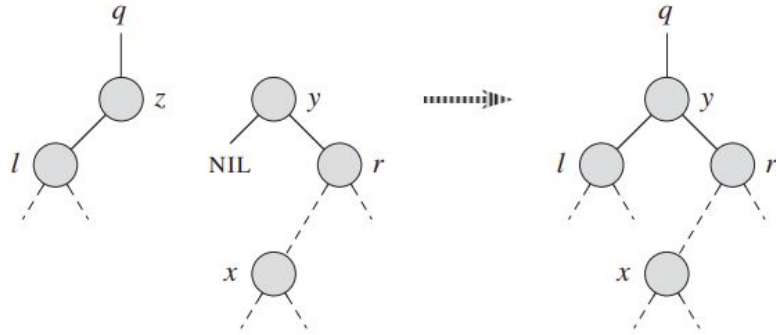
- We want to delete z
- Node z has both left and right child.
- We find z's successor y which lies in z's right subtree and has no left child
- if y is the right child of z, we replace z with y, putting the z's left subtree as the y's left subtree

Deletion: case 4 step a



- We want to delete z
- Node z has both left and right child.
- We find z 's successor y which lies in z 's right subtree and has no left child, but y is not the z 's right child
- we cut the z 's right child, we put x as left child of r , and r as the right child of y

Deletion: case 4 step b



- we put z 's left child as y 's left child
- we replace z with y

Deletion

TREE-DELETE(T, z)

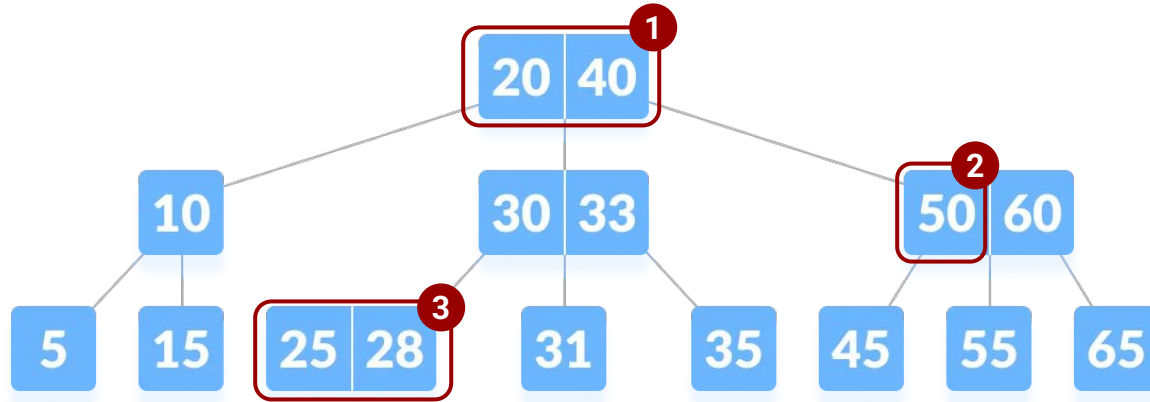
```
1  if  $z.left == \text{NIL}$ 
2      TRANSPLANT( $T, z, z.right$ )
3  elseif  $z.right == \text{NIL}$ 
4      TRANSPLANT( $T, z, z.left$ )
5  else  $y = \text{TREE-MINIMUM}(z.right)$ 
6      if  $y.p \neq z$ 
7          TRANSPLANT( $T, y, y.right$ )
8           $y.right = z.right$ 
9           $y.right.p = y$ 
10     TRANSPLANT( $T, z, y$ )
11      $y.left = z.left$ 
12      $y.left.p = y$ 
```

TRANSPLANT(T, u, v)

```
1  if  $u.p == \text{NIL}$ 
2       $T.root = v$ 
3  elseif  $u == u.p.left$ 
4       $u.p.left = v$ 
5  else  $u.p.right = v$ 
6  if  $v \neq \text{NIL}$ 
7       $v.p = u.p$ 
```

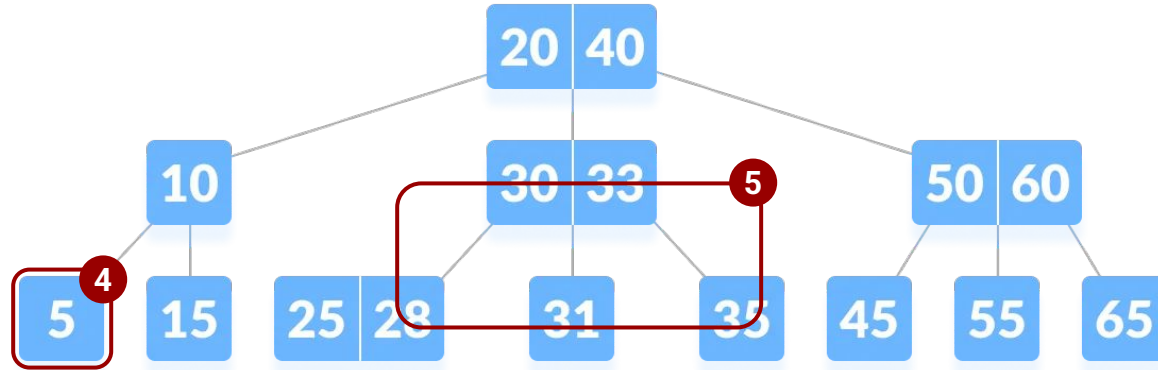
B-Tree

B-Tree: properties



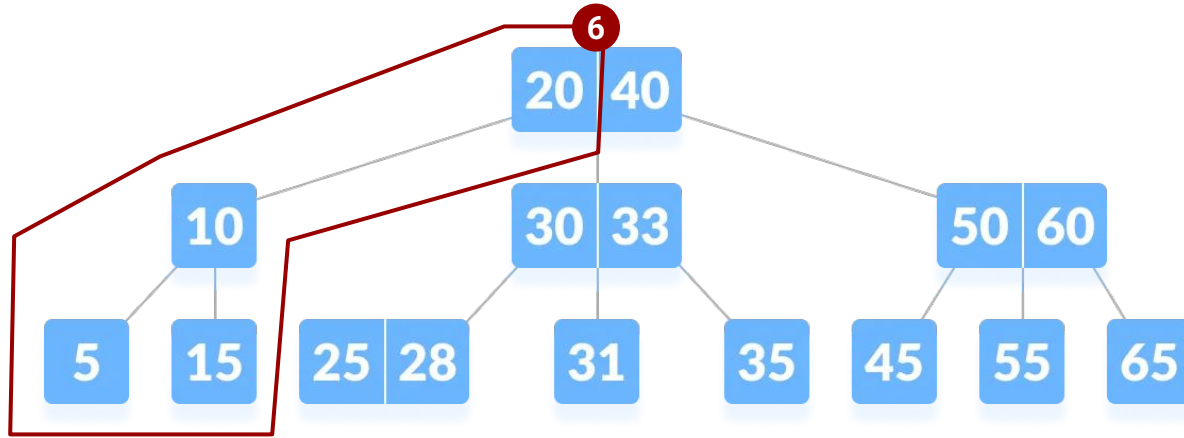
1. A node **x** can have **n** keys, where **x.n** is the number of keys
2. If **x** is a node, **x.key_i** is the *i*-th key of **x**
3. All *n* keys are sorted in non decreasing order: **x.key₁ ≤ x.key₂ ≤ x.key_n**

B-Tree: properties



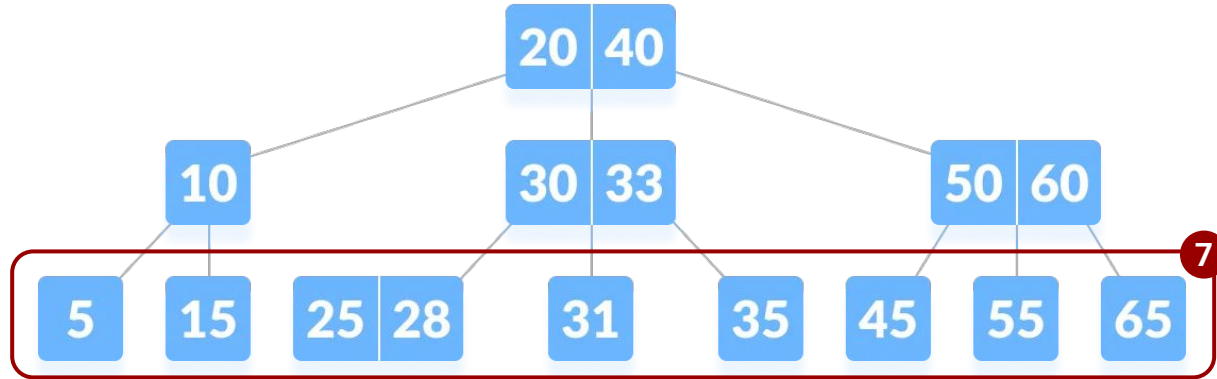
4. **x.leaf = true** if x is a leaf. If x is an internal node, **x.leaf = false**
5. Each internal node x contains **x.n+1** pointers, $x.c_1, x.c_2, \dots, x.c_{n+1}$ to its children

B-Tree: properties



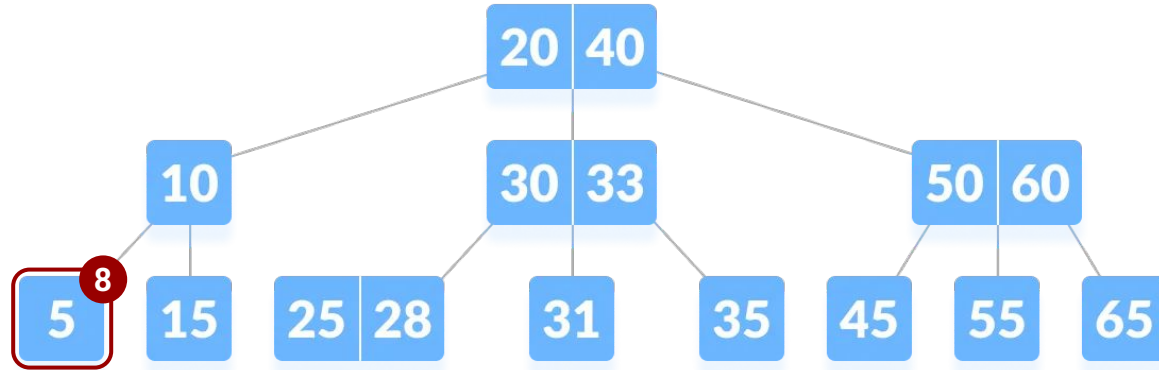
6. If x is a node, $x.\text{key}_i$ is the i -th key of x , all the children in $x.c_i$ are less than or equal to $x.\text{key}_i$

B-Tree: properties



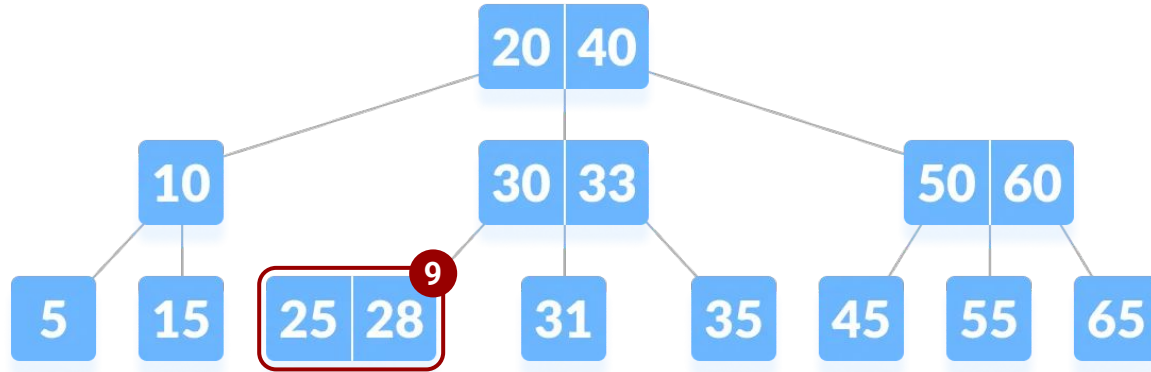
7. All leaves have the same depth, which is the tree's height h

B-Tree: properties



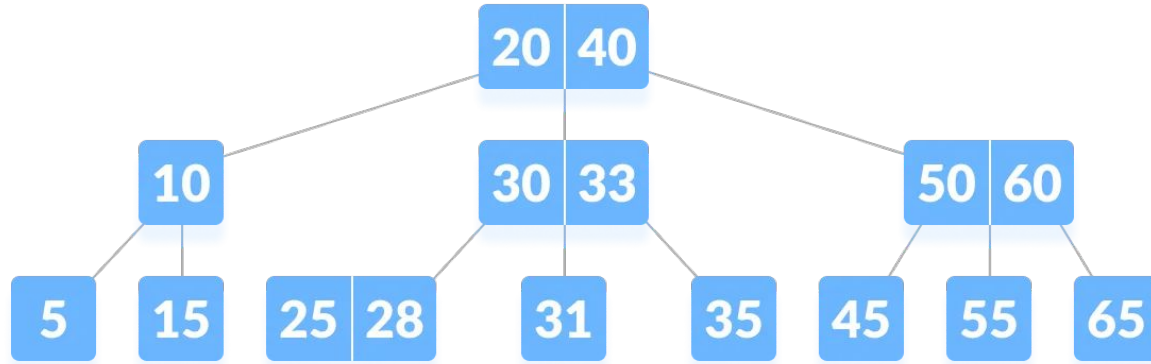
8. Let $t \geq 2$ be an integer, called **minimum degree** of the B-Tree, each node other than the root must have at least $t-1$ keys. The root must have at least one key

Properties of a B-Tree



9. Let $t \geq 2$ be an integer, called **minimum degree** of the B-Tree, each node may contain at most $2t-1$ keys. A node is **full** if it contains exactly $2t-1$ keys

Properties of a B-Tree



This B-Tree has a **minimum degree equals to $t = 2$** . Every internal node then has either 1 or 3 children. In practice, however, larger values of t yield B-trees with smaller height

Theorem

If the **number of keys** $n \geq 1$, for any n -key B-tree T of height h and minimum degree $t \geq 2$,

$$\log_{2t}(n + 1) - 1 \leq h \leq \log_t \frac{n + 1}{2}$$

Theorem: proof

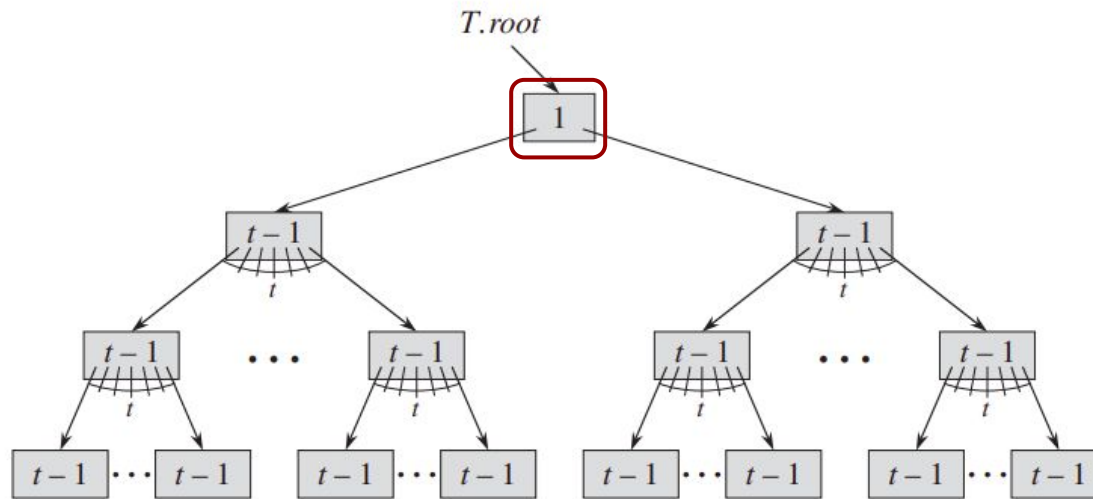
Let's prove that
$$h \leq \log_t \frac{n+1}{2}$$

According to the property 8, having a B-Tree with height ***h*** and a minimum degree equals to ***t***, the minimum number of keys is:

- one key for the root
- $t-1$ keys for the internal node.

See next picture

Theorem: proof

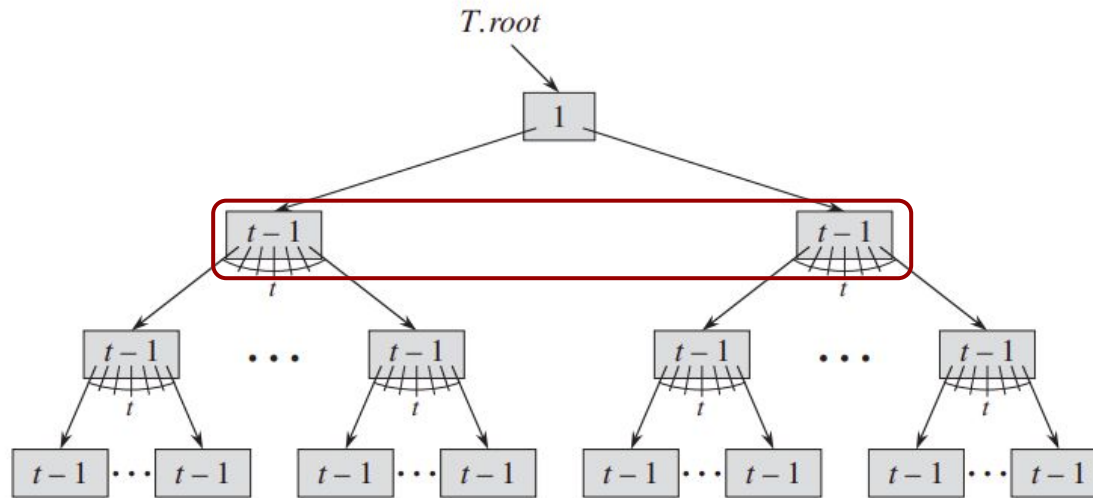


depth	number of nodes
0	1
1	2
2	$2t$
3	$2t^2$

$$n \geq 1 + \dots$$

total number of keys

Theorem: proof

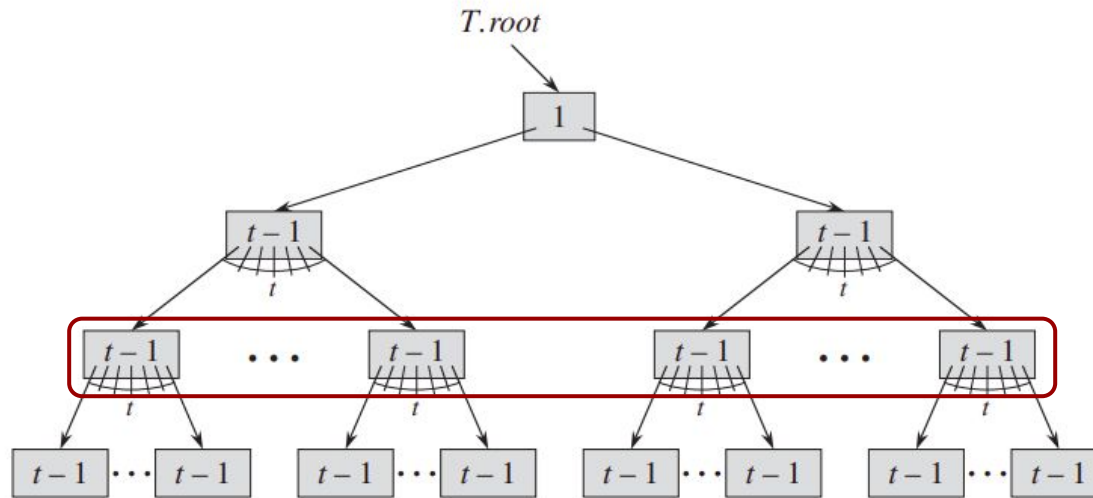


depth	number of nodes
0	1
1	2
2	$2t$
3	$2t^2$

$$n \geq 1 + 2(t-1) + \dots$$

total number of keys

Theorem: proof

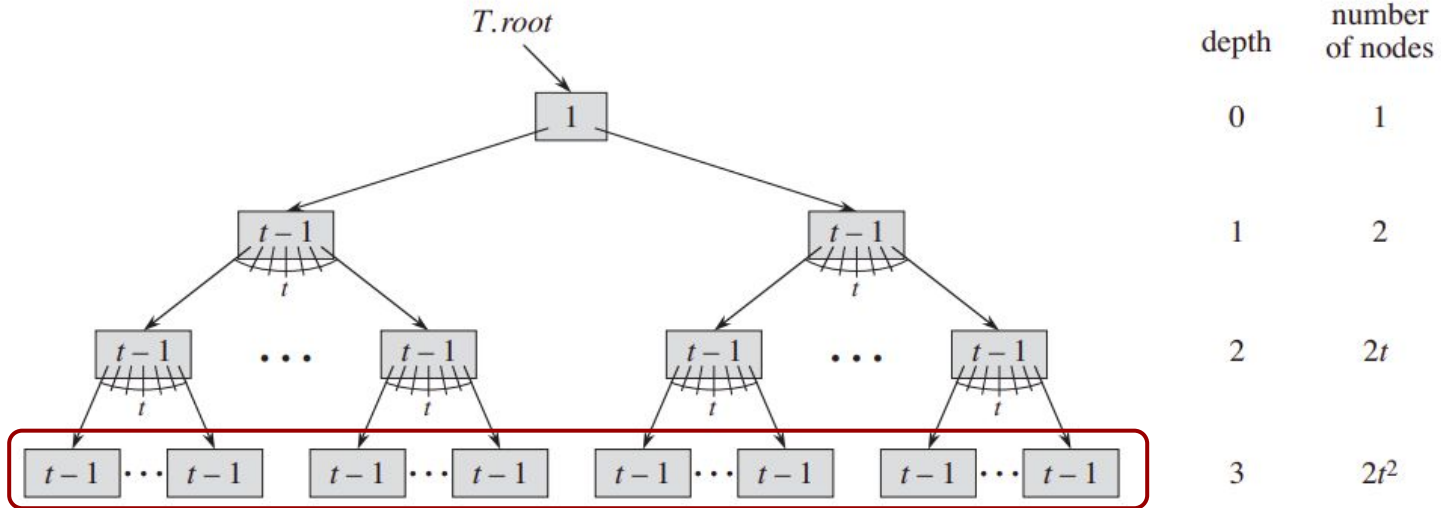


depth	number of nodes
0	1
1	2
2	$2t$
3	$2t^2$

$$n \geq 1 + 2(t-1) + \boxed{2t(t-1)} + \dots$$

total number of keys

Theorem: proof



$$n \geq 1 + 2(t - 1) + 2t(t - 1) + 2t^2(t + 1)$$

total number of keys

Theorem: proof

$$n \geq 1 + 2(t - 1) + 2t(t - 1) + 2t^2(t + 1)$$

$$n \geq 1 + \sum_{i=1}^h 2t^{i-1}(t - 1)$$

$$n \geq 1 + 2(t - 1) \sum_{i=1}^h t^{i-1}$$

Theorem: proof

$$n \geq 1 + 2(t - 1) \sum_{i=1}^h t^{i-1}$$

$$n \geq 1 + 2(t - 1) \left(\frac{t^h - 1}{t - 1} \right) = 1 + 2(t^h - 1)$$

Theorem: proof

$$n \geq 1 + 2(t^h - 1) = 2t^h - 1$$

$$n + 1 \geq 2t^h$$

$$\frac{n + 1}{2} \geq t^h$$

$$h \leq \log_t \frac{n + 1}{2}$$

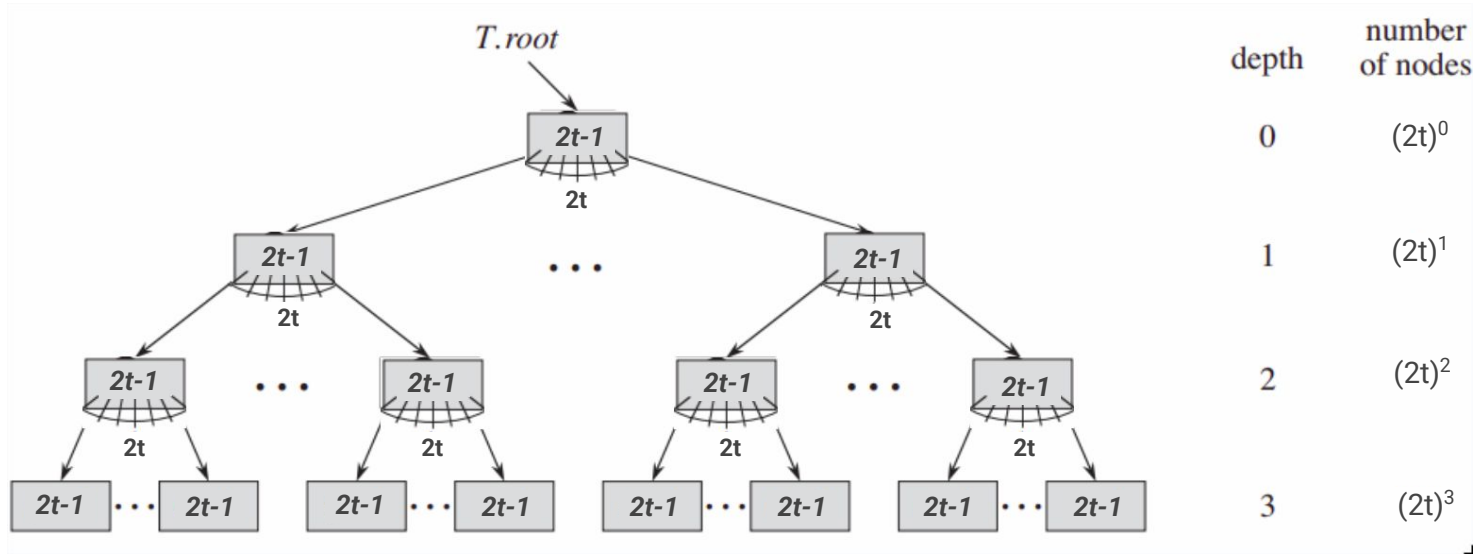
Theorem: proof

Let's prove that $\log_{2t}(n + 1) - 1 \leq h$

According to the property 9, having a B-Tree with height ***h*** and a minimum degree equals to ***t***, the maximum number of keys is $2t - 1$

See next picture

Theorem: proof



$$n \leq (2t - 1)(2t)^0 + (2t - 1)(2t)^1 + (2t - 1)(2t)^2 + (2t - 1)(2t)^3$$

number of keys

maximum number of keys per node

Theorem: proof

$$n \leq (2t-1)(2t)^0 + (2t-1)(2t)^1 + (2t-1)(2t)^2 + (2t-1)(2t)^3$$

$$n \leq (2t-1) \sum_{i=0}^h (2t)^i = (2t-1) \frac{(2t)^{h+1} - 1}{2t-1} = (2t)^{h+1} - 1$$

$$n \leq (2t)^{h+1} - 1$$

Theorem: proof

$$n \leq (2t)^{h+1} - 1$$

$$n + 1 \leq (2t)^{h+1}$$

$$\log_{2t}(n + 1) \leq h + 1$$

$$\log_{2t}(n + 1) - 1 \leq h$$

Creation Operation

B-TREE-CREATE(T)

```
1   $x = \text{ALLOCATE-NODE}()$ 
2   $x.\text{leaf} = \text{TRUE}$ 
3   $x.n = 0$ 
4   $\text{DISK-WRITE}(x)$ 
5   $T.\text{root} = x$ 
```

1. Create an empty node for the new tree T
2. Set the node a leaf
3. Set the number of keys to zero
4. Save the node on the disk
5. Set the root of the tree T

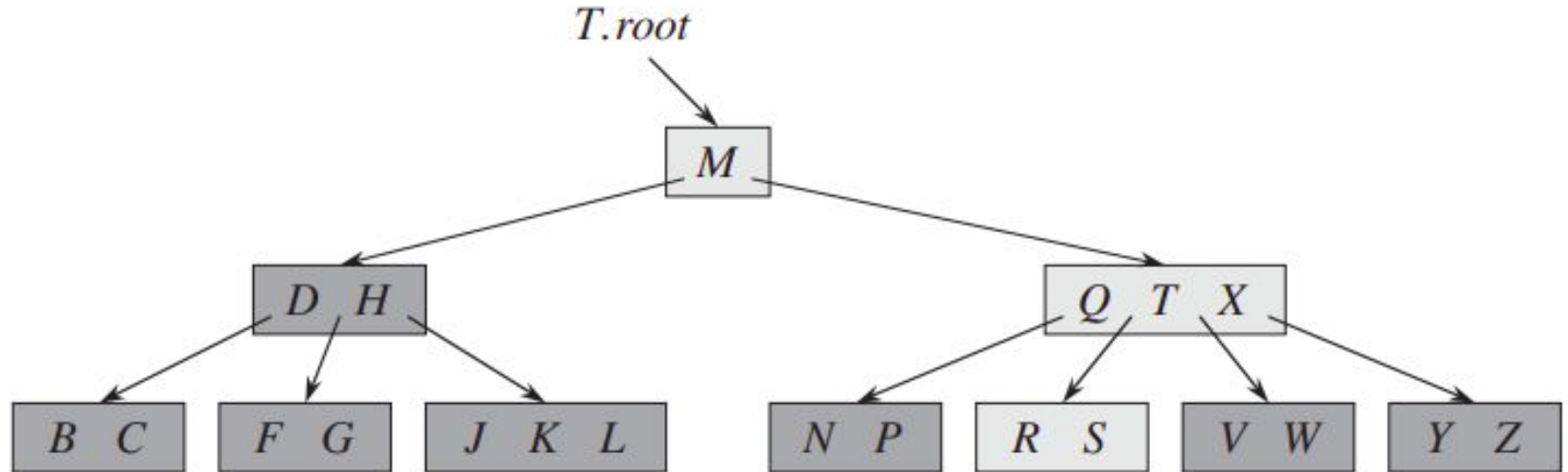
Creation Operation: complexity

B-TREE-CREATE(T)

```
1   $x = \text{ALLOCATE-NODE}()$ 
2   $x.\text{leaf} = \text{TRUE}$ 
3   $x.n = 0$ 
4   $\text{DISK-WRITE}(x)$ 
5   $T.\text{root} = x$ 
```

Execution Time = $O(1)$

Searching Operation for key = R



Searching Operation

B-TREE-SEARCH(x, k)

x : the root the the Tree

k : the key we are looking for

```
1   $i = 1$ 
2  while  $i \leq x.n$  and  $k > x.key_i$ 
3       $i = i + 1$ 
4  if  $i \leq x.n$  and  $k == x.key_i$ 
5      return  $(x, i)$ 
6  elseif  $x.leaf$ 
7      return NIL
8  else DISK-READ( $x.c_i$ )
9      return B-TREE-SEARCH( $x.c_i, k$ )
```

Searching Operation

B-TREE-SEARCH(x, k)

```
1   $i = 1$ 
2  while  $i \leq x.n$  and  $k > x.key_i$ 
3       $i = i + 1$ 
4  if  $i \leq x.n$  and  $k == x.key_i$ 
5      return  $(x, i)$ 
6  elseif  $x.leaf$ 
7      return NIL
8  else DISK-READ( $x.c_i$ )
9      return B-TREE-SEARCH( $x.c_i, k$ )
```

Scan all the keys in the root till we find a key greater than k (**linear scan**)

Searching Operation

B-TREE-SEARCH(x, k)

if we have found the key, just return it

```
1   $i = 1$ 
2  while  $i \leq x.n$  and  $k > x.key_i$ 
3       $i = i + 1$ 
4  if  $i \leq x.n$  and  $k == x.key_i$ 
5      return  $(x, i)$ 
6  elseif  $x.leaf$ 
7      return NIL
8  else DISK-READ( $x.c_i$ )
9      return B-TREE-SEARCH( $x.c_i, k$ )
```

Searching Operation

B-TREE-SEARCH(x, k)

```
1   $i = 1$ 
2  while  $i \leq x.n$  and  $k > x.key_i$ 
3       $i = i + 1$ 
4  if  $i \leq x.n$  and  $k == x.key_i$ 
5      return  $(x, i)$ 
6  elseif  $x.leaf$ 
7      return NIL
8  else DISK-READ( $x.c_i$ )
9      return B-TREE-SEARCH( $x.c_i, k$ )
```

If we haven't found yet the key, and the root is a leaf, then give up

Searching Operation

B-TREE-SEARCH(x, k)

```
1   $i = 1$ 
2  while  $i \leq x.n$  and  $k > x.key_i$ 
3       $i = i + 1$ 
4  if  $i \leq x.n$  and  $k == x.key_i$ 
5      return  $(x, i)$ 
6  elseif  $x.leaf$ 
7      return NIL
8  else DISK-READ( $x.c_i$ )
9      return B-TREE-SEARCH( $x.c_i, k$ )
```

If we haven't found yet the key, and the root is not a leaf, search for the key if the subtree (**recursive call**)

Searching Operation: complexity

B-TREE-SEARCH(x, k)

```
1   $i = 1$ 
2  while  $i \leq x.n$  and  $k > x.key_i$ 
3       $i = i + 1$ 
4  if  $i \leq x.n$  and  $k == x.key_i$ 
5      return  $(x, i)$ 
6  elseif  $x.leaf$ 
7      return NIL
8  else DISK-READ( $x.c_i$ )
9  return B-TREE-SEARCH( $x.c_i, k$ )
```

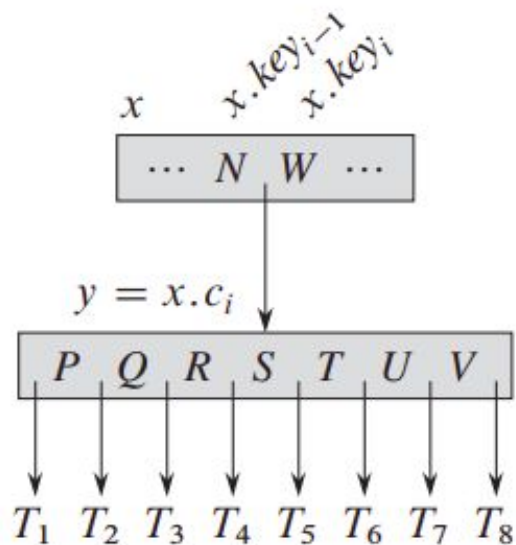
1. The recursive B-Tree-Search function is invoked $\log_t n$
2. Every time the function is invoked, the while cycle is executed $2t$ times the most

Execution time = $O(t \log_t n)$

Splitting Operation

- It is invoked when **a node is full** (the number of keys is $2t - 1$)
- During Splitting Operation, a full node is split in two nodes having $t - 1$ keys each
- After its execution, **the B-Tree grows in height by one**

Splitting Operation

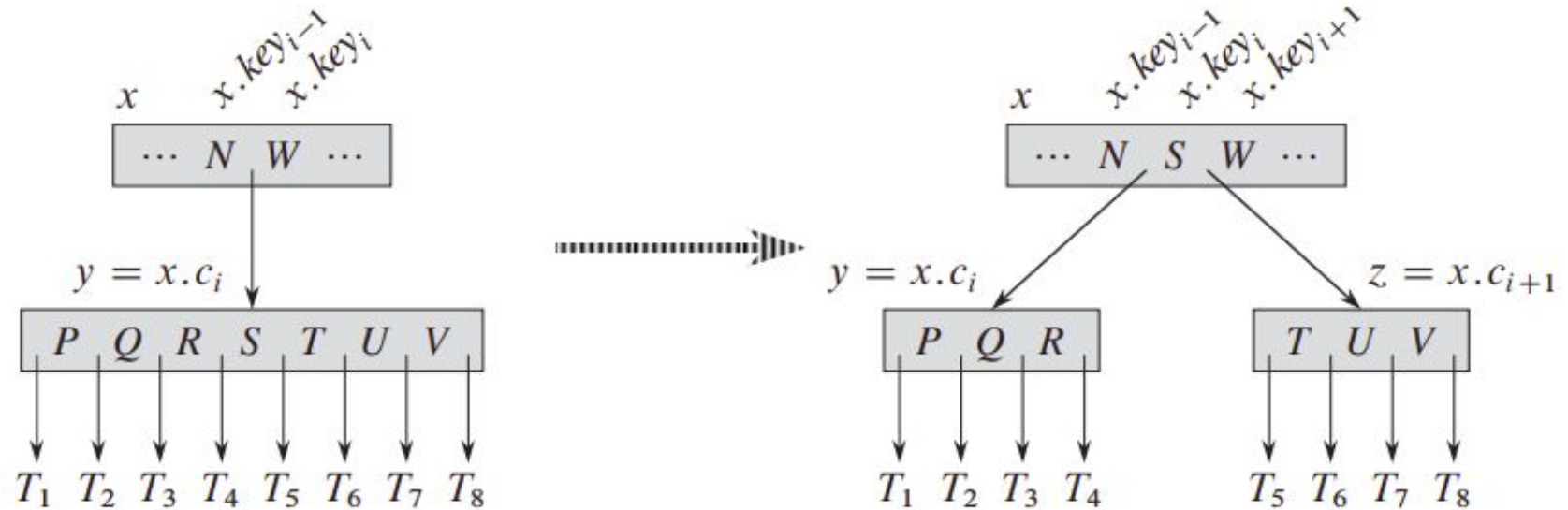


In this example $t = 4$, then a node is full when the number of keys is $2t - 1 = 7$

The node y is full, x instead is a **non-full** node

The key S is the median element in y

Splitting Operation



Splitting Operation

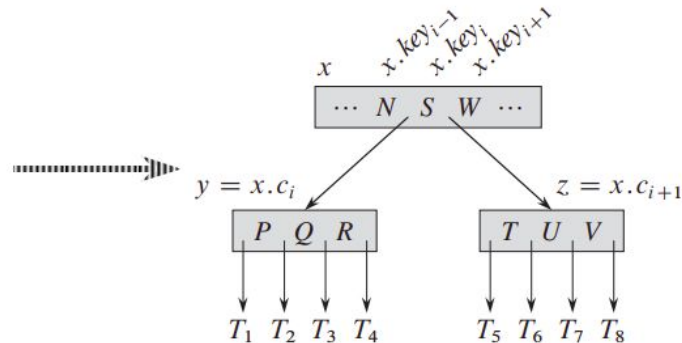
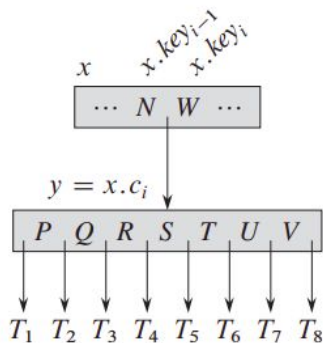
B-TREE-SPLIT-CHILD(x, i)

```
1   $z = \text{ALLOCATE-NODE}()$ 
2   $y = x.c_i$ 
3   $z.\text{leaf} = y.\text{leaf}$ 
4   $z.n = t - 1$ 
5  for  $j = 1$  to  $t - 1$ 
6       $z.\text{key}_j = y.\text{key}_{j+t}$ 
7  if not  $y.\text{leaf}$ 
8      for  $j = 1$  to  $t$ 
9           $z.c_j = y.c_{j+t}$ 
10  $y.n = t - 1$ 
```

x is a nonfull internal node

i is the index such that $x.c_i$ is a full child of x

1. node z is created and initialized

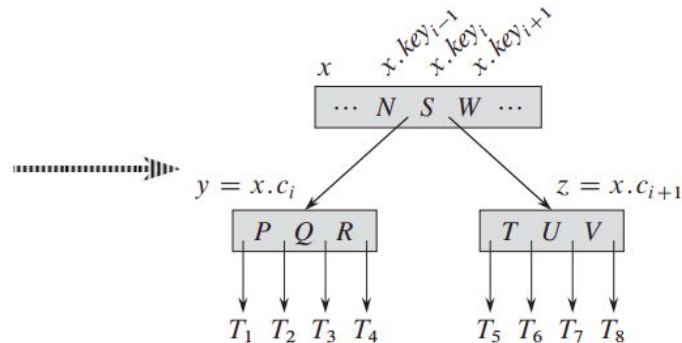
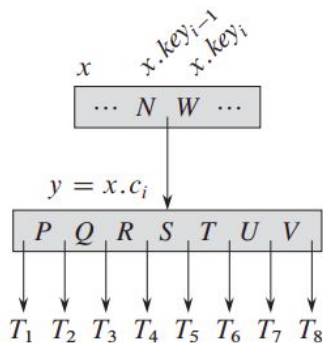


Splitting Operation

B-TREE-SPLIT-CHILD(x, i)

```
1   $z = \text{ALLOCATE-NODE}()$ 
2   $y = x.c_i$ 
3   $z.\text{leaf} = y.\text{leaf}$ 
4   $z.n = t - 1$ 
5  for  $j = 1$  to  $t - 1$ 
6       $z.\text{key}_j = y.\text{key}_{j+t}$ 
7  if not  $y.\text{leaf}$ 
8      for  $j = 1$  to  $t$ 
9           $z.c_j = y.c_{j+t}$ 
10  $y.n = t - 1$ 
```

2. All the keys in z are copied from y
3. All the links to the z 's children are copied from y
4. Adjusts the key count for y



Splitting Operation

```

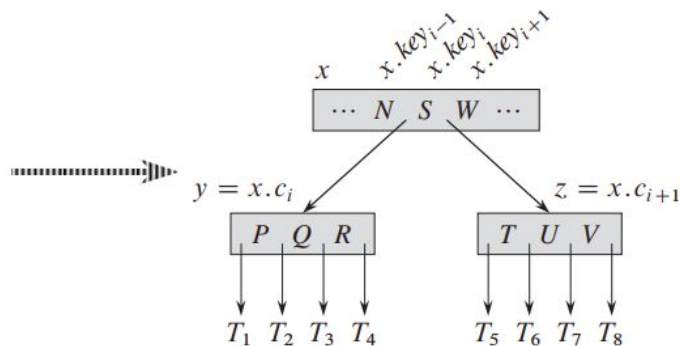
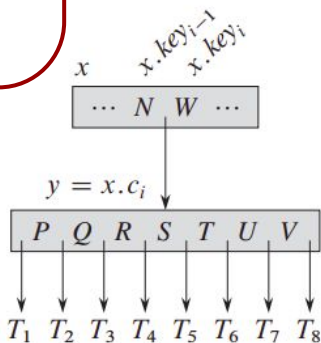
11  for  $j = x.n + 1$  downto  $i + 1$ 
12       $x.c_{j+1} = x.c_j$ 
13   $x.c_{i+1} = z$ 
14  for  $j = x.n$  downto  $i$ 
15       $x.key_{j+1} = x.key_j$ 
16   $x.key_i = y.key_t$ 
17   $x.n = x.n + 1$ 
    
```

```

18  DISK-WRITE( $y$ )
19  DISK-WRITE( $z$ )
20  DISK-WRITE( $x$ )
    
```

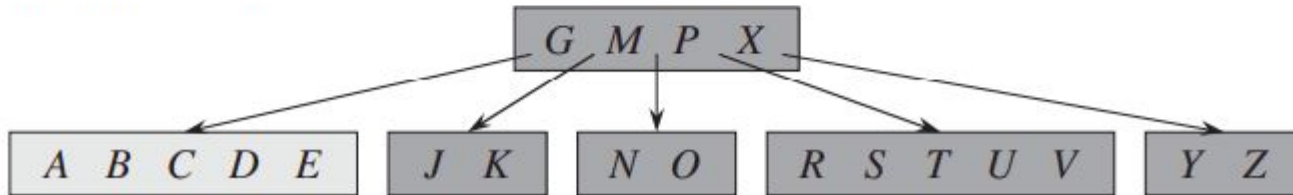
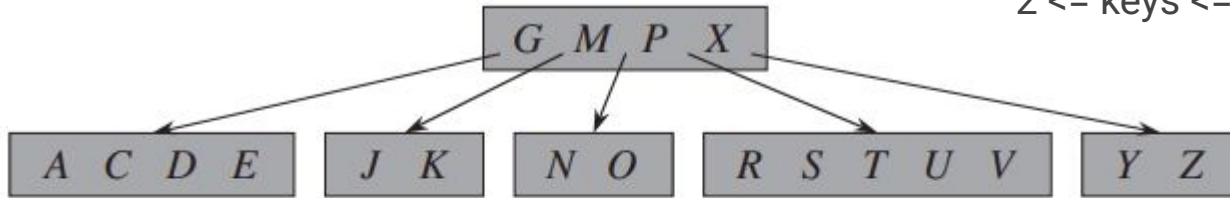
5. Insert z as a child of x , move the median key from y up to x
6. Save permanently all the modified node

Complexity = $O(t)$



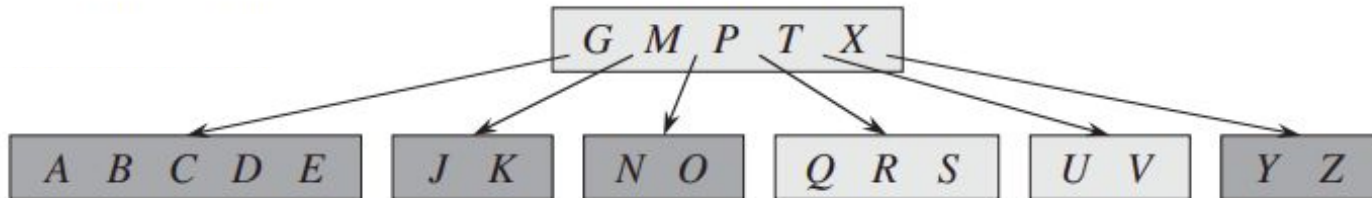
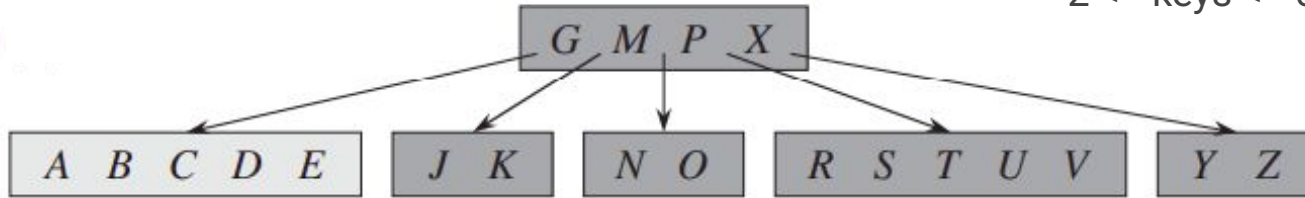
Inserting Operation: inserting B

$t = 3$
 $2 \leq \text{keys} \leq 5$

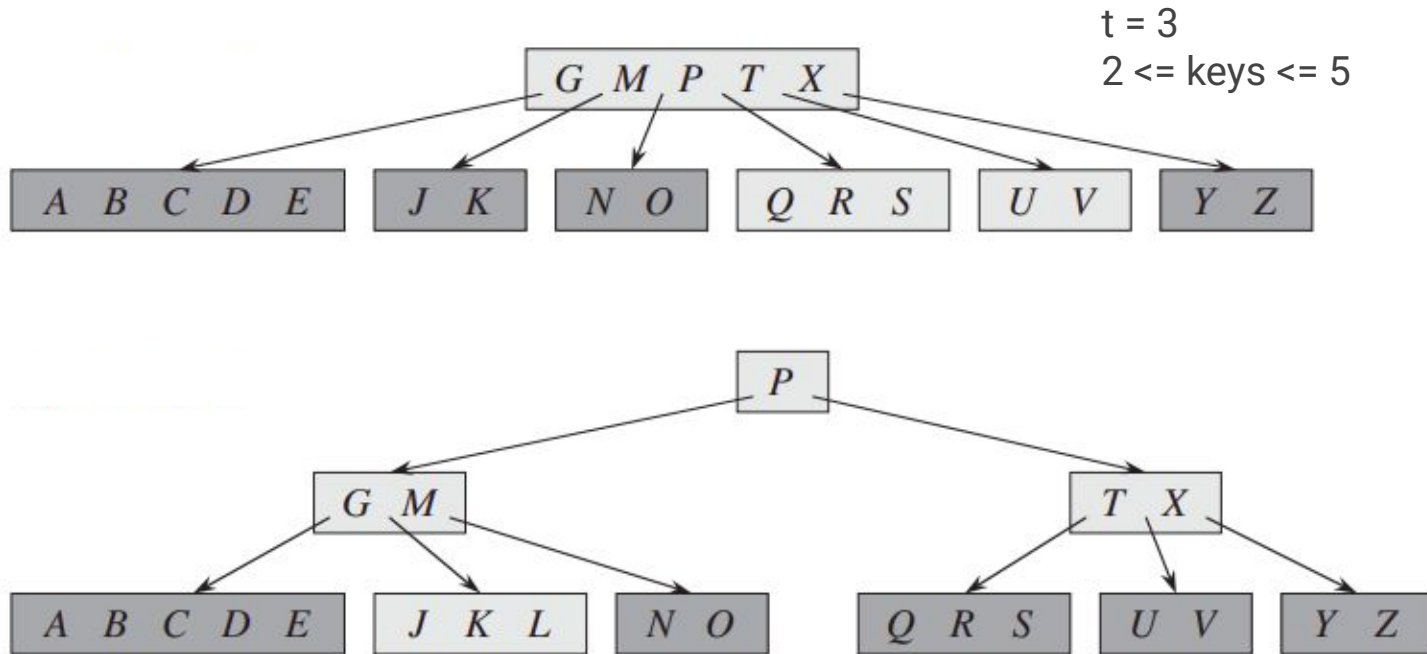


Inserting Operation: inserting Q

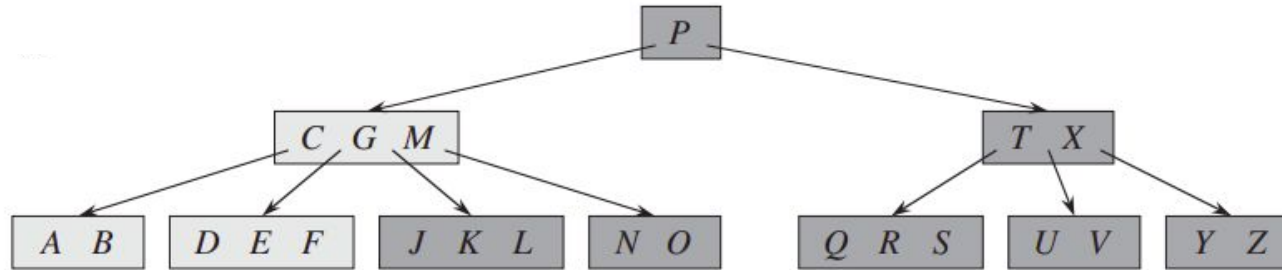
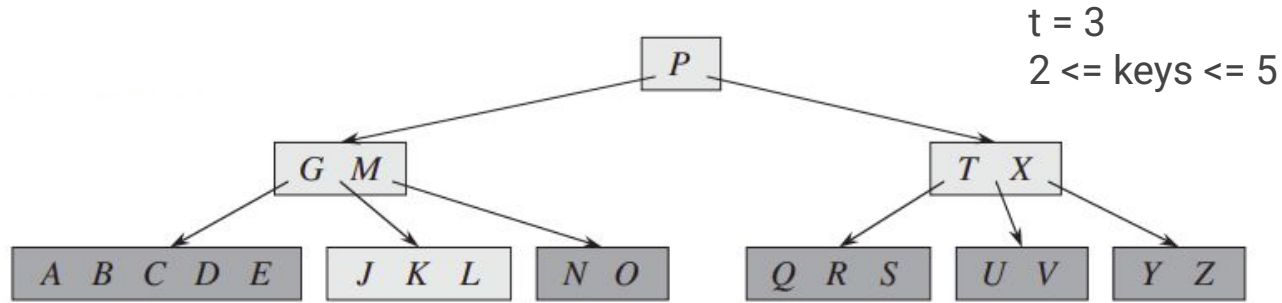
$t = 3$
 $2 \leq \text{keys} \leq 5$



Inserting Operation: inserting L



Inserting Operation: inserting F



Insertion Operation

B-TREE-INSERT(T, k)

```
1   $r = T.root$ 
2  if  $r.n == 2t - 1$ 
3       $s = \text{ALLOCATE-NODE}()$ 
4       $T.root = s$ 
5       $s.leaf = \text{FALSE}$ 
6       $s.n = 0$ 
7       $s.c_1 = r$ 
8  B-TREE-SPLIT-CHILD( $s, 1$ )
9  B-TREE-INSERT-NONFULL( $s, k$ )
10 else B-TREE-INSERT-NONFULL( $r, k$ )
```

T is the B Tree, k is the key to be inserted

1. If the root is full, the root is split, a new node s becomes the root. So, s is a node without any key ($s.n = 0$), with only one pointer pointing to r (the old Tree Root)
2. The first (and unique) child of s is split. This increases the height of a B-tree

Insertion Operation

B-TREE-INSERT(T, k)

```
1   $r = T.root$ 
2  if  $r.n == 2t - 1$ 
3       $s = \text{ALLOCATE-NODE}()$ 
4       $T.root = s$ 
5       $s.leaf = \text{FALSE}$ 
6       $s.n = 0$ 
7       $s.c_1 = r$ 
8      B-TREE-SPLIT-CHILD( $s, 1$ )
9      B-TREE-INSERT-NONFULL( $s, k$ )
10 else B-TREE-INSERT-NONFULL( $r, k$ )
```

3. If the original root is full, call the Tree-Insert-NonFull procedure on s
4. Otherwise, call the procedure on r

Insertion Operation

B-TREE-INSERT-NONFULL(x, k)

```
1   $i = x.n$ 
2  if  $x.leaf$ 
3      while  $i \geq 1$  and  $k < x.key_i$ 
4           $x.key_{i+1} = x.key_i$ 
5           $i = i - 1$ 
6       $x.key_{i+1} = k$ 
7       $x.n = x.n + 1$ 
8      DISK-WRITE( $x$ )
```

1. We are inserting k in a leaf (all the keys are pushed ahead to create the proper space for storing k). The number of keys is increased by 1

Insertion Operation

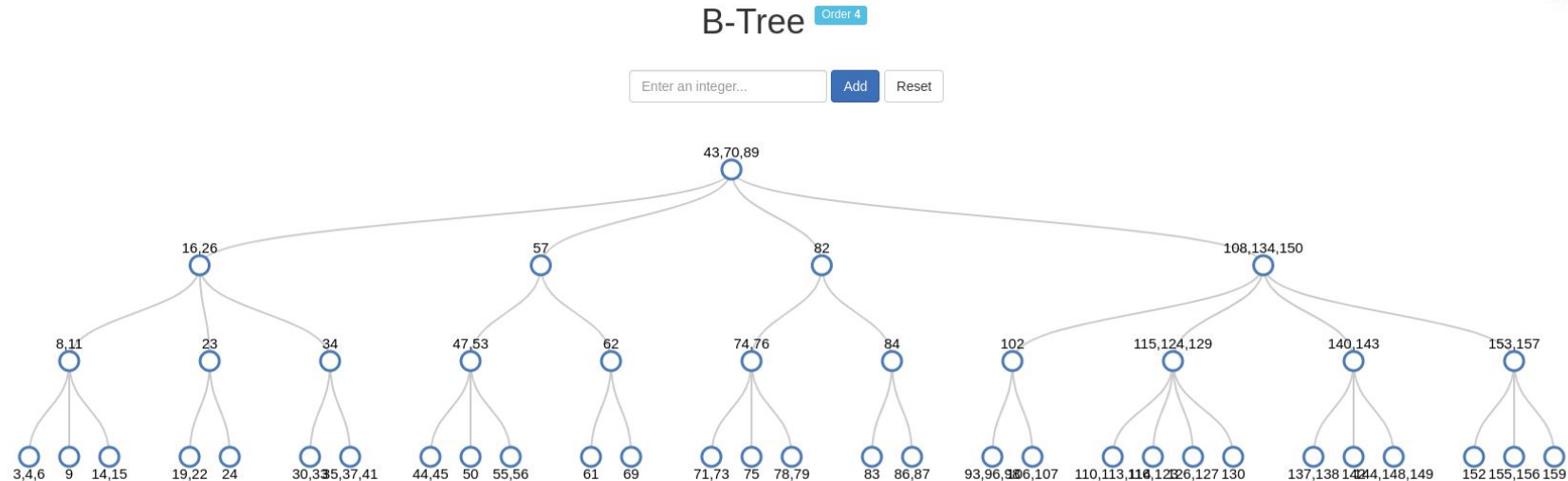
```
9  else while  $i \geq 1$  and  $k < x.key_i$ 
10       $i = i - 1$ 
11       $i = i + 1$ 
12      DISK-READ( $x.c_i$ )
13      if  $x.c_i.n == 2t - 1$ 
14          B-TREE-SPLIT-CHILD( $x, i$ )
15          if  $k > x.key_i$ 
16               $i = i + 1$ 
17      B-TREE-INSERT-NONFULL( $x.c_i, k$ )
```

2. The node is not a leaf, so we need to understand in which subtree k must be added
3. Once we have found the right subtree, we verify whether or not such node is full (and then must be split out)
4. k is then inserted in the right position

Complexity: $O(t \log_t n)$

B-Tree Simulator

<https://yangez.github.io/btree-js/>



B-Tree Simulator

<https://www.cs.usfca.edu/~galles/visualization/BTree.html>

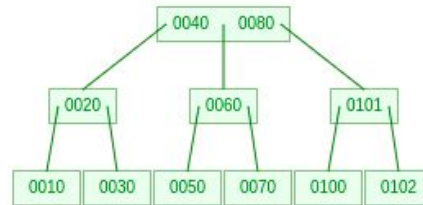
☒ Max. Degree = 3 ☐ Preemptive Split / Merge (Even max degree only)

☐ Max. Degree = 4

☐ Max. Degree = 5

☐ Max. Degree = 6

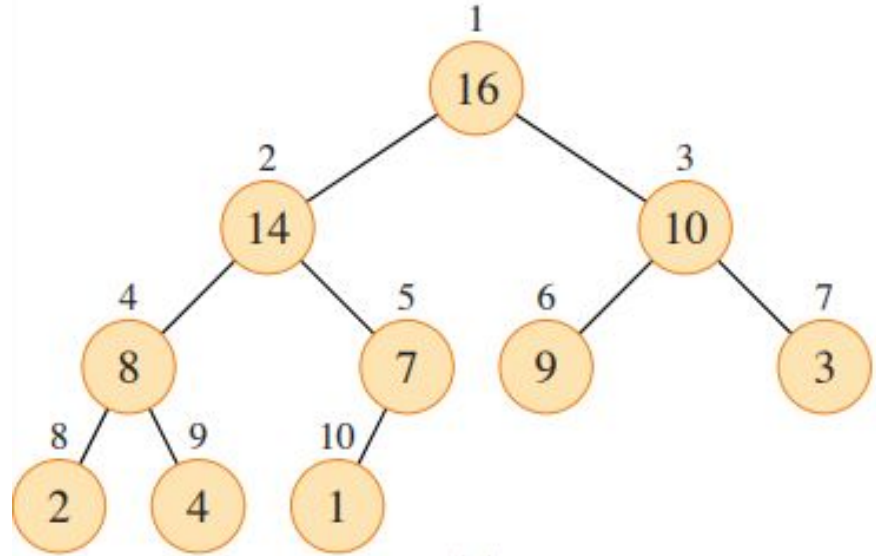
☐ Max. Degree = 7



Binary Heap

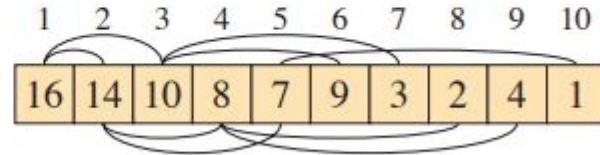
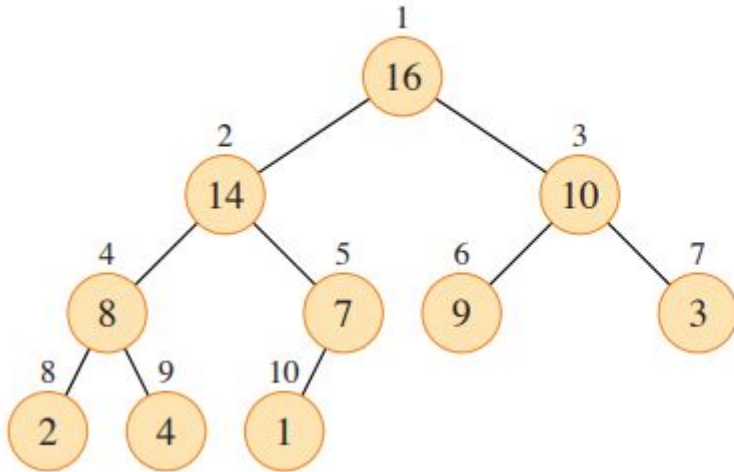
Binary Heap: properties

1. It is a **nearly complete** binary tree
2. It is completely filled on all levels except possibly the lowest, which is filled **from the left up to a point**



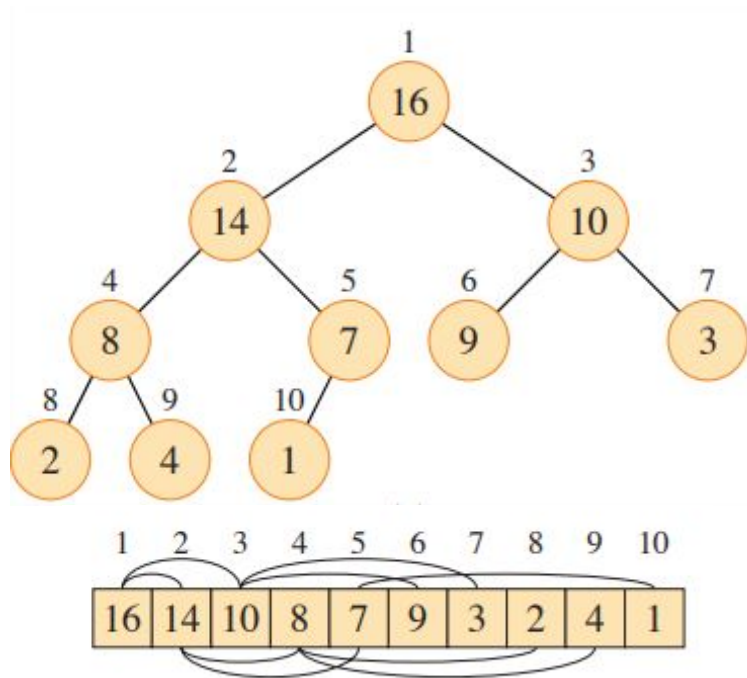
Binary Heap

From the implementation point of view, a Binary Heap is implemented with an array



Binary Heap: basic operation

- Let **A** be an array representing a Binary Heap
- **A.heap-size** is the number of elements in the Heap
- Let **i** be the index of an element (from **1** to **heap-size**)
- **A[1]** is the root of the heap



Binary Heap: basic operation

PARENT(i)

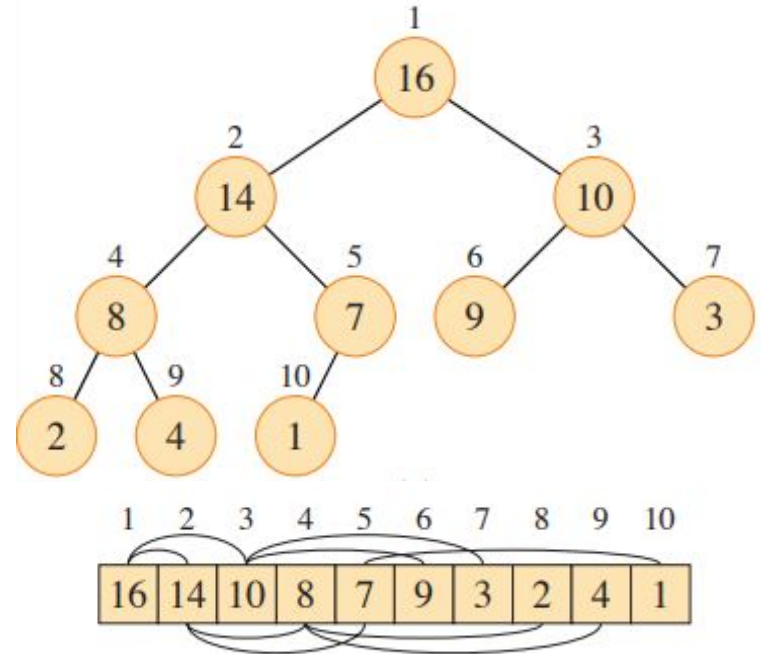
1 **return** $\lfloor i/2 \rfloor$

LEFT(i)

1 **return** $2i$

RIGHT(i)

1 **return** $2i + 1$



Binary Heap: basic operation - Complexity

Parent(i) $O(1)$

Left(i) $O(1)$

Right(i) $O(1)$

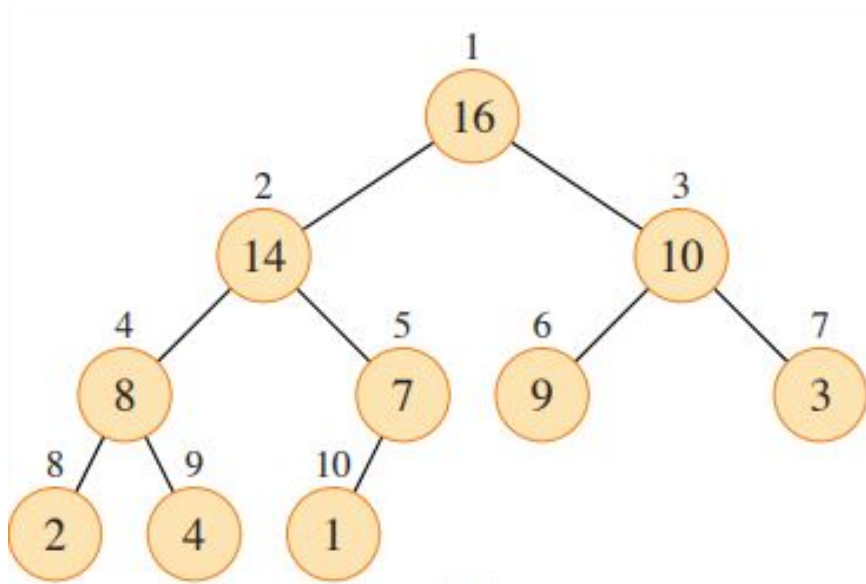
Binary Heap: max-heap and min-heap

There are two kinds of Binary Heap:

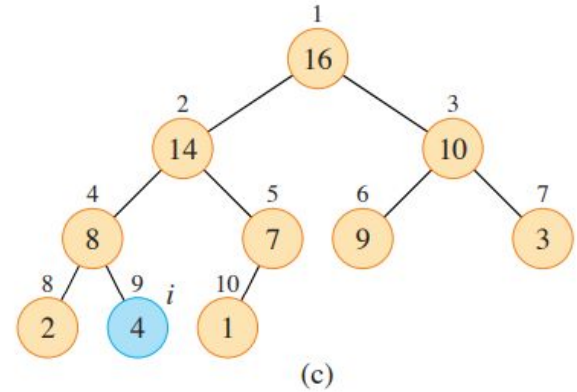
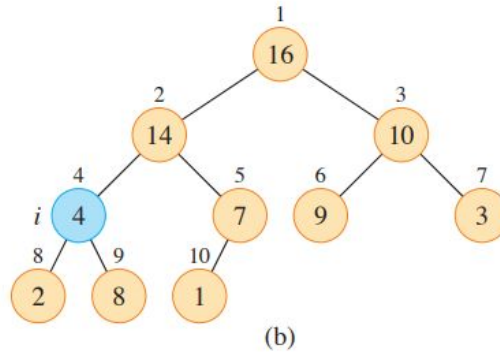
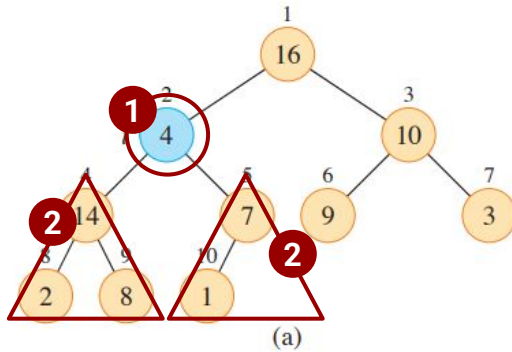
- max-heap
 - $A[\text{Parent}(i)] \geq A[i]$
- min-heap
 - $A[\text{Parent}(i)] \leq A[i]$

In a max-heap, $A[1]$ contains the maximum value; in a min-heap, $A[1]$ contains the minimum value

We will focus on **max-heap**

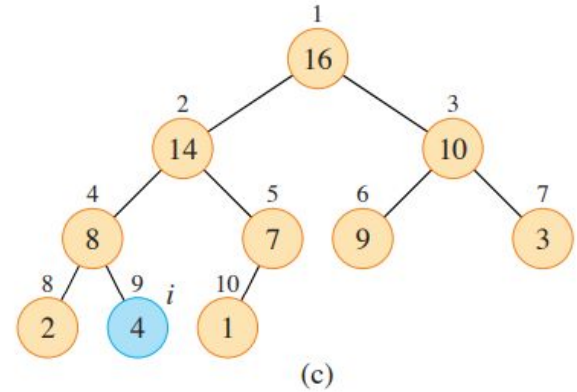
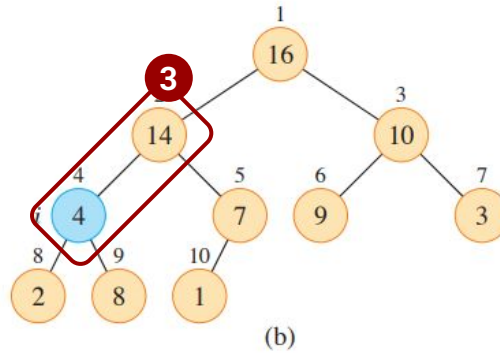
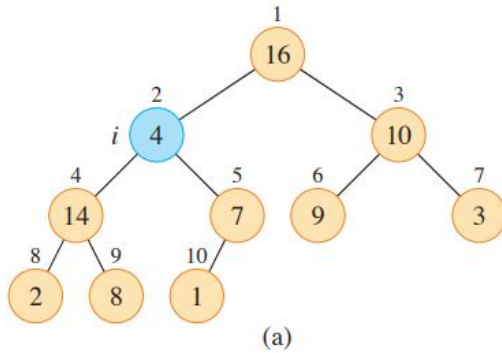


Binary Heap: max-heapify



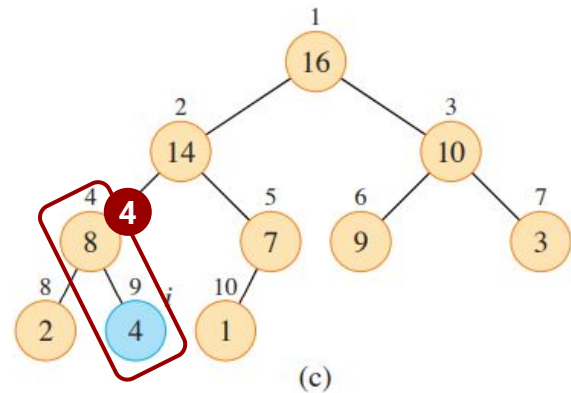
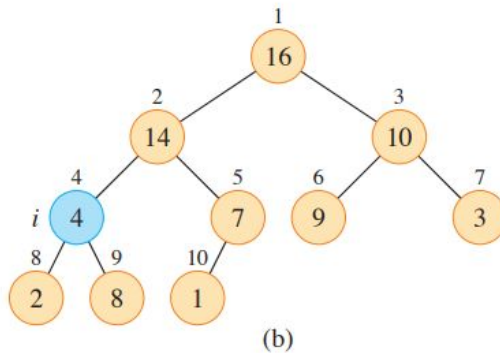
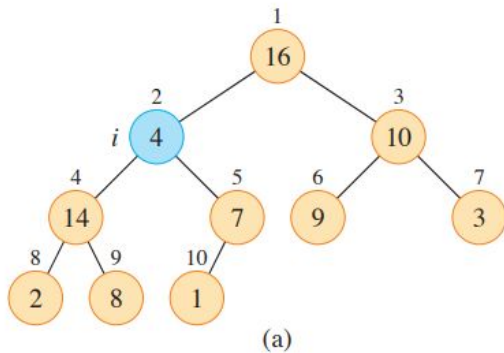
1. We want to run max-heapify on $i=2$ ($A[2] = 4$);
2. $\text{Left}(i)$ and $\text{Right}(i)$ are two Binary Max-Heap

Binary Heap: max-heapify



3. We compare $A[1]$ with $A[\text{Left}(i)]$ and $A[\text{Right}(i)]$ and the maximum value is moved on the root

Binary Heap: max-heapify



3. We run max-heapify recursively only on the $i = 4$ (which is the one which destroys the max-heap property). Then $A[4]$ and $A[9]$ are swapped

Binary Heap: max-heapify

MAX-HEAPIFY(A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $\text{largest} = l$ 
5  else  $\text{largest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{largest}]$ 
7       $\text{largest} = r$ 
8  if  $\text{largest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{largest}]$ 
10     MAX-HEAPIFY( $A, \text{largest}$ )
```

Recursive function

Complexity:

$O(\log_2 n)$

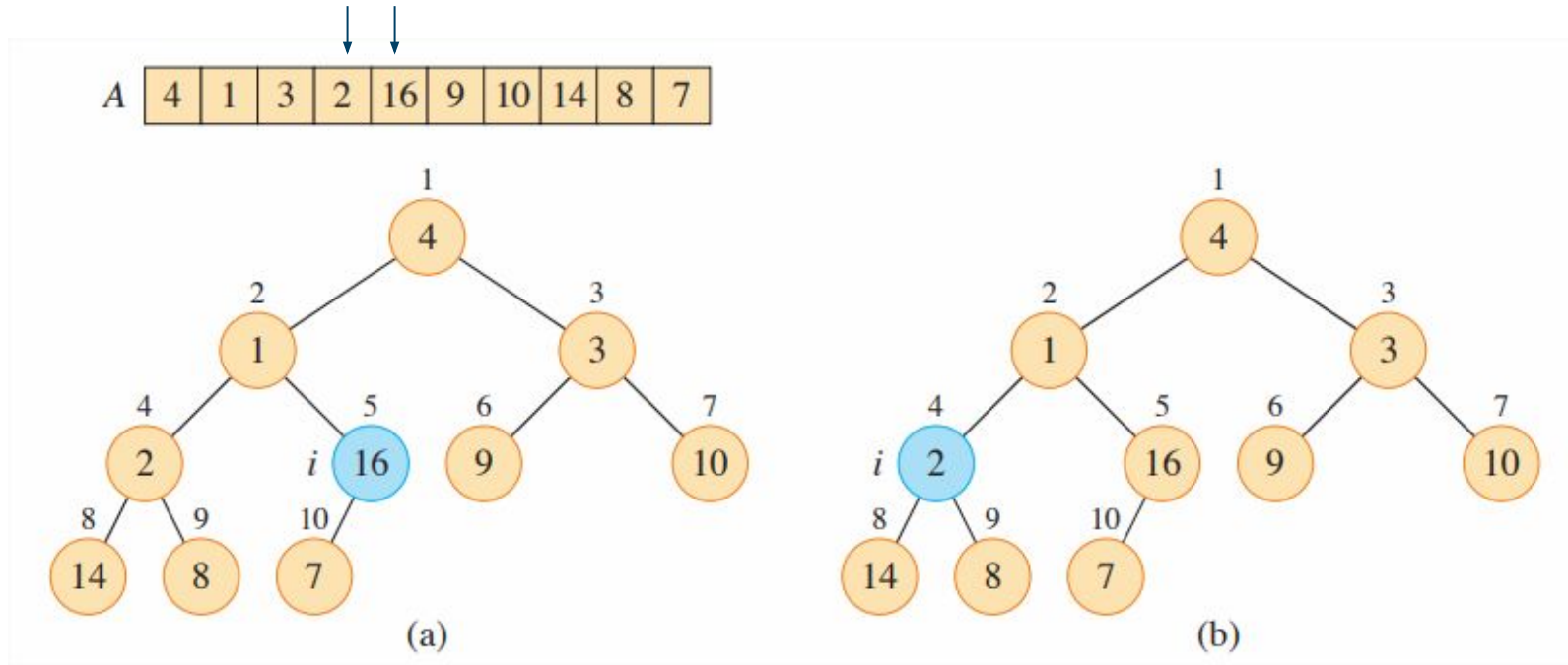
Binary max-heap: build max heap

Let's suppose we have an array which is not a Max-Heap and we would like to turn the array into a max-heap

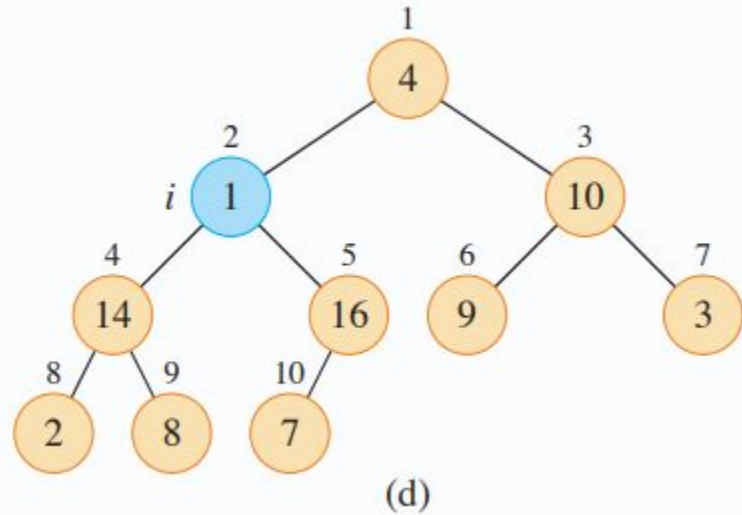
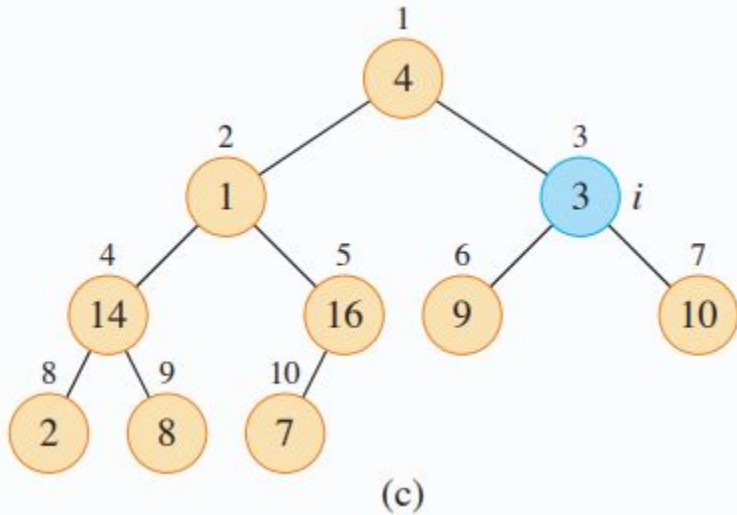
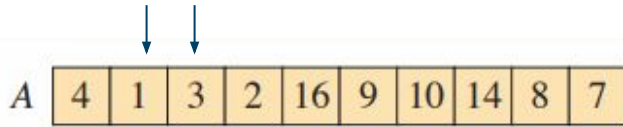
A	4	1	3	2	16	9	10	14	8	7
---	---	---	---	---	----	---	----	----	---	---

We start from the center of the array, which is $\text{floor}(n/2)$, and then run the heapify operation on all elements **from $\text{floor}(n/2)$ to 1**

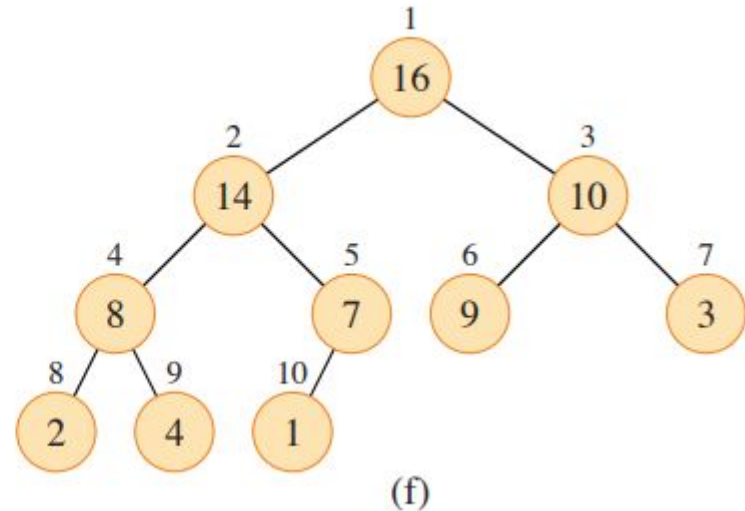
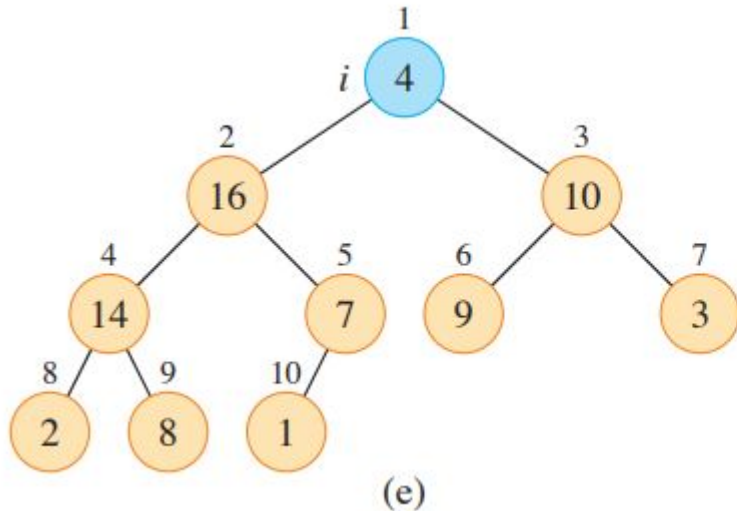
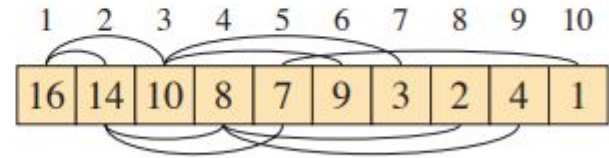
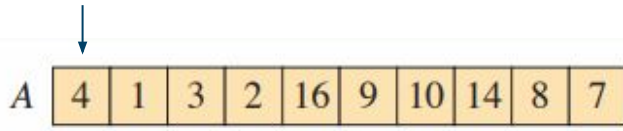
Binary max-heap: build max heap



Binary max-heap: build max heap



Binary max-heap: build max heap



Binary max-heap: build max heap - Complexity

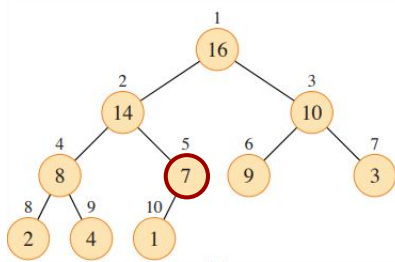
we can build a max-heap from an unordered array in linear time:

$$O(n)$$

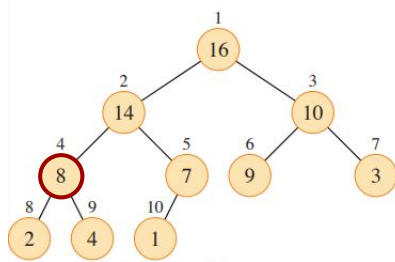
Proof:

1. we start from the middle of the array (from the last non-leaf node) and we move backward till the first element of the array. Total number: $\lfloor n/2 \rfloor$
2. for each element we perform the heapify operation, which takes $O(h)$ where h is the height of the subtree at the current node

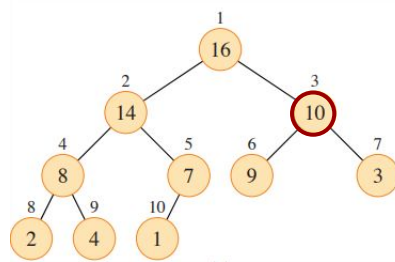
Binary max-heap: build max heap - Complexity



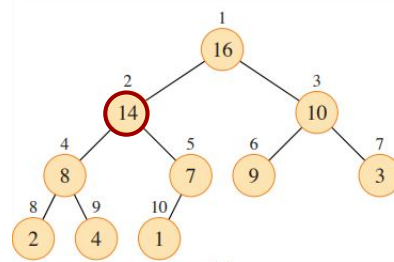
$$h_5 = 1 = \lfloor \log_2 10 \rfloor - \lfloor \log_2 5 \rfloor$$



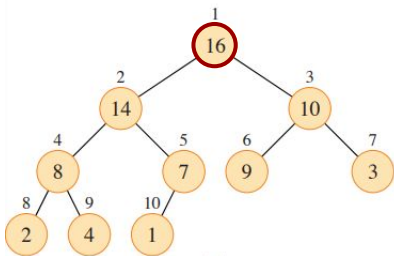
$$h_4 = 1 = \lfloor \log_2 10 \rfloor - \lfloor \log_2 4 \rfloor$$



$$h_3 = 1 = \lfloor \log_2 10 \rfloor - \lfloor \log_2 3 \rfloor$$



$$h_2 = 2 = \lfloor \log_2 10 \rfloor - \lfloor \log_2 2 \rfloor$$



$$h_1 = 2 = \lfloor \log_2 10 \rfloor - \lfloor \log_2 1 \rfloor$$

$$T(n) = \sum_{i=1}^{\lfloor n/2 \rfloor} \lfloor \log_2 n \rfloor - \lfloor \log_2 i \rfloor$$

Binary max-heap: build max heap - Complexity

$$T(n) = \sum_{i=1}^{\lfloor n/2 \rfloor} \lfloor \log_2 n \rfloor - \lfloor \log_2 i \rfloor \leq \sum_{i=1}^{n/2} \log_2 n - \sum_{i=1}^{n/2} \log_2 i = \frac{n}{2} \log_2 n - \log_2 \prod_{i=1}^{n/2} i$$

$$T(n) \leq \frac{n}{2} \log_2 n - \log_2 \frac{n}{2}! = \frac{n}{2} \log_2 \frac{2n}{2} - \log_2 \frac{n}{2}! = \frac{n}{2} (\log_2 \frac{n}{2} + \log_2 2) - \log_2 \frac{n}{2}!$$

$$T(n) \leq \frac{n}{2} \log_2 n - \log_2 \frac{n}{2}! = \frac{n}{2} \log_2 \frac{2n}{2} - \log_2 \frac{n}{2}! = \frac{n}{2} (\log_2 \frac{n}{2} + \log_2 2) - \log_2 \frac{n}{2}!$$

Stirling's approximation
 $\log_e n! \approx n \log_e n - n$

$$T(n) \leq \frac{n}{2} \log_2 \frac{n}{2} + \frac{n}{2} - \frac{n}{2} \log_2 \frac{n}{2} + \frac{n}{2} \log_2 e = \frac{n}{2} (1 + \log_2 e) = O(n)$$

Binary max-heap: max-heap-insert

Insert(A, 15)

	1	2	3	4	5	6	7	8	9	10
A	16	14	10	8	7	9	3	2	4	1

A

	1	2	3	4	5	6	7	8	9	10	$i=11$
	16	14	10	8	7	9	3	2	4	1	15

$A[\text{parent}[i]] < A[i]$

A

	1	2	3	4	$i=5$	6	7	8	9	10	11
	16	14	10	8	15	9	3	2	4	1	7

$A[\text{parent}[i]] < A[i]$

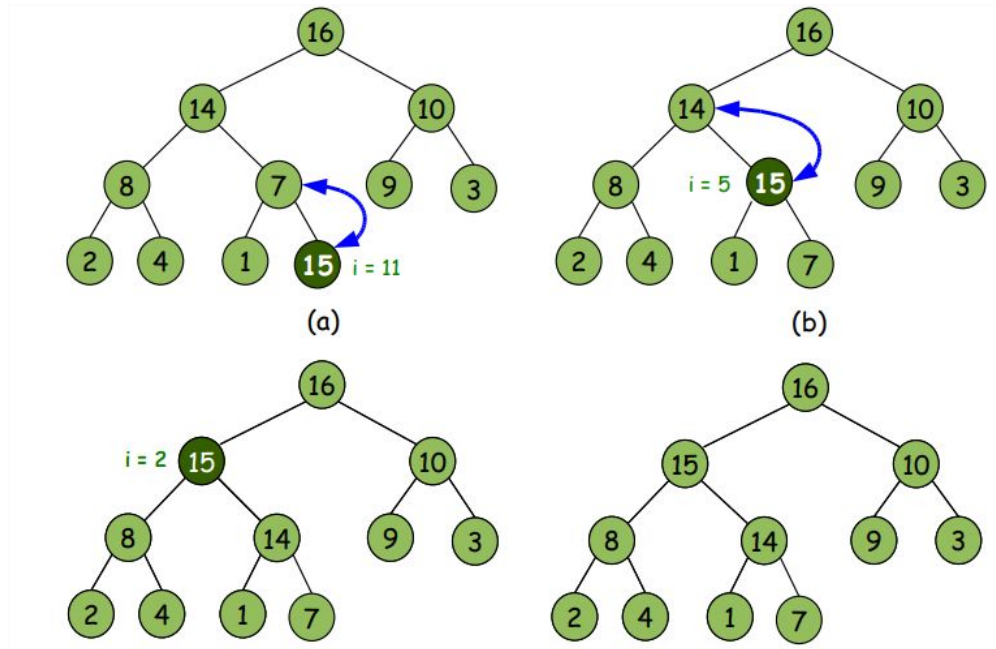
A

	1	$i=2$	3	4	5	6	7	8	9	10	11
	16	15	10	8	14	9	3	2	4	1	7

$A[\text{parent}[i]] \geq A[i]$

- We want to add 15
- We put 15 at the end of the array $A[A.\text{heap-size} + 1]$
- We run max-heapify of $\text{Parent}(A.\text{heap-size} + 1)$

Binary max-heap: max-heap-insert



Binary max-heap: max-heap-insert

Max-Heap-Insert(A, x)

Complexity

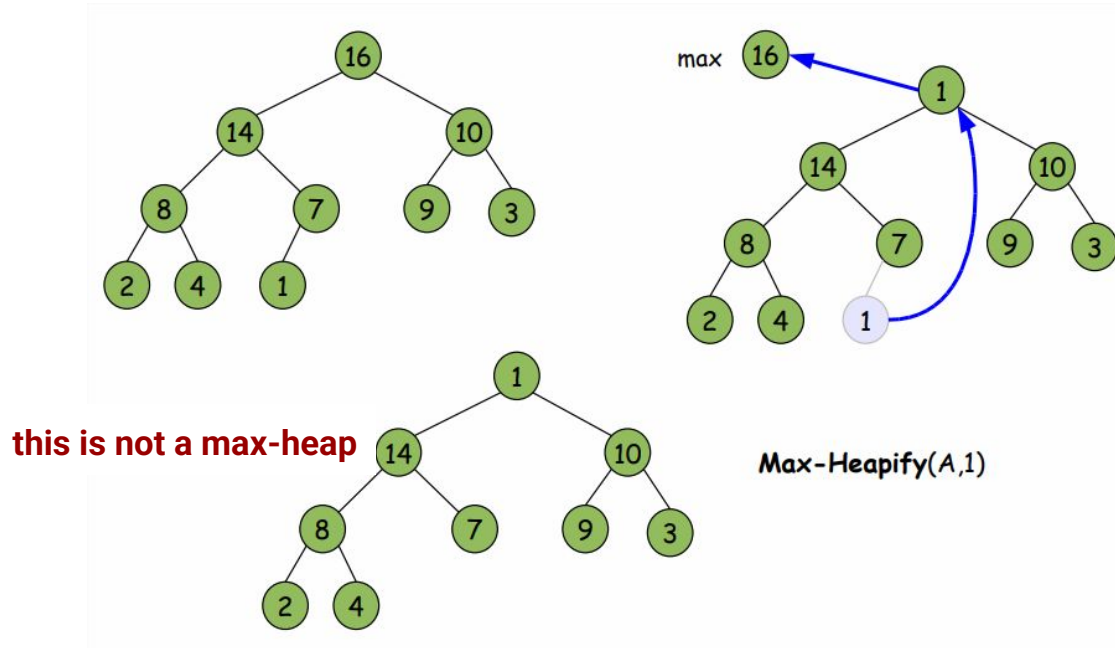
```
{
    A.heap-size = A.heap-size + 1
    i = A.heap-size

    A[i] = x

    while (i > 1 or A[Parent(i)] < A[i])
    {
        swap(A[i], A[Parent(i)])
        i = Parent(i)
    }
}
```

$O(\log_2 n)$

Binary max-heap: max-heap-extraction



Binary max-heap: max-heap-extract

Max-Heap-Extract(A)

Complexity

```
{  
    if A.heap-size < 1 return error;  
  
    max = A[1]  
    A[1] = A[A.heap-size]  
    A.heap-size = A.heap-size - 1  
    Max-Heap-Heapify(A, 1)  
    return max  
}
```

$O(\log_2 n)$

Information Retrieval

Academic Year 2025/2026

Luigi Santangelo
luigi.santangelo@unipv.it

University of Pavia
**Master Degree in
Computer Engineering**

What Information Retrieval is

*Information Retrieval (IR) is **finding material** (usually documents) of an **unstructured nature** (usually text) that satisfies an **information need** from within **large collections** (usually stored on computers)*

With the advent of the Web, IR became an important activity for many people for example when they search for information on the **web** or in the **mail box**.

Structured, Unstructured and SemiStructured

From the definition, IR works on **Unstructured Data**.

Unstructured Data is the opposite to the Structured Data which are suitable to be used in traditional Relational Database.

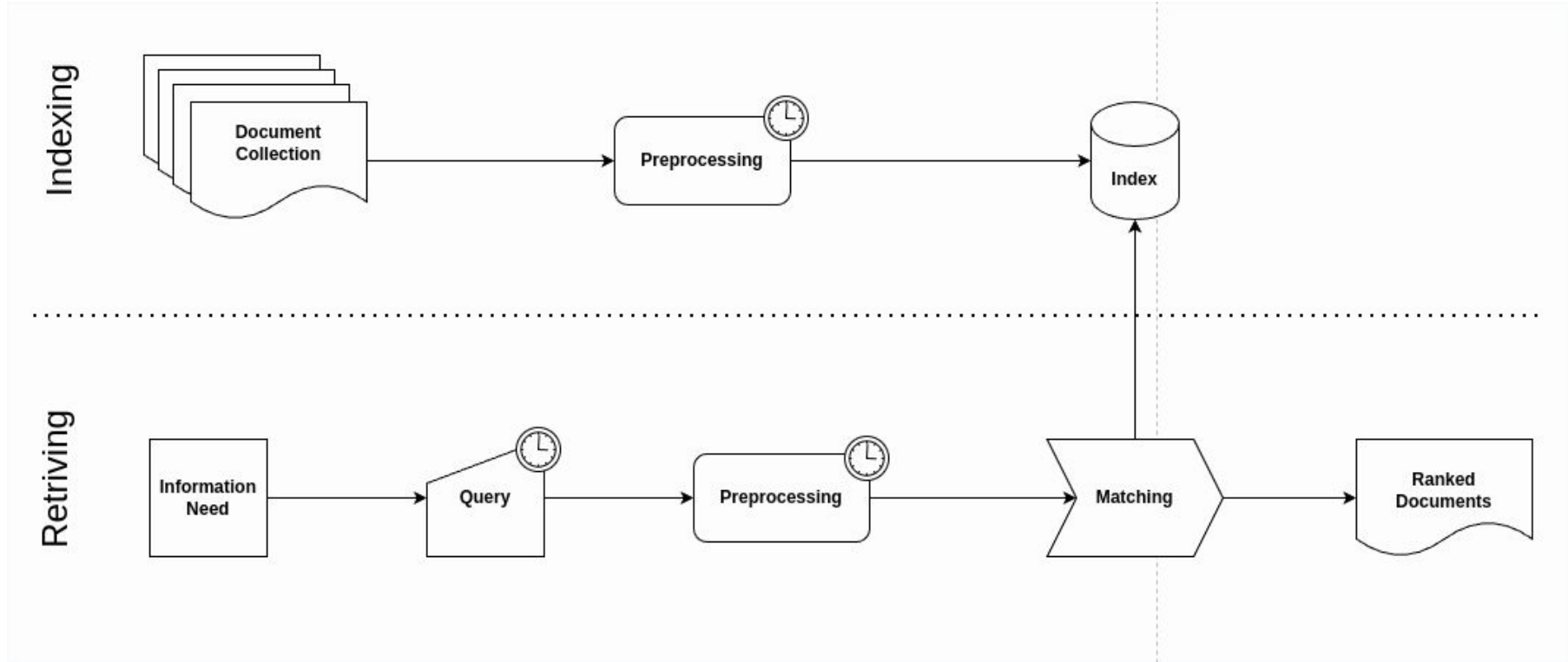
In reality, almost no data are truly “unstructured”. Indeed, if we think about a document, most text has structure, such as headings and paragraphs and footnotes, and this in the web are represented by **tags**.

So, IR facilitates **semi structured** searches and supports further activities on the retrieved documents, such as clustering or classification

What an Information Retrieval System is

An Information Retrieval System aims to provide **documents** belonging to a **collection**, that is **relevant** to an arbitrary **user information need**, which is described by a **query**.

How an Information Retrieval System works



Positive and Negative Results

The **main aim** of a Information Retrieval System is to retrieve all and only those documents that are **relevant**. The **confusion matrix** is a table combining relevant and non relevant documents with those that are retrieved and non retrieved

	Relevant	Non Relevant
Retrieved	<i>True Positive</i>	<i>False Positive</i>
Non Retrieved	<i>False Negative</i>	<i>True Negative</i>

Precision, Recall, Accuracy, F1-Score

An IR system should provide documents that are relevant for the user.

To assess the effectiveness of an IR system we use three indexes:

Precision, Recall, Accuracy, F1-Score

	Relevant	Non Relevant
Retrieved	<i>True Positive</i>	<i>False Positive</i>
Non Retrieved	<i>False Negative</i>	<i>True Negative</i>

$$\text{Precision} = \frac{\sum TP}{\sum TP + FP}$$

$$\text{Recall} = \frac{\sum TP}{\sum TP + FN}$$

$$\text{Accuracy} = \frac{\sum TP + TN}{\sum TP + FP + FN + TN}$$

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Precision, Recall, Accuracy, F1-Score

Precision: What fraction of the returned results are relevant to the information need?

Recall: What fraction of the relevant documents in the collection were returned by the system?

Accuracy, F1-Score: How many predictions did a model get right? Both range between 0 (all bad predictions) and 1 (all good predictions)

	Relevant	Non Relevant
Retrieved	<i>True Positive</i>	<i>False Positive</i>
Non Retrieved	<i>False Negative</i>	<i>True Negative</i>

$$\text{Precision} = \frac{\sum TP}{\sum TP + FP}$$

$$\text{Recall} = \frac{\sum TP}{\sum TP + FN}$$

$$\text{Accuracy} = \frac{\sum TP + TN}{\sum TP + FP + FN + TN}$$

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Example 1

An information retrieval system is used to find relevant documents in a large database of 1000 documents. The system retrieves 50 documents in total. After manual inspection, it's found that 37 of the retrieved documents are relevant, while 15 relevant documents were not retrieved. Based on this information, compute the precision, recall, accuracy, and F1 score of the system.

	Relevant	Non Relevant	TOTAL
Retrieved			
Not Retrieved			
TOTAL			

Example 1

An information retrieval system is used to find relevant documents in a **large database of 1000 documents**. The system retrieves 50 documents in total. After manual inspection, it's found that 37 of the retrieved documents are relevant, while 15 relevant documents were not retrieved. Based on this information, compute the precision, recall, accuracy, and F1 score of the system.

	Relevant	Non Relevant	TOTAL
Retrieved			
Not Retrieved			
TOTAL			1000

Example 1

An information retrieval system is used to find relevant documents in a large database of 1000 documents. The system **retrieves 50 documents in total**. After manual inspection, it's found that 37 of the retrieved documents are relevant, while 15 relevant documents were not retrieved. Based on this information, compute the precision, recall, accuracy, and F1 score of the system.

	Relevant	Non Relevant	TOTAL
Retrieved			50
Not Retrieved			
TOTAL			<i>1000</i>

Example 1

An information retrieval system is used to find relevant documents in a large database of 1000 documents. The system retrieves 50 documents in total. After manual inspection, it's found that **37 of the retrieved documents are relevant**, while 15 relevant documents were not retrieved. Based on this information, compute the precision, recall, accuracy, and F1 score of the system.

	Relevant	Non Relevant	TOTAL
Retrieved	37		50
Not Retrieved			
TOTAL			1000

Example 1

An information retrieval system is used to find relevant documents in a large database of 1000 documents. The system retrieves 50 documents in total. After manual inspection, it's found that 37 of the retrieved documents are relevant, while **15 relevant documents were not retrieved**. Based on this information, compute the precision, recall, accuracy, and F1 score of the system.

	Relevant	Non Relevant	TOTAL
Retrieved	37		50
Not Retrieved	15		
TOTAL			1000

Example 1

An information retrieval system is used to find relevant documents in a large database of 1000 documents. The system retrieves 50 documents in total. After manual inspection, it's found that 37 of the retrieved documents are relevant, while 15 relevant documents were not retrieved. Based on this information, compute the precision, recall, accuracy, and F1 score of the system.

	Relevant	Non Relevant	TOTAL
Retrieved	37	?	50
Not Retrieved	15	?	?
TOTAL	?	?	1000

Example 1

An information retrieval system is used to find relevant documents in a large database of 1000 documents. The system retrieves 50 documents in total. After manual inspection, it's found that 37 of the retrieved documents are relevant, while 15 relevant documents were not retrieved. Based on this information, compute the precision, recall, accuracy, and F1 score of the system.

	Relevant	Non Relevant	TOTAL
Retrieved	37	13	50
Not Retrieved	15	935	950
TOTAL	52	948	1000

Example 1

An information retrieval system is used to find relevant documents in a large database of 1000 documents. The system retrieves 50 documents in total. After manual inspection, it's found that 37 of the retrieved documents are relevant, while 15 relevant documents were not retrieved. Based on this information, compute the precision, recall, accuracy, and F1 score of the system.

	Relevant	Non Relevant	TOTAL
Retrieved	37	13	50
Not Retrieved	15	935	950
TOTAL	52	948	1000

$$TP = 37$$

$$TN = 935$$

$$FP = 13$$

$$FN = 15$$

Example 1

- Precision = $TP / (TP + FP) = 37 / (37+13) = 0.74$
- Recall = $TP / (TP + FN) = 37 / (37 + 15) \approx 0.71$
- Accuracy = $(TP + TN) / (TP + TN + FP + FN) = (37+935)/1000 \approx 0.97$
- F1-score = $2*Precision*Recall/(Precision + Recall) = 2*(0.74 * 0.71)/(0.74 + 0.71) \approx 0.73$

	Relevant	Non Relevant	TOTAL
Retrieved	37	13	50
Not Retrieved	15	935	950
TOTAL	52	948	1000

TP = 37

TN = 935

FP = 13

FN = 15

Example 2

A bank uses a model to predict if a loan applicant will default on their loan. From a sample of 2000 applications, the model's performance is as follows: 1500 applicants who did not default were correctly identified, and 150 who did default were also correctly identified. However, the model incorrectly predicted that 200 applicants would default when they didn't, and it failed to identify 150 applicants who actually defaulted. Calculate the precision, recall, accuracy, and F1 score.

True Positive	150
True Negative	1500
False Positive	200
False Negative	150

	Relevant	Non Relevant	TOTAL
Retrieved	150	200	350
Not Retrieved	150	1500	1650
TOTAL	300	1700	2000

Example 2

- Precision = $TP / (TP + FP) = 150 / (150 + 200) \approx 0.43$
- Recall = $TP / (TP + FN) = 150 / (150 + 150) = 0.50$
- Accuracy = $(TP + TN) / (TP + TN + FP + FN) = (1650) / 2000 \approx 0.83$
- F1-score = $2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall}) = 2 * (0.43 * 0.50) / (0.43 + 0.50) \approx 0.46$

True Positive	150
True Negative	1500
False Positive	200
False Negative	150

	Relevant	Non Relevant	TOTAL
Retrieved	150	200	350
Not Retrieved	150	1500	1650
TOTAL	300	1700	2000

What does relevance mean?

Relevance is the degree to which a query response satisfies a user who is searching for information.

It depends on the the context in which the query was performed.

A single search application may be used in different contexts by users with **different needs and expectations**

What does relevance mean?

Relevant documents are subjective:

For example:

Suppose we have a search engine of **climate data**.

- a **university researcher** is interested in studying long-term climate trend
- a **farmer** is interested in calculating the likely date of the last frost of spring
- a **civil engineer** is interested in rainfall patterns and the frequency of floods
- a **student** is interested in planning a vacation to a region

The simplest Information Retrieval System

Suppose we have all the Shakespeare's Works.

We want to find all documents containing the words

Caesar and Brutus but not Calpurnia.

The simplest form of document retrieval is for a computer to do this sort of **linear scan** through documents.

This process is known as **grepping**. Remind the **Unix grep command**.

Information retrieval problems

- When we have a small document collection (like all Shakespeare's works, containing 38 plays with less than one million words of text in total) this is not a big deal, but having billions or trillions of documents, search time becomes important.
- Sometimes we would like to search for documents using not an exact pattern, but for example “***search for the word Romans near to countrymen***” (where near means “within 5 words”)
- In many case there are many documents satisfying the query, and we would like to have **a rank**

Linear vs Indexed Search

Unix **grep command** works linearly (and this is the reason of its slowness).

The alternative Search is an **Indexed Search**, which makes use of Index.

Indexing documents means building a **term-document incidence matrix**.

Zeros and Ones indicate whether a term is found in the document

		Documents						
		Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	..
Terms	Antony	1	1	0	0	0	1	
	Brutus	1	1	0	1	0	0	
	Caesar	1	1	0	1	1	1	
	Calpurnia	0	1	0	0	0	0	
	Cleopatra	1	0	0	0	0	0	
	mercy	1	0	1	1	1	1	
	worser	1	0	1	1	1	0	
...								

1 = term is contained in the document
0 = term is NOT contained in the document

Boolean Retrieval Model

By looking at the term-document matrix it will be easy to find all documents containing one or more terms.

In the query, terms are combined with operators **AND**, **OR** and **NOT**.

This retrieval is named **Boolean Retrieval Model**

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	..
Terms	1	1	0	0	0	1	
Antony	1	1	0	1	0	0	
Brutus	1	1	0	1	0	0	
Caesar	1	1	0	1	1	1	
Calpurnia	0	1	0	0	0	0	
Cleopatra	1	0	0	0	0	0	
mercy	1	0	1	1	1	1	
worser	1	0	1	1	1	0	
...							

1 = term is contained in the document
0 = term is NOT contained in the document

The big problem of the term-doc matrix

Let's suppose we have a collection of 1 million documents.

Suppose each document is about 1000 words (3 pages)

Suppose all documents have 500.000 distinct terms

The size of the term-document incidence matrix is: $10^6 * 5 * 10^5 = 5 * 10^{11}$ bit,
more than 465Gb (approx 58GB):

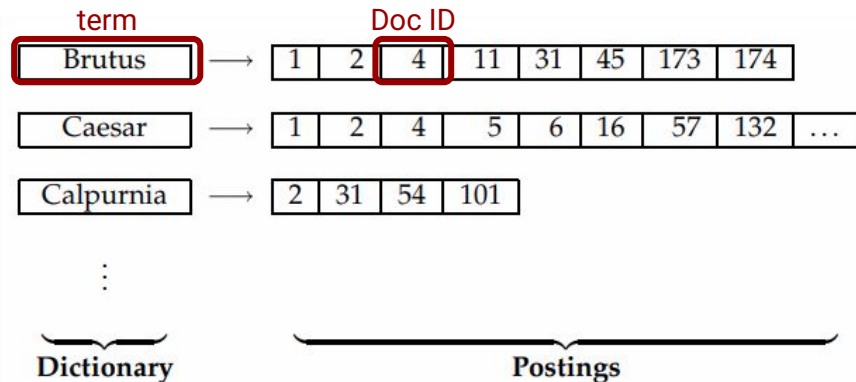
- very big to keep it in memory
- the matrix is sparse (99.8% of cells are zero)

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
Antony	1	1	0	0	0	1	
Brutus	1	1	0	1	0	0	
Caesar	1	1	0	1	1	1	
Calpurnia	0	1	0	0	0	0	
Cleopatra	1	0	0	0	0	0	
mercy	1	0	1	1	1	1	
worser	1	0	1	1	1	0	
...							

The solution: inverted index

This is the major concept in Information Retrieval: **the inverted index**

We have a **dictionary** of terms, for each term we have a list that records which documents the term occurs in. This is named **postings** or **postings list**



Dictionary vs Vocabulary

Dictionary and Vocabulary are often used as synonyms

Here instead we use a convention:

- **Dictionary**: is the data structure containing words
- **Vocabulary**: is the set of words

How to build an inverted index

To gain benefit of indexing at retrieval time, we have to **build the index in advance**:

1. Collect the documents to be indexed

Friends, Romans, countrymen. So let it be with Caesar ...

2. Tokenize the text, turning each document into a list of tokens

Friends Romans countrymen So ...

How to build an inverted index

To gain benefit of indexing at retrieval time, we have **to build the index in advance**:

3. Do linguistic preprocessing, producing a list of **normalized tokens**, which are the indexing **terms**

friend	roman	countryman	so	...
--------	-------	------------	----	-----

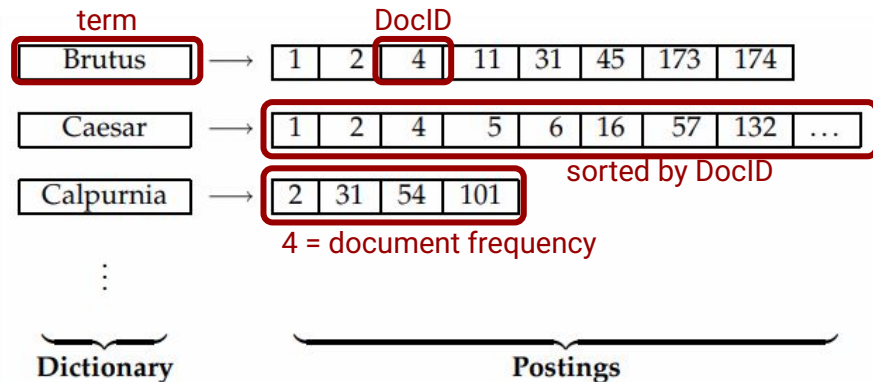
4. Index the documents that each term occurs in by creating an **inverted index**, consisting of a **dictionary and postings**.

How to build an inverted index

During inverted index creation,
documents are identified univocally
by an ID (docID)

Terms in the dictionary and DocID in
the postings are **sorted** and merged

The dictionary contains also some
statistics, for example the **document**
frequency = the length of a posting



An example

1. Collect all documents
2. Terms are extracted, tokenized, preprocessed and tagged with the DocID
3. Terms are sorted alphabetically
4. Instances of the same term are then grouped by word and then by documentID
5. For each unique term, postings list is built
6. Some information (such the document frequency) are stored (for improving query efficiency or for **ranking results**)

Doc 1
I did enact Julius Caesar: I was killed i' the Capitol; Brutus killed me.

Doc 2
So let it be with Caesar. The noble Brutus hath told you Caesar was ambitious:

term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
me	1
noble	2
so	2
the	1
the	2
told	2
told	2
you	2
you	2
was	1
was	2
was	2
with	2

term	doc. freq.	→	postings list
ambitious	1	→	2
be	1	→	2
brutus	2	→	1 → 2
capitol	1	→	1
caesar	2	→	1 → 2
did	1	→	1
enact	1	→	1
hath	1	→	2
I	1	→	1
i'	1	→	1
it	1	→	2
julius	1	→	1
killed	1	→	1
let	1	→	2
me	1	→	1
noble	1	→	2
so	1	→	2
the	2	→	1 → 2
told	1	→	2
you	1	→	2
was	2	→	1 → 2
with	1	→	2

Challenges for inverted index

- Which data structure is the best for managing Dictionary and Postings?
 - Single Linked List
 - Variable length array
 - ...
- Where should I store both Dictionary and Postings?
 - In Memory
 - In Disk

We will discuss about this details which **affect the performance** of a IR system.

Processing Boolean Conjunctive Queries

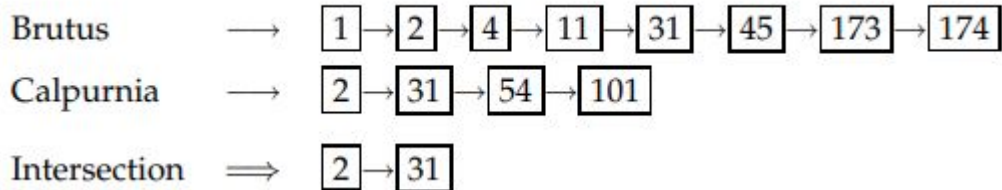
How a Query can work on Boolean Retrieval System?

Suppose we want to process this **simple conjunctive query**:

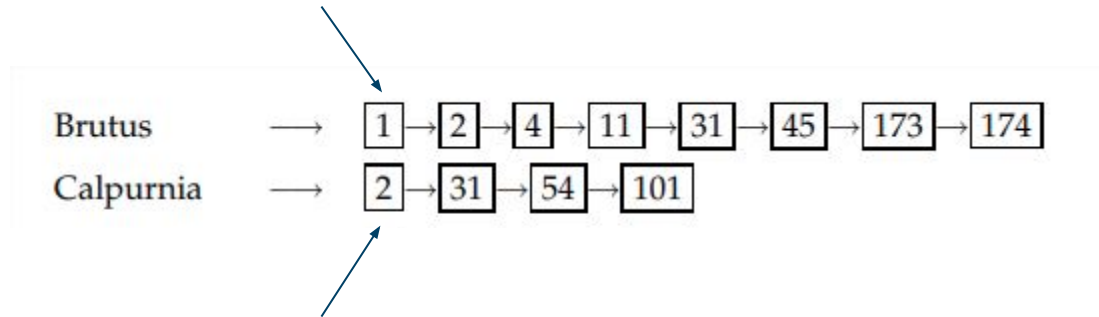
Brutus AND Calpurnia

What we need to do?

1. Locate Brutus on Dictionary
2. Retrieve its postings
3. Locate Calpurnia on Dictionary
4. Retrieve its postings
5. Intersect the two postings lists



Implementing Intersection



We maintain **pointers** into both lists and walk through the two postings lists simultaneously

Implementing Intersection

Computing the intersection between two Postings **must be efficient**.

Here is a well known efficient algorithm

```
INTERSECT( $p_1, p_2$ )
1   $answer \leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $\text{ADD}(answer, \text{docID}(p_1))$ 
5           $p_1 \leftarrow \text{next}(p_1)$ 
6           $p_2 \leftarrow \text{next}(p_2)$ 
7      else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
8          then  $p_1 \leftarrow \text{next}(p_1)$ 
9          else  $p_2 \leftarrow \text{next}(p_2)$ 
10 return  $answer$ 
```

Implementing Intersection: complexity

If x and y are the lengths of the postings p_1 and p_2 , the complexity of the algorithm is $O(x + y) = O(N)$ where N is the number of documents in the collection.

So the **algorithm is linear**.

```
INTERSECT( $p_1, p_2$ )
1   $answer \leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $\text{ADD}(answer, \text{docID}(p_1))$ 
5           $p_1 \leftarrow \text{next}(p_1)$ 
6           $p_2 \leftarrow \text{next}(p_2)$ 
7      else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
8          then  $p_1 \leftarrow \text{next}(p_1)$ 
9          else  $p_2 \leftarrow \text{next}(p_2)$ 
10 return  $answer$ 
```

Processing Boolean Disjunctive Queries

How a Query can work on Boolean Retrieval System?

Suppose we want to process this **simple disjunctive query**:

Brutus OR Calpurnia

What we need to do?

1. Locate Brutus on Dictionary
2. Retrieve its postings
3. Locate Calpurnia on Dictionary
4. Retrieve its postings
5. Merge the two postings lists

Brutus

→ 1 → 2 → 4 → 11 → 31 → 45 → 173 → 174

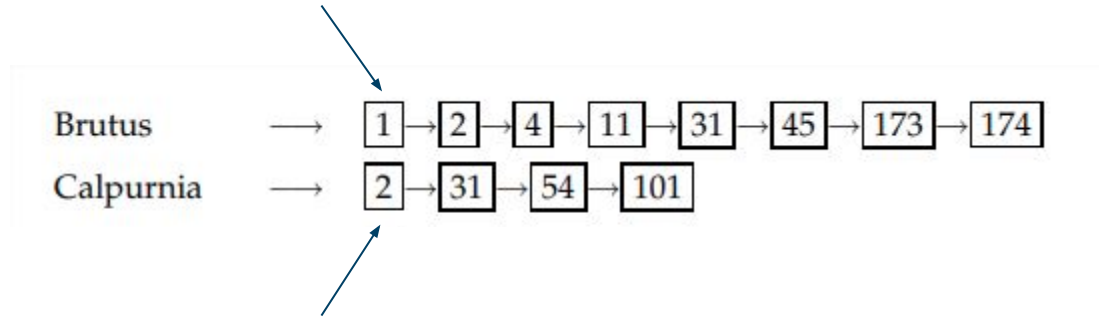
Calpurnia

→ 2 → 31 → 54 → 101

Union

→ 1 2 4 11 31 45 54 101 173 174

Implementing Union



We maintain **pointers** into both lists and walk through the two postings lists simultaneously

Implementing Union

Computing union between two
Postings is very easy and efficient.

Here is a well known efficient algorithm

```
1 union(p1, p2)
2 {
3     answer = NULL;
4     while (p1 <> NULL and p2 <> NULL)
5     {
6         if (docId(p1) == docId(p2))
7         {
8             add(answer, p1);
9             p1 = next(p1);
10            p2 = next(p2);
11        }
12        else if (docId(p1) < docId(p2))
13        {
14            add(answer, p1);
15            p1 = next(p1);
16        }
17        else
18        {
19            add(answer, p2);
20            p2 = next(p2);
21        }
22    }
23    return answer;
24 }
```

Implementing Union: complexity

If x and y are the lengths of the postings p_1 and p_2 , the complexity of the algorithm is $O(x + y) = O(N)$ where N is the number of documents in the collection.

So the **algorithm is linear**.

```
1 union(p1, p2)
2 {
3     answer = NULL;
4     while (p1 <> NULL and p2 <> NULL)
5     {
6         if (docId(p1) == docId(p2))
7         {
8             add(answer, p1);
9             p1 = next(p1);
10            p2 = next(p2);
11        }
12        else if (docId(p1) < docId(p2))
13        {
14            add(answer, p1);
15            p1 = next(p1);
16        }
17        else
18        {
19            add(answer, p2);
20            p2 = next(p2);
21        }
22    }
23    return answer;
24 }
```

Query Optimization

Sometimes, queries are complex and involves many terms, such as:

Brutus AND Caesar AND Calpurnia

Query optimization is the process to organize the work of answering a query so that the least total amount of work needs is done by the system.

An heuristic is to process terms in order of **increasing document frequency**

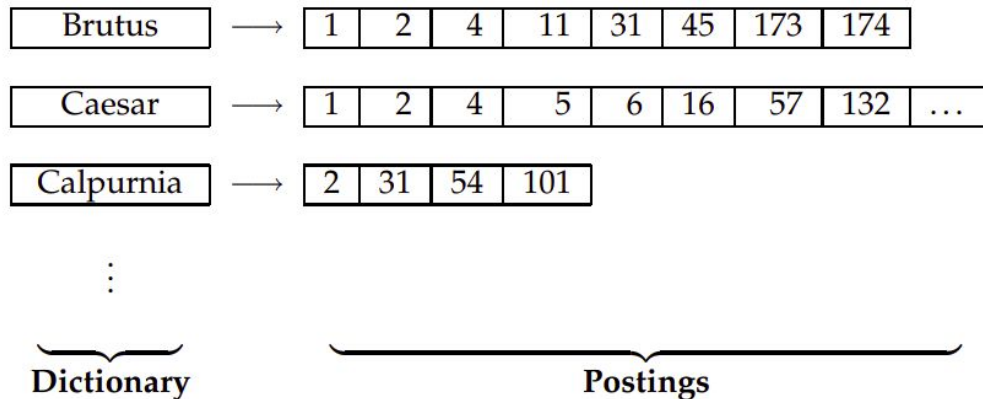
Query Optimization

So, for example, supposing that the the document frequency is the one showed, the query

Brutus AND Caesar AND Calpurnia

becomes

(Calpurnia AND Brutus) AND Caesar



Optimized Intersection

Line 1: compute a sorting of the terms by increasing document frequency

Line 5: make an intersection between two posting list (the current intermediate result and the posting list of the next term having lower document frequency)

```
INTERSECT( $\langle t_1, \dots, t_n \rangle$ )  
1  terms  $\leftarrow$  SORTBYINCREASINGFREQUENCY( $\langle t_1, \dots, t_n \rangle$ )  
2  result  $\leftarrow$  postings(first(terms))  
3  terms  $\leftarrow$  rest(terms)  
4  while terms  $\neq$  NIL and result  $\neq$  NIL  
5  do result  $\leftarrow$  INTERSECT(result, postings(first(terms)))  
6    terms  $\leftarrow$  rest(terms)  
7  return result
```

Optimized Intersection

Notice that:

- *result* is a variable always kept in memory
- *postings(first(terms))* is a function that reads from disk

```
INTERSECT( $\langle t_1, \dots, t_n \rangle$ )  
1  terms  $\leftarrow$  SORTBYINCREASINGFREQUENCY( $\langle t_1, \dots, t_n \rangle$ )  
2  result  $\leftarrow$  postings(first(terms))  
3  terms  $\leftarrow$  rest(terms)  
4  while terms  $\neq$  NIL and result  $\neq$  NIL  
5  do result  $\leftarrow$  INTERSECT(result, postings(first(terms)))  
6    terms  $\leftarrow$  rest(terms)  
7  return result
```

More complex queries

Some query can be more complex, like this

(madding OR crowd) AND (ignoble OR strife) AND (killed OR slain)

we will get the frequencies for all terms, and we can then (conservatively) estimate the size of each **OR** by the **sum of the frequencies of its disjuncts**.

We can then process the query in increasing order of the size of each disjunctive term.

However, in many circumstances, a query is **purely conjunctive**

Preferred IR features

Information Retrieval having only Boolean Operations (AND, OR, NOT) **are very limited**. Often we would like to have other features

- **Proximity operators**: a way to specify how close or far two terms must appear into the documents
- **Error Tolerance**: prevent or adjust **mistakes** and inconsistent words
- **Compounds and Phrases**: allow using **phrases** that denote a concept such as “operating system”
- **Statistics**: retrieving documents depends on the **term frequency**
- **Ranking**: the returned documents should be sorted by rank

Term Vocabulary and Postings List

Academic Year 2024/2025

Luigi Santangelo
luigi.santangelo@unipv.it

University of Pavia
**Master Degree in
Computer Engineering**

Term Vocabulary

The Document

Building a IR system starts from a collection of documents to be indexed.

But how we can define documents?

In general, we can say that a document is a **sequence of text**, but before using them for indexing, there are many issues that we need to face out:

- **coding**: usually an english document is encoded in ASCII, but there are many other encoding schemes such as Unicode UTF-8 or national or vendor-specific standards
- **format**: text, XML with tags, MS Word, PDF, zip

The Document Unit

It is also important to define the **document unit**.

It is easy to think to the document unit like a single document, and this is correct when talk about files in a folder. But think about mails: many applications store several email in a single file stored in the file system. What is the document unit here? The file or the email? Or again, an email can attach several documents, or a zip file containing several documents. Again, what is the document unit here?

The Document Unit

But we need to face the same problem even when we want to index an entire book.

It is not a good idea to consider the whole book as a single document (if I'm looking for the word *italian car*, the book might have *italian* in the first chapter and *car* in the last chapter, but they are not relevant for the search). Perhaps, for such a kind of documents, the best Document Unit is a single chapter

The Document Unit

But keeping going on with this reasoning, why don't we decide to use each single paragraph like a Document Unit? So, in conclusion, defining the right Document Unit is not simple at all.

It becomes clear that there is a **precision/recall tradeoff** here.

Too small Document Unit: we miss some important information because terms are distributed over several mini-documents

Too large Document Unit: relevant information is hard to find for the user needs

The Document Unit

An IR system should be designed to offer **choices of granularity**.

The person who is deploying the system must have a good understanding of the document collection, the users, and their likely information needs and usage patterns.

Tokenization: the general rule

Once the Document Unit has been defined and the Document has been converted in a sequence of text (by decoding the text), **the document is tokenized**.

A **token** is a sequence of characters grouped together as a **useful semantic unit** for processing.

Usually, a token is separate from the others by a **white space** or by a **punctuation sign**.

Input: Friends, Romans, Countrymen, lend me your ears;

Output:

Friends	Romans	Countrymen	lend	me	your	ears
---------	--------	------------	------	----	------	------

Token, Type and Term

We should not make confusion about Token, Type and Term

- A **Token** is a sequence of characters grouped together as a **useful semantic unit** for processing.
- A **Type** is the class of all tokens containing the same character sequence
- A **Term** is a type that is included in the IR system's dictionary

Token, Type and Term

For example:

Document to be indexed

To sleep, perchance to dream

5 Tokens: *to, sleep, perchance, to, dream*

4 Types: *to, sleep, perchance, dream* (since there are 2 instances of *to*)

3 Terms: *sleep, perchance, dream* (since *to* is omitted from the index)

Tokenization

Some **cleaning operations** are done during tokenization (for example punctuation is dropped).

Input: Friends, Romans, Countrymen, lend me your ears;

Output:

Friends	Romans	Countrymen	lend	me	your	ears
---------	--------	------------	------	----	------	------

Tokenization brings along **a lot of problems**, sometimes strictly related to the language characteristics, as we see in the follow.

What are correct tokens?

The previous example of tokenization was very obvious. But defining the right tokenization is not always obvious.

For example, consider this sentence:

Mr. O'Neill thinks that the boys' stories about Chile's capital aren't amusing.

What is **the best tokenization** of *O'Neill* and *aren't*?

oneill	
o'neill	
o'	neill
o	neill ?

aren't	
arent	
are	n't
aren	t ?

What are correct tokens?

The simple strategy is **to split on all non-alphanumeric characters**:

o neill

aren t

But the first looks OK

The second instead looks intuitively bad.

oneill
o'neill
o' neill
o neill?

aren't
arent
are n't
aren t?

What are correct tokens?

The choice of tokenization determines which Boolean queries will match.

For example, if the user searches for the term *neill AND capital* this will match in two cases but not the other two.

oneill	
o'neill	
o'	neill
o	neill?

aren't	
arent	
are	n't
aren	t?

Language and Domain Identification

Tokenization is language-specific.

Before starting tokenizing, it is required to **identify the language**. This problem can be addressed using Classification Methods

Even the **domain** is important. Documents describing **Computer Technology** could contain specific IP address (*142.32.48.231*) which should be tokenized like as single token. Documents describing **Aircraft** might contain names like *B-52*, and again it should be considered like a single token.

Hyphenation

English language makes use of **Hyphen**.

For example: co-education, Hewlett-Packard, a six-minute activity

How those words should be considered? Only one token or two tokens?

Of course, the choice restricts what people can search for.

One heuristic can be to allow short hyphenated prefixes on words, but not longer hyphenated forms

Apostrophe

Many latin-based languages (like Italian, French, Spanish) makes use of apostrophe.

For example: *l'albero*

It would be preferred that documents containing *l'albero* and *un albero* are indexed under *albero*

White Space

Another aspect that makes tokenization harder is **the presence of white space** between words that should not be considered as two or more tokens.

For example: *Los Angeles, San Francisco, New York University*

The same happens for phone numbers (*800 2342333*) or dates (*Mar 11, 1983*)

Compound Nouns

Other languages make the problem harder in new ways.

German writes **compound names** without spaces

For example:

Lebensversicherungsgesellschaftsangestellter = Life Insurance Company Employee

Here, we can use a ***compound-splitter module***: by using a vocabulary the compound noun is splitted in words when they appear in the vocabulary

Stop Words: dropping common terms

Not all tokens should be taken into account.

Many of them, indeed, are excluded from the vocabulary (because for example they are very frequent).

Those common terms are named **Stop Words**

Which token should be ignored? The general strategy is to sort all terms by **collection frequency** (total number of times each term appears in the document collection) and drop all those that are most common

Stop List: Dropping common terms

To avoid dropping domain-related terms that are frequent in the documents, an alternative is a **Stop List**: a list of all terms to be discarded.

a	an	and	are	as	at	be	by	for	from
has	he	in	is	it	its	of	on	that	the
to	was	were	will	with					

Using a Stop List **reduces the number of postings** that the system has to store

Dropping common terms: the side effect

Sadly, dropping common terms has a **side effect**: Search results might not be very precise.

For example, suppose that we want to search for a song. Think about songs like *To be or not to be, Let It Be, I don't want to be.* Many of these tokens are common terms (*to, be, or, not, it, don't*). If we drop them off from indexing, of course **searches are not precise.**

For this reasons, Search Engines makes use of a **very small Stop List** (7-12 terms) or rather no Stop List at all.

Normalization

Tokens should also be normalized:

- **Accents and diacritics** should be removed (In many cases, users will enter queries for words without diacritics, whether for reasons of speed, laziness, limited software, or habits)
- **British vs American:** some British words are slightly different from the equivalent American version, for example *colour* and *color*
- **Date:** In US dates are written in the format *mm/dd/yyyy*, in Europe instead *dd/mm/yyyy*

Capitalization / Case-folding

Case-folding means **reducing all letters to lower case**.

This helps during queries: for example people searching for *ferrari* (lower case) are interested in the *Ferrari car*.

But sometimes the name with the capital letter is a real name (like General Motors) and it should not be put in lowercase.

To deal with this problem, an **heuristic** is to convert to lowercase words at the beginning of a sentence and all words occurring in a title that is all uppercase or in which most or all words are capitalized. Mid-sentence capitalized words are left as capitalized

Stemming and Lemmatization

For grammar reasons, in many languages the same word can be written in different ways, for example: *organize, organizes, organizing*.

A query search using one of these words should retrieve all documents containing the others.

By using **Stemming and Lemmatization** we can reduce a word into a base form

Stemming and Lemmatization

Stemming: dropping off the end of the word hoping to achieve the goal most of the time

Lemmatization: making use of a vocabulary in order to find the base form of a word, which is known as **lemma**

Example: think about the word *saw* (which is the past of *see*)

Stemming: returns *s*

am, are, is $\Rightarrow$ *be*

Lemmatization: returns *see*

car, cars, car's, cars' $\Rightarrow$ *car*

Stemming: the Porter's algorithm

An effective most common algorithm for stemming english words is the Porter's algorithm.

The aim of this algorithm is to remove the suffixes from an English word and obtain its stem in order to reduce the space required for storing the Dictionary

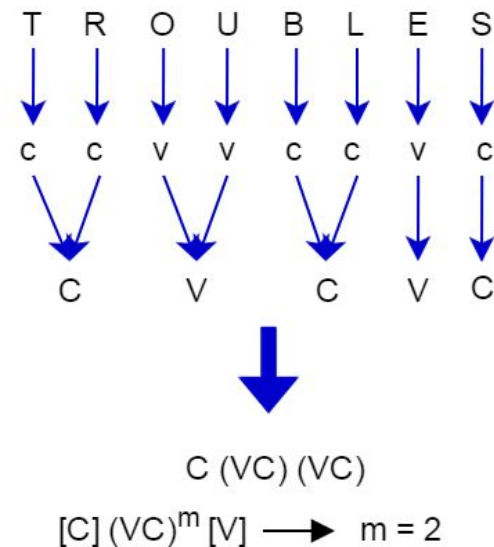
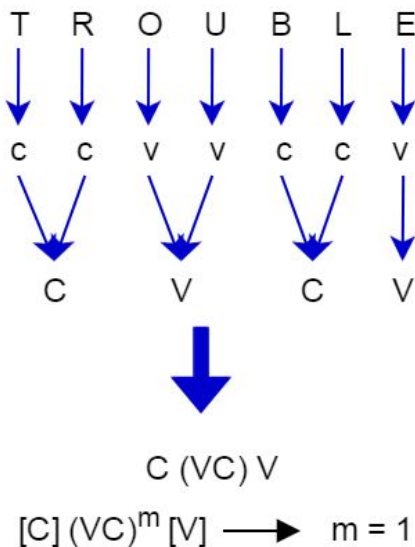
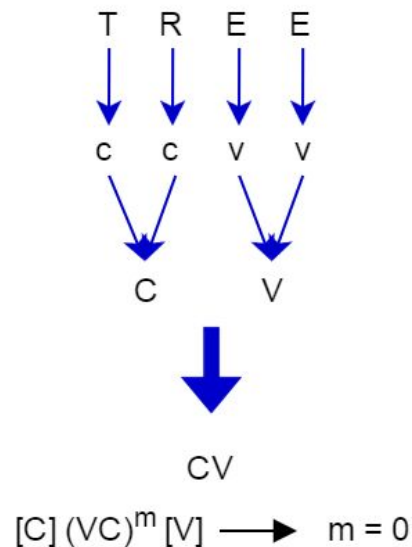
Porter's Algorithm: Consonant and Vowels

All English words can be considered as a sequence of consonant and vowel. All of them share the same pattern

$$[C](VC)^m[V]$$

m is called **measure** and represent the number of pairs Vowel-Consonant appearing in the original word.

Porter's Algorithm: Consonant and Vowels



Porter's Algorithm: Rules

This algorithm consists of **5 steps** of word reductions, applied **sequentially**.

On each step, some **rules** are applied on the word by replacing the ending part of the word according to the rule

A rule has the following form:

(condition) S1 -> S2

where condition (optional) define when the rule can be applied, S1 is the suffix in the original word, S2 is the suffix to replace S1

Porter's Algorithm: Conditions

A condition in the rule can contain:

- *S: the stem ends with S (and similarly for the other letters)
- *v*: the stem contains a vowel
- *d: the stem ends with a double consonant (e.g. -TT, -SS)
- *o: the stem ends cvc, where the second c is not W, X or Y (e.g. -WIL, -HOP)
- m>k: where m is the measure and k is a constant
- one or more previous rules with **AND, OR, NOT**

Porter's Algorithm: Step 1a

Step 1a

1. SSES	→	SS
2. IES	→	I
3. SS	→	SS
4. S	→	

For example CARESSES -> CARESS

Porter's Algorithm: Step 1b

Step 1b

1. (m>0) EED → EE
2. (*v*) ED →
3. (*v*) ING →

If the second or third of the rules in Step 1b is successful, the following is performed.

1. AT → ATE
2. BL → BLE
3. IZ → IZE
4. (*d and not (*L or *S or *Z)) → single letter
5. (m=1 and *o) → E

Porter's Algorithm: Step 1c

Step 1c

| 1. (*v*) Y → I

Porter's Algorithm: Step 2

Step 2

1. (m>0) ATIONAL

→

ATE

2. (m>0) TIONAL

→

TION

3. (m>0) ENCI

→

ENCE

4. (m>0) ANCI

→

ANCE

5. (m>0) IZER

→

IZE

6. (m>0) ABLI

→

ABLE

7. (m>0) ALLI

→

AL

8. (m>0) ENTLI

→

ENT

9. (m>0) ELI

→

E

10. (m>0) OUSLI

→

OUS

11. (m>0) IZATION

→

IZE

12. (m>0) ATION

→

ATE

13. (m>0) ATOR

→

ATE

14. (m>0) ALISM

→

AL

15. (m>0) IVENESS

→

IVE

16. (m>0) FULNESS

→

FUL

17. (m>0) OUSNESS

→

OUS

18. (m>0) ALITI

→

AL

19. (m>0) IVITI

→

IVE

20. (m>0) BILITI

→

BLE

Porter's Algorithm: Step 3

Step 3

1. (m>0) ICATE	→	IC
2. (m>0) ATIVE	→	
3. (m>0) ALIZE	→	AL
4. (m>0) ICITI	→	IC
5. (m>0) ICAL	→	IC
6. (m>0) FUL	→	
7. (m>0) NESS	→	

Porter's Algorithm: Step 4

Step 4

1. (m>1) AL	→
2. (m>1) ANCE	→
3. (m>1) ENCE	→
4. (m>1) ER	→
5. (m>1) IC	→
6. (m>1) ABLE	→
7. (m>1) IBLE	→
8. (m>1) ANT	→
9. (m>1) EMENT	→
10. (m>1) MENT	→

11. (m>1) ENT	→
12. (m>1 and (*S or *T)) ION	→
13. (m>1) OU	→
14. (m>1) ISM	→
15. (m>1) ATE	→
16. (m>1) ITI	→
17. (m>1) OUS	→
18. (m>1) IVE	→
19. (m>1) IZE	→

Porter's Algorithm: Step 5

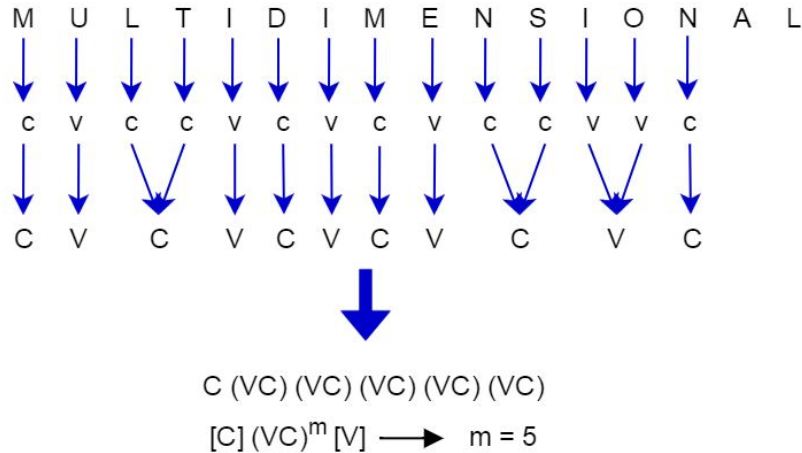
Step 5a

- 1. (m>1) E →
- 2. (m=1 and not *o) E →

Step 5b

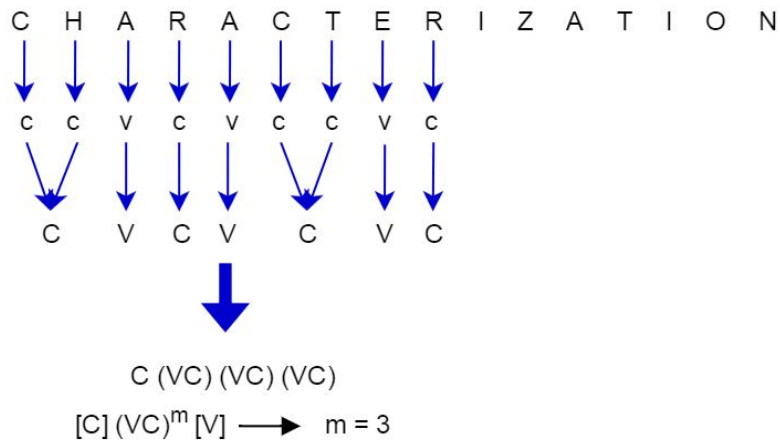
- 1. (m > 1 and *d and *L) → single letter

Porter's Algorithm: Example 1



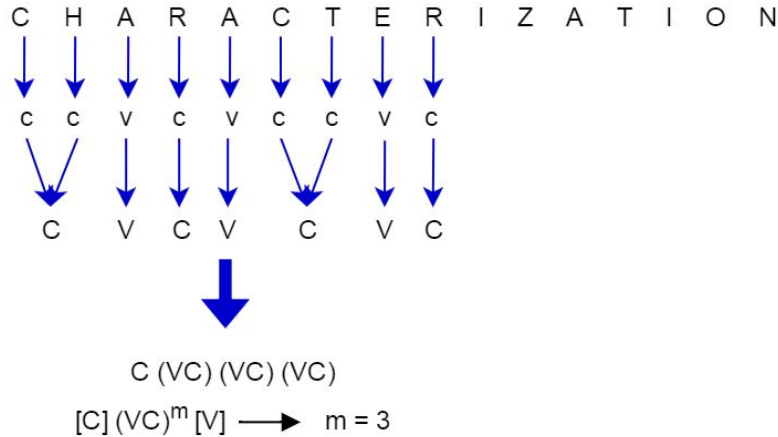
1. The suffix will not match any of the cases found in steps 1, 2 and 3.
2. Then it comes to step 4.
3. The stem of the word has $m > 1$ (since $m = 5$) and ends with "AL".
4. Hence in step 4, "AL" is deleted (replaced with null).
5. Calling step 5 will not change the stem further.
6. Finally the output will be MULTIDIMENSION.

Porter's Algorithm: Example 2



1. The suffix will not match any of the cases found in step 1.
2. So it will move to step 2.
3. The stem of the word has $m > 0$ (since $m = 3$) and ends with "IZATION".
4. Hence in step 2, "IZATION" will be replaced with "IZE".
5. Then the new stem will be CHARACTERIZE.

Porter's Algorithm: Example 2



6. Step 3 will not match any of the suffixes and hence will move to step 4.
7. Now $m > 1$ (since $m = 3$) and the stem ends with "IZE".
8. So in step 4, "IZE" will be deleted (replaced with null).
9. No change will happen to the stem in other steps.
10. Finally the output will be CHARACTER.

Stemming: other algorithms

Sample text: Such an analysis can reveal features that are not easily visible from the variations in the individual genes and can lead to a picture of expression that is more biologically transparent and accessible to interpretation

Lovins stemmer: such an analysis can reveal features that are not easily visible from the variations in the individual genes and can lead to a picture of expression that is more biologically transparent and accessible to interpretation

Porter stemmer: such an analysis can reveal features that are not easily visible from the variations in the individual genes and can lead to a picture of expression that is more biologically transparent and accessible to interpretation

Paice stemmer: such an analysis can reveal features that are not easily visible from the variations in the individual genes and can lead to a picture of expression that is more biologically transparent and accessible to interpretation

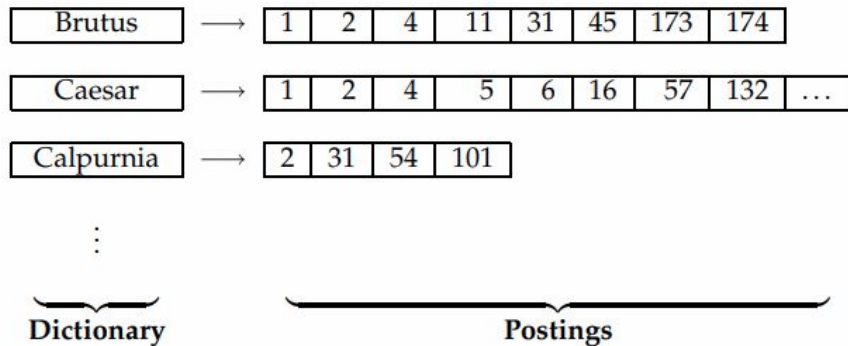
Postings List

Postings List

The Postings List is a list of all documents that are indexed with a term.

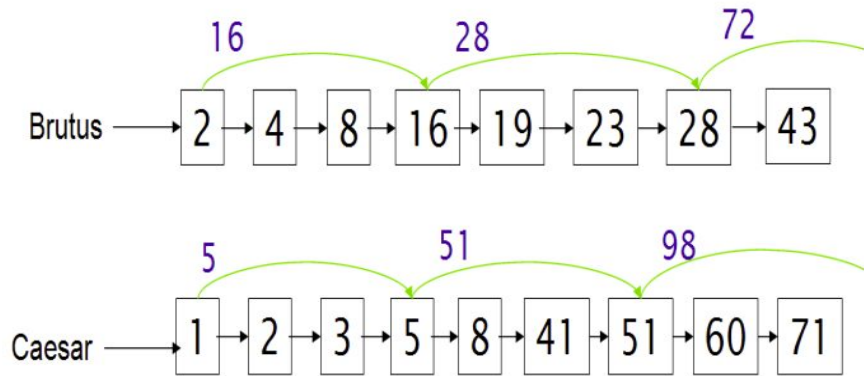
Previously, we discussed about the time for the **intersection operation**, which takes $O(n+m)$, where n and m are the length of two postings list.

Can we do better?



Skip List

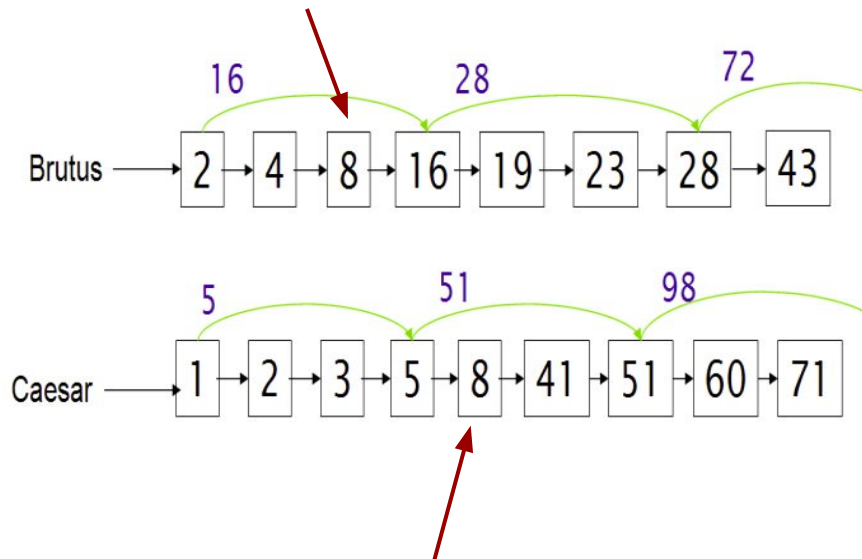
At indexing time (when index is built) we can add some **shortcuts**, that are points to other elements of the Postings Lists, used to make intersection faster (which means taking **sublinear time**)



Skip List

Let's see how skip pointers can effectively make intersection faster:

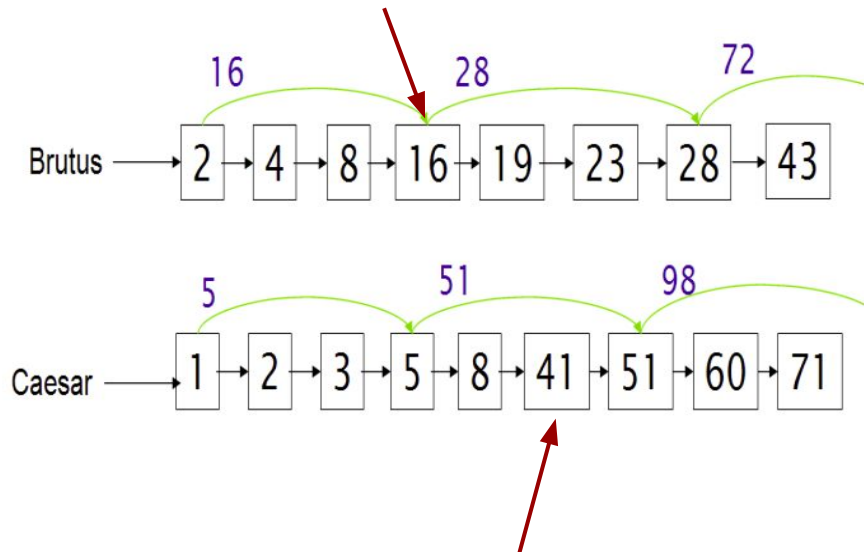
- Suppose we stepped through the lists until we have matched 8
- we move forward the pointers on both list (the first goes to 16, the second to 41)



Skip List

Let's see how skip pointers can effectively make intersection faster:

- Rather than simply moving the first pointer throughout all elements till we reach 41 (if available), we can easily jump to directly 28 (items are **ordered**)



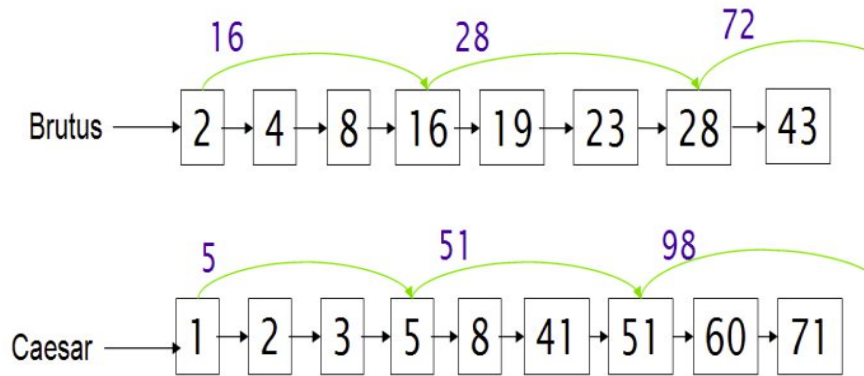
Intersection with Skip

INTERSECTWITHSKIPS(p_1, p_2)

```
1  answer  $\leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $\text{ADD}(\text{answer}, \text{docID}(p_1))$ 
5           $p_1 \leftarrow \text{next}(p_1)$ 
6           $p_2 \leftarrow \text{next}(p_2)$ 
7  else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
8      then if  $\text{hasSkip}(p_1)$  and  $(\text{docID}(\text{skip}(p_1)) \leq \text{docID}(p_2))$ 
9          then while  $\text{hasSkip}(p_1)$  and  $(\text{docID}(\text{skip}(p_1)) \leq \text{docID}(p_2))$ 
10             do  $p_1 \leftarrow \text{skip}(p_1)$ 
11             else  $p_1 \leftarrow \text{next}(p_1)$ 
12  else if  $\text{hasSkip}(p_2)$  and  $(\text{docID}(\text{skip}(p_2)) \leq \text{docID}(p_1))$ 
13      then while  $\text{hasSkip}(p_2)$  and  $(\text{docID}(\text{skip}(p_2)) \leq \text{docID}(p_1))$ 
14          do  $p_2 \leftarrow \text{skip}(p_2)$ 
15          else  $p_2 \leftarrow \text{next}(p_2)$ 
16  return answer
```

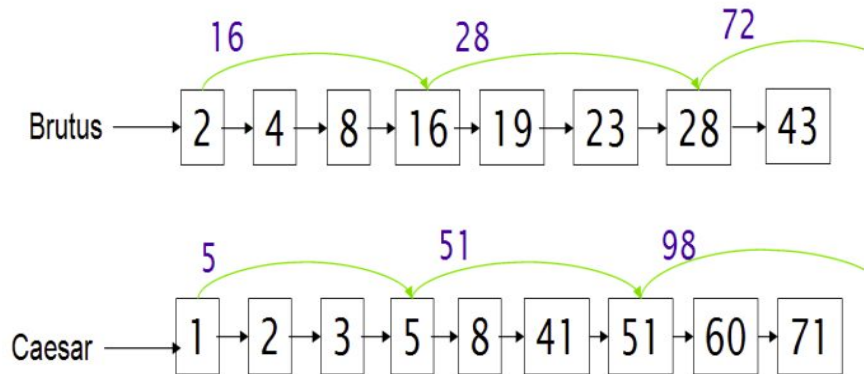

Considerations on Skip List

- Skip List only works with *AND* queries, and not for *OR* queries
- Where a Skip Pointer should be put?
 - In the example, a Skip Point is put each 4 items. Is it a good choice?
 - There is a **tradeoff**



Considerations on Skip List

- Shorter Skip Span means:
 - more Skip Pointers
 - more comparisons of pointers
 - more space to store pointers
 - more opportunities to skip



A **simple heuristic** for placing skips, which has been found to work well in practice, is that for a posting list of length P , use **$\sqrt{P}$** evenly-spaced skip pointers.

The hardness of managing Skip List

If an index **changes hardly ever**, dealing with Skip List is quite simple.

Frequent updates in the index makes management very complex because we need **to update all pointers** when a new item is added to the postings list.

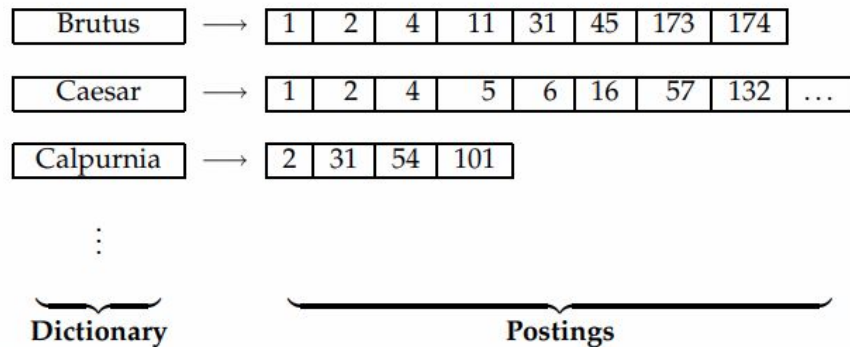
Remind: **Postings Lists are sorted** by Document ID

The problem of single terms

So far we have been thinking about the Dictionary like a **single term** structure.

This works fine but sometimes query results can be unexpected

For example:



The problem of single terms

Suppose we have the following document:

The inventor Stanford Ovshinsky never went to university

and we want to search for *Stanford University*.

Being the words *Stanford* and *University* into the document, the document is retrieved by the Search Engine, although it actually does not refer to the Standard University.

The problem of single terms

This problem has been partially solved by the modern Search Engine by using **Phrase Queries**

A Phrase Query is a sequence of words wrapped by **double quotes**. For example *"Stanford University"*.

To be able to support such a kind of Phrase Queries we need to **extends the Postings List**.

There are two approaches: **BiWord Index** and **Positional Index**

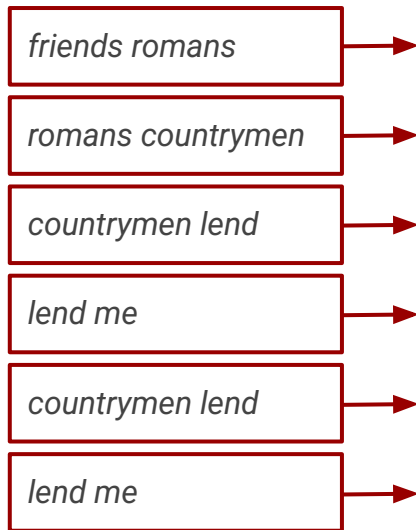
BiWord Index

The idea is very simple: every pair of consecutive terms in a document (**biword**) is considered as a phrase.

Each biword is treated as a **vocabulary term**.

Example

Friends, Romans, Countrymen, lend me your ears;



BiWord Index

Sometimes words to be considered as a Biword are not close to each other (like in *Stanford University*) but they are divided by other words (such as articles, prepositions, etc).

Consider the following example:

the abolition of the slavery or the renegotiation of the constitution

We would like to have the following biword: *abolition slavery* and *renegotiation constitution*, but those names are separated by articles and prepositions

BiWord Index

So, the idea is **to tag** all words in nouns (N) and non-nouns (X), such as articles and prepositions.

For example:

renegotiation	of	the	constitution
N	X	X	N

A biword is any string of terms of the form **NX*N**

So, *renegotiation constitution* is then used as vocabulary term.

BiWord Index

Another example:

The document containing the phrase

cost overruns on a power plant

will be indexed this the following three biwords

cost overruns

overruns power

power plant

Considerations on using BiWord Index

- searches for a single term **are not naturally handled** in a biword index
- to allow single term searches we also need **single-word index**
- **false positive** matches (non relevant documents that are retrieved) becomes very small
- storing biwords makes the **dictionary larger**
- biword index is a convenient method for implementing **proximity operators**

Phrase Index

The concept of Biword Index can be extended to **three or more words**.

If the number of the words considered like a atomic word is variable, we can talk about **Phrase Index**.

The consideration we did on Biwords must be kept into account even for Phrase Index.

Positional Index

A second approach for solving the single term problem, is the **Positional Index**.

Positional index manages Dictionary and Postings Lists in a different manner:

- vocabulary term
- term frequency in the collection
- document id
- term frequency in the document
- position of the term in the document

to, 993427:

$\langle 1, 6: \langle 7, 18, 33, 72, 86, 231 \rangle;$

$2, 5: \langle 1, 17, 74, 222, 255 \rangle;$

$4, 5: \langle 8, 16, 190, 429, 433 \rangle;$

$5, 2: \langle 363, 367 \rangle;$

$7, 3: \langle 13, 23, 191 \rangle; \dots \rangle$

be, 178239:

$\langle 1, 2: \langle 17, 25 \rangle;$

$4, 5: \langle 17, 191, 291, 430, 434 \rangle;$

$5, 3: \langle 14, 19, 101 \rangle; \dots \rangle$

Positional Index

During Queries, Positional Index works in a similar manner of general:

We scan the inverted index starting from terms less frequent. In the **Intersection Operation**, rather than simply checking that both terms are in a document, we also need to check that their positions of appearance in the document are **compatible with the phrase** query being evaluated.

to, 993427:

$\langle 1, 6: \langle 7, 18, 33, 72, 86, 231 \rangle;$
 $2, 5: \langle 1, 17, 74, 222, 255 \rangle;$
 $4, 5: \langle 8, 16, 190, 429, 433 \rangle;$
 $5, 2: \langle 363, 367 \rangle;$
 $7, 3: \langle 13, 23, 191 \rangle; \dots \rangle$

be, 178239:

$\langle 1, 2: \langle 17, 25 \rangle;$
 $4, 5: \langle 17, 191, 291, 430, 434 \rangle;$
 $5, 3: \langle 14, 19, 101 \rangle; \dots \rangle$

Positional Index

An example: suppose we want to search for *to be or not to be*. We look for the terms *to*, *be*, *not*, or in the posting list.

Now, let's consider the intersection between *to* and *be*.

Documents 1, 4, 5 contain both terms.

to, 993427:

$\langle 1, 6: \langle 7, 18, 33, 72, 86, 231 \rangle;$
 $2, 5: \langle 1, 17, 74, 222, 255 \rangle;$
 $4, 5: \langle 8, 16, 190, 429, 433 \rangle;$
 $5, 2: \langle 363, 367 \rangle;$
 $7, 3: \langle 13, 23, 191 \rangle; \dots \rangle$

be, 178239:

$\langle 1, 2: \langle 17, 25 \rangle;$
 $4, 5: \langle 17, 191, 291, 430, 434 \rangle;$
 $5, 3: \langle 14, 19, 101 \rangle; \dots \rangle$

Positional Index

The aim of the Positional Index is to give a **measure of relevance**: documents having both terms closer are more relevant to others.

Then we look the posting list of *be* to see if there is an index **one higher** than a position of *to*

to, 993427:

⟨ 1, 6: ⟨7, 18, 33, 72, 86, 231⟩;

2, 5: ⟨1, 17, 74, 222, 255⟩;

4, 5: ⟨8, 16, 190, 429, 433⟩;

5, 2: ⟨363, 367⟩;

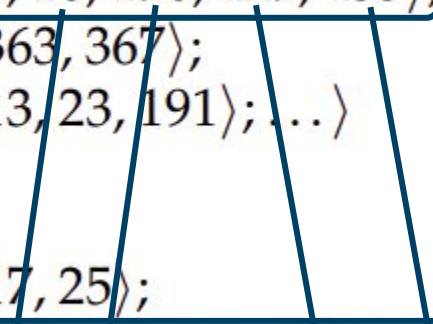
7, 3: ⟨13, 23, 191⟩; ... ⟩

be, 178239:

⟨ 1, 2: ⟨17, 25⟩;

4, 5: ⟨17, 191, 291, 430, 434⟩;

5, 3: ⟨14, 19, 101⟩; ... ⟩



Positional Index

In this example, we used $K = 1$ (we looked for an instance of *be* having an index position **one higher** of *to*)

We can also generalize the algorithm for any K .

The following pseudocode shows the Positional Intersection with a generic K

to, 993427:

$\langle 1, 6: \langle 7, 18, 33, 72, 86, 231 \rangle;$

$2, 5: \langle 1, 17, 74, 222, 255 \rangle;$

$4, 5: \langle 8, 16, 190, 429, 433 \rangle;$

$5, 2: \langle 363, 367 \rangle;$

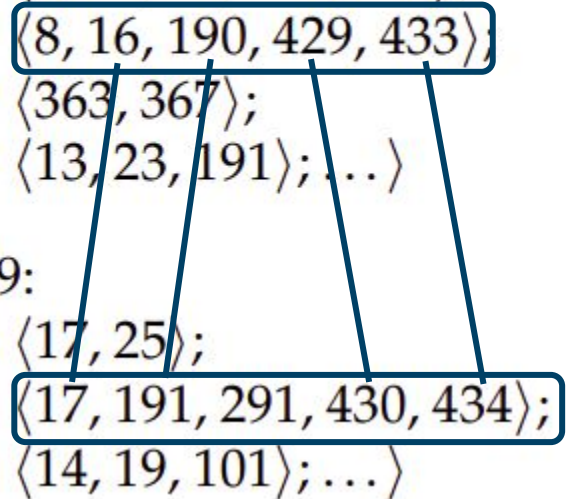
$7, 3: \langle 13, 23, 191 \rangle; \dots \rangle$

be, 178239:

$\langle 1, 2: \langle 17, 25 \rangle;$

$4, 5: \langle 17, 191, 291, 430, 434 \rangle;$

$5, 3: \langle 14, 19, 101 \rangle; \dots \rangle$



Positional Index

```
POSITIONALINTERSECT( $p_1, p_2, k$ )
1   $answer \leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $l \leftarrow \langle \rangle$ 
5           $pp_1 \leftarrow \text{positions}(p_1)$ 
6           $pp_2 \leftarrow \text{positions}(p_2)$ 
7          while  $pp_1 \neq \text{NIL}$ 
8          do while  $pp_2 \neq \text{NIL}$ 
9              do if  $|\text{pos}(pp_1) - \text{pos}(pp_2)| \leq k$ 
10                  then  $\text{ADD}(l, \text{pos}(pp_2))$ 
11                  else if  $\text{pos}(pp_2) > \text{pos}(pp_1)$ 
12                      then break
13                       $pp_2 \leftarrow \text{next}(pp_2)$ 
14                  while  $l \neq \langle \rangle$  and  $|l[0] - \text{pos}(pp_1)| > k$ 
15                      do  $\text{DELETE}(l[0])$ 
16                      for each  $ps \in l$ 
17                          do  $\text{ADD}(answer, \langle \text{docID}(p_1), \text{pos}(pp_1), ps \rangle)$ 
18                           $pp_1 \leftarrow \text{next}(pp_1)$ 
19                       $p_1 \leftarrow \text{next}(p_1)$ 
20                       $p_2 \leftarrow \text{next}(p_2)$ 
21                  else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
22                      then  $p_1 \leftarrow \text{next}(p_1)$ 
23                      else  $p_2 \leftarrow \text{next}(p_2)$ 
24  return  $answer$ 
```

Consideration on Positional Index

- Using Positional Index makes the **size of the inverted index larger**
 - the index size depends on the average document size
- The asymptotic complexity of a postings **intersection operation** changes
 - from **$O(N)$ to $O(T)$** where N is the number of documents, while T is the number of tokens
- It does not work with BiWord indexes
- Positional index is another convenient method for implementing **proximity operators**

Combining schemas

The strategies of biword indexes and positional indexes can be fruitfully **combined**.

For example, for certain queries we can use Phrase Index or Biword Index, for others instead we use Positional Index.

Terms to be included in a Phrase Index are those that are known to be common **based on recent querying behavior**.

Dictionary and Tolerant Retrieval

Academic Year 2025/2026

Luigi Santangelo
luigi.santangelo@unipv.it

University of Pavia
**Master Degree in
Computer Engineering**

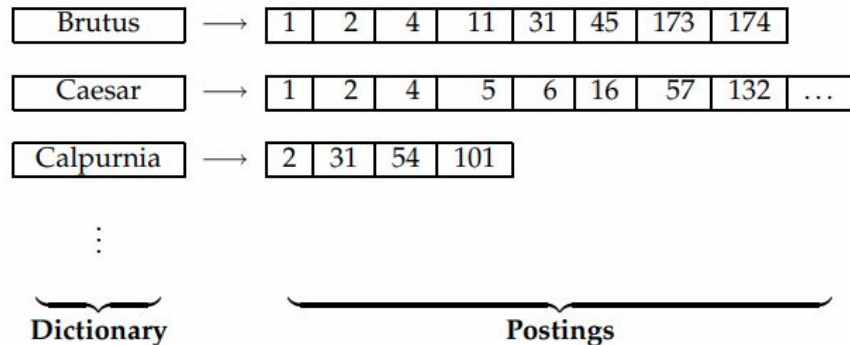
Data Structure for Dictionaries

Structures for Dictionaries

In previous sections we have been talking about different implementation of Inverted Index.

Here instead different data structures for dictionaries are described.

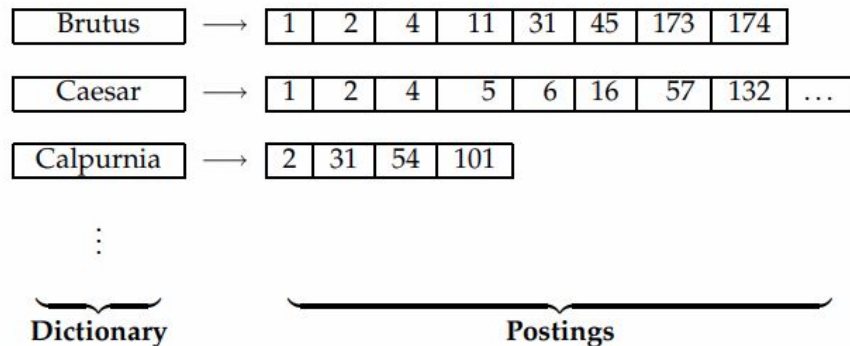
Usually, two different data structures are used: **hashing** and **search tree**.



Hashing

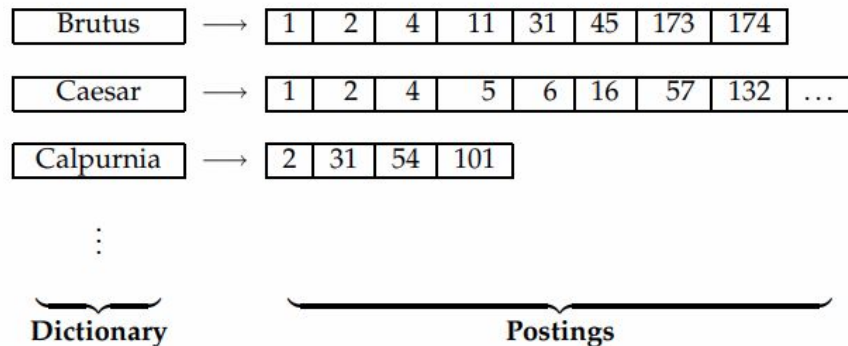
With the Hashing Data Structure, during indexing, each term in the Vocabulary is hashed into an integer large enough, **such that hash collisions are unlikely**

At **Query Time**, each term is hashed and then we get the Postings List by using the hash value.



Considerations with Hashing

- little **variation** in the term (even adding an accent) changes the hash value
- it is not possible to search for all terms beginning with a **prefix**
- for **small Dictionaries**, hashing may work fine, but for big Dictionary (like the Web) it might not be a good choice.



Search Trees

Search Trees overcome many of the issues introduced by Hashing.

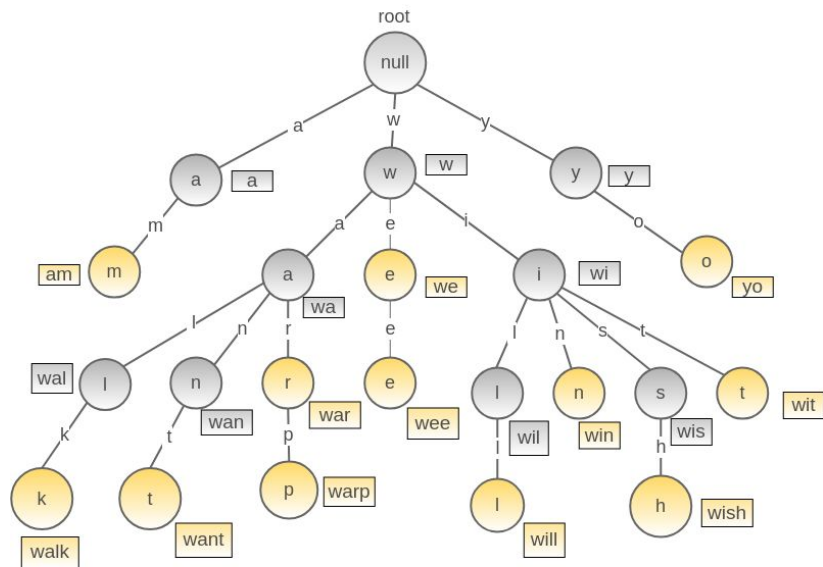
For instance they permit to enumerate all terms beginning with **a prefix**

There are many implementations of Search Trees:

- Prefix Tree (also known as Trie)
- Prefix Binary Tree
- Prefix B-Tree

Prefix Tree

- It is a type of **k-ary** search tree
- Keys are strings
- **Searched Terms** can be in the leaves or in the internal nodes
- A search operation can go **one way or the others**, depending on the value to search for
- Big problems:
 - the tree is not **balanced**
 - the arity is **26⁺** (if not just letters are considered)



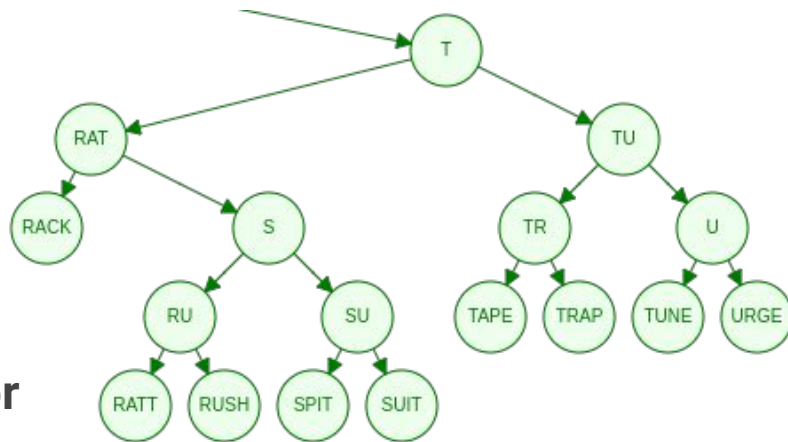
Word(s) with prefix **wa**: walk, want, war, warp

Word(s) with prefix **we**: wee

Word(s) with prefix **wi**: will, win, wish, wit

Prefix Binary Tree

- It is a **binary tree**
- Keys are **strings**
- Each **internal node** represent a test (no real terms in the collection)
- The **leaves** represent the terms we can search for
- A search operation can go **one way or the other**
- Big problems:
 - the tree is not **balanced**
 - **auxiliary terms** must be introduced



Prefix Binary Tree: Separator (1)

To build a proper Prefix Binary Tree, we need to introduce the meaning of **Separator**.

Let S_j be a term already stored in the Tree. Let S_i be the new term to be stored into the tree as a **left child** of S_j .

A separator S is the shortest string derived from S_j such that $S_i < S \leq S_j$

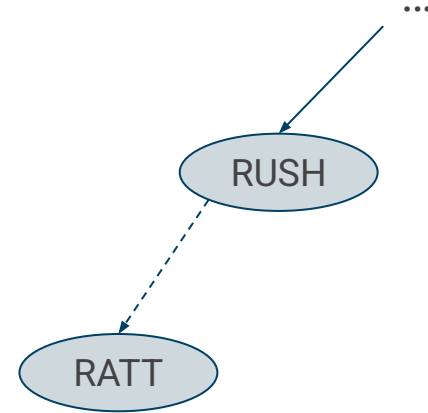
Before inserting S_i into the tree, the Separator must be inserted and the Tree must be rearranged

Prefix Binary Tree: Separator - Example 1

Let RUSH be a term already stored in the Tree.

Let RATT the new term to be stored into the tree

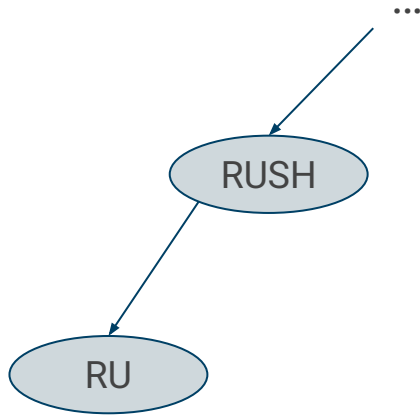
Being $RATT < RUSH$, RATT will be inserted as the left child of RUSH



Prefix Binary Tree: Separator - Example 1

Before inserting RATT, we need:

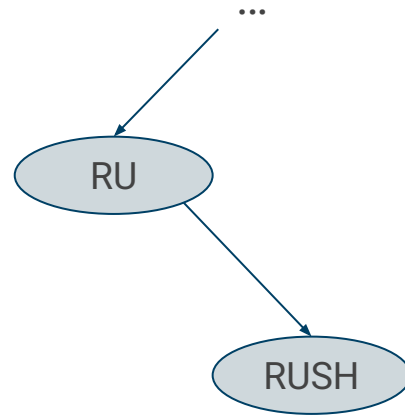
1. Defining the selector S, here RU
2. Adding RU as left child of RUSH



Prefix Binary Tree: Separator - Example 1

Before inserting RATT, we need:

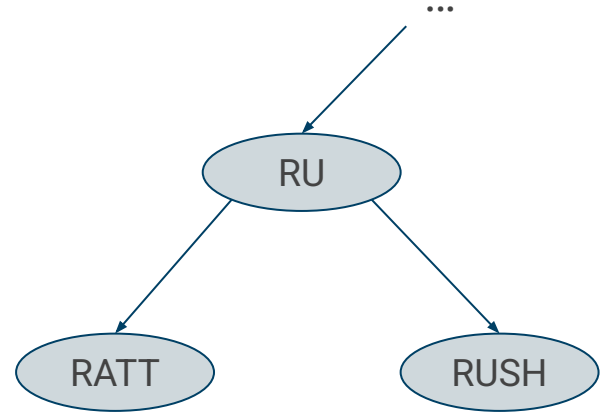
1. Defining the selector S, here RU
2. Adding RU as left child of RUSH
3. Rotating the subtree, pivoting on RUSH



Prefix Binary Tree: Separator - Example 1

Before inserting RATT, we need:

1. Defining the selector S, here RU
2. Adding RU as left child of RUSH
3. Rotating the subtree, pivoting on RUSH
4. Inserting the new term RATT



Prefix Binary Tree: Separator (2)

To build a proper Prefix Binary Tree, we need to introduce the meaning of **Separator**.

Let S_i be a term already stored in the Tree. Let S_j be the new term to be stored into the tree as a **right child** of S_i .

A separator S is the shortest string derived from S_i such that $S_i < S \leq S_j$

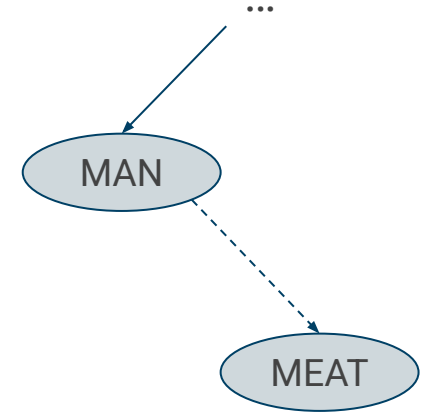
Before inserting S_j into the tree, the Separator must be inserted and the Tree must be rearranged

Prefix Binary Tree: Separator - Example 2

Let MAN be a term already stored in the Tree.

Let MEAT the new term to be stored into the tree

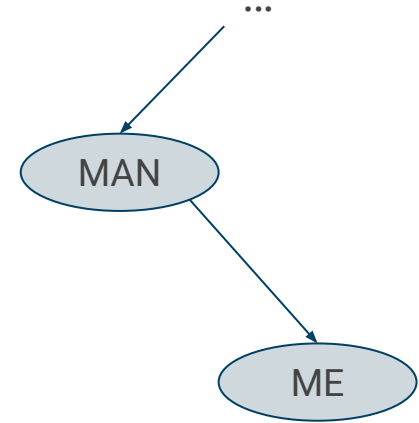
Being $MEAT > MAN$, MEAT will be inserted as the righty child of MAN



Prefix Binary Tree: Separator - Example 2

Before inserting MEAT, we need:

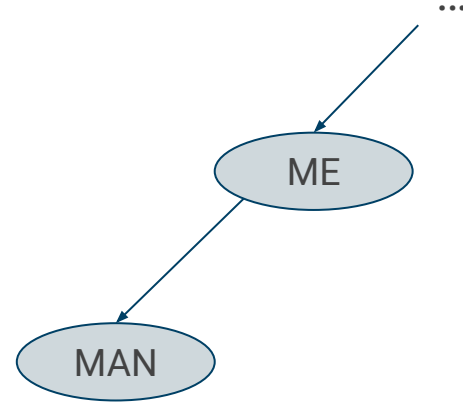
1. Defining the selector S, here ME
2. Adding ME as right child of MAN



Prefix Binary Tree: Separator - Example 2

Before inserting MEAT, we need:

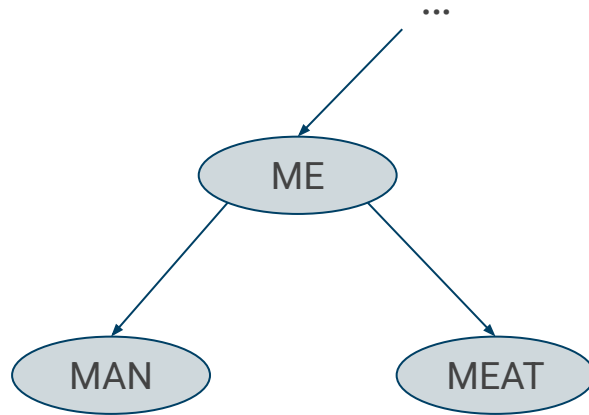
1. Defining the selector S, here ME
2. Adding ME as right child of MAN
3. Rotating the subtree, pivoting on MAN



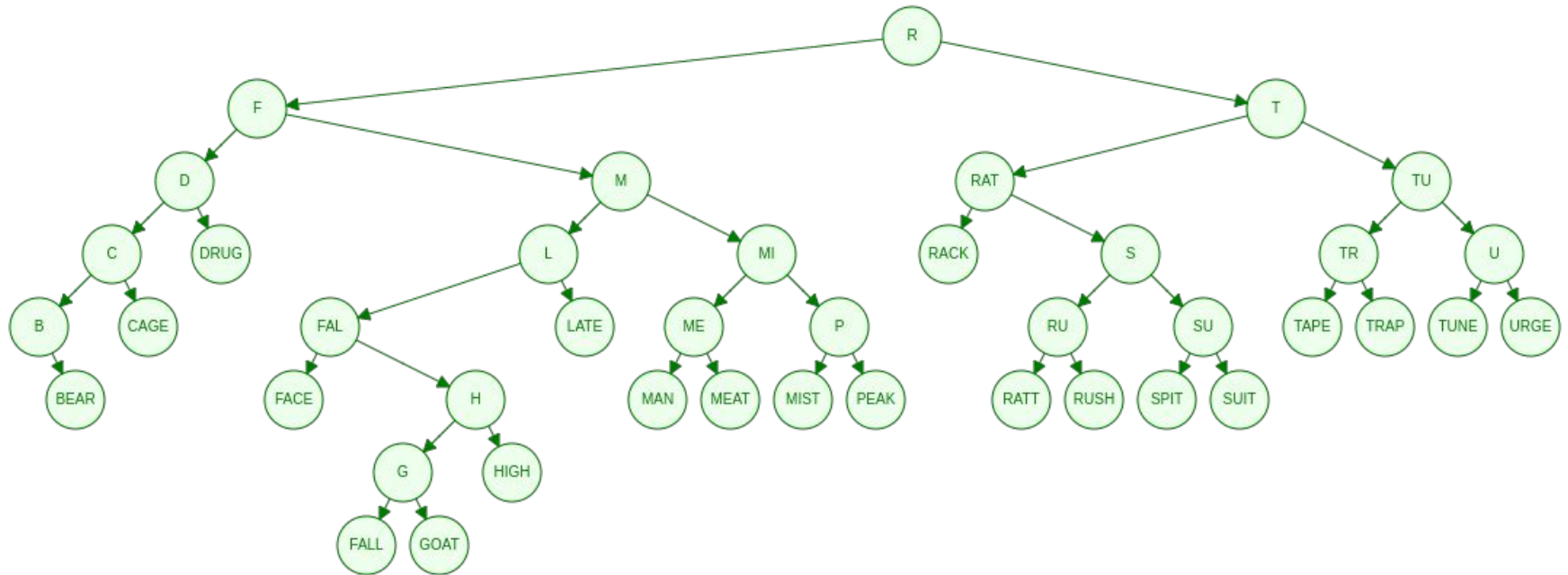
Prefix Binary Tree: Separator - Example 2

Before inserting MEAT, we need:

1. Defining the selector S, here ME
2. Adding ME as right child of MAN
3. Rotating the subtree, pivoting on MAN
4. Inserting the new term MEAT

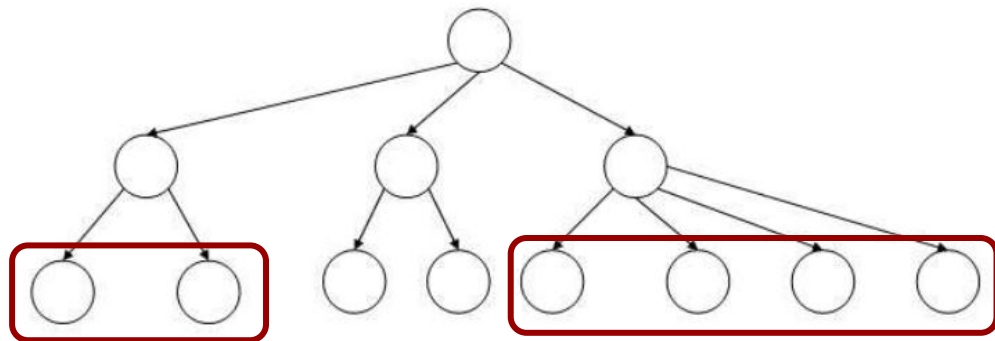


Prefix Binary Tree



Prefix B-Tree

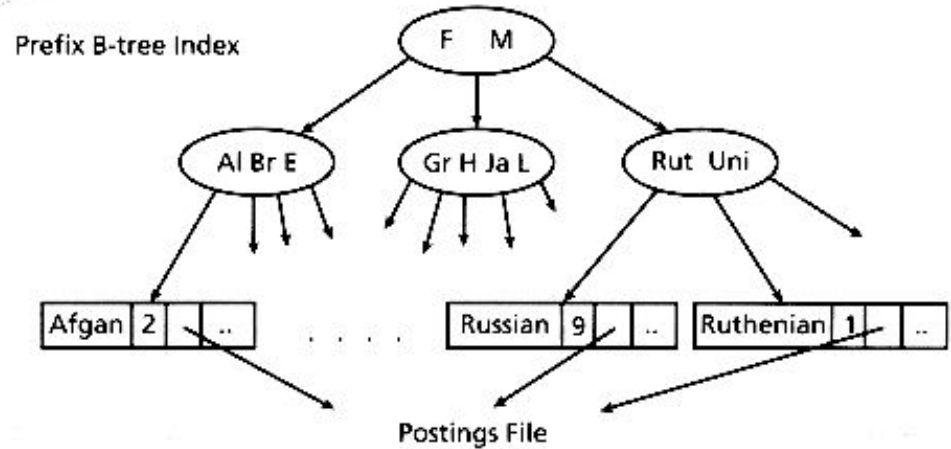
- It solves the problem of **rebalancing** tree
- It satisfies all the properties already seen for the **traditional B-Tree** (for example the **minimum degree** and others)



Prefix B-Tree

Other properties for the Prefix B-Tree

- Leaves contains the terms to store in the dictionary
- Internal nodes represent a test for a sequence of characters
- The strings in the **internal nodes** are named **separators** and are derived the terms



Prefix B-Tree: separator

Definition of **separator**:

If S_{m-1} and S_m are two keys in a full node such that S_m is the **median key** and S_{m-1} is its predecessor, a separator S is the **shortest string derived** from S_m such that

$$S_{m-1} < S \leq S_m$$

Prefix B-Tree: separator



For example: we have a **full leaf** so we need to split it into two nodes.

The median value is MET, the predecessor is EXPOSE. We can use M as the shortest string derived from MET.

The separator M is added to the leaf parent and the leaf is splitted

Prefix B-Tree: separator

Of course there are several separators that can be used but it is more appropriate to select the shortest separator possible because **shorter separator reduce the height and the size** of the B-Tree

Prefix B-Tree: an example

This is the list of keys we want to insert into our B-Tree: *rack, face, meat, tune, mist, late, ratt, hear, drug, peak, suit, fall, spit, cage, high, bear, goat, trap, urge, rush, tape*

Let consider a B-Tree with a minimum degree = 2 (this means the number of keys in each node ranges between 1 and 3)

Prefix B-Tree: an example

*rack, face, meat, tune, mist,
late, ratt, man, drug, peak,
suit, fall, spit, cage, high,
bear, goat, trap, urge, rush,
tape*

After adding the first three keys, our B-Tree looks like this



Prefix B-Tree: an example

*rack, face, meat, tune, mist,
late, ratt, man, drug, peak,
suit, fall, spit, cage, high,
bear, goat, trap, urge, rush,
tape*

Now we want to add the fourth key (*TUNE*), but the node is full. So, before inserting the new key, a separator must be defined. The median key is MEAT. The shortest string between MEAT and FACE is M. So we add M and split the node



Prefix B-Tree: an example

rack, face, meat, tune, mist,
late, ratt, man, drug, peak,
suit, fall, spit, cage, high,
bear, goat, trap, urge, rush,
tape

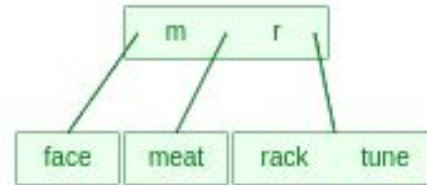
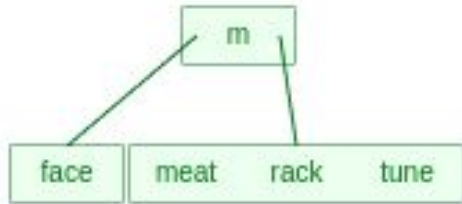
Now we add the following key (*TUNE*)



Prefix B-Tree: an example

*rack, face, meat, tune, mist,
late, ratt, man, drug, peak,
suit, fall, spit, cage, high,
bear, goat, trap, urge, rush,
tape*

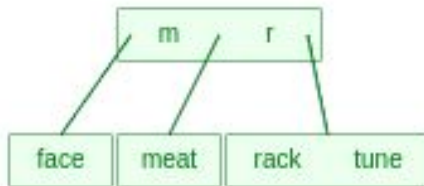
Now we want to add MIST but the node is full.
So we take the median key (RACK) and its
previous (MEAT), and we use R as separator.
We add R, splitting the node into two nodes



Prefix B-Tree: an example

*rack, face, meat, tune, mist,
late, ratt, man, drug, peak,
suit, fall, spit, cage, high,
bear, goat, trap, urge, rush,
tape*

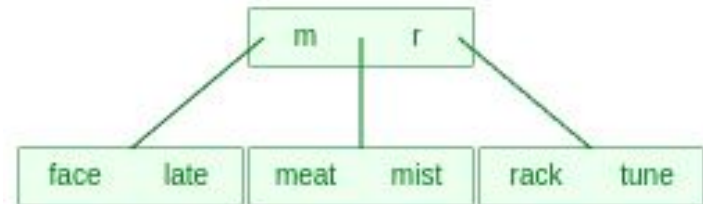
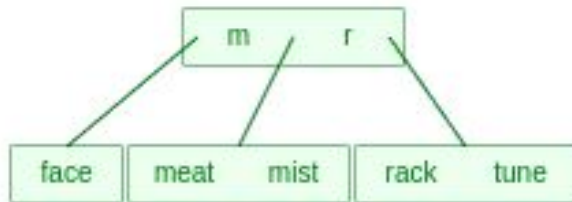
We can now add MIST



Prefix B-Tree: an example

*rack, face, meat, tune, mist,
late, ratt, man, drug, peak,
suit, fall, spit, cage, high,
bear, goat, trap, urge, rush,
tape*

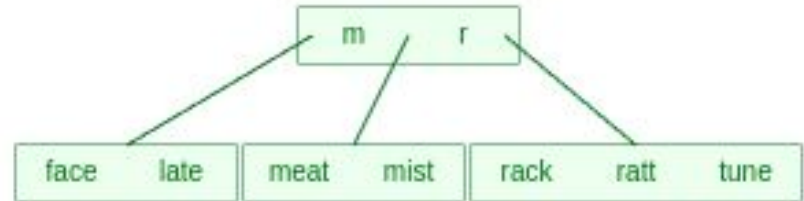
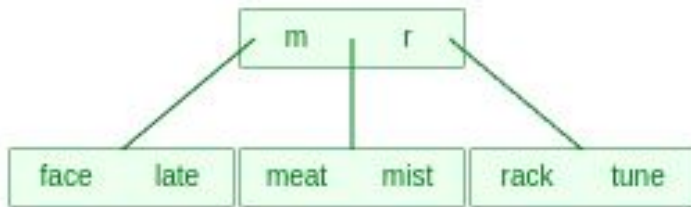
We now want add LATE. The key will fall into the first node, which is not full. So the key can be added



Prefix B-Tree: an example

*rack, face, meat, tune, mist,
late, ratt, man, drug, peak,
suit, fall, spit, cage, high,
bear, goat, trap, urge, rush,
tape*

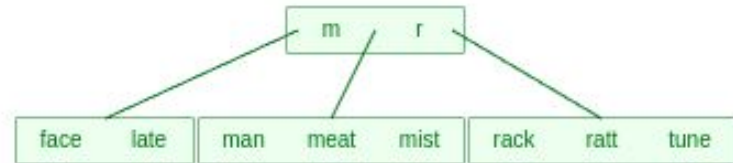
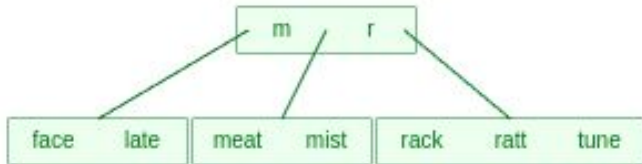
We now want add RATT. The key will fall into the third node, which is not full. So the key can be added.



Prefix B-Tree: an example

*rack, face, meat, tune, mist,
late, ratt, man, drug, peak,
suit, fall, spit, cage, high,
bear, goat, trap, urge, rush,
tape*

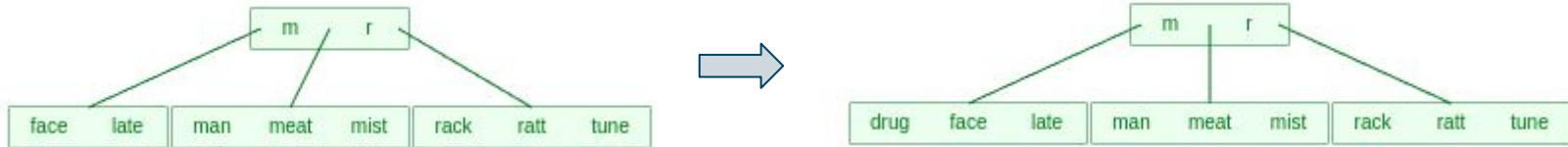
We now want add MAN. The key will fall into the second node, which is not full. So the key can be added.



Prefix B-Tree: an example

*rack, face, meat, tune, mist,
late, ratt, man, drug, peak,
suit, fall, spit, cage, high,
bear, goat, trap, urge, rush,
tape*

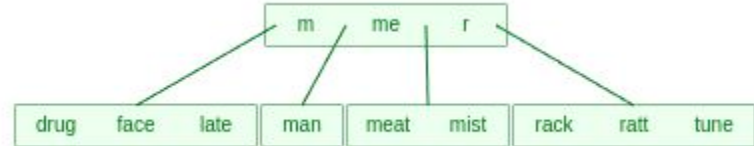
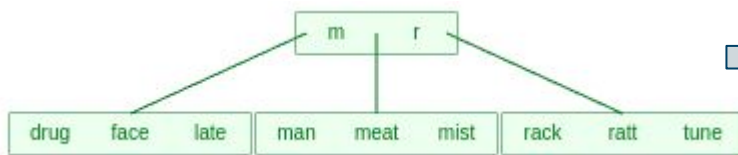
We now want add DRUG. The key will fall into the first node, which is not full. So the key can be added.



Prefix B-Tree: an example

*rack, face, meat, tune, mist,
late, ratt, man, drug, peak,
suit, fall, spit, cage, high,
bear, goat, trap, urge, rush,
tape*

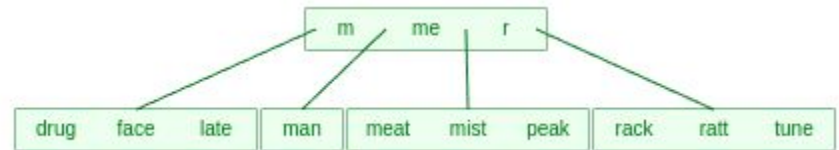
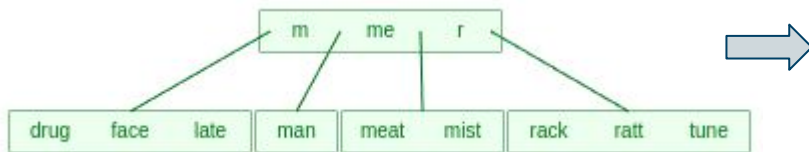
We now want add PEAK. The key will fall into the second node, which is full. So we need to add a separator. The median key is MEAT, the previous key is MAN. The separator is ME. We add ME to the tree and split the second node.



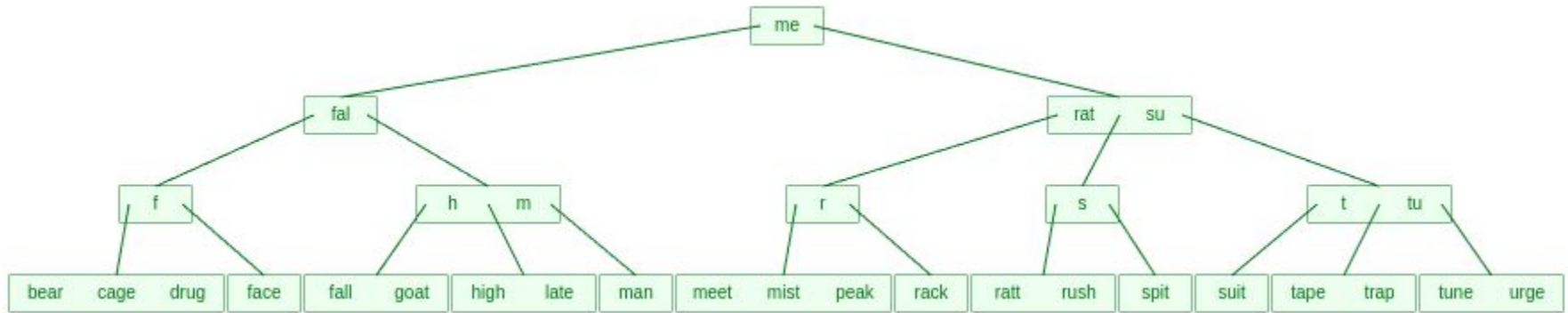
Prefix B-Tree: an example

*rack, face, meat, tune, mist,
late, ratt, man, drug, peak,
suit, fall, spit, cage, high,
bear, goat, trap, urge, rush,
tape*

We now want add PEAK. The key will fall into the third node, which is not full.



Prefix B-Tree: an example



Sequence of terms and separators: rack, face, meat, m, r, mist, late, ratt, man, drug, me, peak, rat, suit, f, fall, s, spit, cage, fal, high, bear, h, goat, su, trap, t, urge, rush, tu, tape

WildCard Query

WildCard Query

During queries, users can use wildcards (such as *):

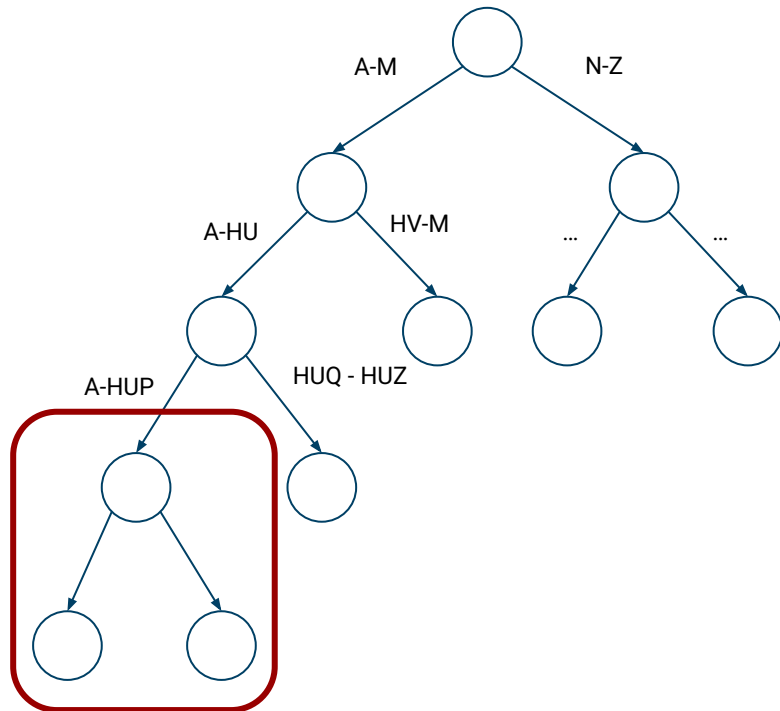
1. when they are uncertain about the spelling of a word (Sidney vs Sydney)
2. when they are aware about the different spelling of a word (color vs colour)
3. when they are searching for all document containing a word beginning with a prefix (judicia* to search for judicial, judiciary, ...)

Wildcard Query

All Prefix Trees work fine when the wildcard is at the end of a word. For example, if I use

*com**

in a query, it means I'm looking for all documents containing words beginning with *com*. I can walk down the tree, following the symbols *c*, *o* and *m*, then I can get all leaves of such subtree



WildCard Query

But how we can use Search Trees when wildcard is at the beginning of a word?
For example

**mon*

Solution: using a **Reversed Search Tree**

Reversed Search Tree is a tree where terms in the dictionary are written in **backwards**

For example, the term *lemon* in the B-Tree would be represented by the path
root-n-o-m-e-l

WildCard Query

A combination of Search Tree and Reversed Search Tree can be used to manage a more general usage of WildCard.

For example, suppose we are looking for the word

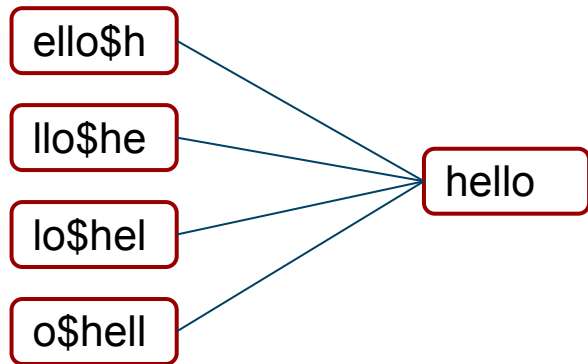
*se*mon*

we can use Search Tree to find all document with a term beginning with se, then we can use Reversed Search Tree to find all documents with a term ending with mon, then, we can **intersect** all results. Finally we use the **Standard Inverted Index** to retrieve all documents containing any term in this intersection

Permuterm Index

Another solution to manage wildcard queries is by using the **Permuterm Index**

Permuterm index is an index containing all rotations of all terms (augmented by \$)

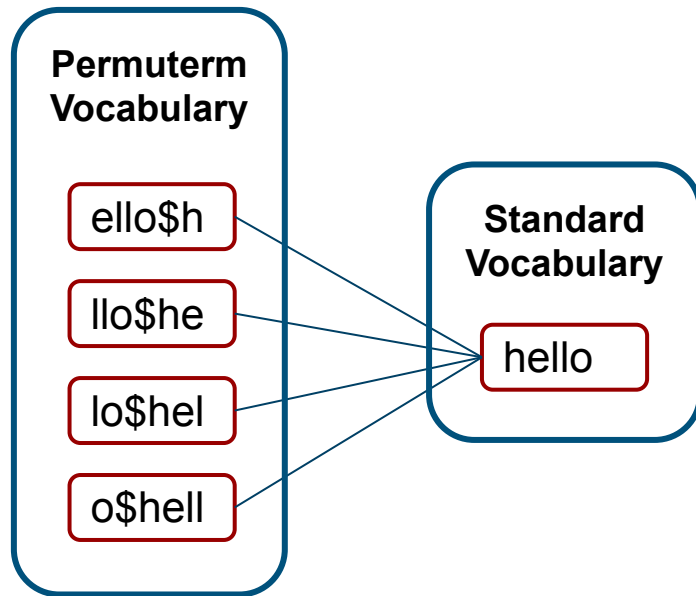


Permuterm Index

All permuterms are stored into a **permuterm vocabulary**

The permuterm index is managed as the same as the **standard inverted index** (for example B-Tree).

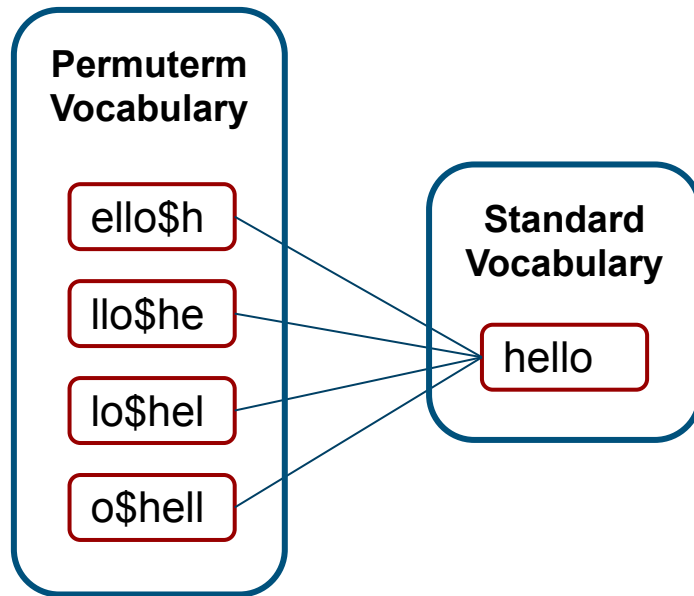
Each term in the permuterm index is **linked to the original vocabulary term**



Permuterm Index

Now let's suppose we are looking for all documents containing $m*n$:

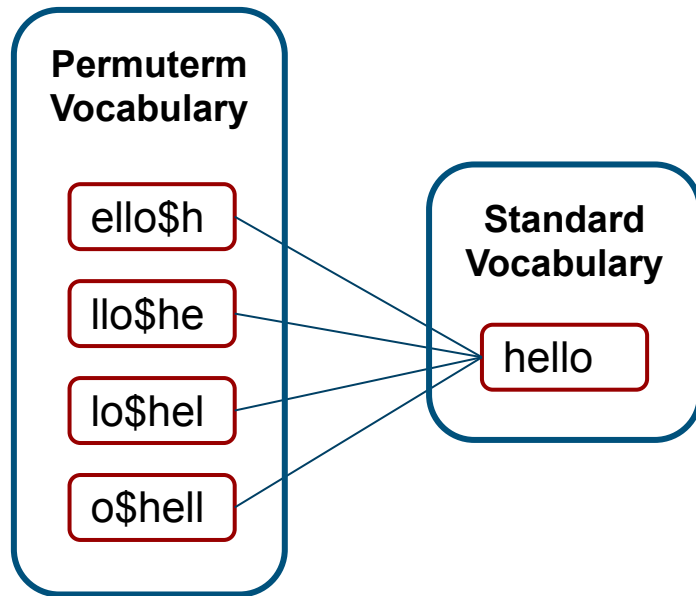
1. we add \$ at the end of the query term: $m*n\$$
2. we rotate $m*n\$$ till * is at the end of the term: $n\$m^*$
3. we search into the permuterm index for all terms beginning with $n\$m$



Permuterm Index

Now let's suppose we are looking for all documents containing $m*n$:

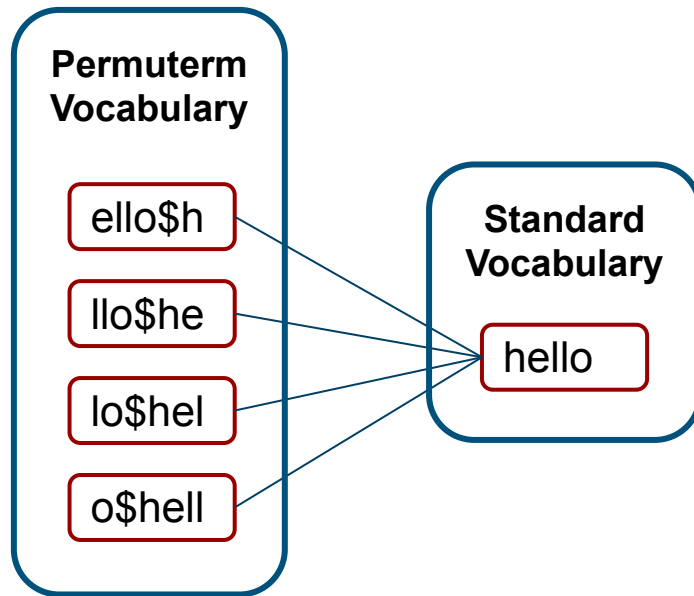
4. we retrieve all permuterms satisfying the constraints
5. by using links, we identify all original terms matching the wildcard (for example, among others, *man* and *moron*)



Permuterm Index

Now let's suppose we are looking for all documents containing $m*n$:

6. we use the original term and by using the standard inverted index, we retrieve all documents containing those terms



Permuterm Index

Ok, this works when $*$ is used in the middle of any term, but what about a query term containing more than one wildcards? Such as $fi*mo*er$?

1. **Filtering Step:** We first look for the query term $fi*er$
 - a. add $\$$ at the end of the query term: $fi*er\$$
 - b. rotate the term till the $*$ is at the end: $er\$fi*$
 - c. search for the permuterm $er\$fi$ into the permuterm index
 - d. retrieve all permuterm satisfying the constraints
 - e. identify all original terms matching the wildcard
2. **Post-filtering step:** We look all original terms to see if they contain mo
 - a. terms not containing mo are discarded

Considerations with Permuterm

- Two indexes are required
 - permuterm index and standard inverted index must both exist
- Permuterm vocabulary is quite large, as it includes all rotations of the term
 - an alternative is the **K-gram index**

K-Gram Index

At indexing time, a **k-gram index** is built.

A **k-gram** is a sequence of k characters.

Starting from a term, we can build all k -grams, after adding \$ at the beginning and at the end of the term

Usually $k = 3$, but it can be whatsoever.

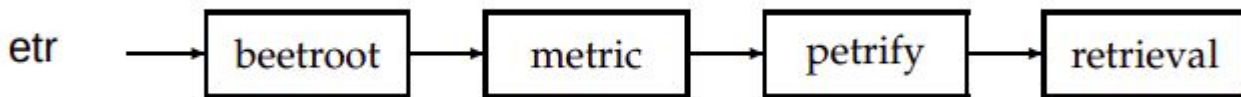
Example:

term	3-grams
● <i>castle</i>	● \$ca
	● cas
	● ast
	● stl
	● tle
	● le\$

K-Gram Index

A k-gram index contains all k-grams that occur in any term in the vocabulary.

The Postings List of each k-gram points not to the document but to the the term containing that k-gram. For example the 3-gram *etr* appears in four different terms



K-Gram Index

Now, suppose that at query time we are looking for the query ***re*ve***.

Instead of looking for *re*ve*, we can look for those documents containing terms starting with *re* and ending with *ve*, that is: ***\$re AND ve\$***

This query is run on the k-gram index, which provides a list of terms (for example *relieve*, *remove*, *retrieve*). Each term is then used in the standard inverted index for retrieving documents

K-Gram Index

A problem of the k-gram index.

Suppose we are looking for *red**, we can inquire our k-gram index with the query:

\$re AND red

BUT even *retired* is matched, although it should not be.

Solution: using a **post-filtering step**, where each term retrieved from the k-gram index is checked against the original query *red**

Only those terms that survive to the post-filtering step are then searched in the standard inverted index

Spelling correction

Misspelling

Often, users make mistake when they write queries

A good Search Engine should be able to identify mistakes and apply a correction

Spelling Errors are divided into **two categories**:

- non-word misspelling: the error leads to a non existing word (e.g. guitar → guittar)
- real-word misspelling: the error leads to an existing word (e.g. their → there)

Misspelling

For non-word misspelling, usually two techniques are used: **Edit Distance** and **K-Gram Overlap**.

Both methods compute distance between the typed query term and vocabulary terms to see which of them is the **nearest one**

Edit Distance

Given two strings s_1 and s_2 , the **edit distance** is the minimum number of edit operations required to transform s_1 into s_2 .

Edit Distance is sometimes known as **Levenshtein Distance**.

For edit operation we mean:

- insert a character
- delete a character
- replace a character

Edit Distance

EDITDISTANCE(s_1, s_2)

```
1  int  $m[i, j] = 0$ 
2  for  $i \leftarrow 1$  to  $|s_1|$ 
3  do  $m[i, 0] = i$ 
4  for  $j \leftarrow 1$  to  $|s_2|$ 
5  do  $m[0, j] = j$ 
6  for  $i \leftarrow 1$  to  $|s_1|$ 
7  do for  $j \leftarrow 1$  to  $|s_2|$ 
8      do  $m[i, j] = \min\{m[i-1, j-1] + \text{if } (s_1[i] = s_2[j]) \text{ then } 0 \text{ else } 1, \text{fi},$ 
9           $m[i-1, j] + 1,$ 
10          $m[i, j-1] + 1\}$ 
11  return  $m[|s_1|, |s_2|]$ 
```

bidimensional matrix $(M+1) \times (N+1)$
($M = \text{length}(s_1)$; $N = \text{length}(s_2)$)

the distance to convert the first i characters
of s_1 into the first j characters of s_2

Edit Distance: an example

		F	A	S	T
C					
A					
T					
S					

We want to compute the distance between *fast* and *cats*.

This is the Levenshtein Matrix

$(M+1) \times (N+1)$ size

Edit Distance: an example

		F	A	S	T
	0	0	0	0	0
C	0	0	0	0	0
A	0	0	0	0	0
T	0	0	0	0	0
S	0	0	0	0	0

Initialization: all elements are set to 0 (line 1)

```
EDITDISTANCE( $s_1, s_2$ )
1  int  $m[i, j] = 0$ 
2  for  $i \leftarrow 1$  to  $|s_1|$ 
3  do  $m[i, 0] = i$ 
4  for  $j \leftarrow 1$  to  $|s_2|$ 
5  do  $m[0, j] = j$ 
6  for  $i \leftarrow 1$  to  $|s_1|$ 
7  do for  $j \leftarrow 1$  to  $|s_2|$ 
8      do  $m[i, j] = \min\{m[i-1, j-1] + \text{if } (s_1[i] = s_2[j]) \text{ then } 0 \text{ else } 1, m[i-1, j] + 1, m[i, j-1] + 1\}$ 
10
11 return  $m[|s_1|, |s_2|]$ 
```


Edit Distance: an example

		F	A	S	T
	0	1	2	3	4
C	1	0	0	0	0
A	2	0	0	0	0
T	3	0	0	0	0
S	4	0	0	0	0

Initialization:

first column from 1 to M

first row from 1 to N

EDITDISTANCE(s_1, s_2)

```
1  int  $m[i, j] = 0$ 
2  for  $i \leftarrow 1$  to  $|s_1|$ 
3  do  $m[i, 0] = i$ 
4  for  $j \leftarrow 1$  to  $|s_2|$ 
5  do  $m[0, j] = j$ 
6  for  $i \leftarrow 1$  to  $|s_1|$ 
7  do for  $j \leftarrow 1$  to  $|s_2|$ 
8     do  $m[i, j] = \min\{m[i-1, j-1] + \text{if } (s_1[i] = s_2[j]) \text{ then } 0 \text{ else } 1, m[i-1, j] + 1, m[i, j-1] + 1\}$ 
9
10 return  $m[|s_1|, |s_2|]$ 
```

Edit Distance: an example

		F	A	S	T
	0	1	2	3	4
C	1	1	0	0	0
A	2	0	0	0	0
T	3	0	0	0	0
S	4	0	0	0	0

Computing distance
(lines 6-10):

We compare C with F. They
are different. So, the value
for $m(1, 1) =$

$$\min(0+1, 1+1, 1+1) = 1$$

```
EDITDISTANCE( $s_1, s_2$ )
1  int  $m[i, j] = 0$ 
2  for  $i \leftarrow 1$  to  $|s_1|$ 
3  do  $m[i, 0] = i$ 
4  for  $j \leftarrow 1$  to  $|s_2|$ 
5  do  $m[0, j] = j$ 
6  for  $i \leftarrow 1$  to  $|s_1|$ 
7  do for  $j \leftarrow 1$  to  $|s_2|$ 
8     do  $m[i, j] = \min\{m[i-1, j-1] + \text{if } (s_1[i] = s_2[j]) \text{ then } 0 \text{ else } 1, m[i-1, j] + 1,$ 
9         $m[i, j-1] + 1\}$ 
10
11 return  $m[|s_1|, |s_2|]$ 
```

Edit Distance: an example

		F	A	S	T
	0	1	2	3	4
C	1	1	2	0	0
A	2	0	0	0	0
T	3	0	0	0	0
S	4	0	0	0	0

Computing distance
(lines 6-10):

We compare C with A. They
are different. So, the value
for $m(1, 2) =$

$$\min(1+1, 2+1, 1+1) = 2$$

```
EDITDISTANCE( $s_1, s_2$ )
1  int  $m[i, j] = 0$ 
2  for  $i \leftarrow 1$  to  $|s_1|$ 
3  do  $m[i, 0] = i$ 
4  for  $j \leftarrow 1$  to  $|s_2|$ 
5  do  $m[0, j] = j$ 
6  for  $i \leftarrow 1$  to  $|s_1|$ 
7  do for  $j \leftarrow 1$  to  $|s_2|$ 
8     do  $m[i, j] = \min\{m[i-1, j-1] + \text{if } (s_1[i] = s_2[j]) \text{ then } 0 \text{ else } 1, m[i-1, j] + 1,$ 
9         $m[i, j-1] + 1\}$ 
10
11 return  $m[|s_1|, |s_2|]$ 
```

Edit Distance: an example

		F	A	S	T
		0	2	3	4
C	1	1	2	3	4
A	2	2	1	0	0
T	3	0	0	0	0
S	4	0	0	0	0

Computing distance
(lines 6-10):

We compare A with A. They **are the same**. So, the value for $m(2, 2) =$

$$\min(1+0, 2+1, 2+1) = 1$$

```
EDITDISTANCE( $s_1, s_2$ )
1  int  $m[i, j] = 0$ 
2  for  $i \leftarrow 1$  to  $|s_1|$ 
3  do  $m[i, 0] = i$ 
4  for  $j \leftarrow 1$  to  $|s_2|$ 
5  do  $m[0, j] = j$ 
6  for  $i \leftarrow 1$  to  $|s_1|$ 
7  do for  $j \leftarrow 1$  to  $|s_2|$ 
8     do  $m[i, j] = \min\{m[i-1, j-1] + \text{if } (s_1[i] = s_2[j]) \text{ then } 0 \text{ else } 1, m[i-1, j] + 1,$ 
9         $m[i, j-1] + 1\}$ 
10
11 return  $m[|s_1|, |s_2|]$ 
```

Edit Distance: an example

		F	A	S	T	
		0	1	2	3	4
C		1	1	2	3	4
A		2	2	1	2	3
T		3	3	2	2	2
S		4	0	0	0	0

Computing distance
(lines 6-10):

We compare T with T. They
are the same. So, the value
for $m(3, 4) =$

$$\min(2+0, 2+1, 3+1) = 2$$

EDITDISTANCE(s_1, s_2)

```
1  int m[i, j] = 0
2  for i ← 1 to |s1|
3  do m[i, 0] = i
4  for j ← 1 to |s2|
5  do m[0, j] = j
6  for i ← 1 to |s1|
7  do for j ← 1 to |s2|
8     do m[i, j] = min{m[i-1, j-1] + if (s1[i] = s2[j]) then 0 else 1,
9                    m[i-1, j] + 1,
10                   m[i, j-1] + 1}
11 return m[|s1|, |s2|]
```

Edit Distance: an example

		F	A	S	T
	0	1	2	3	4
C	1	1	2	3	4
A	2	2	1	2	3
T	3	3	2	2	2
S	4	4	3	2	3

Return distance (line 11):

The returned value is the distance between two words (*fast* and *cats*)

```
EDITDISTANCE( $s_1, s_2$ )
1   $\text{int } m[i, j] = 0$ 
2  for  $i \leftarrow 1$  to  $|s_1|$ 
3  do  $m[i, 0] = i$ 
4  for  $j \leftarrow 1$  to  $|s_2|$ 
5  do  $m[0, j] = j$ 
6  for  $i \leftarrow 1$  to  $|s_1|$ 
7  do for  $j \leftarrow 1$  to  $|s_2|$ 
8     do  $m[i, j] = \min\{m[i-1, j-1] + \text{if } (s_1[i] = s_2[j]) \text{ then } 0 \text{ else } 1, m[i-1, j] + 1, m[i, j-1] + 1\}$ 
9
10
11 return  $m[|s_1|, |s_2|]$ 
```

Edit Distance: another example

		F	R	I	E	D
	0	1	2	3	4	5
F	1	0	1	2	3	4
R	2	1	0	1	2	3
E	3	2	1	1	1	2
S	4	3	2	2	2	2
H	5	4	3	3	3	3

Distance between FRIED and
FRESH: 3

Considerations on Edit Distance

The **complexity** is $O(M*N)$ where $M = |S_1|$ and $N = |S_2|$

The spelling correction problem is more than just computing distance. Indeed having V terms in the Vocabulary, we need to compute the distance between the **query term and each term in the vocabulary**. This takes a lot of time.

Two heuristics are usually used.

Heuristic 1

Statistically, the error **hardly ever occurs at the beginning of the term**. So we can restrict computing distances only with those terms in the vocabulary beginning with the same letters of the query term.

Heuristic 2

Computing distance only with those terms retrieved from a “**refined**” **Permuterm Index**:

1. Let q be the query term
2. Let $r_1, r_2, \dots, r_n$ be all possible rotations of q
3. Let l be an integer, representing the length of the suffix we ignore on each rotation
4. Let $t_1, t_2, \dots, t_n$ be all rotations without the last l characters
5. For each t_i we look for such a term into the Permuterm Index to find all vocabulary terms
6. Computing distance only with those terms

Heuristic 2

Example:

1. we write **recieve** (instead of **receive**)
2. We compute all rotations:

recieve\$
ecieve\$r
cieve\$re
ieve\$rec

eve\$reci
ve\$recie
e\$reciev

Heuristic 2

Example:

3. we set $l = 4$

4. We get all rotations without the last l characters:

reci

ecie

ciev

ieve

eve\$

ve\$r

e\$re

Heuristic 2

5. For each term we look into the Permuterm Index

reci → [appreciated, imprecision, recipe, precise, ...]

eice → [multispecies, species, ...]

ciev → []

ieve → [achieve, believe, relieve, thieves, ...]

eve\$ → [believe, relieve, ...]

ve\$r → [radioactive, rave, receive, reflective, resolve, ...]

e\$re → [realize, realtime, recharge, receive, recipe, ...]

K-Gram Overlap

A second method is named K-Gram Overlap.

This method aims:

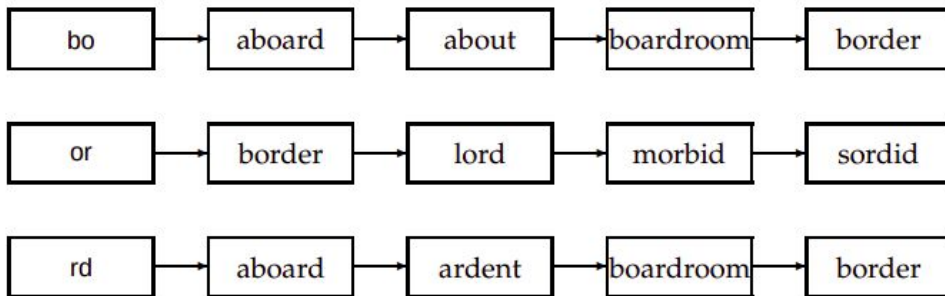
- computing distance between two terms
 - here the distance between two terms describes a measure of **overlapping** (an index to describe how many k-grams are in common)
- reducing the amount of vocabulary terms for which we compute distance

K-Gram Overlap

Suppose we are looking for the misspelled query term *bord*.

Step 1: build a list of k-grams for the misspelled term (suppose $K = 2$ we get: *bo*, *or*, *rd*)

Step 2: by using the K-Gram Index, let's retrieve the posting list for all k-grams



K-Gram Overlap

Step 3: merge all retrieved postings lists in one



Step 4: for each term t in the merged posting list, we compute the distance between t and the query q , by using the **Jaccard Similarity**

Step 5: if the coefficient exceeds a preset threshold, we add t to the output; if not, we move on to the next term in the postings.

Jaccard Similarity Coefficient

It is a statistical index for comparing similarity between two sets.

Let A and B be two sets, the Jaccard Similarity Coefficient can be computed as

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

Jaccard Similarity Coefficient is equal to 1 if A and B are the same, 0 if they are different

Combining Jaccard Similarity with K-Grams

Example 1:

q = Query Term: *bord*

t = Vocabulary Term t : *aboard*

A = Set of 2-Grams for q : $\{bo, or, rd\}$

B = Set of 2-Grams for t : $\{ab, bo, oa, ar, rd\}$

$$J(A, B) = 2/6 = 0.33$$

Combining Jaccard Similarity with K-Grams

Example 2:

q = Query Term: *bord*

t = Vocabulary Term t : *boardroom*

A = Set of 2-Grams for q : $\{bo, or, rd\}$

B = Set of 2-Grams for t : $\{bo, oa, ar, rd, dr, ro, oo, om\}$

$$J(A, B) = 2/9 = 0.22$$

Isolated term correction

Both techniques described so far (**Edit Distance** and **K-gram Overlap**) are real-word misspelling error detectors. They are also named **Isolated Term Correction**.

Suppose for example that we would look for the sentence *fly from Heathrow*, but instead of typing from we type form. This mistake won't be identified by both method being *form* a non misspelled word.

For such a correction, we need to use another type of methods named **context sensitive correction**

Context Sensitive Correction

The idea is to replace each term t in the query q with another term d which is closer to t even though the term t is correct.

For each such substitute phrase, the search engine runs the query and determines the number of matching results.

If the number of documents retrieved using the substitute phrase is larger than the original one, the search engine may like to offer the corrected query.

Spelling Suggestion

When the Search Engine realizes that user might have typed the query term in the wrong way, it should suggest to the user the possible query to be used. For example saying: *“Did you mean carrot instead of carot?”*

Phonetic Correction: Soundex Algorithm

Often, terms are misspelled because of their sound. Users typically type a query that sounds like the target term.

For example *cigarette* vs *sigarette*?

To address this problem, we can build an inverted index using the sound as hash key. This kind of index is named **soundex index**

Phonetic Correction: Soundex Algorithm

The following is a typical algorithm for building the **hash key**:

1. Let t be the term. Let α be the first character of t
2. Replace all characters in t with a digit
 - a. A, E, I, O, U, H, W, Y $\Rightarrow$ 0
 - b. B, F, P, V $\Rightarrow$ 1
 - c. C, G, J, K, Q, S, X, Z $\Rightarrow$ 2
 - d. D, T $\Rightarrow$ 3
 - e. L $\Rightarrow$ 4
 - f. M, N $\Rightarrow$ 5
 - g. R $\Rightarrow$ 6

Phonetic Correction: Soundex Algorithm

3. Repeatedly remove one out of each pair of consecutive identical digits.
4. Remove all zeros from the resulting string.
5. Add α at the beginning of the sequence of digits
6. Return the first 4 characters

For example: Apple and Appoll are both mapped into A14.

Phonetic Correction: Metaphone 3

Metaphone 3 is an algorithm which returns a rough approximation of how an English word sounds. Names and Words that are pronounced similarly will have a similar value. It has been release on 2009

There are many implementation. For java see <https://commons.apache.org/>

Phonetic Correction: Metaphone 3

The Algorithm defines 22 steps:

1. Drop duplicate adjacent letters, except for **c**
2. Drop the first letter if the string begins with **AE** , **GN** , **KN** , **PN** OR **WR**
3. Drop **B** if after **M** at the end of the string
4. **c** transforms into
 - **x** if followed by **IA** OR **H**
 - **s** if followed by **I** , **E** , OR **Y**
 - **k** otherwise
5. **d** transforms into
 - **j** if followed by **GE** , **GY** , OR **GI**
 - **t** otherwise

Phonetic Correction: Metaphone 3

6. Drop G

- if followed by H and H is not at the end or before a vowel
- if followed by N or NED and is at the end of the string

7. G transforms into

- J if before I, E or Y and is not a GG
- K otherwise

8. Drop H

- if after a vowel and not before a vowel
- if after C, S, P, T or G

9. Drop K if after C

10. PH transforms into F

11. Q transforms into K

12. S transforms into X if followed by H, IO or IA

Phonetic Correction: Metaphone 3

13. `T` transforms into `X` if followed by `IA` or `IO`
14. `TH` transforms into `0` (zero)
15. Drop `T` if followed by `CH`
16. `V` transforms into `F`
17. Drop `W` if not followed by a vowel
18. `WH` transforms into `W` if at the beginning of the string
19. `X` transforms into
 - `S` if at the beginning
 - `KS` otherwise
20. Drop `Y` if not followed by a vowel
21. `Z` transforms into `S`
22. Drop all vowels unless it is the beginning character



Index Construction



Academic Year 2025/2026



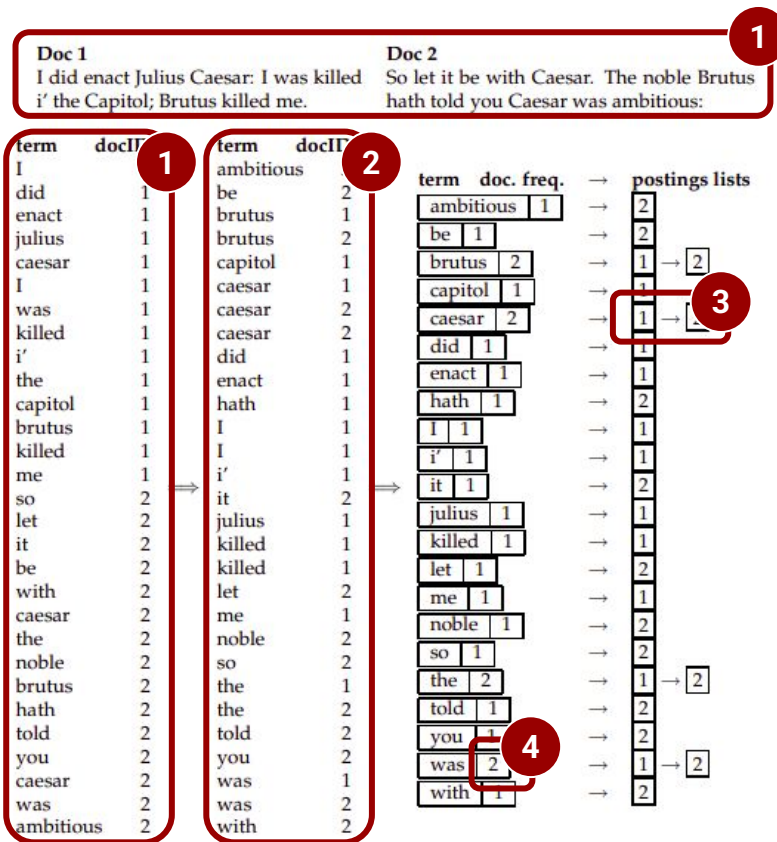
Luigi Santangelo
luigi.santangelo@unipv.it

University of Pavia
**Master Degree in
Computer Engineering**

Building an Index

How already discussed, building an index requires:

1. **Scanning** all documents and assembling all term–docID pairs
2. **Sorting** the pairs with the term as primary key and the DocID as secondary
3. **Organizing** all DocIDs for each term into a Postings List
4. **Computing** statistics like term and document frequency



In-memory vs Disk-saved implementation

A Standard Inverted Index can be built and **kept in memory** only if the number of documents and terms is quite limited.

Nowadays it is easy to have a collection of documents which cannot be kept in memory.

An in-memory indexer **does not scale at all**.

We need to keep data on the disk, but writing and reading data from disk is **very very slow**

Blocked Sort-Based Indexing Algorithm

The solution is to use the **BSBI Algorithm**.

This algorithm aims to reduce the **number of disk seeks** during sorting.

To make the algorithm **more efficient**, instead of using Terms, it assigns each Term to a **TermId**. A TermId and a DocId are represented by an integer (4 bytes), so, for example, having a collection of documents with 100 million of tokens (remind: a token is a sequence of characters semantically grouped together) we need **0.8GB of RAM** to keep all pairs (TermId, DocId) in memory.

Blocked Sort-Based Indexing Algorithm

1. We define a size for a block. We choose the block size to fit comfortably into memory to permit a **fast in-memory sort**
2. Line 4 retrieves the next block to be parsed. **Block** contains all termID-DocID pairs which fit into the memory

BSBINDEXCONSTRUCTION()

```
1   $n \leftarrow 0$ 
2  while (all documents have not been processed)
3  do  $n \leftarrow n + 1$ 
4     $block \leftarrow \text{PARSENEXTBLOCK}()$ 
5    BSBI-INVERT( $block$ )
6    WRITEBLOCKTODISK( $block, f_n$ )
7  MERGEBLOCKS( $f_1, \dots, f_n; f_{\text{merged}}$ )
```

Blocked Sort-Based Indexing Algorithm

3. Line 5 builds the inverted index for the terms in the block. This means:
- Sorting** all termID-DocID (first on the TermID, then on the DocID)
 - Organizing** all DocIDs for each term into a Postings List
 - Computing** statistics like term and document frequency

BSBINDEXCONSTRUCTION()

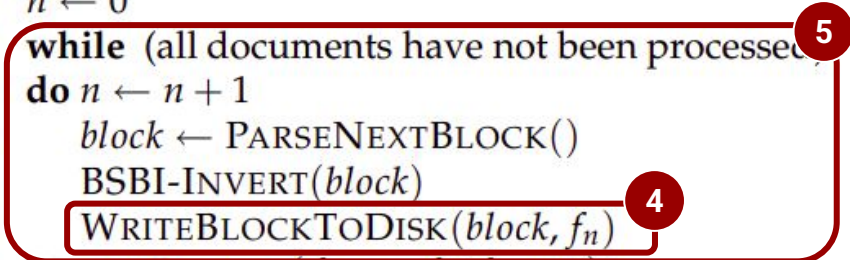
```
1   $n \leftarrow 0$ 
2  while (all documents have not been processed)
3  do  $n \leftarrow n + 1$ 
4      $block \leftarrow \text{PARSENEXTBLOCK}()$ 
5     BSBI-INVERT( $block$ )
6      $\text{WRITEBLOCKTODISK}(block, f_n)$ 
7   $\text{MERGEBLOCKS}(f_1, \dots, f_n; f_{\text{merged}})$ 
```

Blocked Sort-Based Indexing Algorithm

4. Line 6 writes intermediate sorted results on disk on a different file (related to the processed block). fn is a **partial inverted index**.
5. Repeating 4-6 till all documents are not processed

BSBINDEXCONSTRUCTION()

```
1   $n \leftarrow 0$ 
2  while (all documents have not been processed)
3  do  $n \leftarrow n + 1$ 
4       $block \leftarrow \text{PARSENEXTBLOCK}()$ 
5       $\text{BSBI-INVERT}(block)$ 
6       $\text{WRITEBLOCKTODISK}(block, f_n)$ 
7   $\text{MERGEBLOCKS}(f_1, \dots, f_n; f_{\text{merged}})$ 
```



Blocked Sort-Based Indexing Algorithm

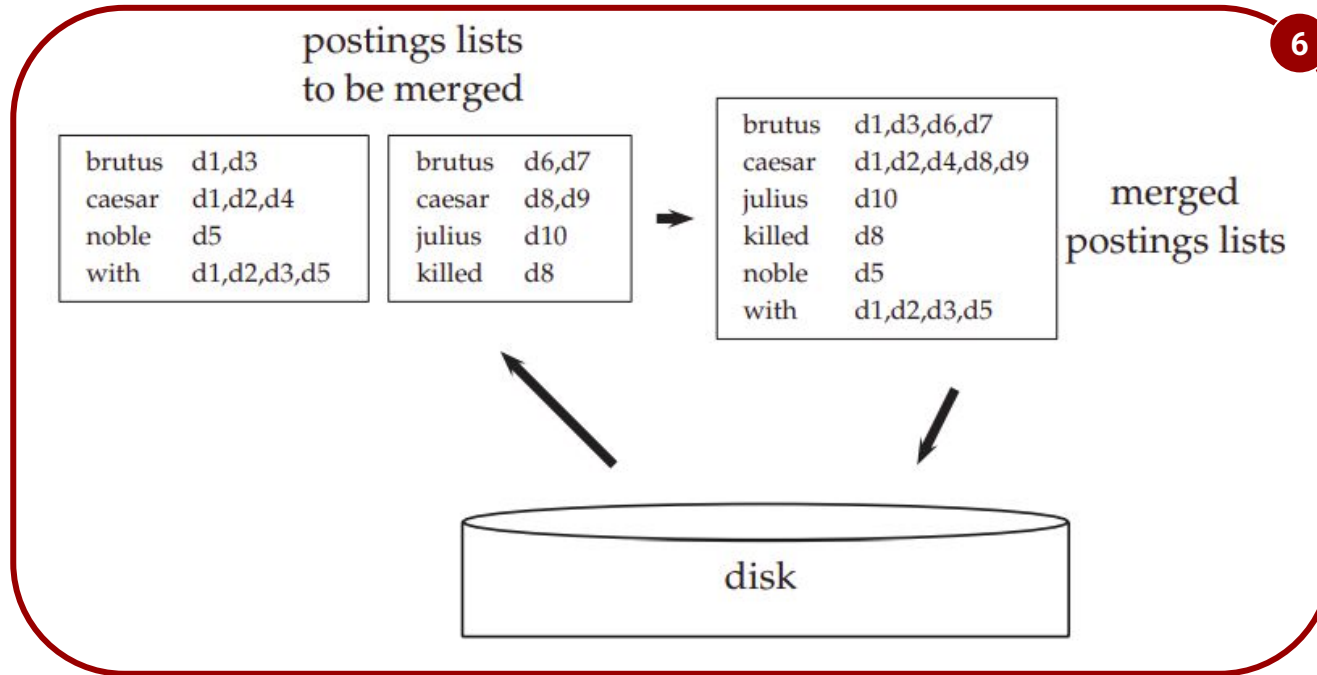
6. Line 7 reads all n written files (representing the partial inverted index) and merge results all together into the final inverted index (saved back to the disk)

BSBINDEXCONSTRUCTION()

```
1   $n \leftarrow 0$ 
2  while (all documents have not been processed)
3  do  $n \leftarrow n + 1$ 
4       $block \leftarrow \text{PARSENEXTBLOCK}()$ 
5       $\text{BSBI-INVERT}(block)$ 
6       $\text{WRITEBLOCKTODISK}(block, f_n)$ 
7   $\text{MERGEBLOCKS}(f_1, \dots, f_n; f_{\text{merged}})$ 
```

6

Blocked Sort-Based Indexing Algorithm



Blocked Sort-Based Indexing Algorithm

Let's have a deep dive into the **Merging Operation**. How we can do this operation in an efficient way?

1. We **open all n files** containing a partial inverted index (remind that all terms are already sorted)
2. We **use a pointer** to the next Term not processed yet (n pointers for n files)
3. In each iteration, we select the lowest Term that has not been processed yet
4. All postings lists for this term are read and merged
5. The merged list is written back to disk

Complexity of BSBI Algorithm

Let T be the **number of tokens**

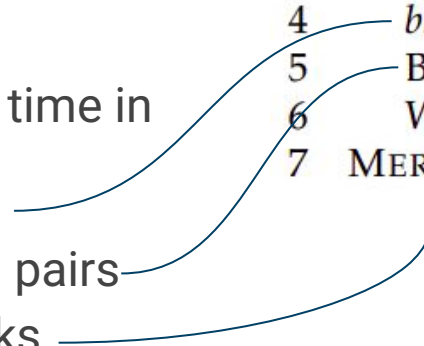
The complexity time for the algorithm is **$O(T \log T)$**

The algorithm spends more time in

- parsing the documents
- sorting (TermID, DocID) pairs
- merging all partial blocks

BSBIINDEXCONSTRUCTION()

```
1   $n \leftarrow 0$ 
2  while (all documents have not been processed)
3  do  $n \leftarrow n + 1$ 
4     $block \leftarrow \text{PARSENEXTBLOCK}()$ 
5     $\text{BSBI-INVERT}(block)$ 
6     $\text{WRITEBLOCKTODISK}(block, f_n)$ 
7     $\text{MERGEBLOCKS}(f_1, \dots, f_n; f_{\text{merged}})$ 
```



Drawbacks of the BSBI Algorithm

- Each pair makes use of TermID. This means that we need a data structure for mapping Terms into TermIDs. For very large collections, this data structure **does not fit into memory**.
- It requires **sorting** all (TermID, DocID) pairs

Single-pass in-memory indexing

The SPIMI algorithm is more efficient than the Blocked Sort-Based Indexing Algorithm because:

- It uses Terms instead of TermID, so it does not need any data structure for mappings
- It does not require sorting (Term, DocID) pairs (which is done by BSBI-Invert function in BSBI Algorithm)
 - Sorting operation is postponed and applied on the dictionary

Single-pass in-memory indexing

SPIMI-INVERT (*token_stream*)

```
1  output_file = NEWFILE()
2  dictionary = NEWHASH()
3  while (free memory available)
4  do token ← next(token_stream)
5      if term(token) ∉ dictionary
6          then postings_list = ADDTODICTIONARY(dictionary, term(token))
7          else postings_list = GETPOSTINGSLIST(dictionary, term(token))
8      if full(postings_list)
9          then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
10     ADDTOPOSTINGSLIST(postings_list, docID(token))
11 sorted_terms ← SORTTERMS(dictionary)
12 WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13 return output_file
```

- *token_stream*: a collection of (Token, DocID) pairs
- *dictionary* is an Hash Table

Single-pass in-memory indexing

SPIMI-INVERT(*token_stream*)

```
1  output_file = NEWFILE()
2  dictionary = NEWHASH()
3  while (free memory available)
4  do token ← next(token_stream)
5      if term(token) ∉ dictionary
6          then postings_list = ADDTODICTIONARY(dictionary, term(token))
7      else postings_list = GETPOSTINGSLIST(dictionary, term(token))
8      if full(postings_list)
9          then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
10     ADDTOPOSTINGSLIST(postings_list, docID(token))
11 sorted_terms ← SORTTERMS(dictionary)
12 WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13 return output_file
```

- Tokens are processed one by one
- When a term occurs for the first time, it is added to the dictionary. The corresponding (empty) Postings List is returned

Single-pass in-memory indexing

SPIMI-INVERT(*token_stream*)

```
1  output_file = NEWFILE()
2  dictionary = NEWHASH()
3  while (free memory available)
4  do token ← next(token_stream)
5      if term(token) ∉ dictionary
6          then postings_list = ADDTODICTIONARY(dictionary, term(token))
7          else postings_list = GETPOSTINGSLIST(dictionary, term(token))
8      if full(postings_list)
9          then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
10     ADDTOPOSTINGSLIST(postings_list, docID(token))
11 sorted_terms ← SORTTERMS(dictionary)
12 WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13 return output_file
```

- If the term is already in the dictionary, the corresponding Postings List is returned
- Using Hash Table, managing Dictionary is very efficient

Single-pass in-memory indexing

SPIMI-INVERT(*token_stream*)

```
1  output_file = NEWFILE()
2  dictionary = NEWHASH()
3  while (free memory available)
4  do token ← next(token_stream)
5      if term(token) ∉ dictionary
6          then postings_list = ADDTODICTIONARY(dictionary, term(token))
7          else postings_list = GETPOSTINGSLIST(dictionary, term(token))
8          if full(postings_list)
9              then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
10         ADDTOPOSTINGSLIST(postings_list, docID(token))
11 sorted_terms ← SORTTERMS(dictionary)
12 WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13 return output_file
```

- The Posting List is managed like a List having predefined size. This instruction verifies if the List is full (all available seats for terms are busy). If so, the size of the is doubled.

Single-pass in-memory indexing

SPIMI-INVERT(*token_stream*)

```
1  output_file = NEWFILE()
2  dictionary = NEWHASH()
3  while (free memory available)
4  do token ← next(token_stream)
5      if term(token) ∉ dictionary
6          then postings_list = ADDTODICTIONARY(dictionary, term(token))
7          else postings_list = GETPOSTINGSLIST(dictionary, term(token))
8      if full(postings_list)
9          then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
10     ADDTOPOSTINGSLIST(postings_list, docID(token))
11 sorted_terms ← SORTTERMS(dictionary)
12 WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13 return output_file
```

The Document ID is added to the Postings List.

Single-pass in-memory indexing

SPIMI-INVERT(*token_stream*)

```
1  output_file = NEWFILE()
2  dictionary = NEWHASH()
3  while (free memory available)
4  do token ← next(token_stream)
5      if term(token) ∉ dictionary
6          then postings_list = ADDTODICTIONARY(dictionary, term(token))
7          else postings_list = GETPOSTINGSLIST(dictionary, term(token))
8          if full(postings_list)
9              then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
10         ADDTOPOSTINGSLIST(postings_list, docID(token))
11 sorted_terms ← SORTTERMS(dictionary)
12 WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13 return output_file
```

Processing Terms continues until we still have memory available (Posting List and Dictionary grow and they can fill all memory)

Single-pass in-memory indexing

SPIMI-INVERT(*token_stream*)

```
1  output_file = NEWFILE()
2  dictionary = NEWHASH()
3  while (free memory available)
4  do token ← next(token_stream)
5      if term(token) ∉ dictionary
6          then postings_list = ADDTODICTIONARY(dictionary, term(token))
7          else postings_list = GETPOSTINGSLIST(dictionary, term(token))
8      if full(postings_list)
9          then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
10     ADDTOPOSTINGSLIST(postings_list, docID(token))
11     sorted_terms ← SORTTERMS(dictionary)
12     WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13 return output_file
```

When memory is exhausted 1) we sort the terms in the dictionary (for the final merging operation of all blocks) and 2) write the block on the disk

Distributed Indexing

When the number of documents to process is very high, search engines use **distributed indexing algorithms** for index construction.

The result of the construction process is a distributed index that is partitioned **across several machines**.

Distributed Indexing

Building a distributed index requires a **cluster of machines** cooperating all together.

The job is **split into chunks** and **distributed** among all **workers**

A **master** directs the process of assigning **task**

Distributed Indexing

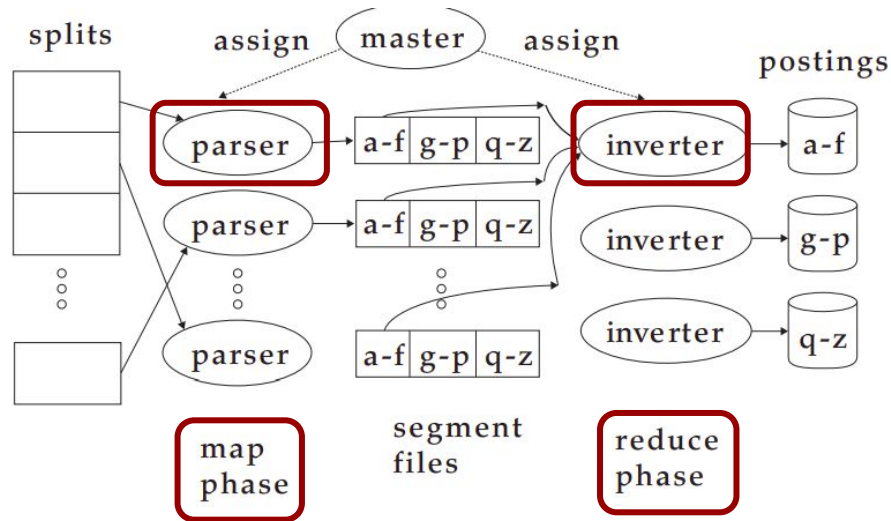
- The master process splits collection of documents into n **chunks**
- Each chunk should not be **neither too small nor too large** (16MB or 64MB are good sizes in distributed indexing)
- Each chunk is sent to a worker. Chunks are **not preassigned**
- As soon as a process finishes processing one chunk, the master assigns the next one to it
- If a machine **dies due to hardware problems**, the chunk it was working on is simply reassigned to another machine.
- The strategy adopted by the workers is based on the **MapReduce Algorithm**

Distributed Indexing

MapReduce: it is a programming strategy for computing huge amount of data in parallel

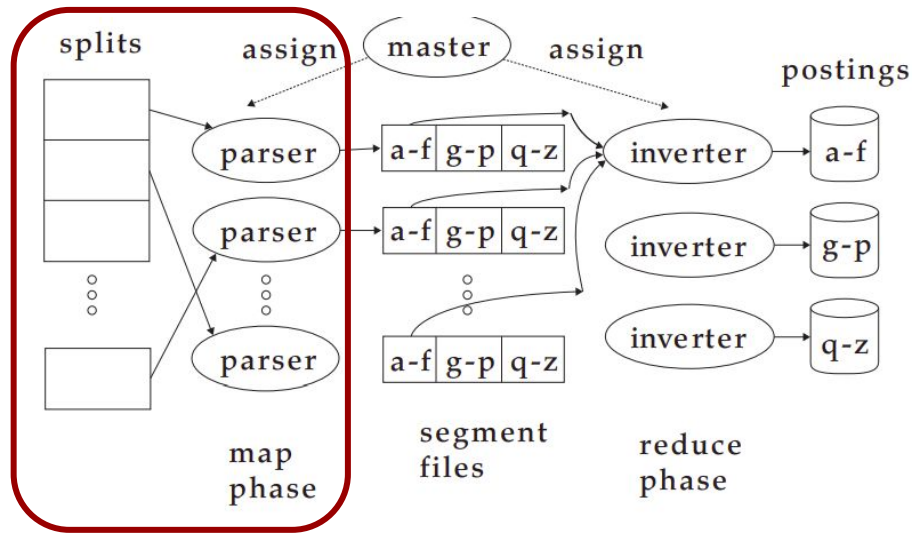
Programmers never write a MapReduce application. They just write two functions: **map and reduce functions**

Workers executing map/reduce phase are usually called **parser/inverter**



Distributed Indexing

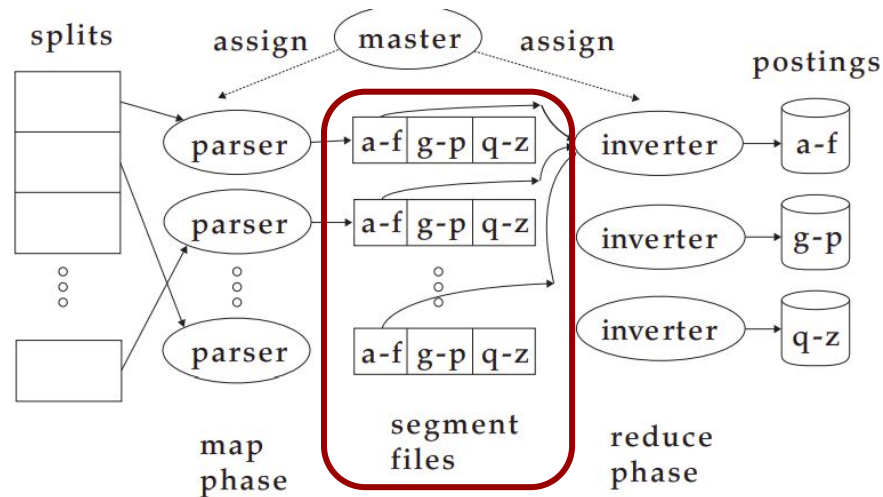
Map phase: operates on the input data (the chunk of web pages) and generates **key-value pairs** having the form of (TermID-DocID). Of course all workers must share a consistent mapping of Term and TermID



Distributed Indexing

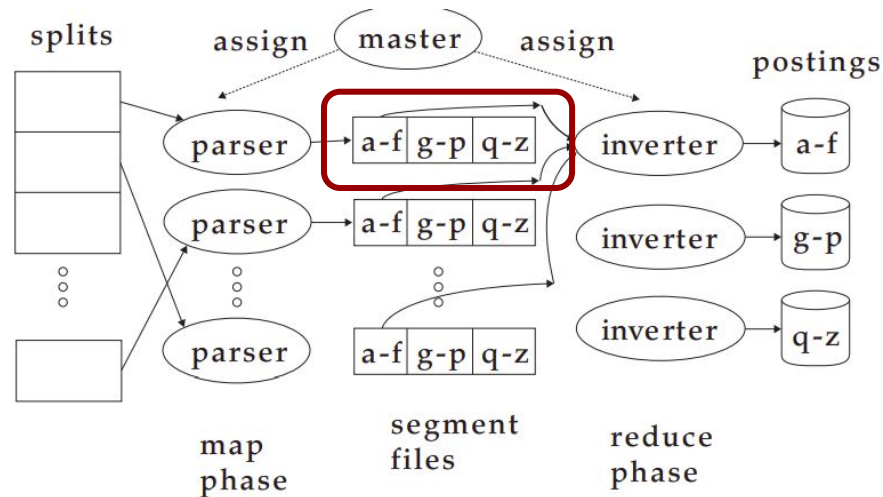
Map phase: the parser starts processing the chunk received by the master. It creates pairs of **key-value**, and then stores them into local intermediate file called **segment files**.

Each segment file is a partition of key-value pairs



Distributed Indexing

Map phase: we want all values for a given key to be stored close together, such that for a given key we can have the list of all documents containing that key. To do so in parallel, all the keys **are partitioned** in j parts (in the pictures all keys have been partitioned in 3 parts according to the first letter (a-f), (g-p) and (q-z), but there might be several different ways to partition all keys

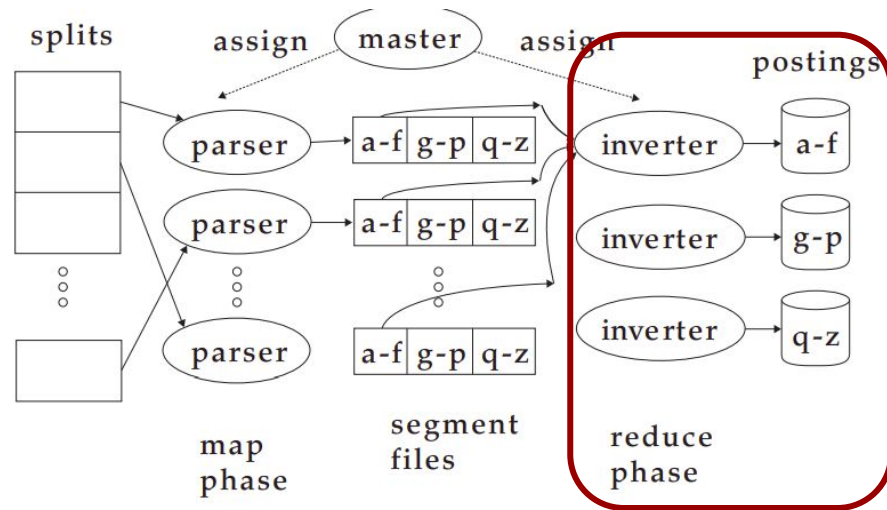


Distributed Indexing

Reduce phase: collecting all values for a given key into one list is the task of the **inverter**. The master assigns each term partition to a different inverter.

In case of **failure**, the task can be reassigned to another inverter.

Finally, the list of values is sorted for each key and written to the final sorted postings list



Term-partitioned Distributed Indexing

Parsers and Inverters are not separate sets of machines.

The master identifies **idle machines** and assigns tasks to them.

The same machine **can be both parser and inverter** during map phase and reduce phase.

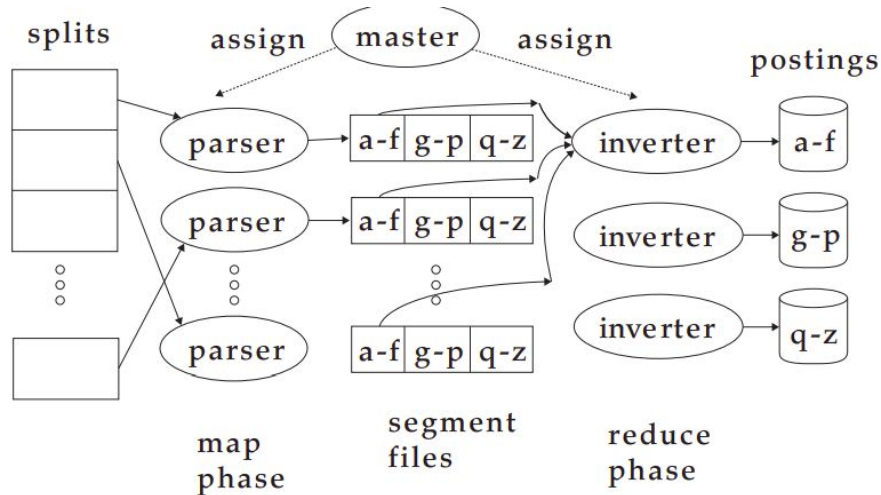
The distributed index described so far is named **term-partitioned distributed index**

Problems using distributed indexes

For example if I'm looking for "apple"
I'll send the search only to the node
containing the posting list [A-F]

But there are two big problems that we
need to face: **Multi-term queries**

A possible solution would be to **share
postings lists** between sets of nodes
for merging, but this can be very
**expensive in terms of Network Traffic
and Computation**



Document-partitioned Distributed Indexing

Why not to use a document-partitioned distributed index?

- Each node contains the index for a subset of all documents.
- Each query is distributed to all nodes.
- The results from various nodes are merged before presentation to the user.

This approach makes **global statistics** used in scoring harder to be computed, because they should be computed across the entire document collection (but each node contains only a small subset of all documents).

For such a kind of global statistics, **a distributed background process** is usually used. The process refreshes periodically the node indexes with fresh global statistics

Document Partitioning vs Term Partitioning

(a) Document partitioning

		Documents								
		D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇	D ₈	D ₉
Terms	T ₁	X		X	X		X			X
	T ₂		X			X				
	T ₃		X	X					X	
	T ₄				X			X		
	T ₅	X					X			X
	T ₆	X						X	X	
	T ₇		X		X		X			
	T ₈			X					X	
		Node 1			Node 2			Node 3		

(b) Term partitioning

		Documents								
		D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇	D ₈	D ₉
Terms	T ₁	X		X	X		X			X
	T ₂		X			X				
	T ₃		X	X					X	
	T ₄				X			X		
	T ₅	X					X			X
	T ₆	X						X	X	
	T ₇		X		X		X			
	T ₈			X					X	
		Node 1			Node 2			Node 3		

Dynamic Indexing

Document collection can **change over the time**. New documents can be added, existing documents can be updated or modified.

An easy way to manage changes in the Collection is to build a **new index** while the old one is available only for queries until the new is ready.

Of course, building a new index can be **time consuming** and requires computational resources.

Dynamic Indexing

If we need that the document is included quickly in the index, we can maintain **two indexes**:

1. the **main index** (quite large)
2. an **auxiliary index** that stores new or updated documents. This, being **small**, is kept in memory

Searches are run on both indexes and **results are merged**.

Document deletions are managed by using an **invalidation bit vector**. Results related to **deleted documents are filtered**

Dynamic Indexing

When the auxiliary index is too large, it **is merged** with the main index and then stored into the disk.

Merging the auxiliary index with the main index means **merging posting lists**.

Merging can be time consuming, depending on the number of files and their size used to store the main index.

Logarithmic Merging

An efficient algorithm for merging indexes is the **Logarithmic Merging**.

Let T be the number of total postings

Let n be the maximum number of postings to be stored into the auxiliary index.

The idea behind the Logarithmic Merging Algorithm is to create $k = \log_2(T/n)$ indexes, named $I_0, I_1, I_2, \dots, I_{k-1}$ having size (number of postings to be stored) respectively $2^0n, 2^1n, 2^2n, \dots, 2^{k-1}n$ which are **permanently stored** into disk, and an **auxiliary index** Z_0 having size (number of posting to be stored) equals to n . The auxiliary index is kept **in memory**.

Logarithmic Merging

Note that

$$\begin{aligned} 2^0 n + 2^1 n + 2^2 n + \dots + 2^{k-1} n &= \sum_{i=0}^{k-1} 2^i n \\ &= \frac{n(2^k - 1)}{2 - 1} = n(2^k - 1) \\ &= n(2^{\log_2 \frac{T}{n}} - 1) = n\left(\frac{T}{n} - 1\right) = T - n \end{aligned}$$

This means that all persistent indexes are able to store only $T - n$ postings, because the remainder n are kept into the in-memory index

Logarithmic Merging

The Logarithmic Merging works as follows:

- at the beginning, the first n postings are accumulated into Z_0 (in-memory auxiliary index)
- When the limit n is reached, all postings in Z_0 are transferred to a new persistent I_0 . Z_0 is then set empty.
- Z_0 is then filled again and when the limit is reached again, Z_0 should be transferred to I_0 but this persistent index is not empty so Z_0 and I_0 are merged together. The result is temporarily stored into Z_1 and then moved into I_1 . The index I_0 is then set empty.

Logarithmic Merging

First Few Steps

n	$2^0 \times n$	$2^1 \times n$	$2^2 \times n$	$2^3 \times n$	$\dots$
Z_0 $Z_0 \rightarrow l_0$					
	l_0				
Z_0	l_0 $Z_0 \cup l_0 \rightarrow Z_1 \rightarrow l_1$				
		l_1			
Z_0 $Z_0 \rightarrow l_0$		l_1			
	l_0	l_1			
Z_0	l_0 $Z_0 \cup l_0 \rightarrow Z_1$	l_1			
		l_1 $Z_1 \cup l_1 \rightarrow Z_2 \rightarrow l_2$			
			l_2		

Logarithmic Merging

LOGARITHMICMERGE()

1 $Z_0 \leftarrow \emptyset$ (Z_0 is the in-memory index.)

2 $indexes \leftarrow \emptyset$

3 **while** true

4 **do** LMERGEADDTOKEN($indexes, Z_0, \text{GETNEXTTOKEN}()$)

- Auxiliary Index Z_0 is empty
- The set of all persistent indexes is empty

Logarithmic Merging

One token at a time is added to the auxiliary index

```
LOGARITHMICMERGE()  
1   $Z_0 \leftarrow \emptyset$   ( $Z_0$  is the in-memory index.)  
2   $indexes \leftarrow \emptyset$   
3  while true  
4  do LMERGEADDTOKEN( $indexes, Z_0, \text{GETNEXTTOKEN}()$ )
```



Logarithmic Merging

LMERGEADDTOKEN(*indexes*, Z_0 , *token*)

```
1   $Z_0 \leftarrow \text{MERGE}(Z_0, \{token\})$ 
2  if  $|Z_0| = n$ 
3      then for  $i \leftarrow 0$  to  $\infty$ 
4          do if  $I_i \in \text{indexes}$ 
5              then  $Z_{i+1} \leftarrow \text{MERGE}(I_i, Z_i)$ 
6                  ( $Z_{i+1}$  is a temporary index on disk.)
7                   $\text{indexes} \leftarrow \text{indexes} - \{I_i\}$ 
8              else  $I_i \leftarrow Z_i$  ( $Z_i$  becomes the permanent index  $I_i$ .)
9                   $\text{indexes} \leftarrow \text{indexes} \cup \{I_i\}$ 
10                 BREAK
11      $Z_0 \leftarrow \emptyset$ 
```

The new token is added to the auxiliary index

Logarithmic Merging

Temporary and persistent indexes are created

```
LMERGEADDTOKEN(indexes,  $Z_0$ , token)
1   $Z_0 \leftarrow \text{MERGE}(Z_0, \{\text{token}\})$ 
2  if  $|Z_0| = n$ 
3    then for  $i \leftarrow 0$  to  $\infty$ 
4      do if  $I_i \in \text{indexes}$ 
5        then  $Z_{i+1} \leftarrow \text{MERGE}(I_i, Z_i)$ 
6          ( $Z_{i+1}$  is a temporary index on disk.)
7           $\text{indexes} \leftarrow \text{indexes} - \{I_i\}$ 
8        else  $I_i \leftarrow Z_i$  ( $Z_i$  becomes the permanent index  $I_i$ .)
9           $\text{indexes} \leftarrow \text{indexes} \cup \{I_i\}$ 
10         BREAK
11    $Z_0 \leftarrow \emptyset$ 
```


Logarithmic Merging

- Index Construction Time = $O(T \log(T/n))$
- Logarithmic Merging is more efficient in merging but
 - **query processing** now requires the merging of $O(\log T/n)$ indexes, whereas it is $O(1)$ if we just have a main and auxiliary index

Index Compression

Academic Year 2025/2026

Luigi Santangelo
luigi.santangelo@unipv.it

University of Pavia
**Master Degree in
Computer Engineering**

Compression

In our discussion so far, we have been talking about vocabulary and postings lists but all of them were **not compressed**.

Tradeoffs of compression:

- less disk **space**
- faster transfer data from disk to **memory**
- being the data compressed, IR systems can benefit from **caching**
- decompression introduces a **penalty** during queries
 - but there are many efficient compression methods so the **penalty is small**

Reducing the index size using statistics

Many researches have demonstrated that some **preprocessing activities** can reduce the number of terms and postings.

- Stemming and case-folding reduce the number of terms by 17%
- Stemming and case-folding reduce the number of postings by 3% to 4%
- Rule of 30: 30 most common words account for 30% of the tokens in written text
 - Considering the first 150 most common words as **Stop Words** cuts 25% to 30% of Postings

Lossy and Lossless Compression

Stemming, Case Folding and Stop Word are forms of **lossy compression**.

There are many other lossy techniques that, under some conditions, can be used for compression without any reduction in effectiveness (**lossless compression**).

Estimating the number of terms

Before describing compression methods, it is important to have an estimate of the number of terms.

Oxford English Dictionary (OED) defines other than 600.000 words

But the vocabulary of most **collections is much larger than OED** (OED does not include most names of people, locations, products, or scientific entities like genes. All of these must be included in an inverted index).

We can estimate the number of terms by using the **Heaps Law**

Estimating the number of terms: the Heaps Law

Let T be the number of tokens in the collection

Let k be a parameter ranging from 30 to 100

Let b be another parameter about having value about 0.50

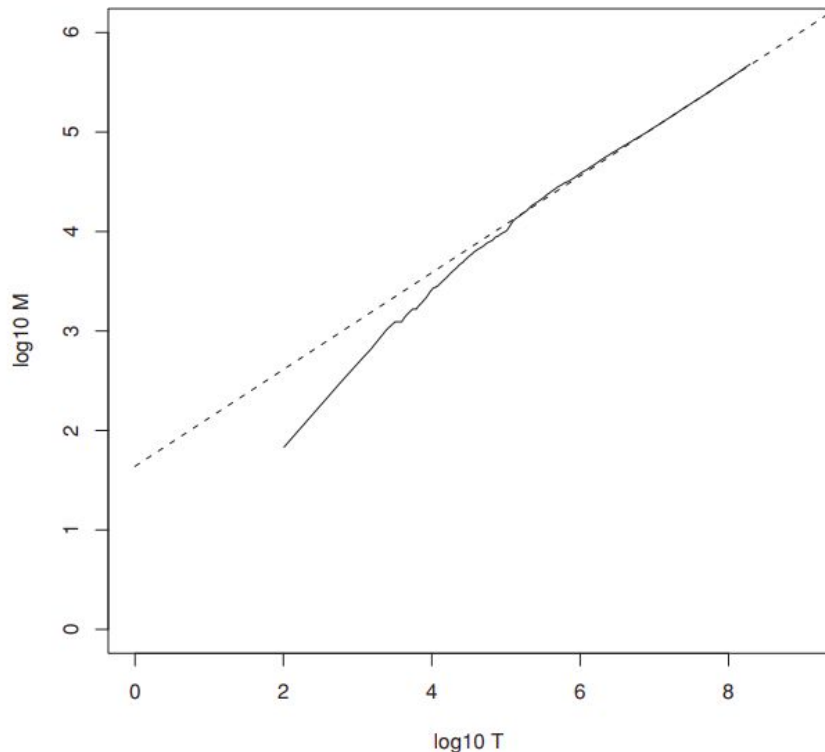
We can compute the size of the vocabulary M using the Heaps Law:

$$M = kT^b$$

Estimating the number of terms: the Heaps Law

The Heaps Law comes from the observation that the relationship between Token size T and Vocabulary size M is **linear in log-log space**.

The picture shows the relationship between M and T using a real document collection, using $k = 44$ and $b = 0.49$



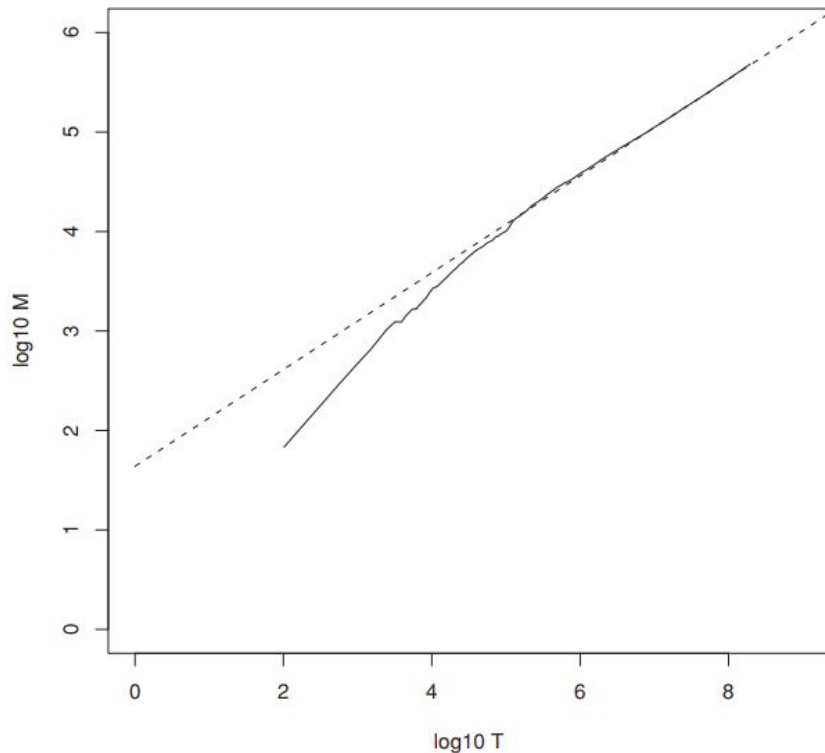
Estimating the number of terms: the Heaps Law

After processing about one million tokens, the Heaps Law predicted 38,323 of terms, whilst the real number of terms was **38,365**, which is very close to the predicted one.

$$M = kT^b$$

$$M = 44 * T^{0.49}$$

$$M = 44 * 10^{6^{0.49}} \approx 38,323$$



Estimating the number of terms: the Heaps Law

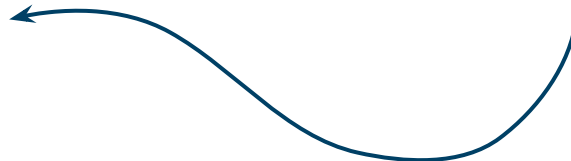
Collection for Reuters-RCV1*	
Documents	806,791
Number of Tokens	96,969,056
Tokens per documents	222
Distinct Terms	391,523
Average bytes per Term	7.5

$$T = 96,969,056$$

$$k = 50$$

$$b = 0.4875$$

$$M = kT^b = 391,249$$



* <https://archive.ics.uci.edu/dataset/259/reuters+rcv1+rcv2+multilingual+multiview+text+categorization+test+collection>

Estimating the number of terms: the Heaps Law

In conclusion, Heaps Law suggests:

1. the dictionary size continues to increase with more documents in the collection (**no maximum size is reached**)
2. the size of the dictionary is quite large for large collections

This highlights **the importance of compressed dictionary** for an effective information retrieval system

Estimate the distribution of terms

Another interesting information we need to know to build an effective algorithm for compressing postings is the **distribution of terms**.

For this purpose there is another law: the **Zipf's law**, representing a commonly used model of the distribution of terms in a collection

Estimate the distribution of terms: Zipf's law

Zipf's law states that if t_1 is the most common term, t_2 the next most common term and so on, then the collection frequency of the i -th most common term (cf_i) is

$$cf_i \propto \frac{1}{i}$$

Estimate the distribution of terms: Zipf's law

Said in other words:

The first most common word appears twice as often as the second most common word, three times as often as the third and so on.

The intuition behind this law is that **frequency decreases very rapidly with rank**.

Estimate the distribution of terms: Zipf's law

For example, consider this text:

The purpose of this paper is to investigate the applicability of Zipf's law to English text. Zipf's law states that the frequency of a word in a text is inversely proportional to its rank. In other words, the most frequent word will appear twice as often as the second most frequent word, three times as the third most frequent word, and so on.

the: 6 occurrences

of: 3 occurrences

is: 2 occurrences

Index size and estimation

As discussed, Indexes are **crucial** for a Search Engine

The larger the index, the **more complete** is the search for a user

Of course, it is not possible to know the fraction of the web indexed by a Search Engine, because there are many **dynamic web pages** making the content changed very time.

Anyway, some techniques have been defined **to estimate the ratio** of the index sizes of two Search Engines E_1 and E_2

Index size and estimation

The basic hypothesis underlying these techniques is that each Search Engine indexes a **finite subset** of the Web chosen **independently and uniformly at random**.

With this assumption, we can invoke the classical estimation technique known as the **capture-recapture method**.

This method is not new in computer science, as it is usually used for estimating population.

Capture-Recapture Method

1. Let E_1 and E_2 two Search Engines
2. Pick a page from E_1 's index (**picking phase**) and check if the link is also in E_2 's index (**checking phase**)
3. Symmetrically, pick a page from E_2 's index and test if the link is also in E_1 's index
4. Repeat steps 2 and 3 many times
5. Let x the fraction of pages in E_1 that are also in E_2 , and y the fraction of pages in E_2 that are also in E_1

Capture-Recapture Method

With these assumptions, the Capture-Recapture Method says that

$$x |E_1| \approx y |E_2|$$

$$\frac{|E_1|}{|E_2|} \approx \frac{y}{x}$$

where $|E_1|$ and $|E_2|$ are the size of the index respectively in E_1 and E_2

Page Similarity

Page Duplication

Many contents in the web are duplicated. According to some research, **30-40% of contents are duplicated.**

Many of these duplications are **legitimate copies** (pages are mirrored to provide **redundancy** and **availability**)

Some of these duplications are not legitimate copies (**plagiarism**)

Page Duplication

Having to manage duplicate pages is not convenient for Search Engines:

- More pages means larger index
- More duplicate pages means weak results (results will show different pages having the same content)

Where similarity detection is used

- Search Engine should avoid indexing and showing multiple copies of the same content (for performance and storage reasons)
- Shop Online (i.e. Amazon) can consider two customers are similar if their sets of purchased items is similar
- Moving Ratings (i.e. Netflix): two movies are considered similar if they are rated highly by many of the same customers

Detect page duplication

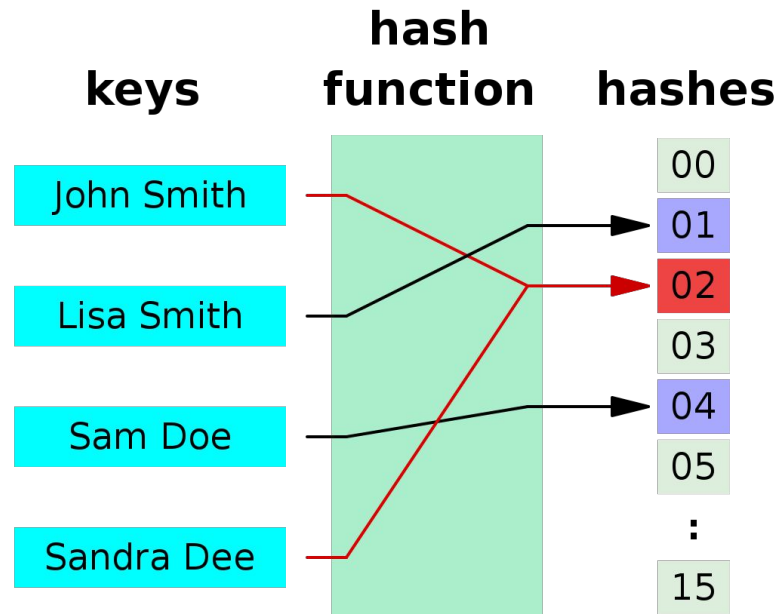
An easy way of identifying duplicated pages is to use a **fingerprint**: little variation on the page brings large variation in the fingerprint

If two pages have the same fingerprint, they are likely to be identical (**near duplication**)

How Fingerprinting works

Fingerprinting is based on Hash Functions.

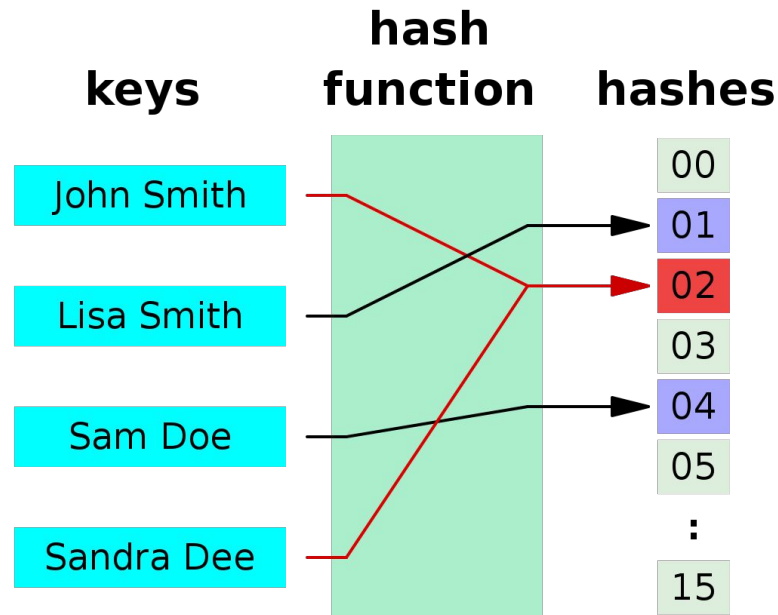
Hash Functions are able to map data of arbitrary size (named key) to a fixed-size value (named hash value or digest).



How Fingerprinting works

Being the hash valued limited in size, there might be **collisions** (two different keys produce the same hash value).

A good Hash Function distributes the possible hash-key values **approximately evenly** among buckets, to reduce the number of collision



How Fingerprinting works

Typical Hash Functions are:

- SHA (Secure Hash Algorithm)
- CRC (Cyclic Redundancy Check)
- MD5 (Message Digest 5)
- MurmurHash
- BLAKE2
- Argon2

```
$ md5sum file.pdf  
eb9d2993f2c27224ffc4e44d95dd6918  file.pdf
```

Detect page duplication

This simplistic approach does not work fine when even just the publishing date and time is updated.

The solution is a techniques known as **shingling**

Detect page duplication

Shingling works on **character-level similarity**

Not “similar meaning”

Shingling

- Let k be a positive integer and d a web document
- Let define k -shingle of d be the set of all consecutive sequences of k characters in d

An example

d = "the lazy dog jumped over the quick brown fox"

k = 4

k -shingle: "the ", "he l", "e la", "laz ", "lazy", "azy ", "zy d", ...

Shingling

When two documents can be considered as similar?

Two documents are near duplicate if the sets of shingles generated from them are nearly the same

Before giving a more formal description of the Shingling, let's introduce the **Jaccard Similarity Coefficient**

Jaccard Similarity Coefficient

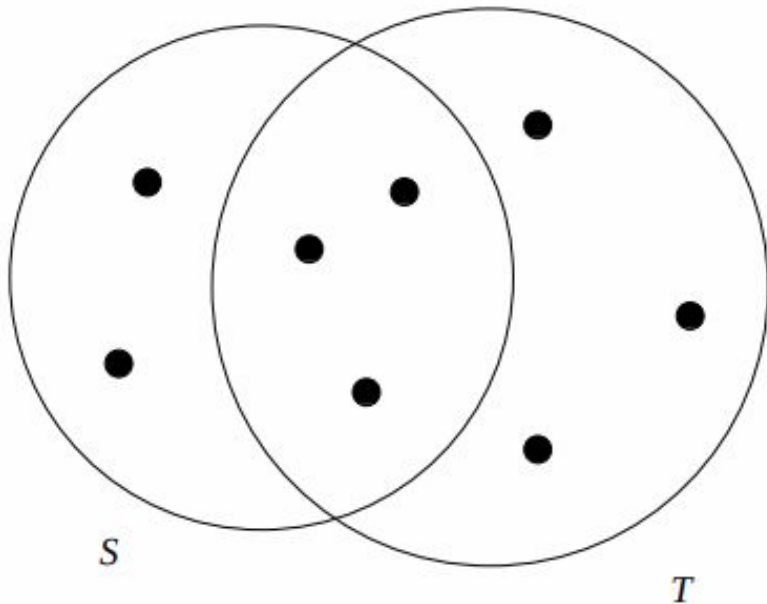
It is a statistical index for comparing similarity between two sets.

Let A and B be two sets, the Jaccard Similarity Coefficient can be computed as

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

Jaccard Similarity Coefficient is equal to 1 if A and B are the same, 0 if they are different

Jaccard Similarity Coefficient



- Two sets: S and T
- $|S| = 5$
- $|T| = 6$
- $|S \cup T| = 8$
- $|S \cap T| = 3$
- $J(S, T) = 3/8$

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

Shingling

- Let d_1 and d_2 be two documents
- Let $S(d_1)$ and $S(d_2)$ be the *k-shingle* sets for both documents d_1 and d_2
- The Jaccard Coefficient of both sets can be computed as

$$J(S(d_1), S(d_2)) = \frac{|S(d_1) \cap S(d_2)|}{|S(d_1) \cup S(d_2)|}$$

- If $J \geq \text{threshold}$ (for example 0.9) we can state both sets are near duplicates and the one of them can be deleted from the index

Shingling

How large k should be?

It depends on how long typical documents are. The important thing to remember is: **the larger the k, the lower the probability of finding the shingle in any document**

Which is the best value of k?

Usually $k = 5$ is fine for emails, 9 for scientific documents. Indeed, considering 26 characters in english alphabet + 1 blank space, using 5 chars we can have $27^5 = 14,348,907$ possible shingles, and a typical email is much smaller than 14 million characters long

Hashing Shingle

Instead of using Shingles as a set of strings having length k , we can use an hash function for mapping each Shingle to an integer.

So, instead of working with shingles of substrings (for example 5 or 9 bytes), we can work with shingles of integers (only 4 byte), **reducing the amount of space** to store shingles.

If we represent an integer with 32 bits, each shingle is mapped into a bucket number ranging between 0 and $2^{32}-1$

Signature

Even if we hash them to four bytes each, the space needed to store a set of shingles is very large.

If we have millions of documents, it might not be possible **to store all shingles** in main memory.

The idea is that to use a **smaller representation** of the large sets. Such a representation is named **signature**

Signature

So, for evaluating the similarity between two documents, instead of using the sets of shingles we can use the signatures of both sets

Of course, we need a way to compare the signatures of two sets and estimate the Jaccard similarity of the underlying sets from the signatures alone.

It is not possible that the signatures give the exact similarity of the sets they represent, because the size of the signature is smaller, but **the larger the signatures the more accurate the estimates**

Signature

Suppose to have 4 different documents D_1, D_2, D_3, D_4 , and we built 4 different hashing shingle sets S_1, S_2, S_3, S_4 , defined as follows $S_1 = \{a, b\}$, $S_2 = \{c\}$, $S_3 = \{b, d, e\}$, $S_4 = \{a, c, d\}$, where $U = \{a, b, c, d, e\}$ is the universal set containing all hashing shingles.

We can build the **characteristic matrix**, putting all the sets in the columns and all the elements in the rows.

Element	S1	S2	S3	S4
a	1	0	0	1
b	0	0	1	0
c	0	1	0	1
d	1	0	1	1
e	0	0	1	0

Characteristic matrix

Shingle sets from the documents

Shingle items

Signature

There is a 1 in row r and column c if the element for row r is a member of the set for column c .

Otherwise the value in position (r, c) is 0.

Starting from the characteristic matrix we can build the signature, by applying several hundred times an operation which is named **minhash**

<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0

Signature

Minhashing:

Step 1: we make a permutation of the rows in the characteristic matrix

<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0



<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0
<i>c</i>	0	1	0	1

Signature

Minhashing:

Step 2: the minhash value of a column S_i is the shingle item of the first row, in the permuted order, in which the column has a 1: $h(S_1)=a$, $h(S_2)=c$; $h(S_3)=b$; $h(S_4)=a$

<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0



<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0
<i>c</i>	0	1	0	1

<i>a</i>			
----------	--	--	--

Signature

Minhashing:

Step 2: the minhash value of a column S_i is the shingle item of the first row, in the permuted order, in which the column has a 1: $h(S_1)=a$, $h(S_2)=c$; $h(S_3)=b$; $h(S_4)=a$

<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0



<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0
<i>c</i>	0	1	0	1

a	c		
----------	----------	--	--

Signature

Minhashing:

Step 2: the minhash value of a column S_i is the shingle item of the first row, in the permuted order, in which the column has a 1: $h(S_1)=a$, $h(S_2)=c$; $h(S_3)=b$; $h(S_4)=a$

<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0



<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0
<i>c</i>	0	1	0	1

a	c	b	
----------	----------	----------	--

Signature

Minhashing:

Step 2: the minhash value of a column S_i is the shingle item of the first row, in the permuted order, in which the column has a 1: $h(S_1)=a$, $h(S_2)=c$; $h(S_3)=b$; $h(S_4)=a$

<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0



<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0
<i>c</i>	0	1	0	1

<i>a</i>	<i>c</i>	<i>b</i>	<i>a</i>
----------	----------	----------	----------

Minhashing and Jaccard Similarity

Theorem

The probability that the minhash function for a random permutation of rows produces the same value for two sets (S_i and S_j) is equal to the Jaccard similarity of those sets.

$$\Pr(h(S_i) = h(S_j)) = J(S_i, S_j)$$

Minhashing and Jaccard Similarity

Demonstration

Let's consider the permutation of the characteristic matrix and restrict ourselves to the columns S_1 and S_4 .

All rows can be divided into three sets:

- X: rows having 1 in both columns
- Y: rows having 1 in one column and 0 in the other
- Z: rows having 0 in both columns

<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>e</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0

Minhashing and Jaccard Similarity

Demonstration

We can also compute those values

- x : number of rows of type X
- y : number of rows of type Y
- z : number of rows of type Z

<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>e</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0

Minhashing and Jaccard Similarity

Demonstration

If 1 is in both columns, it means that the corresponding element belongs to both sets (for example, the element d belongs to both S_1 and S_4 , so it is in the intersection of both sets). This means that:

<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>e</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0

$$J(S_1, S_4) = |S_1 \cap S_4| / |S_1 \cup S_4| = x / (x + y) = 2/3$$

Minhashing and Jaccard Similarity

Demonstration

Now, let's try to compute the probability that $h(S_1) = h(S_4)$:

Two columns have the same minhashing value if the first row, non belonging to Z, belongs to X. And being x the number of X rows and $x+y$ the total number, we can say that:

$$\Pr(h(S_1) = h(S_4)) = x / (x + y) = 2/3$$

which is equal to $J(S_1, S_4)$

<i>Element</i>	<i>S1</i>	<i>S2</i>	<i>S3</i>	<i>S4</i>
<i>e</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0

Signature Matrix

Previously, we saw how to build a Signature from a random permutation of the Characteristic Matrix. But if we permute again and again (let's say thousands of times) instead of having a Signature we have a Signature Matrix

Element	S1	S2	S3	S4
a	1	0	0	1
b	0	0	1	0
d	1	0	1	1
e	0	0	1	0
c	0	1	0	1

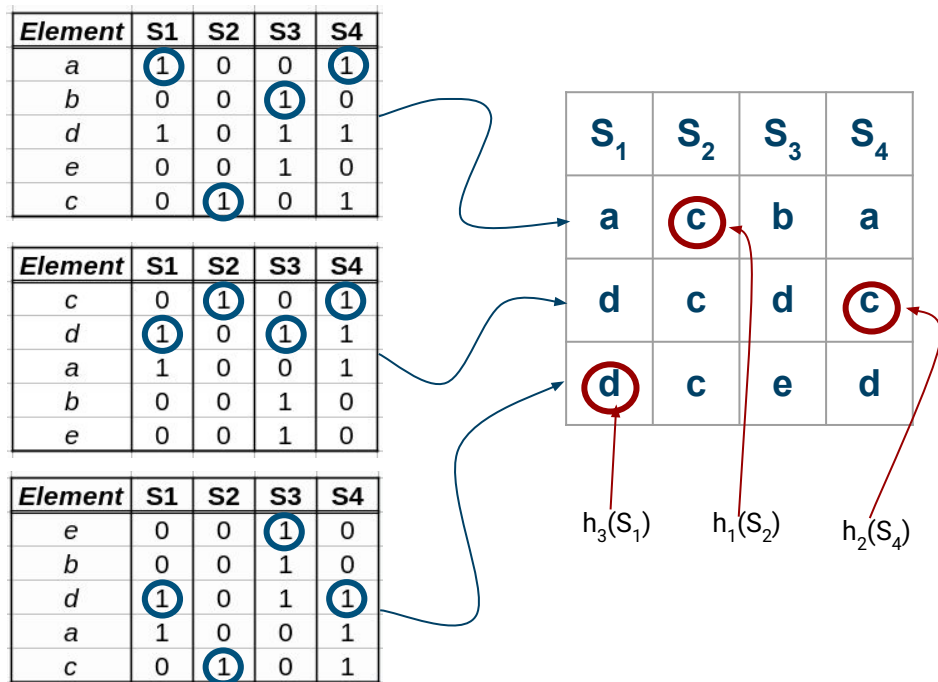
Element	S1	S2	S3	S4
c	0	1	0	1
d	1	0	1	1
a	1	0	0	1
b	0	0	1	0
e	0	0	1	0

Element	S1	S2	S3	S4
e	0	0	1	0
b	0	0	1	0
d	1	0	1	1
a	1	0	0	1
c	0	1	0	1

S ₁	S ₂	S ₃	S ₄
a	c	b	a
d	c	d	c
d	c	e	d

Signature Matrix

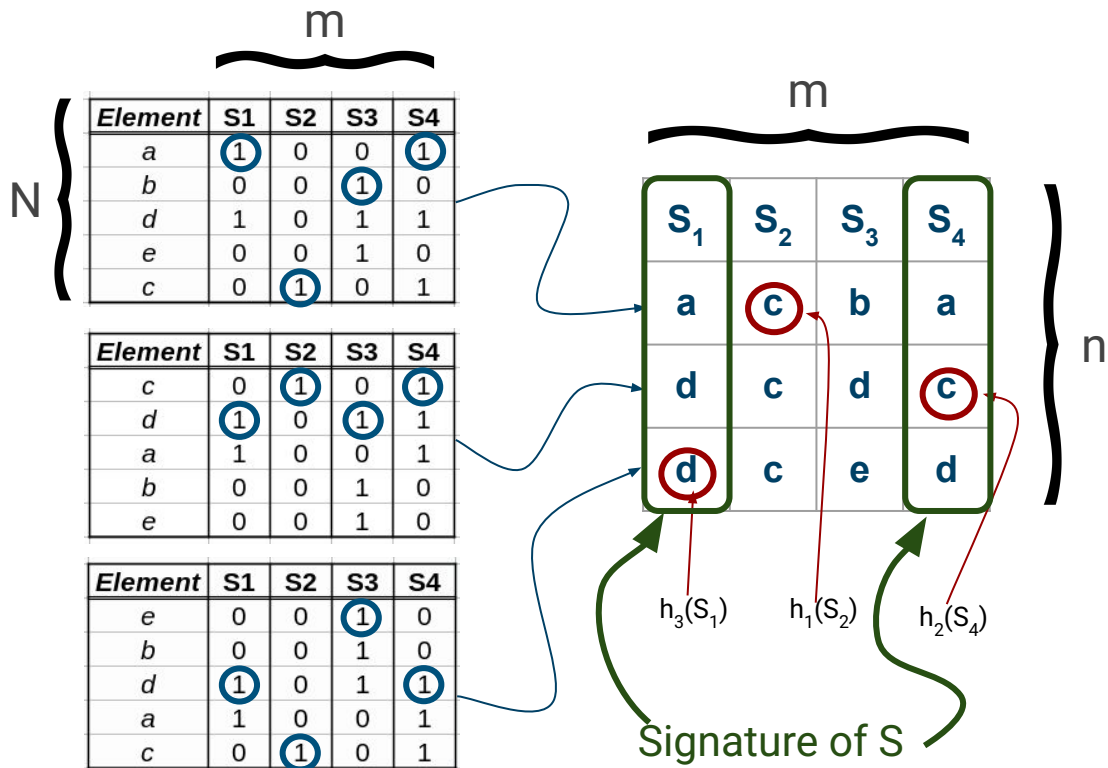
If we consider $h_1, h_2, \dots, h_n$ the permutations, the Signature Matrix is made by m columns (the same number of columns in the Characteristic Matrix) where each element of the column is the minhash value for a permutation.



Signature Matrix

Note that the signature matrix has the same number of columns as Characteristic Matrix M but only n rows. Even if M is not represented explicitly it is normal for the signature matrix to be much smaller than M .

$$N \gg n$$



Characteristic Matrix vs Signature Matrix

- Characteristic Matrix is much larger than Signature Matrix (same column but more rows)
- By the theorem, the Jaccard Similarity between the Shingle sets of two documents is equal to the probability that two columns have the same value in a given row of the signature matrix
- Since the permutations were chosen independently, we can think of each row of the signature matrix as an independent experiment
- The expected number of rows in which two columns agree equals the Jaccard similarity of their corresponding sets

The error of the Signature

The more the number of rows in the signature matrix, the smaller the expected error in the estimate of the Jaccard similarity will be

Computing Minhash Signature

When the Characteristic Matrix is wide (million or billion of rows), even picking a random permutation can be very time-consuming.

Luckily it is possible to simulate the effect of a random permutation by a random hash function h . So we can choose n different hash functions $(h_1, h_2, \dots, h_n)$ and each of them will represent a permutation.

An example of hash function $h(x)$ can be

$$h(x) = 3x + 1 \bmod 5.$$

Then, for the row having index 4, the $h(4) = 3$

Computing Minhash Signature

Let see the algorithm for building the Minhash Signature Matrix

On the left side, there is the Characteristic Matrix (we have just replaced the label for the rows with an integer).

On the right the assigned value depending on two hash functions

Row	S_1	S_2	S_3	S_4	$x + 1 \bmod 5$	$3x + 1 \bmod 5$
0	1	0	0	1	1	1
1	0	0	1	0	2	4
2	0	1	0	1	3	2
3	1	0	1	1	4	0
4	0	0	1	0	0	3

Computing Minhash Signature

We start initializing the signature matrix with infinite values

Row	S_1	S_2	S_3	S_4	$x + 1 \bmod 5$	$3x + 1 \bmod 5$
0	1	0	0	1	1	1
1	0	0	1	0	2	4
2	0	1	0	1	3	2
3	1	0	1	1	4	0
4	0	0	1	0	0	3

	S_1	S_2	S_3	S_4
h_1	∞	∞	∞	∞
h_2	∞	∞	∞	∞

Computing Minhash Signature

Then we start for row 0 in the Characteristic Matrix. We see 1 in column S_1 and S_4 . As $h_1(S_1)$, $h_2(S_1)$, $h_1(S_4)$, $h_2(S_4)$ are less than infinite, we replace infinite with the corresponding value h_1 and h_2 on the Signature Matrix only on all rows in the column S_1 and S_4

Row	S_1	S_2	S_3	S_4	$x + 1 \mod 5$	$3x + 1 \mod 5$
0	1	0	0	1	1	1
1	0	0	1	0	2	4
2	0	1	0	1	3	2
3	1	0	1	1	4	0
4	0	0	1	0	0	3

	S_1	S_2	S_3	S_4
h_1	∞	∞	∞	∞
h_2	∞	∞	∞	∞

	S_1	S_2	S_3	S_4
h_1	1	∞	∞	1
h_2	1	∞	∞	1

Computing Minhash Signature

Then we move on row 1 in the Characteristic Matrix. We see that only S_3 column has 1, so we replace $h_1(S_3)$ and $h_2(S_3)$ in the Signature with respectively 2 and 4, being such values less than infinite.

Row	S_1	S_2	S_3	S_4	$x + 1 \bmod 5$	$3x + 1 \bmod 5$
0	1	0	0	1	1	1
1	0	0	1	0	2	4
2	0	1	0	1	3	2
3	1	0	1	1	4	0
4	0	0	1	0	0	3

	S_1	S_2	S_3	S_4
h_1	∞	∞	∞	∞
h_2	∞	∞	∞	∞

	S_1	S_2	S_3	S_4
h_1	1	∞	∞	1
h_2	1	∞	∞	1

	S_1	S_2	S_3	S_4
h_1	1	∞	2	1
h_2	1	∞	4	1

Computing Minhash Signature

Then we move on row 2 in the Characteristic Matrix. We see that S_2 and S_4 columns have 1, so we replace $h_1(S_2)$ and $h_2(S_2)$ in the Signature with 3 and 2, being such values less than infinite, but we do not replace $h_1(S_4)$ and $h_2(S_4)$ because the respective values (3 and 2) are greater than the current value in the Signature (1 and 1)

Row	S_1	S_2	S_3	S_4	$x + 1 \mod 5$	$3x + 1 \mod 5$
0	1	0	0	1	1	1
1	0	0	1	0	2	4
2	0	1	0	1	3	2
3	1	0	1	1	4	0
4	0	0	1	0	0	3

	S_1	S_2	S_3	S_4
h_1	∞	∞	∞	∞
h_2	∞	∞	∞	∞

	S_1	S_2	S_3	S_4
h_1	1	∞	∞	1
h_2	1	∞	∞	1

	S_1	S_2	S_3	S_4
h_1	1	∞	2	1
h_2	1	∞	4	1

	S_1	S_2	S_3	S_4
h_1	1	3	2	1
h_2	1	2	4	1

Computing Minhash Signature

Next row 3. In the Characteristic Matrix only columns S_1 , S_3 and S_4 have 1, so in the Signature we can put the corresponding value of h_1 and h_2 (that is 4 and 0) on all three columns only if the such values are lower than the value already stored in the Signature Value

Row	S_1	S_2	S_3	S_4	$x + 1 \mod 5$	$3x + 1 \mod 5$
0	1	0	0	1	1	1
1	0	0	1	0	2	4
2	0	1	0	1	3	2
3	1	0	1	1	4	0
4	0	0	1	0	0	3

	S_1	S_2	S_3	S_4
h_1	∞	∞	∞	∞
h_2	∞	∞	∞	∞

	S_1	S_2	S_3	S_4
h_1	1	∞	∞	1
h_2	1	∞	∞	1

	S_1	S_2	S_3	S_4
h_1	1	∞	2	1
h_2	1	∞	4	1

	S_1	S_2	S_3	S_4
h_1	1	3	2	1
h_2	1	2	4	1

	S_1	S_2	S_3	S_4
h_1	1	3	2	1
h_2	0	2	0	0

Computing Minhash Signature

Next row number 4. In the Characteristic Matrix only columns S_3 has 1, so in the Signature we can put the corresponding value of h_1 and h_2 (0 and 3) on all three columns only if the such values are lower than the value already stored in the Signature Value

Row	S_1	S_2	S_3	S_4	$x + 1 \mod 5$	$3x + 1 \mod 5$
0	1	0	0	1	1	1
1	0	0	1	0	2	4
2	0	1	0	1	3	2
3	1	0	1	1	4	0
4	0	0	1	0	0	3

	S_1	S_2	S_3	S_4
h_1	∞	∞	∞	∞
h_2	∞	∞	∞	∞

	S_1	S_2	S_3	S_4
h_1	1	∞	2	1
h_2	1	∞	4	1

	S_1	S_2	S_3	S_4
h_1	1	3	2	1
h_2	0	2	0	0

	S_1	S_2	S_3	S_4
h_1	1	∞	∞	1
h_2	1	∞	∞	1

	S_1	S_2	S_3	S_4
h_1	1	3	2	1
h_2	1	2	4	1

	S_1	S_2	S_3	S_4
h_1	1	3	0	1
h_2	0	2	0	0

Jaccard Similarity on Signature Matrix

This is the final Minhash Signature Matrix.

Underneath instead is the original
Characterization Matrix

By using the Signature Matrix, we can compute similarities between two documents. For example S1 and S4 seem to be very similar (they have $J(S1, S4) = 1$. In the original matrix instead

2/3

	S_1	S_2	S_3	S_4
h_1	1	3	0	1
h_2	0	2	0	0

Element	S1	S2	S3	S4
<i>a</i>	1	0	0	1
<i>b</i>	0	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	1
<i>e</i>	0	0	1	0

Jaccard Similarity on Signature Matrix

The difference in the Jaccard Similarity (computed on the Signature Matrix) and the **true** Jaccard Similarity (computed on the Characteristic Matrix) is because the fraction of rows in the Signature Matrix is only **an estimate of the true Jaccard similarity**, and this example is much too small **for the law of large numbers** to assure that the estimates are close

	S_1	S_2	S_3	S_4
h_1	1	3	0	1
h_2	0	2	0	0

Element	S1	S2	S3	S4
a	1	0	0	1
b	0	0	1	0
c	0	1	0	1
d	1	0	1	1
e	0	0	1	0

Dictionary Compression

Dictionary Compression

If compared with the Posting Lists, Dictionary is very small. So, why is it important to compress it if it is responsible for only a **small percentage of the overall space** requirements of the IR system?

Because a smaller dictionary can stay into the main memory, and this means **less disk seeks** required to process a query.

To allow the dictionary **to stay into the memory**, we need to compress it.

Statistic-driven Compression

Collection for Reuters-RCV1	
Documents	806,791
Number of Tokens	96,969,056
Tokens per documents	222
Distinct Terms	391,523
Average bytes per Term	7.5

Dictionary as a string

The simplest way to manage a Dictionary is by storing terms into the vocabulary and **sort them lexicographically**.

Each term is stored in **fixed size of 20 bytes** (only few english words require more than 20 chars).

We also need 4 bytes for the **document frequency** and 4 byte to **point to the Posting Lists**

term	document frequency	pointer to postings list
a	656,265	→
aachen	65	→
...	...	...
zulu	221	→
20 bytes	4 bytes	4 bytes

$$D_{size} = T(20 + 4 + 4) = 28T$$

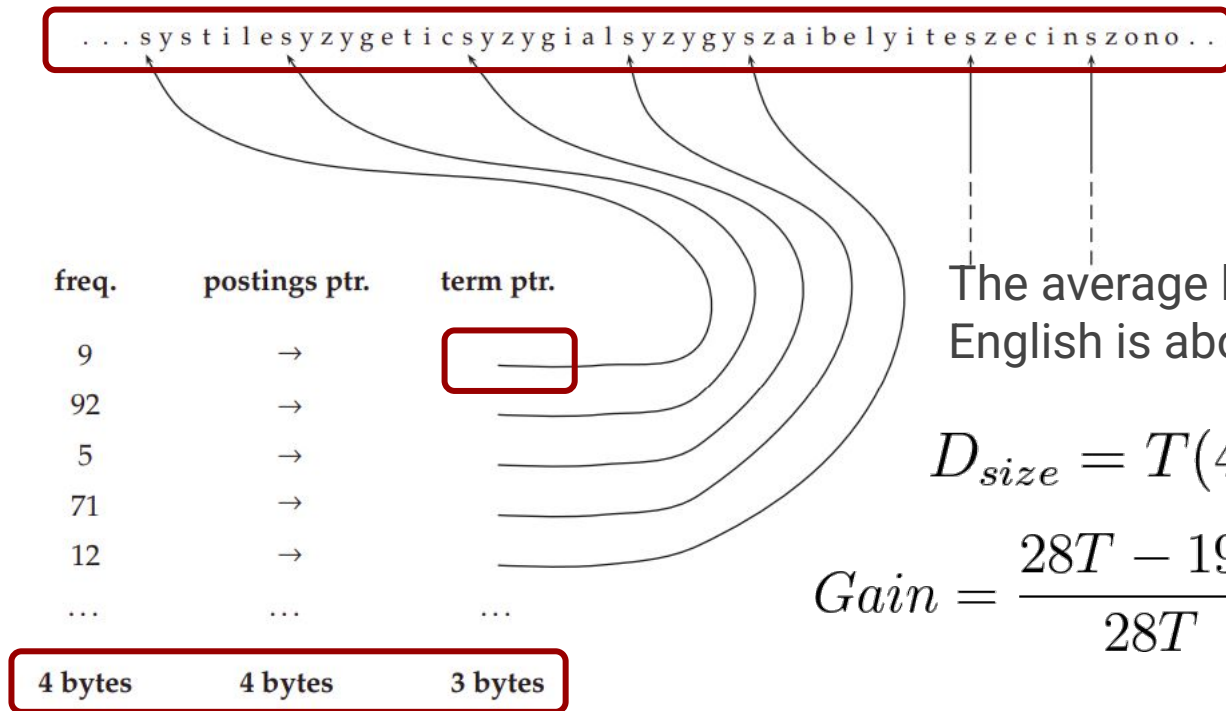
Dictionary as a string

But:

- The average length of a term in English is about **8 characters**, having a fixed size length of 20 chars we lose **12 chars**
- $D_1 = 28T$ $D_2 = 16T$ **Gain** = $(D_1 - D_2)/D_1 = (28T - 16T)/28T \approx 42.86\%$
- We cannot manage longer words

Solution: storing the dictionary terms as one long string of characters

Dictionary as a string



The average length of a term in English is about **eight characters**

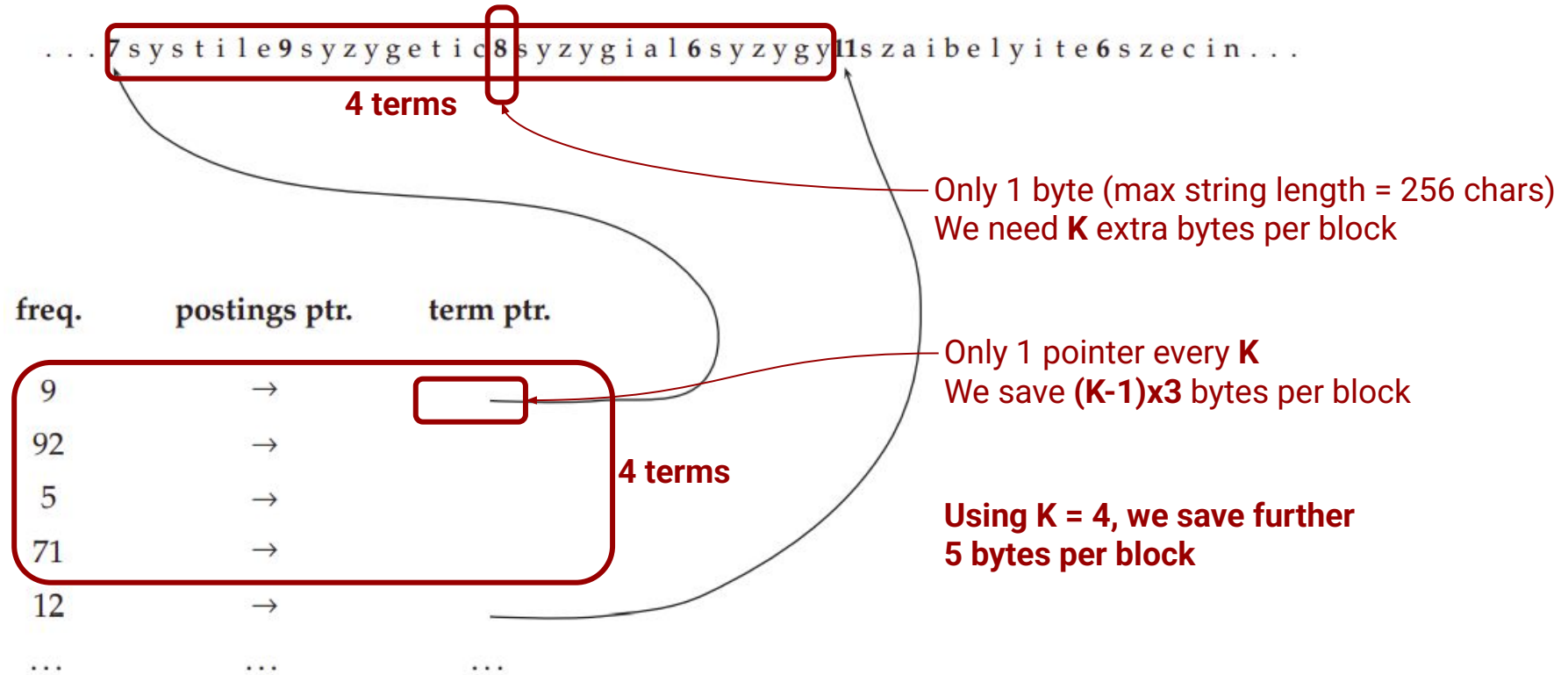
$$D_{size} = T(4 + 4 + 3 + 8) = 19T$$

$$Gain = \frac{28T - 19T}{28T} = \frac{9T}{28T} \approx 0.32 = 32\%$$

Dictionary as blocks of strings

We can even compress the size of the dictionary by **reducing the number of pointers** and managing the long string as **blocks of strings**

Dictionary as blocks of strings



Dictionary as block of strings

The diagram illustrates the formula for dictionary size, D_{size} , with annotations for each part of the expression:

- $\frac{T}{K}$: number of blocks
- K : document frequency
- 4 : postings pointer
- 4 : term average length
- 8 : pointer to block string
- 1 : auxiliary byte for term length
- 3 : pointer to block string

$$D_{size} = \frac{T}{K} (K(4 + 4 + 8 + 1) + 3) = \frac{T}{K} (17K + 3)$$

Dictionary as block of strings

If $K = 4$, then we can get a gain equals to

$$\begin{aligned} Gain &= \frac{28T - \frac{T}{K} (17K + 3)}{28T} = \frac{28T - \frac{T}{4} (17 * 4 + 3)}{28T} \\ &= \frac{28T - 17.75T}{28T} \approx 36.61\% \end{aligned}$$

Tradeoffs

- Managing Dictionary as String or block of string is **more complex**
- The larger is K , the more is the compression factor
- But it takes more time in search
 - because we need to scan the whole block

Dictionary with Front Coding

Observation: One block in blocked compression ($k = 4$) ...
8automata8automate9automatic10automation

As the example shows, when terms are sorted alphabetically, they more often have the **same prefix**.

We can exploit this observation in order to further compress the Dictionary

Dictionary with Front Coding

How does it work?

We can use special character to identify the common prefix

One block in blocked compression ($k = 4$) ...
8automata8automate9automatic10automation



...further compressed with front coding.
8automat*a1♦e2♦ic3♦ion

Comparing all techniques

Collection for Reuters-RCV1	
Documents	806,791
Number of Tokens	96,969,056
Tokens per documents	222
Distinct Terms	391,523
Average bytes per Term	7.5

data structure	size in MB
dictionary, fixed-width	11.2
dictionary, term pointers into string	7.6
~, with blocking, $k = 4$	7.1
~, with blocking & front coding	5.9

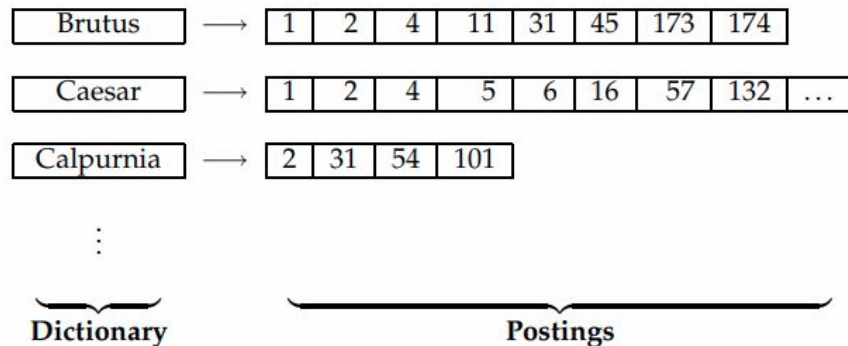
$$Gain = \frac{11.2MB - 5.9MB}{11.2MB} \approx 0.47 = 47\%$$

Postings List Compression

How to reduce posting list size?

Let consider the simplest Posting Lists:

- Each posting is the DocID
- Each ID can be identified by 32 bits
- Using 32 bit we can identify 2^{32} documents (sometimes too much)

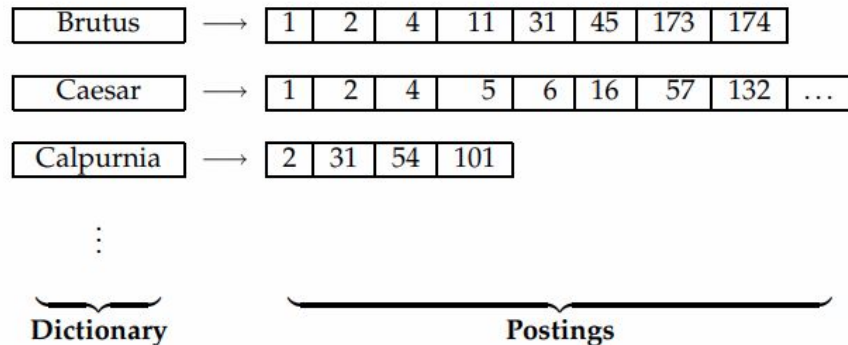


We can use $\log_2 D$ bits for the DocID

How to reduce posting list size?

But we can do better.

Observation: terms that are very frequent appear often in the documents. This means the DocIDs, being sorted, are very **close to each other**.



How to reduce posting list size?

So, instead of storing the DocIDs, why don't we store only the **gap**?

For very recurring Terms, the gap is very small and require a small number of bits

So, instead of $\log_2 D$ bits for the DocIDs, we can use fewer bit but **we cannot know in advance how large is this gap** and then how many bits are required to store the gap.

For this reason, we need a **Variable Byte Encoding**.

Variable Byte Code

A Variable Byte Code is a sequence of 8 bit (that is a byte), where:

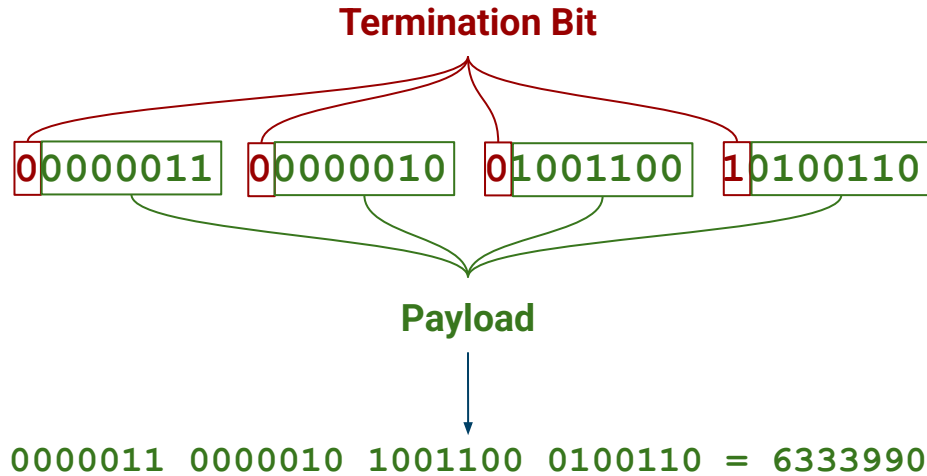
- the first bit is named **termination bit**
- the last seven bits are named **payload**

One or more Variable Byte Codes can be used to describe the gap

The termination Bit

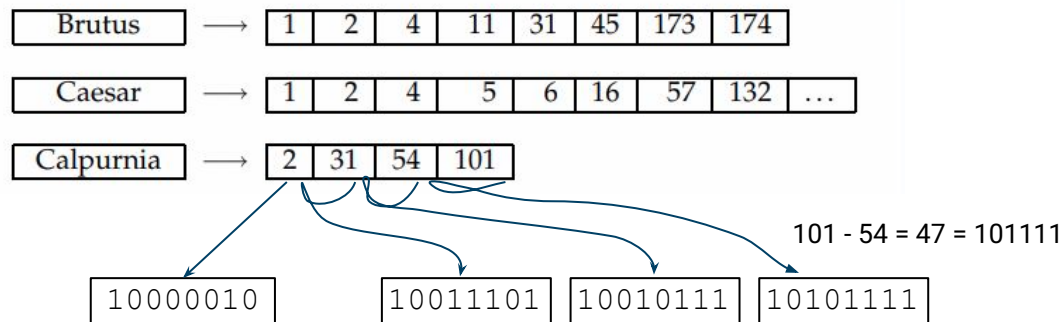
- is set to 0 if the next byte is the continuation of the same Variable Byte Code
- is set to 1 if this byte is the last byte of the Variable Byte Code

Variable Byte Code



Variable Byte Code

For example: suppose we have 65,000 documents. We need 16 bits for each DocID. Let's consider the last Postings List using Variable Byte Code:



Variable Byte Code

Consideration: if the gap was 127, we can represent the gap using only one byte:

11111111

but if the gap was 128 (only one higher), we need to use two bytes:

00000001 **1**0000000

The first 6 zeros are useless

Tradeoffs of Variable Byte Code

Variable Byte Code represents a **bytewise compression**. This means that the gaps are always represented by one or more bytes

Working with a **bitwise compression method**, we can achieve better compression ratio: **Gamma Codes**

Gamma Code

Gamma Code is a bitwise compression method.

It represents the gap with a sequence of bits having an **adaptive length**

The Gamma Code is made by two parts:

- the offset: is the binary representation of the gap **without the heading bit**
- the length: is an **unary code** of the length
 - the unary code of n is a string of n ones followed by a zero, example $5 = 111110$

Gamma Code

An example:

Suppose we want to represent 13 (decimal) in Gamma Code:

- $13_{10} = 1101_2$
- offset = **101**
- $\text{length}(\text{101}) = 3 = \text{1110}$
- $\text{Gamma}(13) = \text{1110101}$

Gamma Code

What is the maximum length of a Gamma Code?

If G is the gap:

- the length of the **offset** $\lfloor \log_2 G \rfloor$
- the length of the **length** $\lfloor \log_2 G \rfloor + 1$

So the maximum length $2 \lfloor \log_2 G \rfloor + 1$

Comparing all techniques

Collection for Reuters-RCV1	
Documents	806,791
Number of Tokens	96,969,056
Tokens per documents	222
Distinct Terms	391,523
Average bytes per Term	7.5

Data Structure	Size in MB
Term Incidence Matrix	40,000
Postings with no compression using 32-bit for DocID	400
Postings with no compression 20-bit for DocID	250
Postings with variable byte encoding	116
Posting with gamma encoding	101

Comparing all techniques

Data Structure	Size in MB	Gain
Term Incidence Matrix	40,000	
Postings with no compression using 32-bit for DocID	400	
Postings with no compression 20-bit for DocID	250	$(400 - 250) / 400 = 0.375 =$ 37.50%
Postings with variable byte encoding	116	$(400 - 116) / 400 = 0.71 =$ 71.00%
Posting with gamma encoding	101	$(400 - 101) / 400 = 0.7475 =$ 74.75%

Scoring, Term Weighting and the Vector Space Model

Academic Year 2025/2026

Luigi Santangelo
luigi.santangelo@unipv.it

University of Pavia
**Master Degree in
Computer Engineering**

Boolean vs Free Text Retrieval Model

All the methods described so far (Single Word Index, Bi- and Multi-Word Index, Positional Index, WildCard, Permuterm, K-Gram) are based on the **Boolean Retrieval Model**. Such model is able to retrieve all those documents containing all terms.

To help users finding all the information they need, Search Engines are also based on a different model, called **Free Text Retrieval Model**, which allows retrieving documents even though they do not contain all terms. Such models are based on **ranking**. The **Vector Space Model** supports ranking.

Boolean vs Free Text Retrieval Model

Feature	Boolean Retrieval	Free Text Retrieval
<i>Approach</i>	Logical Operators	Natural Language
<i>Precision</i>	High	Low
<i>Recall</i>	Lower	Higher
<i>Ranking</i>	Exact match	Relevance Based
<i>Easy to use</i>	Requires Learning	Simpler
<i>Result Scope</i>	Narrower	Broader
<i>Example</i>	Paris AND restaurant AND Michelin star	Best restaurants in Paris
<i>Suitable for</i>	Specific Information Retrieval	Information Exploration, Broader searches

The importance of using scores

Usually, the collection of documents retrieved after a search can be very large.

It is essential for a Search Engine to rank and order the documents matching a query.

To help user finding the right documents, Search Engines makes use of different ideas:

1. Parametric Index and Zone Index
2. Term Weighting
3. Vector Space Scoring

Parametric and Zone Index

Metadata

So far, we have considered a Document as a sequence of Terms. And for each Term in the dictionary we create a Posting List which is used during queries.

But a document is not just a sequence of terms, because sometimes it is **described by many other information**, such as the author, the date of creation, the title, the language

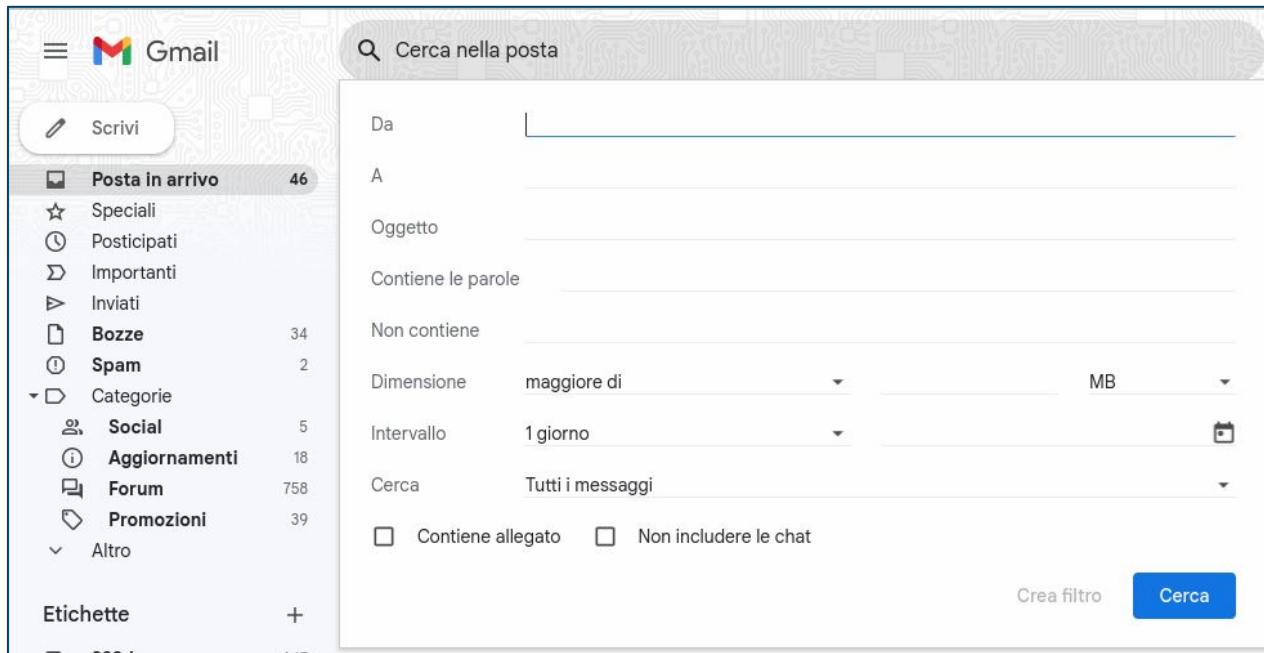
These data are known **Metadata**

Metadata

A metadata can be identified as:

- a **field**: if the value is limited within a set of possible alternatives (such as the author and the year of publishing)
- a **zone**: if the value is an arbitrary free text (such as abstract or title)

Using Metadata during Searches



The screenshot displays the Gmail search interface. On the left, the navigation sidebar includes the Gmail logo, a 'Scrivi' button, and a list of folders: 'Posta in arrivo' (46), 'Speciali', 'Posticipati', 'Importanti', 'Inviati', 'Bozze' (34), 'Spam' (2), 'Categorie', 'Social' (5), 'Aggiornamenti' (18), 'Forum' (758), 'Promozioni' (39), and 'Altro'. Below these are 'Etichette' and a plus sign. The main search area has a header 'Cerca nella posta' and a search bar. Below the search bar are several filter fields: 'Da' (empty), 'A' (empty), 'Oggetto' (empty), 'Contiene le parole' (empty), 'Non contiene' (empty), 'Dimensione' (set to 'maggiore di' and 'MB'), 'Intervallo' (set to '1 giorno' with a calendar icon), and 'Cerca' (set to 'Tutti i messaggi'). At the bottom of the filters are two checkboxes: 'Contiene allegato' and 'Non includere le chat'. In the bottom right corner, there is a 'Crea filtro' link and a blue 'Cerca' button.

Gmail

Cerca nella posta

Scrivi

Posta in arrivo 46

Speciali

Posticipati

Importanti

Inviati

Bozze 34

Spam 2

Categorie

Social 5

Aggiornamenti 18

Forum 758

Promozioni 39

Altro

Etichette +

Da

A

Oggetto

Contiene le parole

Non contiene

Dimensione maggiore di MB

Intervallo 1 giorno

Cerca Tutti i messaggi

☐ Contiene allegato ☐ Non includere le chat

Crea filtro

Cerca

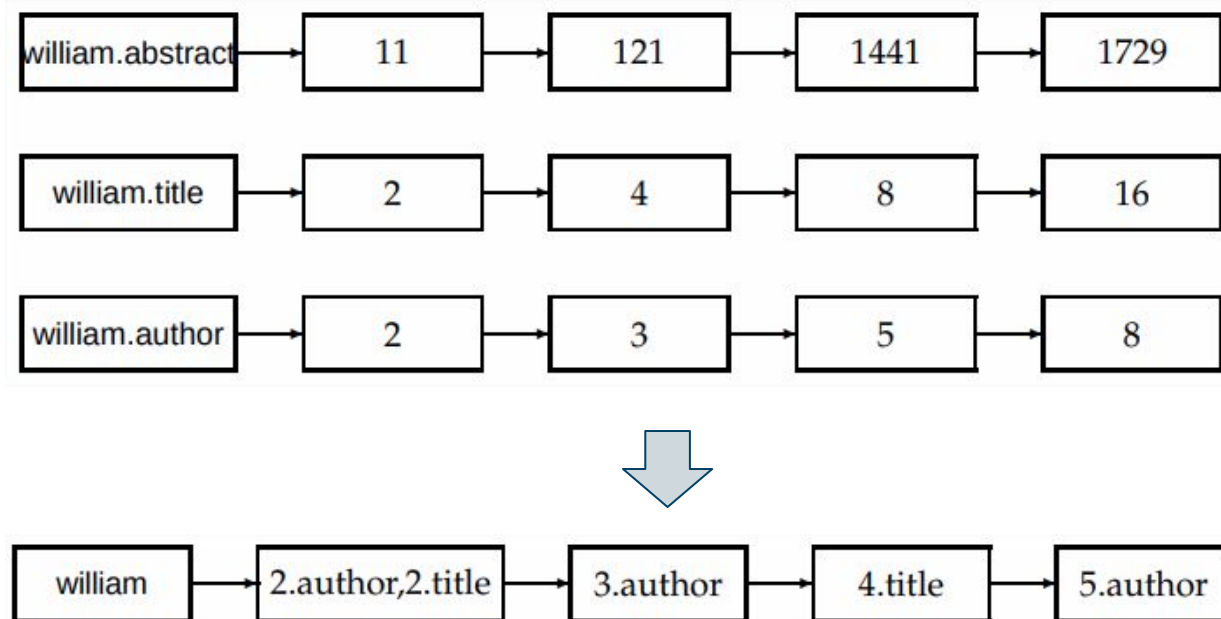
Parametric Index and Zone Index

A Parametric Index is a inverted index which is built using Field.

A Zone Index is a reverted index which is built using Zone.

During queries, Parametric Index, Zone Index and Inverted Index are merged together.

An example of Zone Index



Weighted Zone Scoring

Not all zones are equally important.

Search Engine should consider the weight of each zone.

Given a query term q and a document d , weighted zone scoring assigns to the pair (d, q) a score in the interval $[0, 1]$, by computing a **linear combination of zone scores**

Weighted Zone Scoring

Let's consider l zones. Each zone is assigned to a weight g_i where

$$g_i \in [0, 1] \text{ such that } \sum_{i=1}^l g_i = 1$$

Let's consider also $s_i(\mathbf{d}, \mathbf{q}) \in \{0, 1\}$ where 0 means that the term does not occur in the zone i of the document d , 1 if it does. Then the **weighted zone scoring** can be computed as

$$score(d, q) = \sum_{i=1}^l g_i * s_i(d, q)$$

Weighted Zone Scoring

For example: Let's suppose to have 3 zones: *author*, *title*, *body*, having weight respectively $g_1=0.2$, $g_2=0.3$ and $g_3=0.5$. We want to search for the term $q = \textit{shakespeare}$.

Now, consider a document d where the term appears only on the title and the body. So, the score of this document would be **$0.2*0 + 0.3*1 + 0.5*1 = 0.8$**

How to use the right score for the zones?

An trivial way to assign scores is by asking the user (or an expert)

But assigning the right scores can be viewed as a simple **optimization problem** and we can learn the weights.

Let's start simple. Suppose we have a collection of documents each of them described by two zones: *title* and *body*.

How to use the right score for the zones?

Given a query term q and a document d , we can compute $s_T(d, q)$ and $s_B(d, q)$

both functions will give 0 or 1 depending on the matching of the query to the title and to the body.

If we had the weights for both zones, we could easily compute the score, but we do not have such weights, we need to learn them:

$$score(d, q) = g * s_T(d, q) + (1 - g) * s_B(d, q)$$

How to use the right score for the zones?

Now, our aim is to determine the constant g from **a set of training examples** built by a human editor.

Such set is made by quintuples of the form $\Phi_j = (d, q, s_T(d, q), s_B(d, q), r(d, q))$ where:

- d is the document
- q is the query term
- $s_T(d, q) = 1$ if the title in d contains q , 0 otherwise
- $s_B(d, q) = 1$ if the body in d contains q , 0 otherwise
- $r(d, q)$ is the **relevance judgment** assessed by the human editor,
 - **1 for *relevant*** and **0 for *non relevant***.

How to use the right score for the zones?

Example	DocID	Query	s_T	s_B	Judgment
Φ_1	37	linux	1	1	Relevant
Φ_2	37	penguin	0	1	Non-relevant
Φ_3	238	system	0	1	Relevant
Φ_4	238	penguin	0	0	Non-relevant
Φ_5	1741	kernel	1	1	Relevant
Φ_6	2094	driver	0	1	Relevant
Φ_7	3191	driver	1	0	Non-relevant

How to use the right score for the zones?

Now we can compute the difference between the predicted score and the real score as

$$\epsilon(g, \Phi_j) = (r(d_j, q_j) - \text{score}(d_j, q_j))^2$$

This is the error of the scoring function with weight g . We can compute the total error as

$$\sum_j \epsilon(g, \Phi_j)$$

We want to make the error as **the minimum**

How to use the right score for the zones?

Now, let's split Φ in 8 different parts:

 Φ_{00r} Φ_{00n}

$$\Phi = \Phi_{00r} \cup \Phi_{01r} \cup \Phi_{10r} \cup \Phi_{11r} \cup \Phi_{00n} \cup \Phi_{01n} \cup \Phi_{10n} \cup \Phi_{11n}$$

 Φ_{01r} Φ_{01n} Φ_{10r} Φ_{10n}

Now, let's compute the error for one element belonging each of these subsets

 Φ_{11r} Φ_{11n}

How to use the right score for the zones?

and then consider the cardinality of each subset

- n_{00r} and n_{00n} : number of training example Φ_j where $s_T(d_j, q_j) = 0$ and $s_B(d_j, q_j) = 0$ that are relevant and non relevant respectively
- n_{01n} and n_{01n} : number of training example Φ_j where $s_T(d_j, q_j) = 0$ and $s_B(d_j, q_j) = 1$ that are non relevant and non relevant respectively
- n_{10r} and n_{10n} : number of training example Φ_j where $s_T(d_j, q_j) = 1$ and $s_B(d_j, q_j) = 0$ that are non relevant and non relevant respectively
- n_{11r} and n_{11n} : number of training example Φ_j where $s_T(d_j, q_j) = 1$ and $s_B(d_j, q_j) = 1$ that are non relevant and non relevant respectively

How to use the right score for the zones?

Let's compute the error for one item belonging to Φ_{00r}

$$\epsilon(g, \Phi_{00r_j}) = [r(d_j, q_j) - \text{score}(d_j, q_j)]^2$$

$$\epsilon(g, \Phi_{00r_j}) = [r(d_j, q_j) - (gs_T(d_j, q_j) + (1 - g)s_B(d_j, q_j))]^2$$

$$\epsilon(g, \Phi_{00r_j}) = [1 - (g * 0 + (1 - g) * 0)]^2 = 1$$

How to use the right score for the zones?

Let's compute the error for one item belonging to Φ_{00n}

$$\epsilon(g, \Phi_{00n_j}) = [r(d_j, q_j) - \text{score}(d_j, q_j)]^2$$

$$\epsilon(g, \Phi_{00n_j}) = [r(d_j, q_j) - (gs_T(d_j, q_j) + (1 - g)s_B(d_j, q_j))]^2$$

$$\epsilon(g, \Phi_{00n_j}) = [0 - (g * 0 + (1 - g) * 0)]^2 = 0$$

How to use the right score for the zones?

Let's compute the error for one item belonging to Φ_{01r}

$$\epsilon(g, \Phi_{01r_j}) = [r(d_j, q_j) - \text{score}(d_j, q_j)]^2$$

$$\epsilon(g, \Phi_{01r_j}) = [r(d_j, q_j) - (gs_T(d_j, q_j) + (1 - g)s_B(d_j, q_j))]^2$$

$$\epsilon(g, \Phi_{01r_j}) = [1 - (g * 0 + (1 - g) * 1)]^2 = g^2$$

How to use the right score for the zones?

Let's compute the error for one item belonging to Φ_{01n}

$$\epsilon(g, \Phi_{01n_j}) = [r(d_j, q_j) - \text{score}(d_j, q_j)]^2$$

$$\epsilon(g, \Phi_{01n_j}) = [r(d_j, q_j) - (gs_T(d_j, q_j) + (1 - g)s_B(d_j, q_j))]^2$$

$$\epsilon(g, \Phi_{01n_j}) = [0 - (g * 0 + (1 - g) * 1)]^2 = (1 - g)^2$$

How to use the right score for the zones?

Let's compute the error for one item belonging to Φ_{10r}

$$\epsilon(g, \Phi_{10r_j}) = [r(d_j, q_j) - \text{score}(d_j, q_j)]^2$$

$$\epsilon(g, \Phi_{10r_j}) = [r(d_j, q_j) - (gs_T(d_j, q_j) + (1 - g)s_B(d_j, q_j))]^2$$

$$\epsilon(g, \Phi_{10r_j}) = [1 - (g * 1 + (1 - g) * 0)]^2 = (1 - g)^2$$

How to use the right score for the zones?

Let's compute the error for one item belonging to Φ_{10n}

$$\epsilon(g, \Phi_{10n_j}) = [r(d_j, q_j) - \text{score}(d_j, q_j)]^2$$

$$\epsilon(g, \Phi_{10n_j}) = [r(d_j, q_j) - (gs_T(d_j, q_j) + (1 - g)s_B(d_j, q_j))]^2$$

$$\epsilon(g, \Phi_{10n_j}) = [0 - (g * 1 + (1 - g) * 0)]^2 = g^2$$

How to use the right score for the zones?

Let's compute the error for one item belonging to Φ_{11r}

$$\epsilon(g, \Phi_{11r_j}) = [r(d_j, q_j) - \text{score}(d_j, q_j)]^2$$

$$\epsilon(g, \Phi_{11r_j}) = [r(d_j, q_j) - (gs_T(d_j, q_j) + (1 - g)s_B(d_j, q_j))]^2$$

$$\epsilon(g, \Phi_{11r_j}) = [1 - (g * 1 + (1 - g) * 1)]^2 = 0$$

How to use the right score for the zones?

Let's compute the error for one item belonging to Φ_{11n}

$$\epsilon(g, \Phi_{11n_j}) = [r(d_j, q_j) - \text{score}(d_j, q_j)]^2$$

$$\epsilon(g, \Phi_{11n_j}) = [r(d_j, q_j) - (gs_T(d_j, q_j) + (1 - g)s_B(d_j, q_j))]^2$$

$$\epsilon(g, \Phi_{11n_j}) = [0 - (g * 1 + (1 - g) * 1)]^2 = 1$$

How to use the right score for the zones?

Let's now compute the total error

$$\begin{aligned} \sum_{j \in \Phi} \epsilon(g, \Phi_j) = & \sum_{j \in \Phi_{00n}} \epsilon(g, \Phi_{00r_j}) + \sum_{j \in \Phi_{00n}} \epsilon(g, \Phi_{00n_j}) + \sum_{j \in \Phi_{01n}} \epsilon(g, \Phi_{01r_j}) + \sum_{j \in \Phi_{01n}} \epsilon(g, \Phi_{01n_j}) + \\ & \sum_{j \in \Phi_{10n}} \epsilon(g, \Phi_{10r_j}) + \sum_{j \in \Phi_{10n}} \epsilon(g, \Phi_{10n_j}) + \sum_{j \in \Phi_{11n}} \epsilon(g, \Phi_{11r_j}) + \sum_{j \in \Phi_{11n}} \epsilon(g, \Phi_{11n_j}) \end{aligned}$$

How to use the right score for the zones?

Let's now compute the total error

$$\sum_{j \in \Phi} \epsilon(g, \Phi_j) = n_{00r} * 1 + n_{00n} * 0 + n_{01r} * g^2 + n_{01n} * (1 - g)^2 + \\ n_{10r} * (1 - g)^2 + n_{10n} * g^2 + n_{11r} * 0 + n_{11n} * 1$$

$$\sum_{j \in \Phi} \epsilon(g, \Phi_j) = (1 - g)^2(n_{01n} + n_{10r}) + g^2(n_{01r} + n_{10n}) + n_{00r} + n_{11n}$$

How to use the right score for the zones?

As we want to make the error as minimum as possible, let's compute the derivative of the error

$$\begin{aligned} d[(1 - g)^2(n_{01n} + n_{10r}) + g^2(n_{01r} + n_{10n}) + n_{00r} + n_{11n}] = \\ - 2(1 - g)(n_{01n} + n_{10r}) + 2g(n_{01r} + n_{10n}) \end{aligned}$$

How to use the right score for the zones?

Setting the derivative to zero we can get

$$-2(1 - g)(n_{01n} + n_{10r}) + 2g(n_{01r} + n_{10n}) = 0$$

$$g = \frac{n_{01n} + n_{10r}}{n_{01n} + n_{10r} + n_{01r} + n_{10n}}$$

Term Frequency and Weighting

Weighted Zone Scoring

So far, we have just assigned a score depending on whether or not a term appears in a document.

$$score(d, q) = \sum_{i=1}^l g_i * s_i(d, q)$$

Let's consider also $s_i(d, q) \in \{0, 1\}$ where 0 means that the term does not occur in the zone i of the document d , 1 if it does.

Term Count, Term Frequency, Document Frequency

Sometimes the importance of a document might depend on the frequency a term appear in the collection:

Three scores are usually used:

- **Term Count**
- **Term Frequency**
- **Document Count**
- **Document Frequency**

Term Count, Term Frequency, Document Frequency

The **Term Count** $tc_{t,d}$ is the number of time a term t appears in a document d

The **Term Frequency** $tf_{t,d}$ is the number of time a term t appears in a document d over the number of words in the document d

$$tf_{t,d} = \frac{tc_{t,d}}{\sum_{\tau \in d} tc_{\tau,d}}$$

The **Document Count** d_t is is the number of documents a term appears in

The **Document Frequency** df_t is the number of documents a term appears in over the total number of documents in the collection

Using Term Frequency as Score

We can then use the Term Frequency for computing the score:

$$score(d, q) = tf_{t,d}$$

which can be easily extended if we have ***l* zones**:

$$score(d, q) = \sum_{i=1}^l g_i * tf_{t,d}^i$$

Term Frequency vs Inverse Document Frequency

Using a score based on the term frequency is not always a good idea, because documents containing several occurrences of that term **will have an higher score**.

The idea is to **normalize** the Term Frequency with the **Inverse Document Frequency**.

Inverse Document Frequency

$$idf_t = \log_{10} \frac{N}{d_t}$$

where

- N is the number of documents in the collection
- d_t is the number of documents containing the term t .

This mean that the *idf* of a rare term is high, whereas the *idf* of a frequent term is likely to be low. The Inverse Document Frequency is **a measure of how much information the term provides**

Scaling the Term Frequency

Now we can scale the Term Frequency according to the Document Frequency as follows:

$$tf_idf_{t,d} = tf_{t,d} * idf_t$$

$tf_idf_{t,d}$ is

- highest when t occurs many time within a small number of documents
- lower when t occurs few time in a document
- 0 when t occur in all documents or the term does not appear at all in the document

An example

	Number of terms	Number of time term “dog” appears	tf	idf	tf_idf
<i>Document 01</i>	100	5	0,050	0,000	0,000
<i>Document 02</i>	100	5	0,050	0,000	0,000
<i>Document 03</i>	100	5	0,050	0,000	0,000
<i>Document 04</i>	100	5	0,050	0,000	0,000
<i>Document 05</i>	100	5	0,050	0,000	0,000
<i>Document 06</i>	100	5	0,050	0,000	0,000
<i>Document 07</i>	100	5	0,050	0,000	0,000
<i>Document 08</i>	100	5	0,050	0,000	0,000
<i>Document 09</i>	100	5	0,050	0,000	0,000
<i>Document 10</i>	100	5	0,050	0,000	0,000

An example

	Number of terms	Number of time term “dog” appears	tf	idf	tf_idf
<i>Document 01</i>	100	95	0,950	0,000	0,000
<i>Document 02</i>	100	95	0,950	0,000	0,000
<i>Document 03</i>	100	95	0,950	0,000	0,000
<i>Document 04</i>	100	95	0,950	0,000	0,000
<i>Document 05</i>	100	95	0,950	0,000	0,000
<i>Document 06</i>	100	95	0,950	0,000	0,000
<i>Document 07</i>	100	95	0,950	0,000	0,000
<i>Document 08</i>	100	95	0,950	0,000	0,000
<i>Document 09</i>	100	95	0,950	0,000	0,000
<i>Document 10</i>	100	95	0,950	0,000	0,000

An example

	Number of terms	Number of time term “dog” appears	tf	idf	tf_idf
<i>Document 01</i>	100	5	0,050	0,301	0,015
<i>Document 02</i>	100		0,000	0,301	0,000
<i>Document 03</i>	100	5	0,050	0,301	0,015
<i>Document 04</i>	100		0,000	0,301	0,000
<i>Document 05</i>	100	5	0,050	0,301	0,015
<i>Document 06</i>	100		0,000	0,301	0,000
<i>Document 07</i>	100	5	0,050	0,301	0,015
<i>Document 08</i>	100		0,000	0,301	0,000
<i>Document 09</i>	100	5	0,050	0,301	0,015
<i>Document 10</i>	100		0,000	0,301	0,000

An example

	Number of terms	Number of time term “dog” appears	tf	idf	tf_idf
<i>Document 01</i>	100	95	0,950	0,301	0,286
<i>Document 02</i>	100		0,000	0,301	0,000
<i>Document 03</i>	100	95	0,950	0,301	0,286
<i>Document 04</i>	100		0,000	0,301	0,000
<i>Document 05</i>	100	95	0,950	0,301	0,286
<i>Document 06</i>	100		0,000	0,301	0,000
<i>Document 07</i>	100	95	0,950	0,301	0,286
<i>Document 08</i>	100		0,000	0,301	0,000
<i>Document 09</i>	100	95	0,950	0,301	0,286
<i>Document 10</i>	100		0,000	0,301	0,000

An example

	Number of terms	Number of time term “dog” appears	tf	idf	tf_idf
<i>Document 01</i>	100	5	0,050	1,000	0,050
<i>Document 02</i>	100		0,000	1,000	0,000
<i>Document 03</i>	100		0,000	1,000	0,000
<i>Document 04</i>	100		0,000	1,000	0,000
<i>Document 05</i>	100		0,000	1,000	0,000
<i>Document 06</i>	100		0,000	1,000	0,000
<i>Document 07</i>	100		0,000	1,000	0,000
<i>Document 08</i>	100		0,000	1,000	0,000
<i>Document 09</i>	100		0,000	1,000	0,000
<i>Document 10</i>	100		0,000	1,000	0,000

An example

	Number of terms	Number of time term “dog” appears	tf	idf	tf_idf
<i>Document 01</i>	100	95	0,950	1,000	0,950
<i>Document 02</i>	100		0,000	1,000	0,000
<i>Document 03</i>	100		0,000	1,000	0,000
<i>Document 04</i>	100		0,000	1,000	0,000
<i>Document 05</i>	100		0,000	1,000	0,000
<i>Document 06</i>	100		0,000	1,000	0,000
<i>Document 07</i>	100		0,000	1,000	0,000
<i>Document 08</i>	100		0,000	1,000	0,000
<i>Document 09</i>	100		0,000	1,000	0,000
<i>Document 10</i>	100		0,000	1,000	0,000

An example: to recap

Conditions	tf_idf
Term appearing a few time in all documents	0.000
Term appearing many times in all documents	0.000
Term appearing a few time in few documents	0.015
Term appearing many times in few documents	0.286
Term appearing a few time in one document	0.050
Term appearing many times in one document	0.950
Term not appearing at all	0.000

The new score

So, we can change the way we score a document on a query term. That is, instead of summing up the number of occurrences of each term t in query q , we can use the tf_idf weight of each term in the document d :

$$score(d, q) = \sum_{t \in q} \sum_{i=1}^l g_i * tf_idf_{t,d}$$

The BM25 score function

BM25 (Best Match 25) is one ranking algorithm that has gained significant attention and success.

BM25 is a term-based ranking model that aims to provide accurate and relevant search results by scoring documents based on their term frequencies and document lengths.

It is currently used by many Search Engine, such as Solr and Elasticsearch

The BM25 score function

$$score(q, d) = \sum_{i=1}^{|q|} idf(q_i) \cdot \frac{tf(q_i, d) \cdot (k_1 + 1)}{tf(q_i, d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{avgdl})}$$

$$idf(q_i) = \log \frac{N - df(q_i) + 0.5}{df(q_i) + 0.5}$$

q_i = i^{th} term of the query

tf = term frequency (number of times a particular term appears in a document)

|d| = length of the document in words (terms)

avgdl = the average document length over all the documents of the collection

k_1 , **b** = free parameters, usually chosen as $k_1 = 2.0$ and $b = 0.75$

N = total number of documents in the collection

df(q_i) = the number of documents containing the query term q_i

Vector Space

Vector Space Model

The Vector Space Model supports **free text retrieval**, in which a query is specified as a set of words **without any query operators** connecting them.

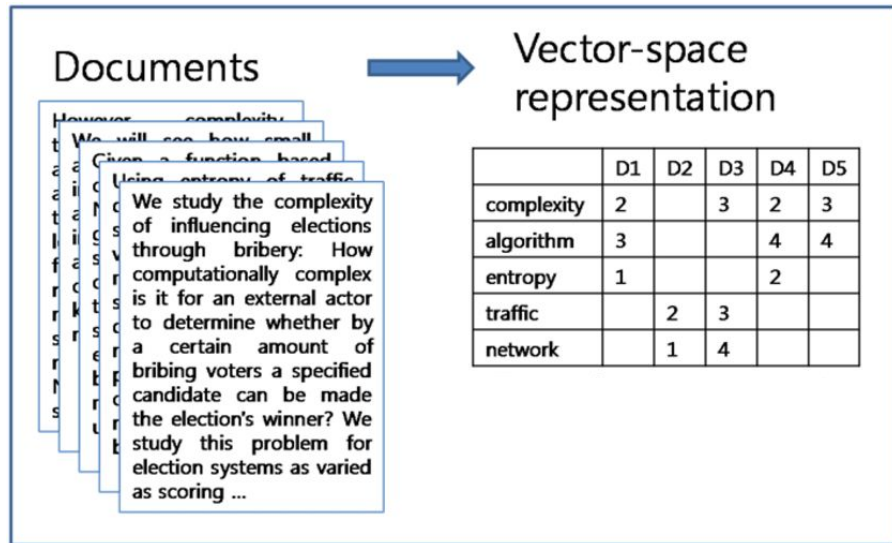
Documents matching the query are **scored** and **ranked**.

The interpretation of such free text queries is that **at least one of the query terms** be present in any retrieved document.

Vector Space Model

A document can be represented by a Vector $V(d)$ with one component for each dictionary term.

The component of the vector are computed using the **tc**, **tf** or **tf_idf** weighting schema



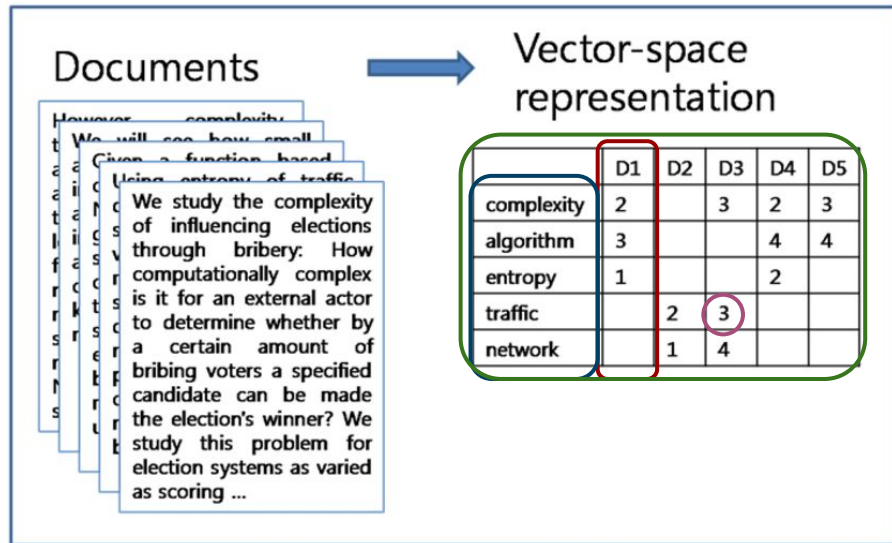
Vector Space Model

Term-Document Matrix

Vocabulary

tc, tf or tf_idf weight

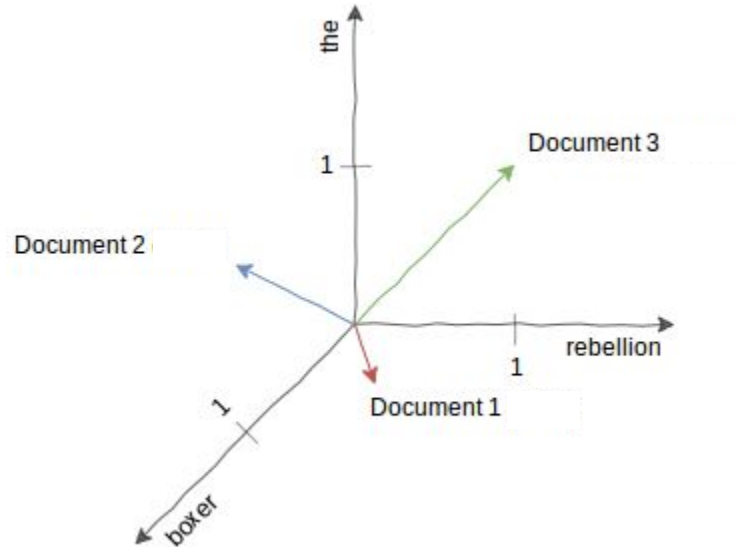
Vector representation for the document



Vector Space Model

We can represent graphically each document by using its own Vector.

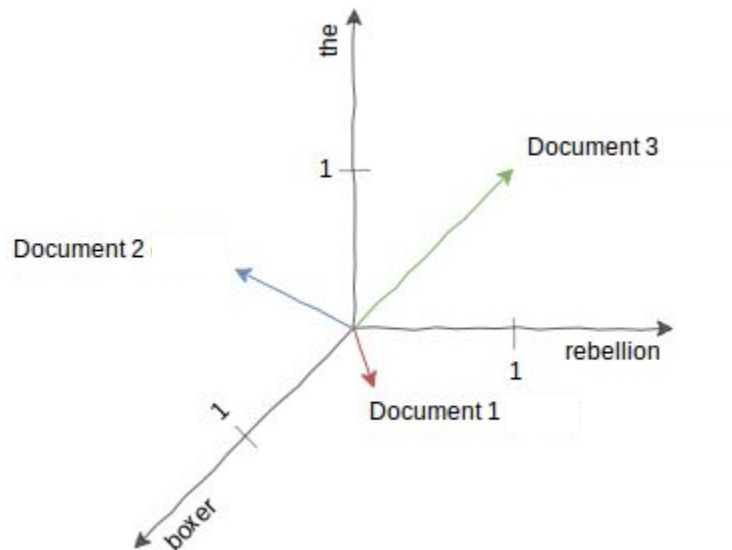
The axis of the space are the terms of the vocabulary. The arrows represent documents



Vector Space Model

Important: with the vector representation we lose the relative ordering of the terms in the documents. This means that the documents *“Mary is quicker than John”* and *“John is quicker than Mary”* are identical in this Vector Space Model.

For this reason this model is also well known in literature as **Bag of Words**



Weaknesses of the Bag of Words model

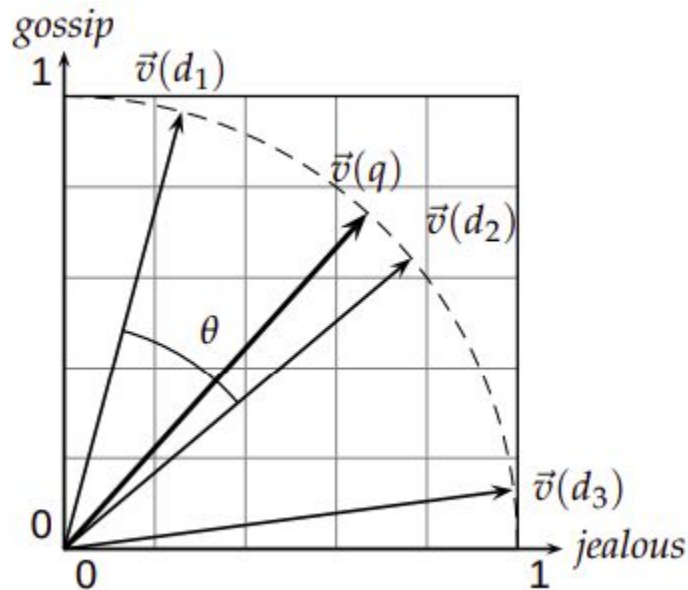
1. BoW treats all words the same, regardless of their order or meaning in a sentence ("time flies" and "flies time" would be represented identically, even though they have opposite meanings)
2. BoW does not take into account the context (the meaning) of each document
3. BoW creates very large vectors (having the same size of the vocabulary)

Solution: **Word Embedding** and **Document Embedding**

Cosine Similarities

We can measure the **similarity between two documents** just using their Vector Model.

Indeed, if the angle of two vectors, $\vec{v}(d_1)$ and $\vec{v}(d_2)$ is equal to 0, both vectors are identical and then both documents are identical (ignoring the order of the terms in the documents)



Cosine Similarities

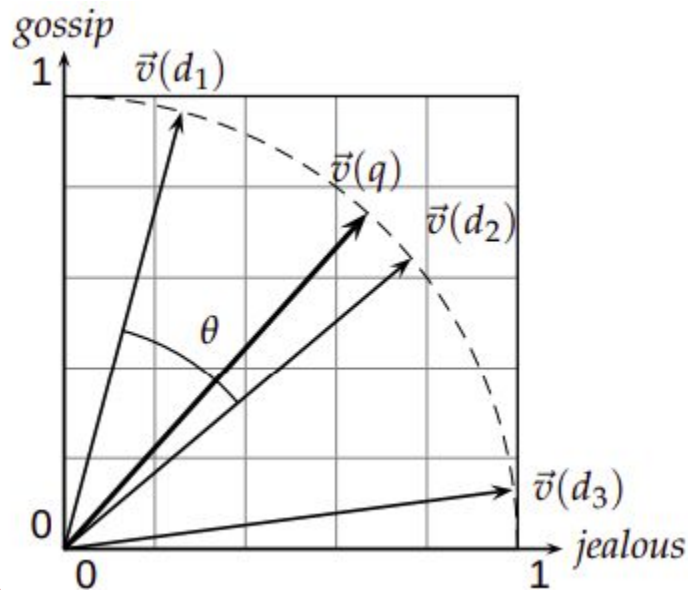
So, the smaller the angle, the more similar are the documents.

How can we compute similarity between two documents?

$$\text{sim}(d_1, d_2) = \cos\theta = \frac{\vec{V}(d_1) \cdot \vec{V}(d_2)}{|\vec{V}(d_1)| |\vec{V}(d_2)|}$$

norm 1

scalar product

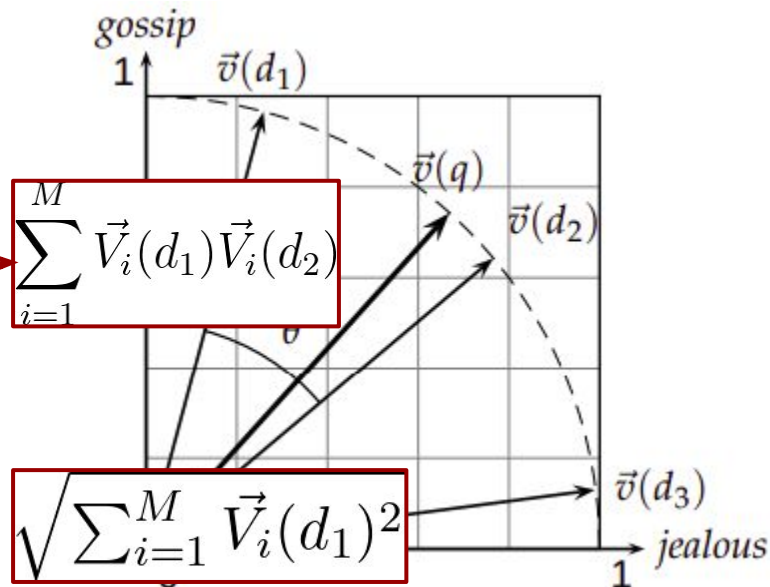


Cosine Similarities

So, the smaller the angle, the more similar are the documents.

How can we compute similarity between two documents?

$$\text{sim}(d_1, d_2) = \cos\theta = \frac{\vec{V}(d_1) \cdot \vec{V}(d_2)}{|\vec{V}(d_1)| |\vec{V}(d_2)|}$$



A real example

Let's suppose we have a collection of three different documents:

- d1: music is a universal language
- d2: music is a miracle
- d3: music is a universal feature of the human experience

Term Count Vector

Term	d1	d2	d3
<i>a</i>	1	1	1
<i>experience</i>	0	0	1
<i>feature</i>	0	0	1
<i>human</i>	0	0	1
<i>is</i>	1	1	1
<i>language</i>	1	0	0
<i>miracle</i>	0	1	0
<i>music</i>	1	1	1
<i>of</i>	0	0	1
<i>the</i>	0	0	1
<i>universal</i>	1	0	1
TOTAL	5	4	9

A real example

Term Count Vector

Term	d1	d2	d3
<i>a</i>	1	1	1
<i>experience</i>	0	0	1
<i>feature</i>	0	0	1
<i>human</i>	0	0	1
<i>is</i>	1	1	1
<i>language</i>	1	0	0
<i>miracle</i>	0	1	0
<i>music</i>	1	1	1
<i>of</i>	0	0	1
<i>the</i>	0	0	1
<i>universal</i>	1	0	1
TOTAL	5	4	9

$$tf_{t,d} = \frac{tc_{t,d}}{\sum_{\tau \in d} tc_{\tau,d}}$$



Term Frequency Vector

Term	d1	d2	d3
<i>a</i>	0,200	0,250	0,111
<i>experience</i>	0,000	0,000	0,111
<i>feature</i>	0,000	0,000	0,111
<i>human</i>	0,000	0,000	0,111
<i>is</i>	0,200	0,250	0,111
<i>language</i>	0,200	0,000	0,000
<i>miracle</i>	0,000	0,250	0,000
<i>music</i>	0,200	0,250	0,111
<i>of</i>	0,000	0,000	0,111
<i>the</i>	0,000	0,000	0,111
<i>universal</i>	0,200	0,000	0,111

A real example

Term Count Vector

Term	d1	d2	d3
<i>a</i>	1	1	1
<i>experience</i>	0	0	1
<i>feature</i>	0	0	1
<i>human</i>	0	0	1
<i>is</i>	1	1	1
<i>language</i>	1	0	0
<i>miracle</i>	0	1	0
<i>music</i>	1	1	1
<i>of</i>	0	0	1
<i>the</i>	0	0	1
<i>universal</i>	1	0	1
TOTAL	5	4	9

$$idf_t = \log_{10} \frac{N}{d_t}$$



IDF

Term	IDF
<i>a</i>	0,000
<i>experience</i>	0,477
<i>feature</i>	0,477
<i>human</i>	0,477
<i>is</i>	0,000
<i>language</i>	0,477
<i>miracle</i>	0,477
<i>music</i>	0,000
<i>of</i>	0,477
<i>the</i>	0,477
<i>universal</i>	0,176

A real example

Term Frequency Vector

Term	d1	d2	d3
a	0,200	0,250	0,111
experience	0,000	0,000	0,111
feature	0,000	0,000	0,111
human	0,000	0,000	0,111
is	0,200	0,250	0,111
language	0,200	0,000	0,000
miracle	0,000	0,250	0,000
music	0,200	0,250	0,111
of	0,000	0,000	0,111
the	0,000	0,000	0,111
universal	0,200	0,000	0,111

IDF

Term	IDF
a	0,000
experience	0,477
feature	0,477
human	0,477
is	0,000
language	0,477
miracle	0,477
music	0,000
of	0,477
the	0,477
universal	0,176

$$tf_idf_{t,d} = tf_{t,d} * idf_f$$



TF_IDF

Term	d1	d2	d3
a	0,000	0,000	0,000
experience	0,000	0,000	0,053
feature	0,000	0,000	0,053
human	0,000	0,000	0,053
is	0,000	0,000	0,000
language	0,095	0,000	0,000
miracle	0,000	0,119	0,000
music	0,000	0,000	0,000
of	0,000	0,000	0,053
the	0,000	0,000	0,053
universal	0,035	0,000	0,020

A real example

Term	d1	d2	d3
<i>a</i>	0,000	0,000	0,000
<i>experience</i>	0,000	0,000	0,053
<i>feature</i>	0,000	0,000	0,053
<i>human</i>	0,000	0,000	0,053
<i>is</i>	0,000	0,000	0,000
<i>language</i>	0,095	0,000	0,000
<i>miracle</i>	0,000	0,119	0,000
<i>music</i>	0,000	0,000	0,000
<i>of</i>	0,000	0,000	0,053
<i>the</i>	0,000	0,000	0,053
<i>universal</i>	0,035	0,000	0,020

Let's compute the similarity between d1 and d3:

$$\text{sim}(d_1, d_2) = \cos\theta = \frac{\vec{V}(d_1) \cdot \vec{V}(d_2)}{|\vec{V}(d_1)| |\vec{V}(d_2)|}$$

= 0.056

A real example

Term	d1	d2	d3
<i>a</i>	0,000	0,000	0,000
<i>experience</i>	0,000	0,000	0,053
<i>feature</i>	0,000	0,000	0,053
<i>human</i>	0,000	0,000	0,053
<i>is</i>	0,000	0,000	0,000
<i>language</i>	0,095	0,000	0,000
<i>miracle</i>	0,000	0,119	0,000
<i>music</i>	0,000	0,000	0,000
<i>of</i>	0,000	0,000	0,053
<i>the</i>	0,000	0,000	0,053
<i>universal</i>	0,035	0,000	0,020

Let's compute the similarity between d1 and d2:

$$\text{sim}(d_1, d_2) = \cos\theta = \frac{\vec{V}(d_1) \cdot \vec{V}(d_2)}{|\vec{V}(d_1)| |\vec{V}(d_2)|}$$

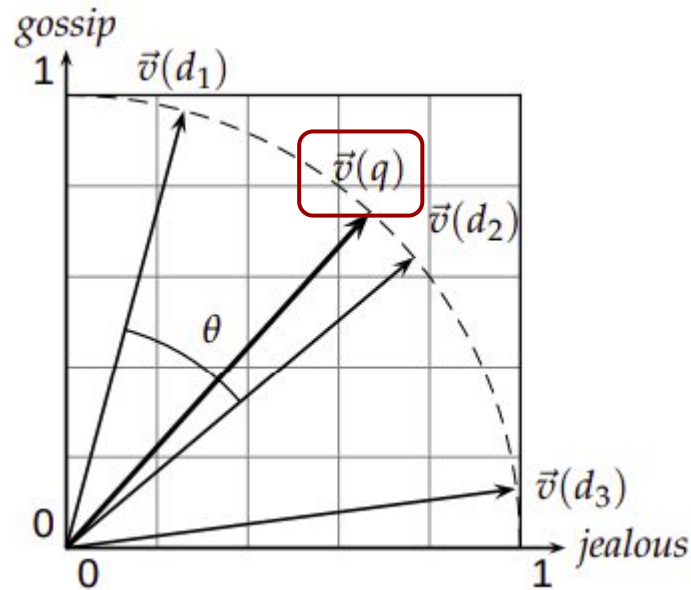
= 0.000

Queries as vectors

As the same as the documents, even **queries** can be considered like a document and then represented by a vector.

Then we can use the cosine similarity between the query vector and a document vector as a measure of the score of the document for that query:

$$score(d, q) = \frac{\vec{V}(q) \cdot \vec{V}(d)}{|\vec{V}(q)| |\vec{V}(d)|}$$



Queries as vectors

A document may have a high cosine score for a query even if it does not contain all query terms.

Queries as vectors

Once the score has been computed between the query and all documents belonging to the collection, **all documents are sorted depending on the score** and then the first K are retrieved.

For many search engines, usually $K = 10$

Computing the cosine similarities between the query vector and each document vector in the collection, sorting the resulting scores and selecting the top K documents **can be expensive**. Will see how to deal with this problem.

Variant tf_idf functions: $\text{wf_idf}_{t,d}$

Many other ways of defining weight have been defined.

Instead of using $\text{tf_idf}_{t,d}$ as weight we can use the **$\text{wf_idf}_{t,d}$**

$$\text{wf-idf}_{t,d} = \text{wf}_{t,d} \times \text{idf}_t$$
$$\text{wf}_{t,d} = \begin{cases} 1 + \log \text{tf}_{t,d} & \text{if } \text{tf}_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$$

the logarithm of the term frequency

Variant tf_idf functions: $ntf_idf_{t,d}$

Another variant:

Instead of using $tf_idf_{t,d}$ as weight we can use the **$ntf_idf_{t,d}$**

$$ntf_idf_{t,d} = ntf_{t,d} \times idf_t$$

The diagram illustrates the components of the $ntf_idf_{t,d}$ function. A blue arrow points from the $ntf_{t,d}$ term in the first equation to the $ntf_{t,d}$ term in the second equation. The second equation, $ntf_{t,d} = a + (1 - a) \frac{tf_{t,d}}{tf_{max}(d)}$, is annotated with three labels: 'smoothing (0.4 or 0.5)' points to the variable a ; 'normalization' points to the fraction $\frac{tf_{t,d}}{tf_{max}(d)}$; and 'maximum term frequency in the document' points to the denominator $tf_{max}(d)$.

$$ntf_{t,d} = a + (1 - a) \frac{tf_{t,d}}{tf_{max}(d)}$$

smoothing (0.4 or 0.5)

normalization

maximum term frequency in the document

Measuring Distances

Measuring distances

Jaccard Similarity Coefficient measures how much two different sets are similar to each other. Sometimes, instead, we are more interested to study how much two sets (or vectors) differ from each other.

Of course Jaccard Similarity can be used to get the **Jaccard Distance**, but it is important to notice that there are many distances measures, but all of them must have some properties:

Measuring distances

Definition: suppose we have a set of point, called space. Let x and y be two points. A distance measure on this between two points is a function $d(x, y)$ which gives a real number representing the distance between both points. Such a function must satisfy these properties:

1. $d(x, y) \geq 0$ [no negative distances]
2. $d(x, y) = 0$ iif $x = y$
3. $d(x, y) = d(y, x)$ [symmetric]
4. $d(x, y) \leq d(x, z) + d(z, y)$ [triangle inequality]

Measuring distances

- Jaccard Distance
- Manhattan and Euclidean Distance
- Cosine Distance
- Edit Distance
- Hamming Distance

Jaccard Distance

Let x and y be two n -dimensional vectors

$$d(x, y) = 1 - J(x, y)$$

Manhattan and Euclidean Distance

Let x and y two n -dimensional vectors

$$d([x_1, x_2, \dots, x_n], [y_1, y_2, \dots, y_n]) = \sqrt[r]{\sum_{i=1}^n |x_i - y_i|^r}$$

if $r = 1$ (L_1 norm): Manhattan Distance

if $r = 2$ (L_2 norm): Euclidean Distance

if $r = \infty$ (L_{∞} norm): defined as the maximum of $|x_i - y_i|$ over all dimensions i .

Cosine Similarity and Cosine Distance

cosine similarity = $S_C(A, B) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}},$

norm 1 points to $\|\mathbf{A}\|$

scalar product points to $\mathbf{A} \cdot \mathbf{B}$

cosine distance = $D_C(A, B) := 1 - S_C(A, B).$

Cosine Similarity and Cosine Distance

An example: Let $x = [1, 2, 1]$ and $y = [2, 1, 1]$ two vectors.

Let's compute the cosine similarity:

$$\frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}, \quad [1*2 + 2*1 + 1*1] / \{ \sqrt{1^2 + 2^2 + 1^2} * \sqrt{2^2 + 1^2 + 1^2} \} =$$
$$3 / \{ \sqrt{6} * \sqrt{6} \} = 3/6 = \frac{1}{2}$$

Then the cosine distance is: $1 - \frac{1}{2} = \frac{1}{2}$

Edit Distance

This distance makes sense when points are strings. The Edit Distance between two strings $x = x_1x_2\dots x_n$ and $y = y_1y_2\dots y_m$ is the smallest number of insertions and deletions of single characters that will convert x to y .

Ad example: $x = abcde$, $y = acfdeg$. To convert x to y we need:

- delete b
- insert f after c
- insert g after e

The Edit distance between x and y is 3

Edit Distance

There is another way for computing Edit Distance. It is based on the **Longest Common Subsequence (LCS)**. An LCS of x and y is a string that is constructed by deleting positions from x and y.

The edit distance can be calculated as the length of x plus the length of y minus twice the length of their LCS.

An example: x = abcde, y = acfdeg. The LCS = acde

Edit Distance = $\text{lenght}(x) + \text{lenght}(y) - 2 * \text{lenght}(\text{LCS}(x, y)) = 5 + 6 - 2 * 4 = 3$

Hamming Distance

The Hamming Distance between two vectors is the number of component they differ to each other. This kind of distance is particularly useful when data are **represented by binary values**

An example: $x = [1, 0, 1, 0, 1]$ and $y = [1, 1, 1, 1, 0]$

The hamming distance is 3

Computing Scores in a Complete Search System

Academic Year 2025/2026

Luigi Santangelo
luigi.santangelo@unipv.it

University of Pavia
**Master Degree in
Computer Engineering**

Computing Scores

In a Information Retrieval System each document in the collection is represented by a vector.

The system computes the distance between the vector representing the Query and each Document, then shows K top documents ordered by **decreasing score**

The Cosine Score Algorithm

COSINESCORE(q)

```
1 float Scores[N] = 0
2 Initialize Length[N]
3 for each query term  $t$ 
4 do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5   for each pair( $d, tf_{t,d}$ ) in postings list
6     do Scores[d] +=  $wf_{t,d} \times w_{t,q}$ 
7 Read the array Length[d]
8 for each  $d$ 
9 do Scores[d] = Scores[d] / Length[d]
10 return Top K components of Scores[]
```

N = number of documents

Score[N] = vector for document scores (initialized to 0)

Length[N] = vector for document length

$$Length[i] = \sqrt{\sum_{j=1}^M \vec{V}_j(d_i)^2}$$

The Cosine Score Algorithm

COSINESCORE(q)

```
1  float Scores[N] = 0
2  Initialize Length[N]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5      for each pair( $d, tf_{t,d}$ ) in postings list
6      do Scores[d] +=  $wf_{t,d} \times w_{t,q}$ 
7  Read the array Length[d]
8  for each  $d$ 
9  do Scores[d] = Scores[d] / Length[d]
10 return Top K components of Scores[]
```

This process of adding in contributions one query term at a time is sometimes known as ***term-at-a-time*** scoring

The Cosine Score Algorithm

COSINESCORE(q)

```
1  float Scores[N] = 0
2  Initialize Length[N]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5    for each pair( $d, tf_{t,d}$ ) in postings list
6    do Scores[d] +=  $wf_{t,d} \times w_{t,q}$ 
7  Read the array Length[d]
8  for each  $d$ 
9  do Scores[d] = Scores[d] / Length[d]
10 return Top K components of Scores[]
```

For each term in the query, the weight $w_{t,q}$ is computed. Such a value is multiplied (line 6) with the weight of the term in the document.

The Cosine Score Algorithm

COSINESCORE(q)

```
1  float Scores[N] = 0
2  Initialize Length[N]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5    for each pair( $d, tf_{t,d}$ ) in postings list
6      do Scores[d] +=  $wf_{t,d} \times w_{t,q}$ 
7  Read the array Length[d]
8  for each  $d$ 
9  do Scores[d] = Scores[d] / Length[d]
10 return Top K components of Scores[]
```

For each term in the query, the posting list is retrieved, then we compute the weight of the term t in the document.

The weight can be either tf or $tf-idf$

The computed weight is stored in an **accumulator**

The Cosine Score Algorithm

COSINESCORE(q)

```
1  float Scores[N] = 0
2  Initialize Length[N]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5      for each pair( $d, tf_{t,d}$ ) in postings list
6      do Scores[d] +=  $wf_{t,d} \times w_{t,q}$ 
7  Read the array Length[d]
8  for each  $d$ 
9  do Scores[d] = Scores[d] / Length[d]
10 return Top K components of Scores[]
```

Scores are normalized

The Cosine Score Algorithm

COSINESCORE(q)

```
1  float Scores[N] = 0
2  Initialize Length[N]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5      for each pair( $d, tf_{t,d}$ ) in postings list
6      do Scores[d] +=  $wf_{t,d} \times w_{t,q}$ 
7  Read the array Length[d]
8  for each  $d$ 
9  do Scores[d] = Scores[d] / Length[d]
10 return Top K components of Scores[]
```

K documents are returned

The Cosine Score Algorithm

COSINESCORE(q)

```
1  float Scores[N] = 0
2  Initialize Length[N]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5    for each pair( $d, tf_{t,d}$ ) in postings list
6    do Scores[d] +=  $wf_{t,d} \times w_{t,q}$ 
7  Read the array Length[d]
8  for each  $d$ 
9  do Scores[d] = Scores[d] / Length[d]
10 return Top K components of Scores[]
```

$$\text{sim}(d_1, d_2) = \cos\theta =$$

$$\frac{\vec{V}(d_1) \cdot \vec{V}(d_2)}{\|\vec{V}(d_1)\| \|\vec{V}(d_2)\|}$$

$$\sum_{i=1}^M \vec{V}_i(d_1) \vec{V}_i(d_2)$$

This is an invariant because it is the same for all documents (being d_2 here the query)

The Cosine Score Algorithm: increasing performance

COSINESCORE(q)

```
1 float Scores[N] = 0
2 Initialize Length[N]
3 for each query term  $t$ 
4 do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5   for each pair( $d, tf_{t,d}$ ) in postings list
6     do Scores[d] +=  $wf_{t,d} \times w_{t,q}$ 
7 Read the array Length[d]
8 for each  $d$ 
9 do Scores[d] = Scores[d] / Length[d]
10 return Top K components of Scores[]
```

Instead of computing the length of the vector representing a document each time the CosineScore function is called, we can precompute this vector only once, during indexing time, because such value is independent to the query

$$Length[i] = \sqrt{\sum_{j=1}^M \vec{V}_j(d_i)^2}$$

The Cosine Score Algorithm: increasing performance

COSINESCORE(q)

```
1  float Scores[N] = 0
2  Initialize Length[N]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5    for each pair( $d, tf_{t,d}$ ) in postings list
6      do Scores[d] +=  $wf_{t,d} \leftarrow w_{t,q}$ 
7  Read the array Length[d]
8  for each  $d$ 
9  do Scores[d] = Scores[d] / Length[d]
10 return Top K components of Scores[]
```

To increase execution time, we avoid computing the weight each time.

Instead, we can store idf_t in the head of the postings for t . The value $tf_{t,d}$ instead is stored on each posting entry.

The Cosine Score Algorithm: increasing performance

COSINESCORE(q)

```
1  float Scores[N] = 0
2  Initialize Length[N]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5      for each pair( $d, tf_{t,d}$ ) in postings list
6      do Scores[d] +=  $wf_{t,d} \times w_{t,q}$ 
7  Read the array Length[d]
8  for each  $d$ 
9  do Scores[d] = Scores[d] / Length[d]
10 return Top K components of Scores[]
```

Sorting all documents
depending on the scores can
take much time.

An alternative is to use an Max
Heap

The Cosine Score Algorithm: the big problem!

COSINESCORE(q)

```
1  float  $Scores[N] = 0$ 
2  Initialize  $Length[N]$ 
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5    for each pair( $d, tf_{t,d}$ ) in postings list
6    do  $Scores[d] += w_{t,q} \times tf_{t,d}$ 
7  Read the array  $Length[d]$ 
8  for each  $d$ 
9  do  $Scores[d] = Scores[d] / Length[d]$ 
10 return Top  $K$  components of  $Scores[]$ 
```

For each term in the query, the corresponding posting list is retrieved and for each item the score is computed.

Computing the score for millions of documents takes a huge amount of time.

Increasing Performance

We would like to have a method for retrieving K documents that **are likely to be** among the K highest scoring documents for a query.

The idea is to eliminate all those documents from the collection that probably are not interesting.

To identify those documents to be eliminated, some **heuristics** are followed

Increasing Performance

Heuristic 1: We only consider documents containing terms whose *idf* exceeds a preset threshold. The idf value is stored in the **head of the Postings List**. This value is lower when the number of documents where the term appears is large. Low-idf terms are treated as **stop words** and do not contribute to scoring.

Increasing Performance

Heuristic 2: We only consider documents **that contain all (or many)** of the query terms. A danger of this scheme is that by requiring all query terms to be present in a document, we may end up with **fewer than K candidate** documents in the output

Increasing Performance

Heuristic 3: for each term t in the dictionary, we precompute a **list of r documents** having the highest weight (for example tf_idf) for that term t . This list is called **champion list**

We build a set A , given as the union of all champion lists for all terms.

Given a query q , we compute the score between the query q and the documents in A

Of course **r must be much greater than K** . One issue here is that the value r is set at **the time of index construction**, whereas K is application dependent and may not be available until the query is received.

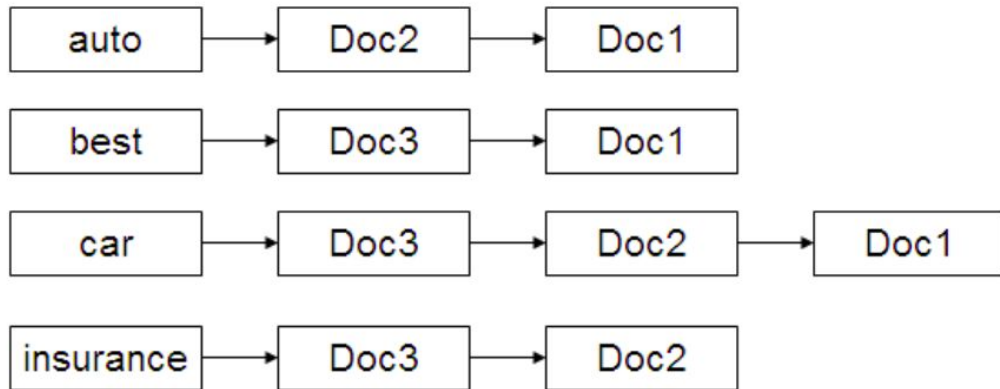
Increasing Performance

Heuristic 4: we assign a **static quality score** to each document $g(d)$.

Such a score is independent from the query and is usually assigned statically to a document. It is a **value between 0 and 1** and it represents the quality of the document.

We can **sort all Posting Lists**, not depending on the ID of the document but depending on the Static Quality Score:

Increasing Performance



In this example, the Posting Lists are sorted by static quality index:

$$g(\text{Doc1}) = 0.25$$

$$g(\text{Doc2}) = 0.5$$

$$g(\text{Doc3}) = 1$$

Increasing Performance

Similarly to the Heuristic 3, here too, we create a Champion List with the first r documents having the largest static quality score.

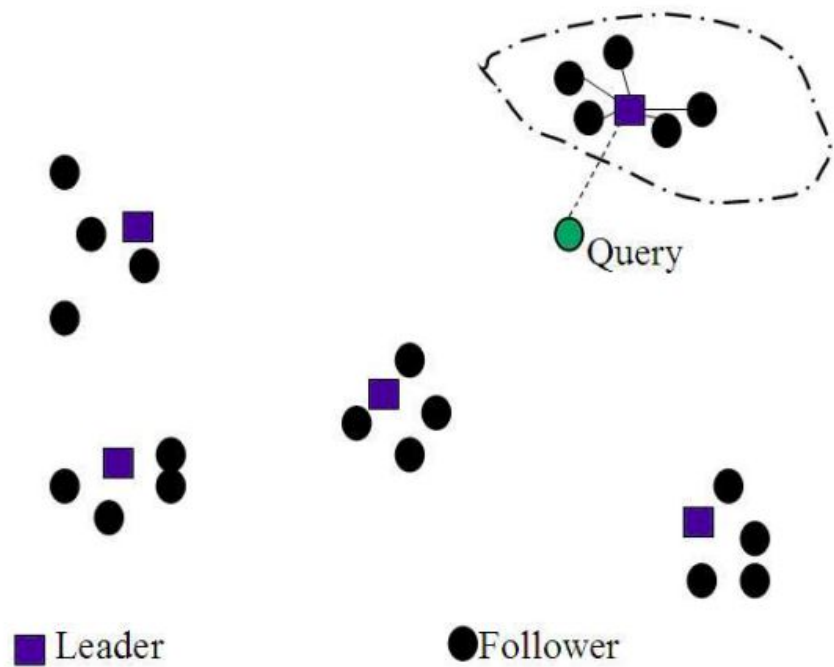
Then at query time, we only compute the score for all the documents belonging to the union of all Champion Lists. The score can be computed as a combination between the quality score and the cosine score

$$\text{net-score}(q, d) = g(d) + \frac{\vec{V}(q) \cdot \vec{V}(d)}{|\vec{V}(q)| |\vec{V}(d)|}$$

Increasing Performance

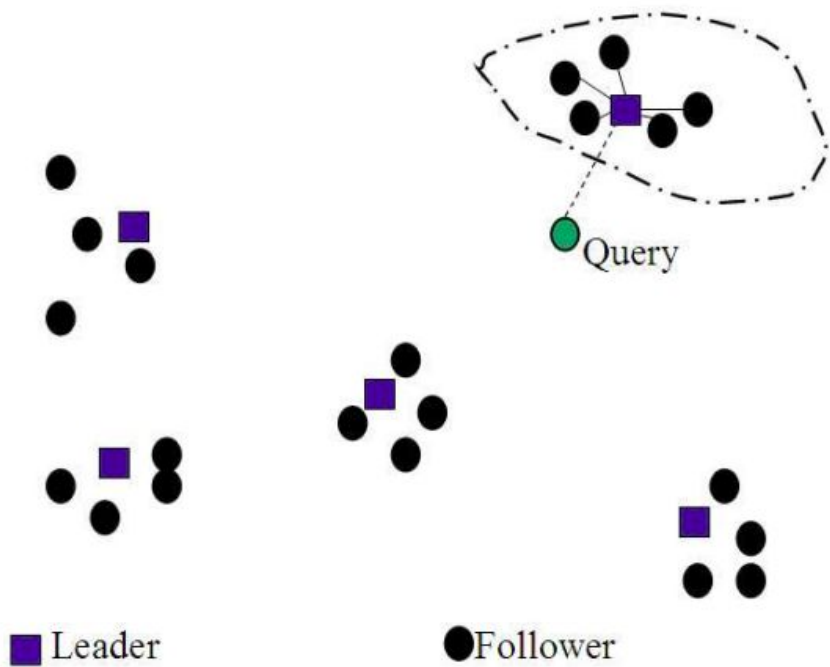
Heuristic 5: we have two preprocessing steps during which document vectors are clustered, then at query time we compute cosine score only with those documents belonging to some clusters as candidate.

This method is called **Cluster Pruning**



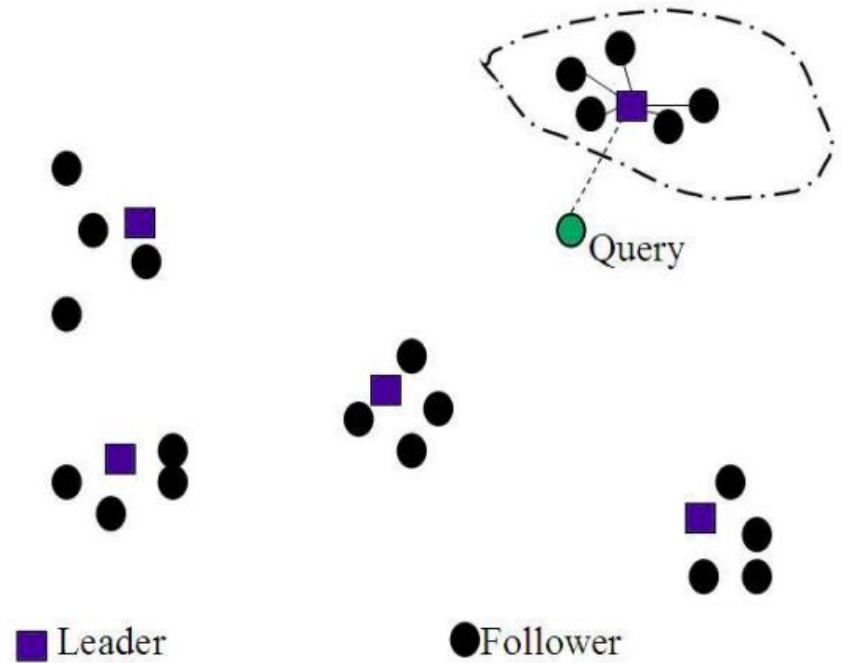
Increasing Performance

1. pick $\sqrt{N}$ documents at random from the collection. Call these **leaders**
2. For each document that is not a leader (called **follower**), we compute its nearest leader (this means computing cosine distance)



Increasing Performance

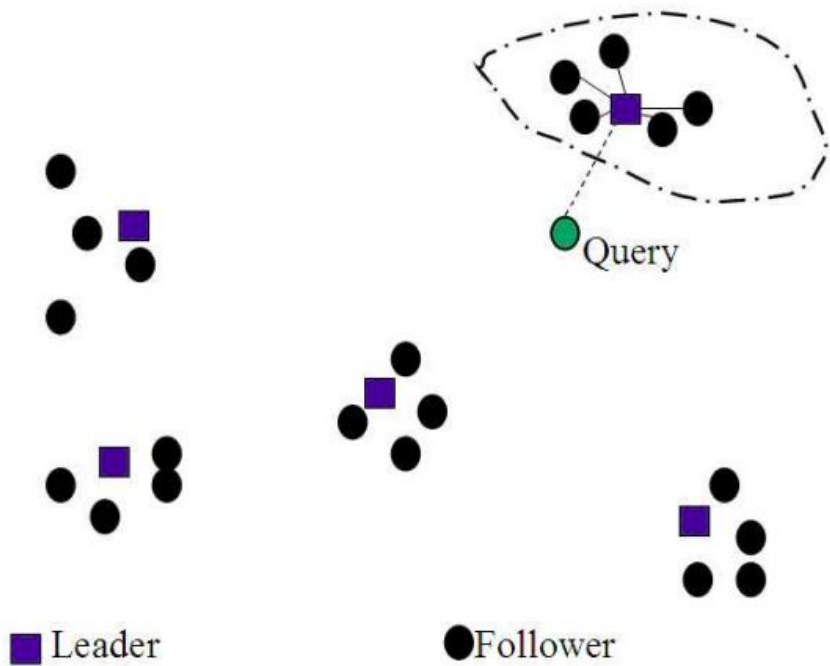
Being the leaders selected randomly,
we expect that number of followers of
each leader is approximately equals to
 $N / \text{sqrt}(N) = \text{sqrt}(N)$



Increasing Performance

Now at query time we can:

1. using the query q , find the leader L which is closest to q . This means that we need to compute the cosine a number of time equals to $\sqrt{N}$
2. build the set A will contain L and its followers
3. compute cosine score for all documents in this candidate set



Drawback of using heuristics

All heuristics aim to reduce the set of documents for which compute the cosine score.

To cut the collection, they all make use of parameter (for example r). Sometimes depending on the terms of the query and the parameters, **the number of documents to the candidate set A is lower than k**

Solution: **Tiered Index**

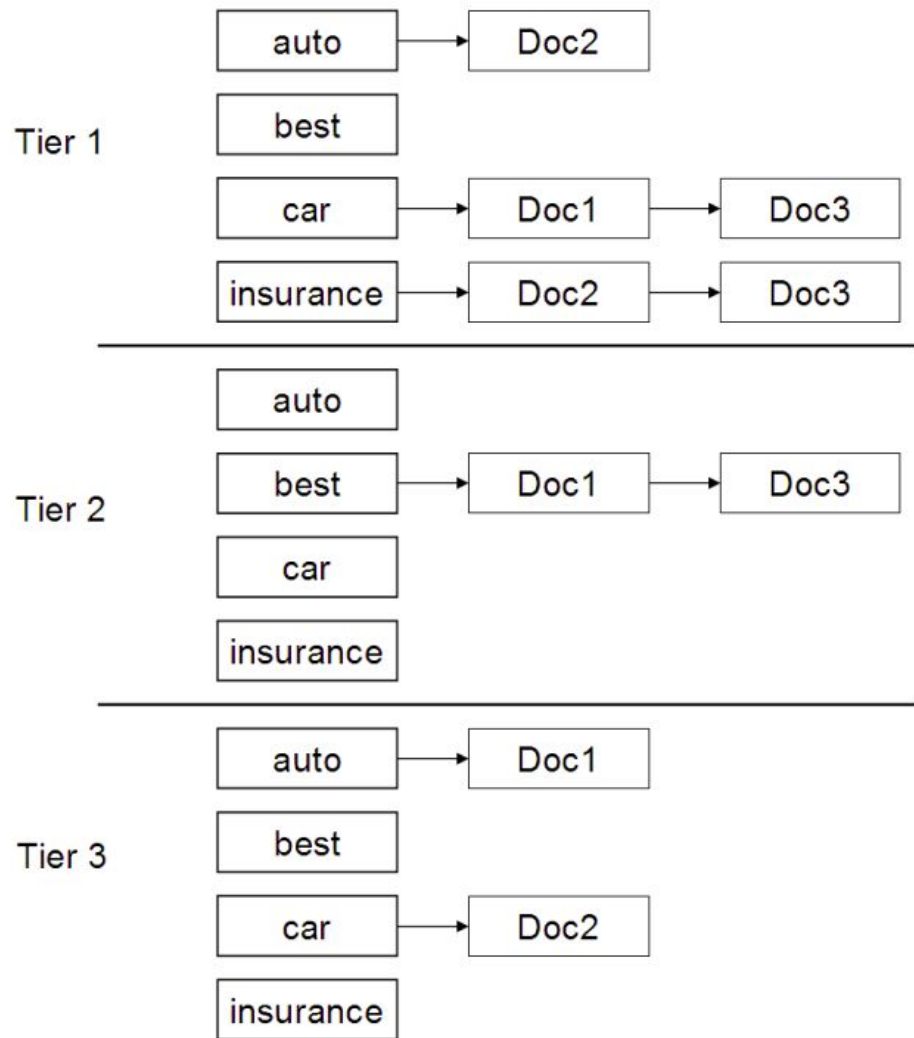
Tiered Index

The picture showed a 3-tiered Index.

Tier 1 indexes all documents having, for example, a `tf_idf` greater 20

Tier 2 indexes all documents having a `tf_idf` between 10 and 20

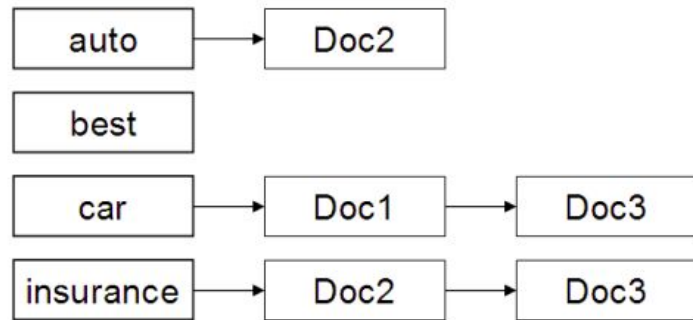
Tier 3 indexes all documents having a `tf_idf` lower than 10



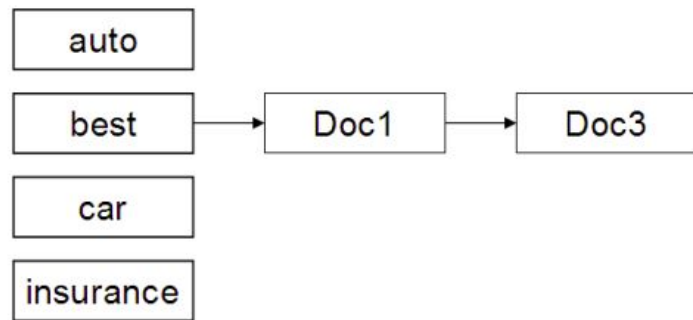
Tiered Index

If we fail to get K results from tier 1, query processing “falls back” to tier 2, and so on

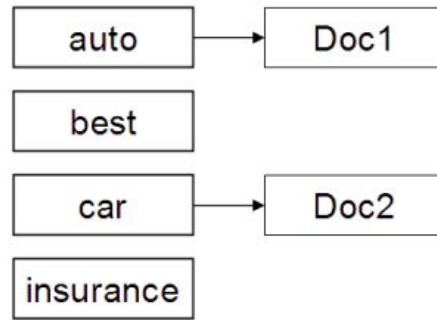
Tier 1



Tier 2

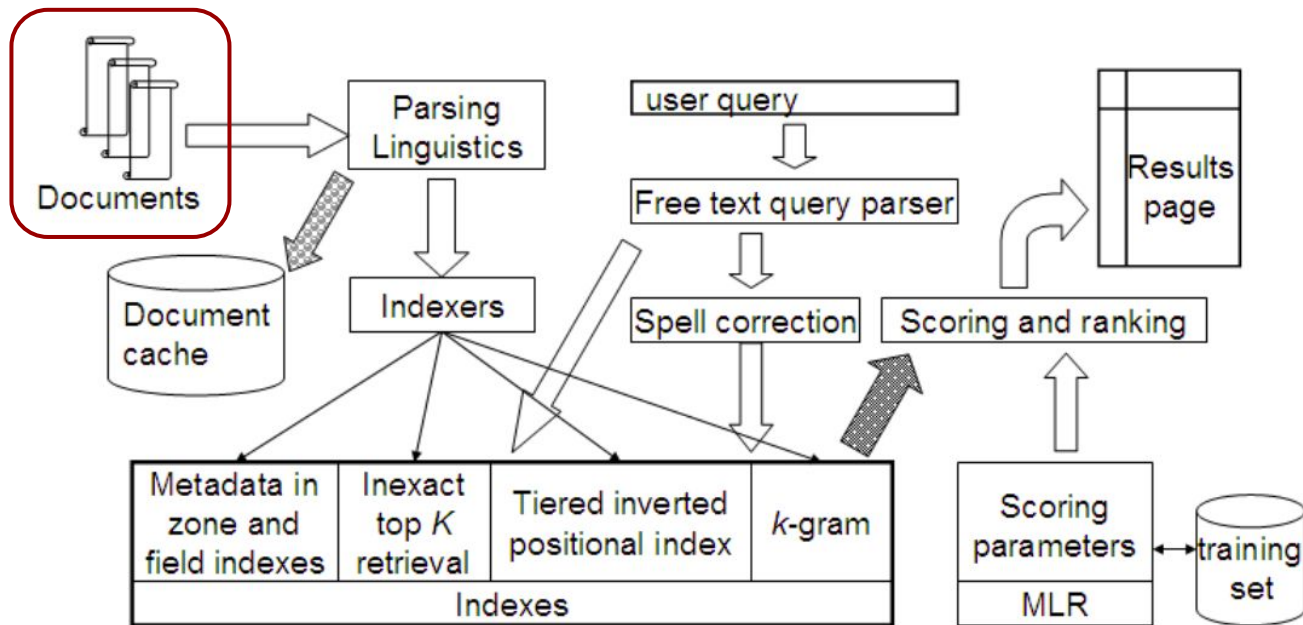


Tier 3



Components of an information retrieval system

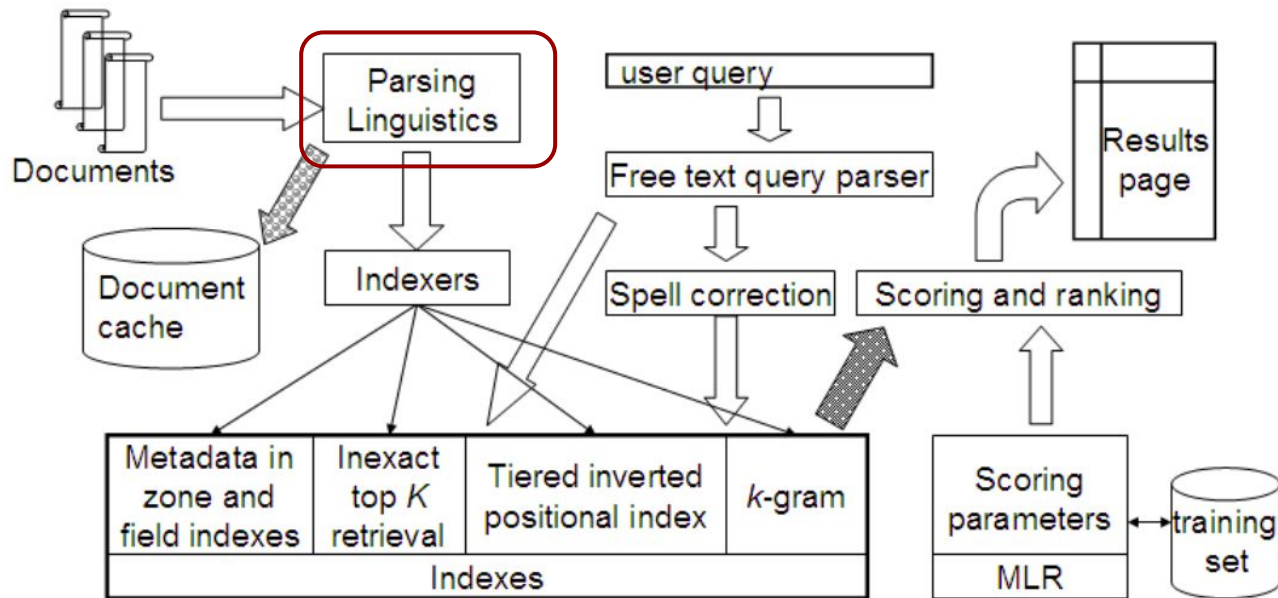
Putting them all together



Collection of documents to be parsed.

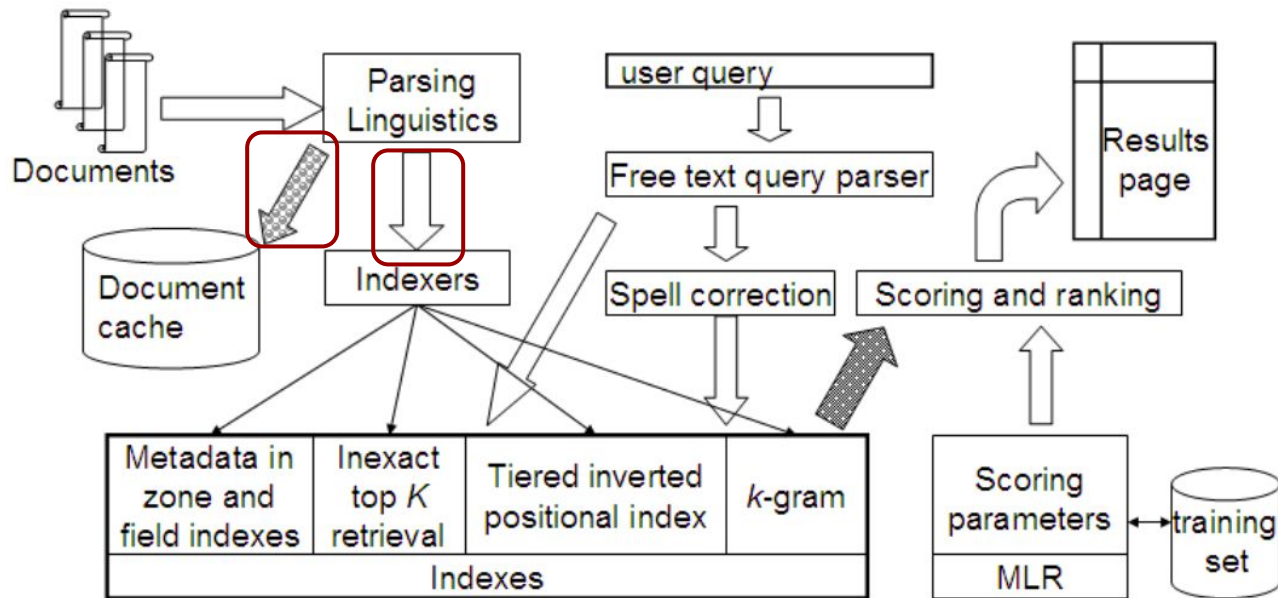
Text is extracted and sent to the Linguistic Preprocessor

Putting them all together



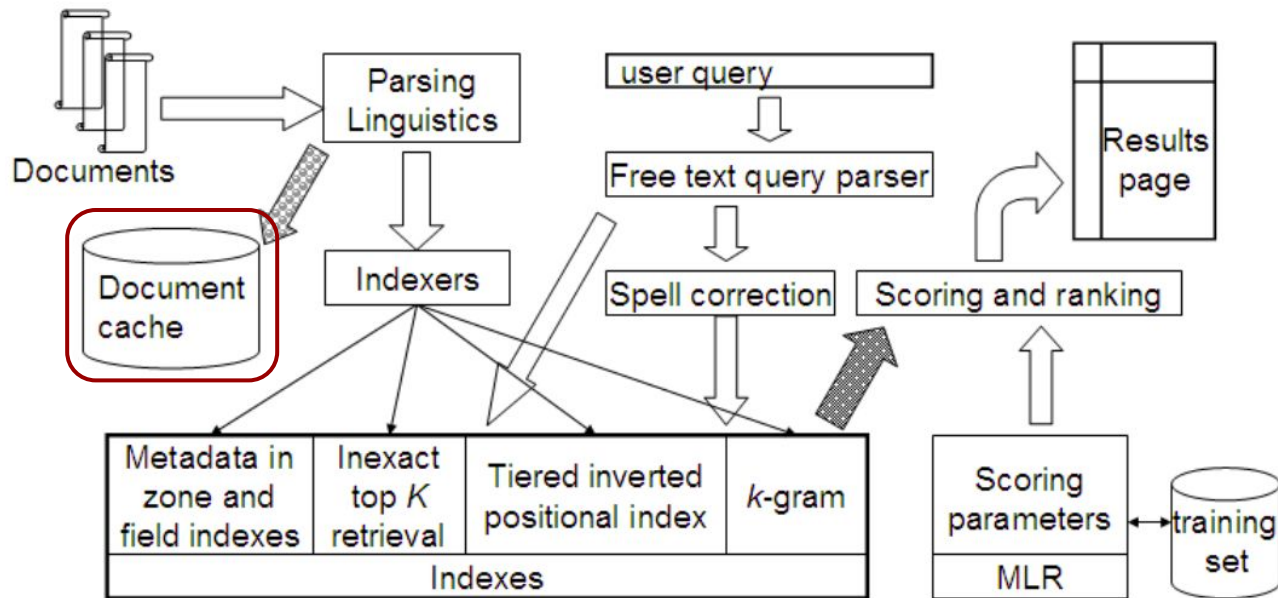
1. detect format
2. detect language
3. tokenization
4. stemming

Putting them all together



All tokens are sent to the indexers and to the document cache

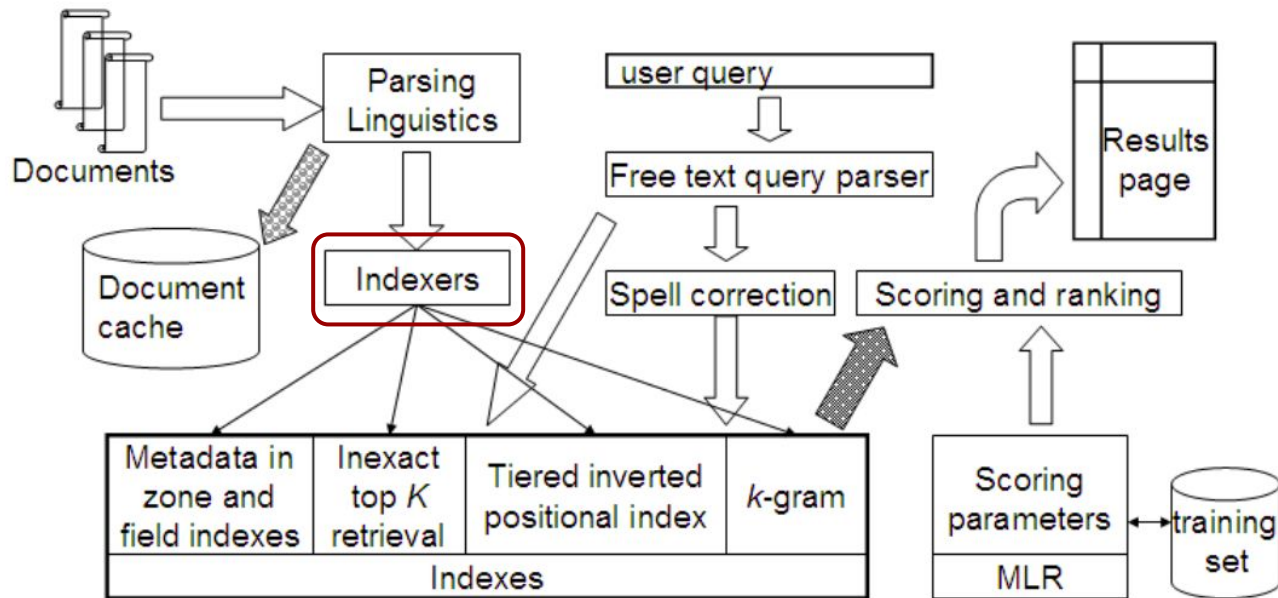
Putting them all together



The Document Cache keeps the parsed document. This is used to create **snippets** of text in the query results.

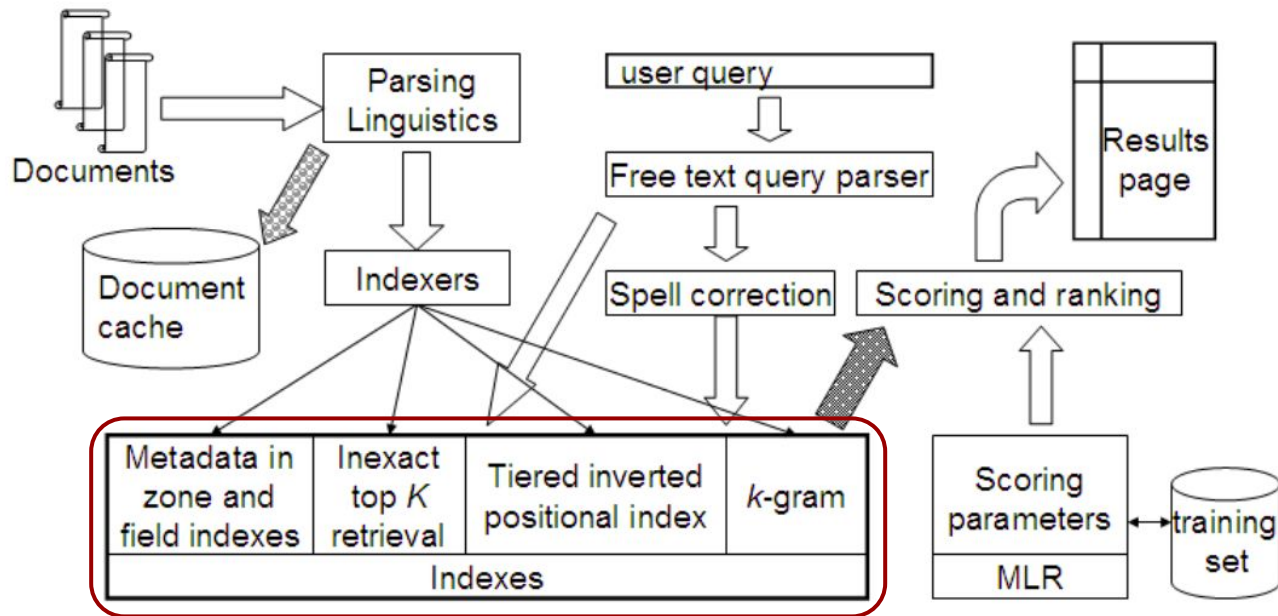
Snippet tries to give an explanation to the user of why the document matches the query

Putting them all together



Tokens are turned into Terms and are stored into the vocabulary. Posting Lists are created.

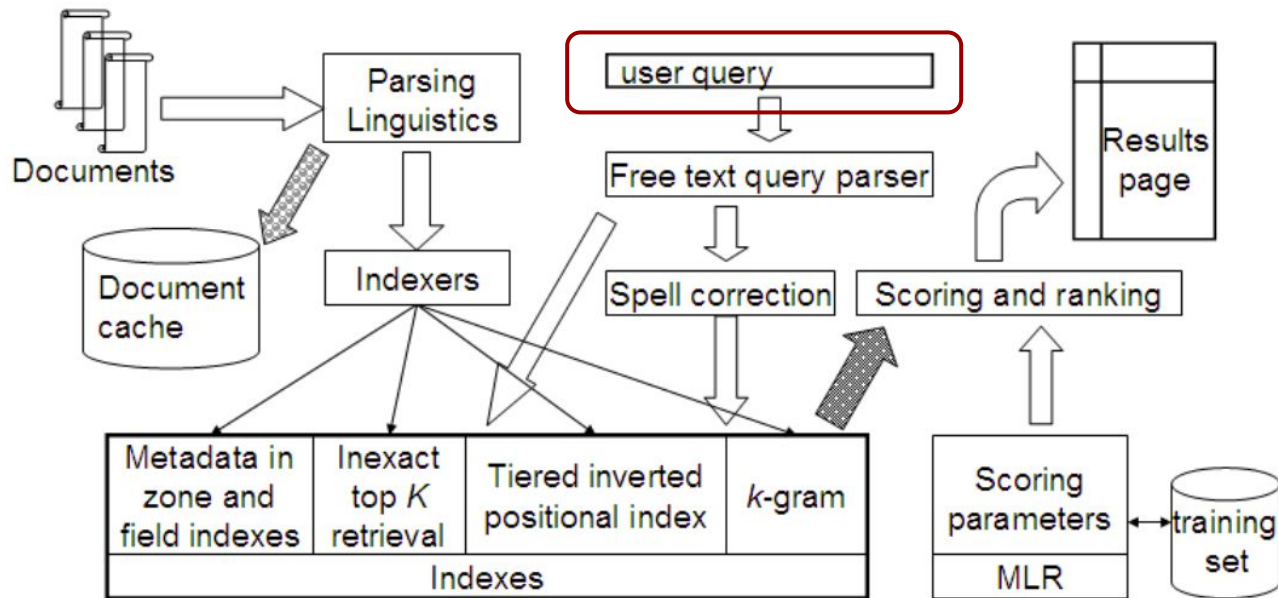
Putting them all together



Several Indexes are built:

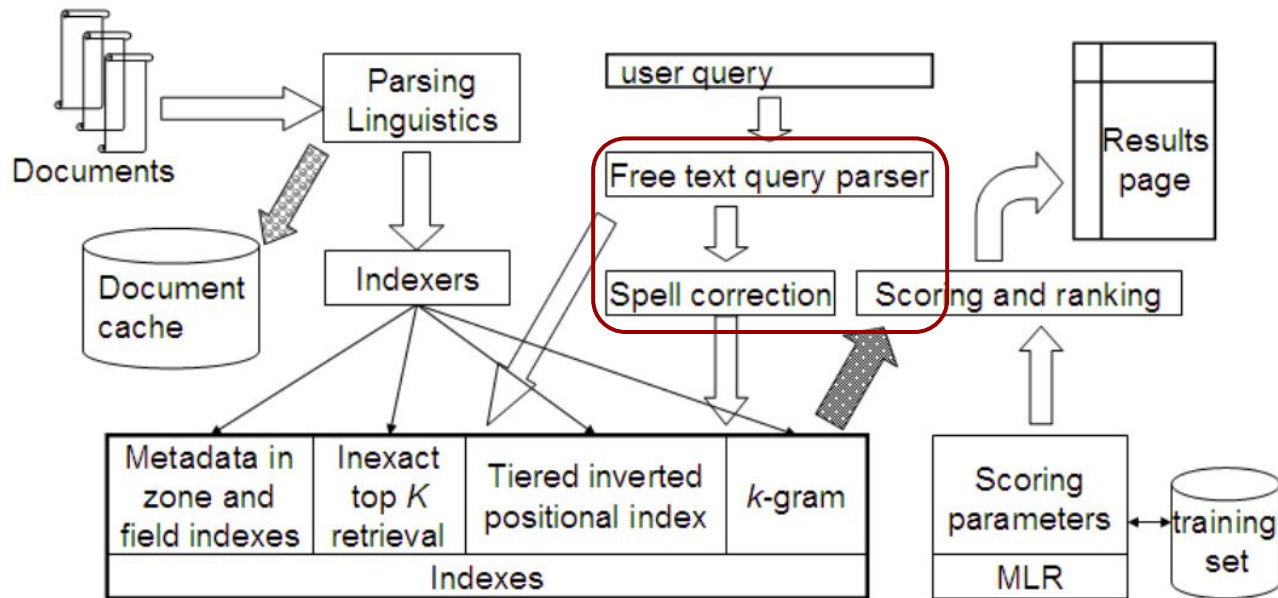
1. Zone Index
2. Field Index
3. Tiered Inverted Positional Index
4. Structures for accelerating inexact top-K retrieval
5. K-gram index

Putting them all together



User writes the query according to his information need

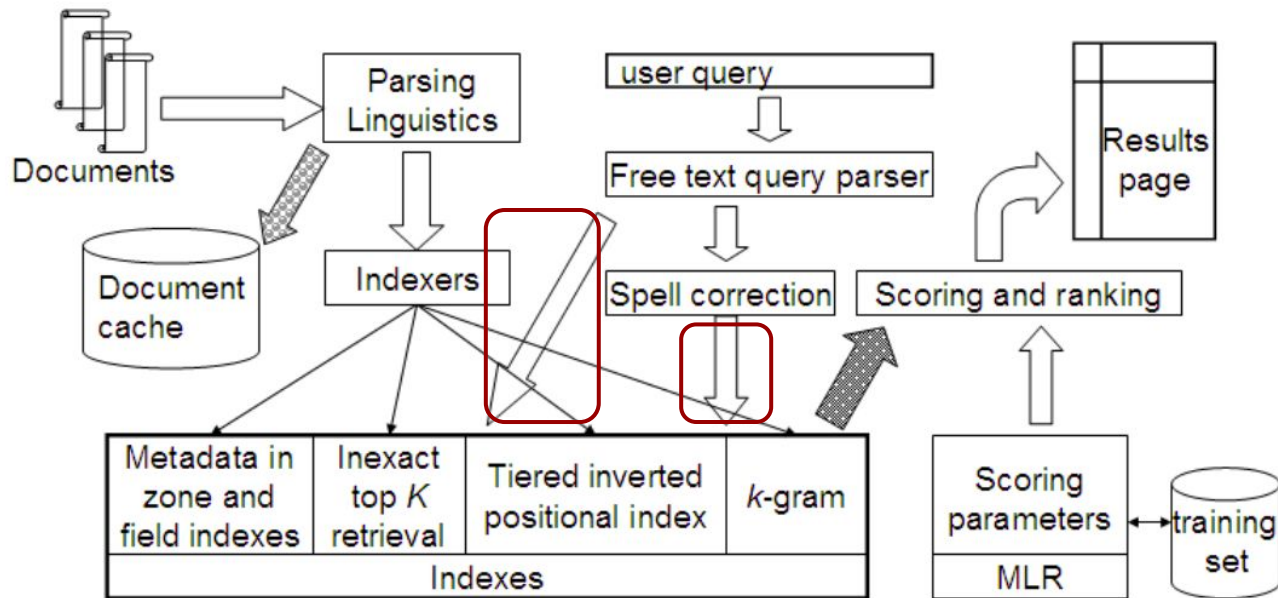
Putting them all together



The query is parsed,
tokens are extracted;

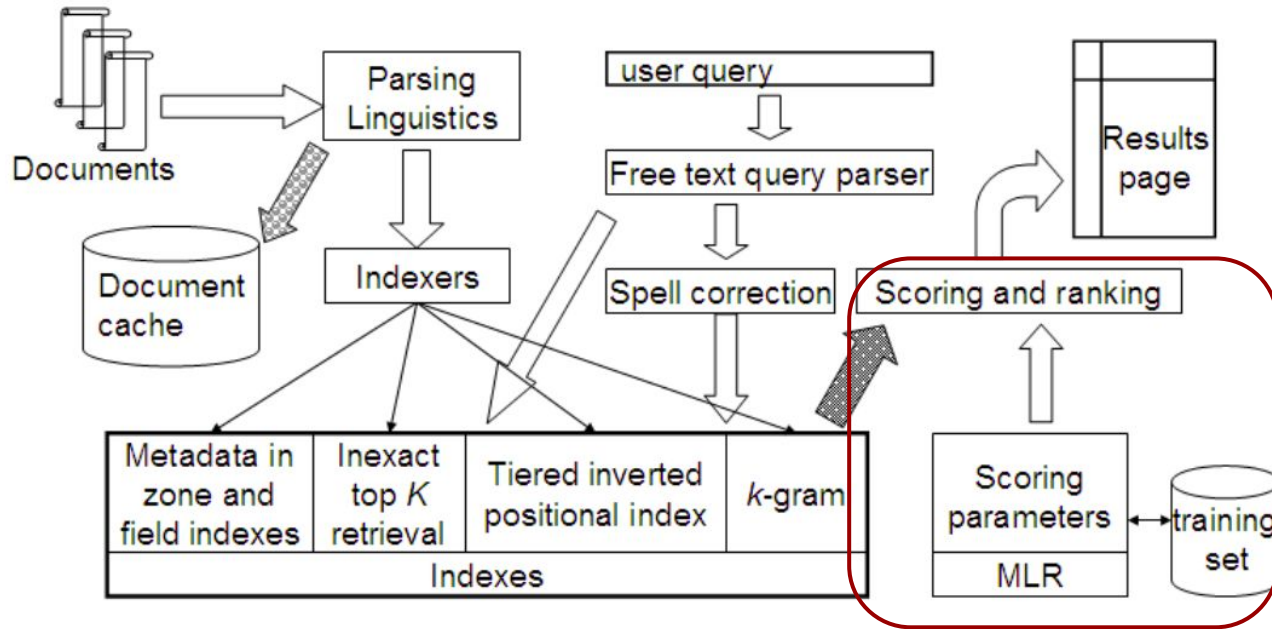
The Spell Correction
module may
optionally be invoked
only when the
original query fails
to retrieve enough
results

Putting them all together



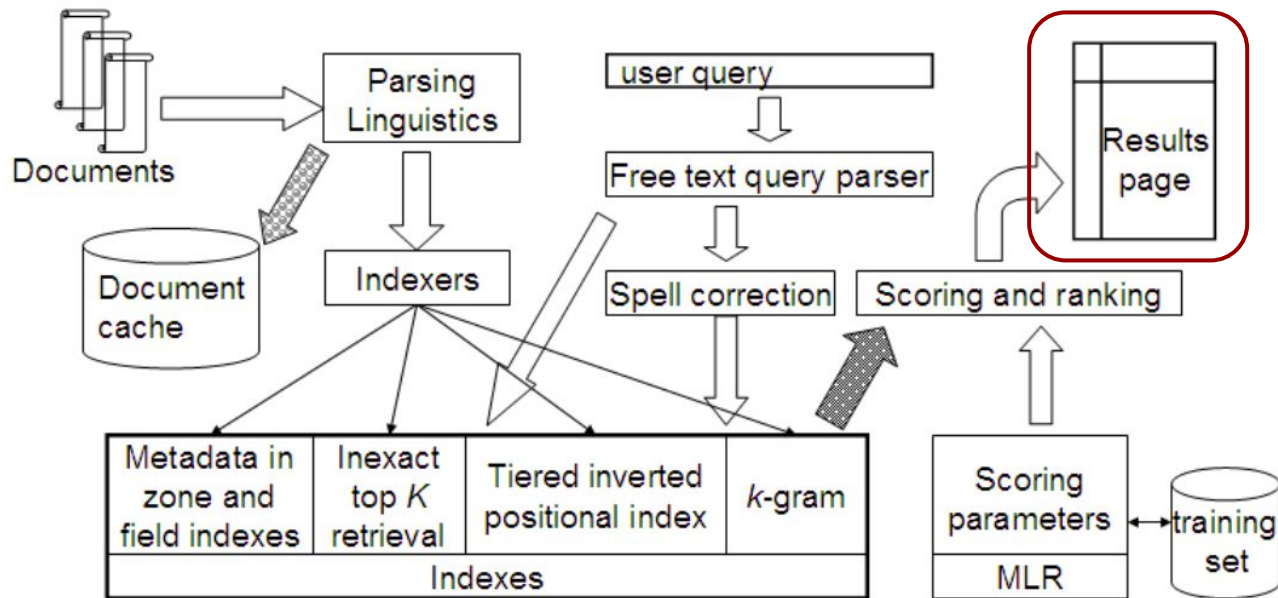
The query (or the corrected query) is used to retrieve documents

Putting them all together



Using the weight of the zones and fields, documents are scored and ranked. Zone weights are assigned using ML techniques

Putting them all together



Finally, these ranked documents are rendered as a results page

Web Crawling and Indexes

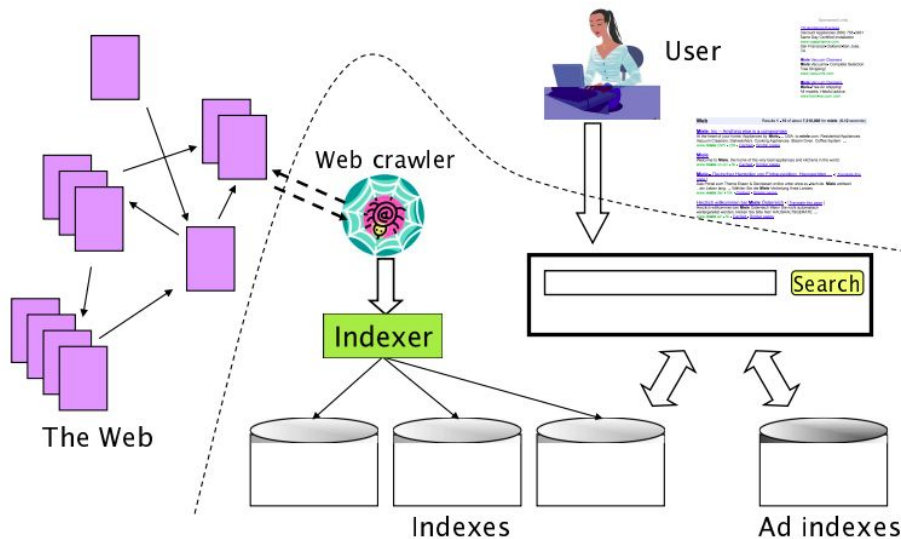
Academic Year 2025/2026

Luigi Santangelo
luigi.santangelo@unipv.it

University of Pavia
**Master Degree in
Computer Engineering**

Overview

- **Web Crawling** is the process by which we gather pages from the web in order to index them and support a Web Engine
- Web Crawler (also named **Spider**) is in charge of this job
- The complexity of the web makes such a job **very complex**



Features a crawler MUST provide

- Robustness: some Web Server might generate ***spider trap***, which are pages generated to mislead crawlers into getting stuck fetching an infinite number of pages in a particular domain. Not all traps are malicious, some of them are side effects of faults made by developers. Crawlers must **be resilient** to such traps;
- Politeness: Web servers have implicit and explicit policies **regulating the rate** at which a crawler can visit them (not too often, not too late). These policies must be respected

Features a crawler SHOULD provide

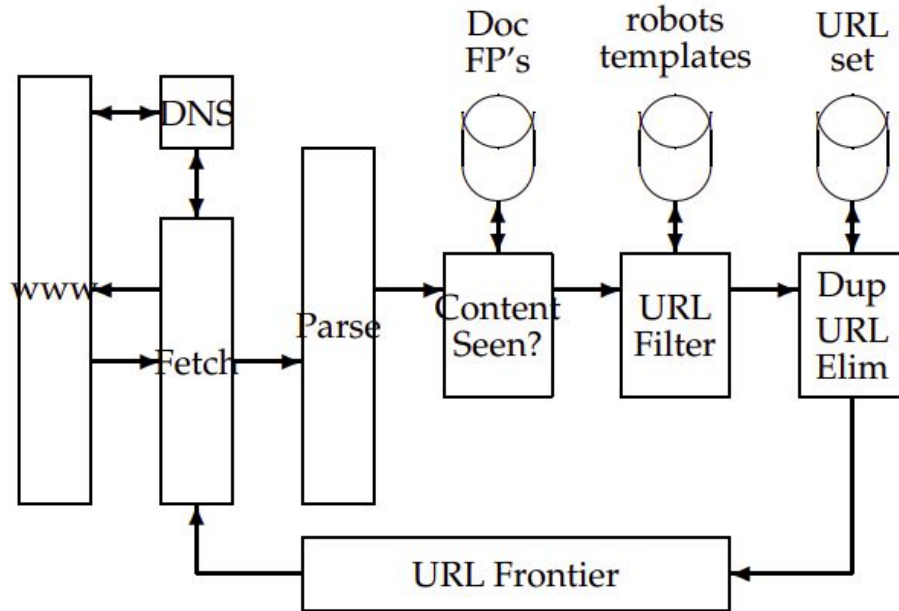
- Distributed: crawlers should be able to run gathering web pages in a distribute fashion
- Scalable: the crawler architecture should permit scaling up, by adding extra machines
- Performance and Efficiency: the crawler system should use efficiently all the resource such as bandwidth, processor, storage, ecc.

Features a crawler SHOULD provide

- Quality: the crawler should start **fetching useful pages first**
- Freshness: a crawler should gather pages with a frequency that approximate **the rate of change of the page**
- Extensible: the crawler should be easily extensible to work properly even with new data format, new fetch protocols. This demands that the crawler architecture be **modular**

The Crawler Architecture

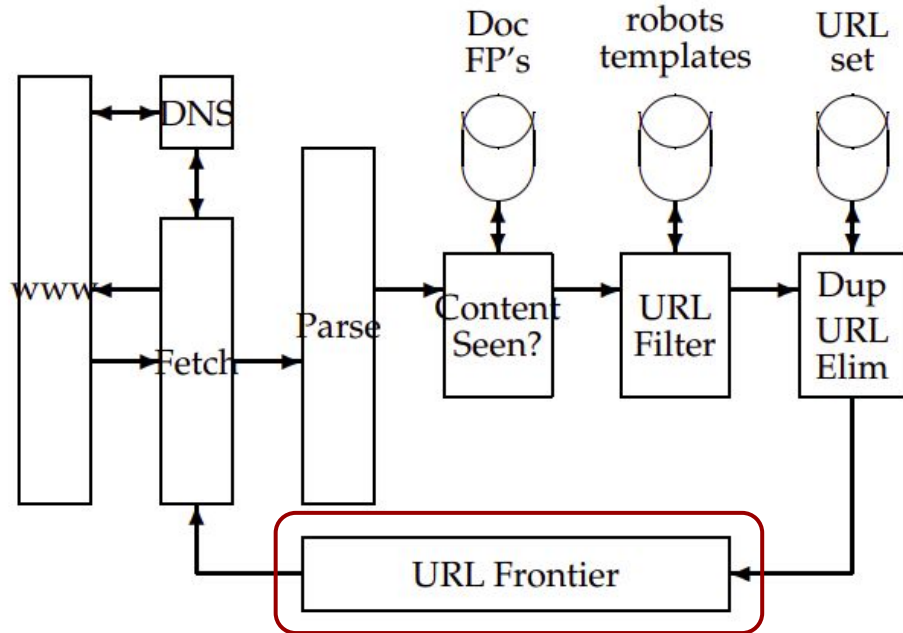
- URL Frontier
- DNS Resolution module
- Fetch module
- Parsing Module
- Duplicate Elimination Module



How a crawler works

The crawler owns a “**seed set**”, a small static list of possible websites to crawl

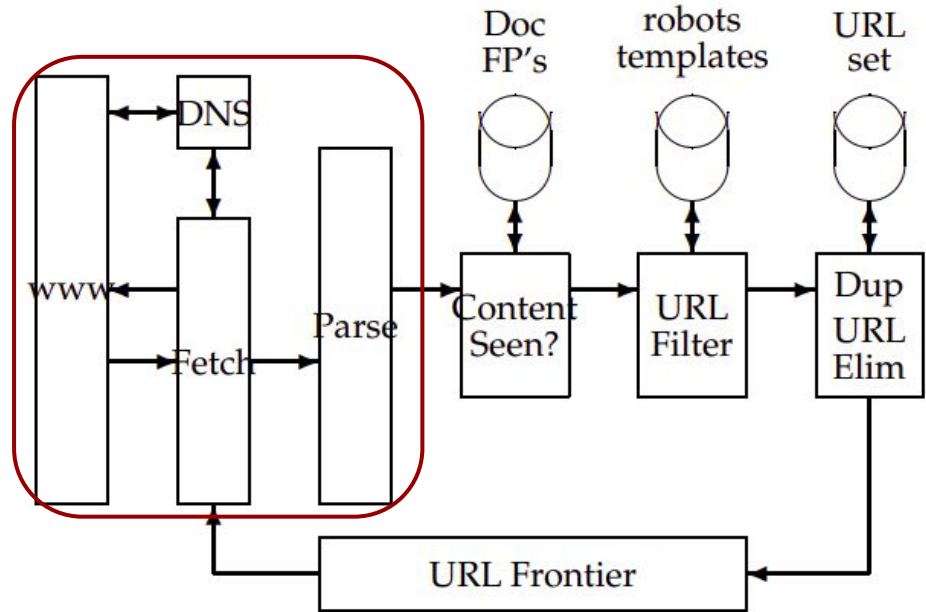
The seed set is then stored in the **URL Frontier**, representing all URL to be fetched



How a crawler works

The Crawler Thread takes a URL from the URL Frontier and fetch the web pages using HTTP protocol

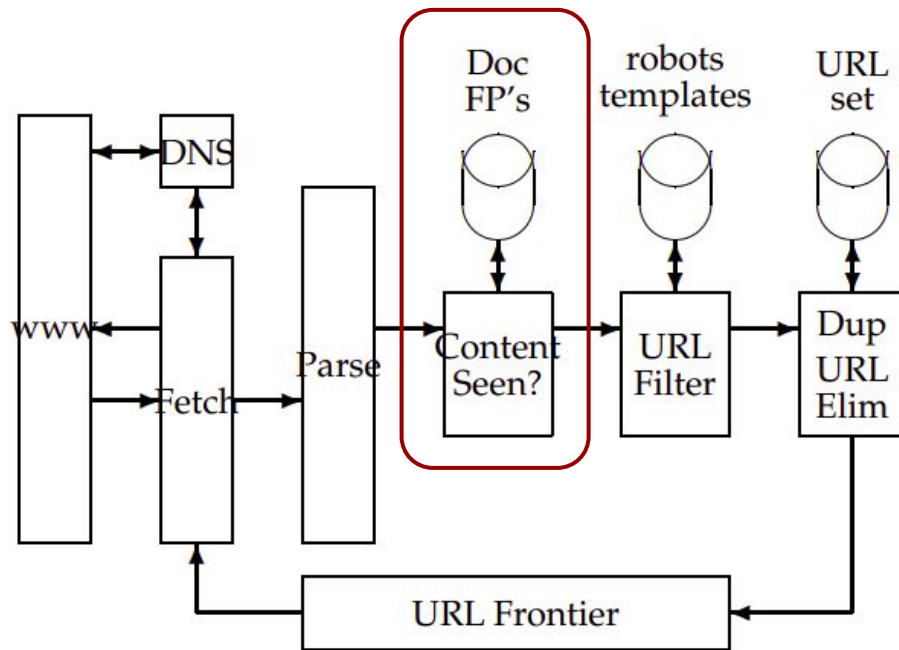
The page is stored in a temporary area, and then it is **parsed**. Text and Links are extracted and sent to the **Indexer**



How a crawler works

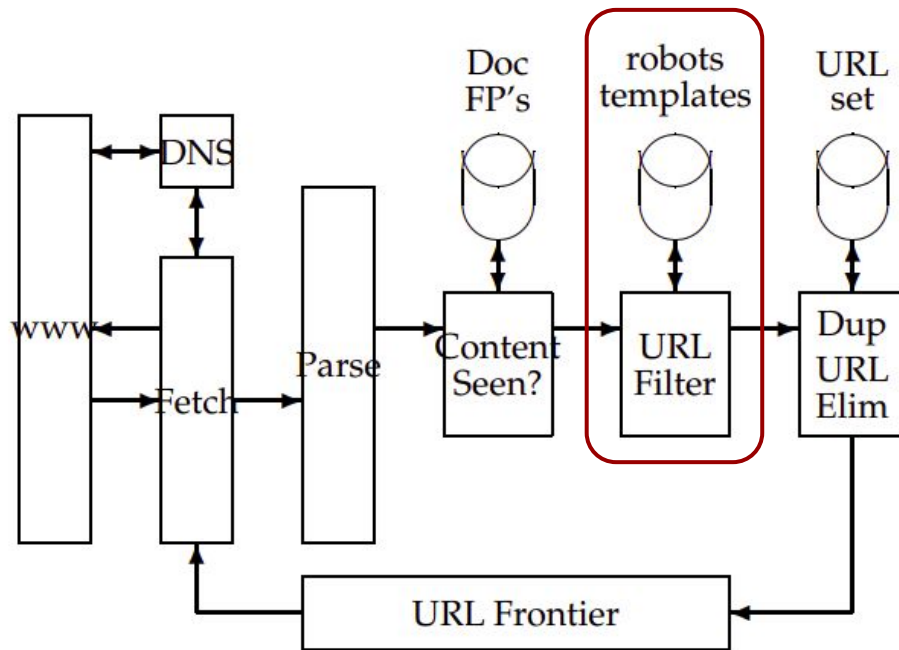
Text is analyzed to test whether a web page is similar to another already indexed (Doc FP's contains all the Fingerprint for all web pages already indexed)

Some implementations, instead of using **Fingerprint**, make use of **Shingles**



How a crawler works

All links are also tested to determine if they should be added to the URL frontier. The URL Filter is used to determine whether the extracted URL should be excluded from the frontier based on one of several tests. It depends on the results of **several rules** (see later)



How a crawler works

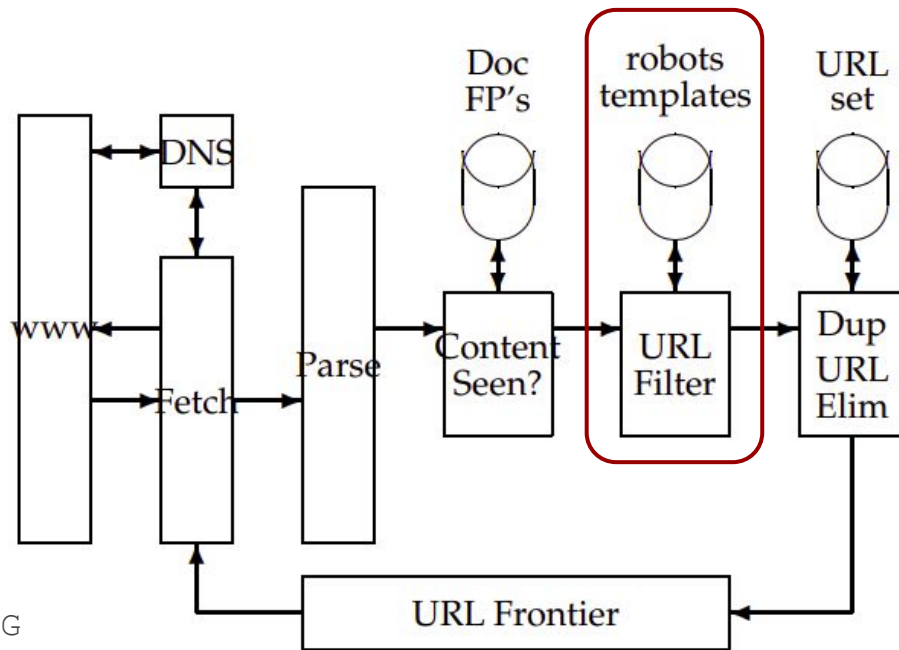
Once the URL has been selected to be put into the URL Frontiers, the link is **normalized**

For example, the following tag

```
<a href=
"/wiki/Wikipedia:General_disclaimer"
title="Wikipedia:General
disclaimer">Disclaimers</a>
```

is normalized in

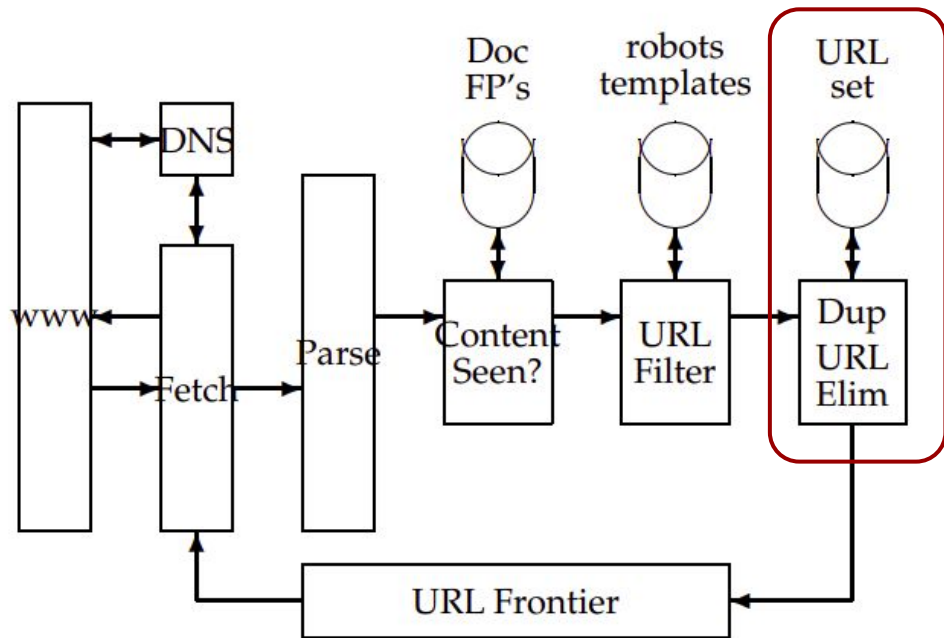
```
http://en.wikipedia.org/wiki/Wikipedia:G
eneral_disclaimer
```



How a crawler works

The Duplicate URL Elimination Module is in charge of verify if an URL is already in the URL Frontier or not. If it is, the URL is discard. Otherwise, it is added.

Being the number of possible URL too high, **an efficient way of detecting duplicate URL** should be used.

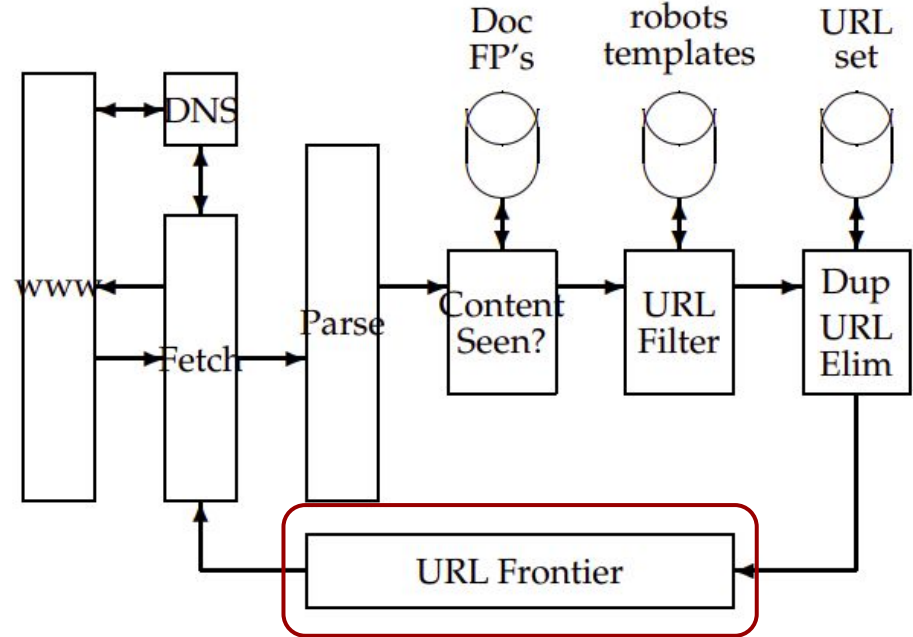


How a crawler works

Finally the link is put into the URL Frontier only if the same normalized link has not been already **stored into the URL Frontier**

When the link is stored in the URL Frontier, **a priority** is assigned.

Links having an **higher priority** will be fetched before those having a lower priority



Filtering URL

As we saw previously, the URL Filter is a component of a Crawling system which is used to determine whether the extracted URL should be excluded from the frontier based on rules.

Rules can be:

- **domain-based rules:** for example all .com domain
- **robots-based rules:** some web pages can be kept off-limit to crawling. This can be done making the file robots.txt available to the search engine

Robot Exclusion Protocol

```
User-agent: *  
Disallow: /yoursite/temp/
```

```
User-agent: searchengine  
Disallow:
```

In this example we do not allow any robot to visit any page under /yoursite/temp, except for the robot called "searchengine"

Multithreading

Crawling systems are usually developed in **multithreading or multiprocessing** fashion, allowing them to work even in distributed system (shared memory or distributed memory).

One or more threads or processes implement each component

But there are also some dedicated threads/processes

- logging
- statistics
- URL frontier snapshot (for crawler failure)

Distributing the crawler

Modern Crawling Systems are also **geographically distributed**.

How the whole system can manage fetching operations?

URL are scattered among all distributed processes

The rules for splitting are different:

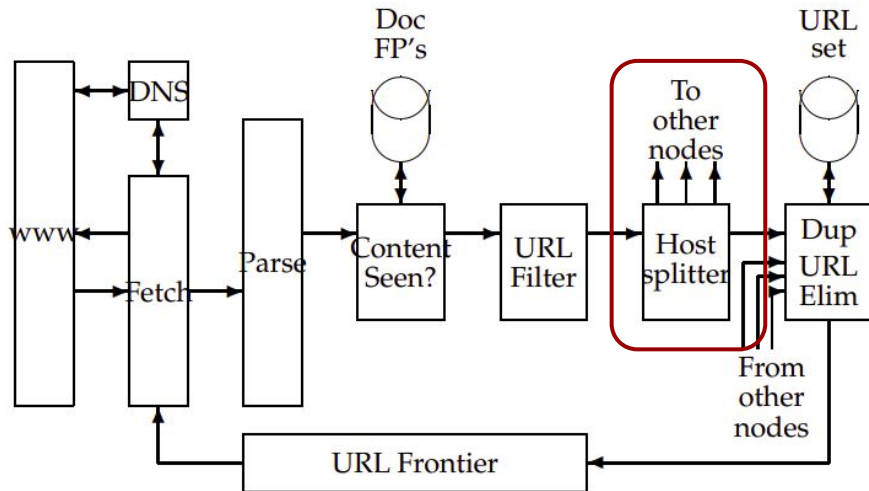
- domain based
- URL hashing

Distributing the crawler

A new component is added: **Host Splitter**

The output of the Host Splitter goes to the Duplicate URL Eliminator block of the local system or in the remote nodes

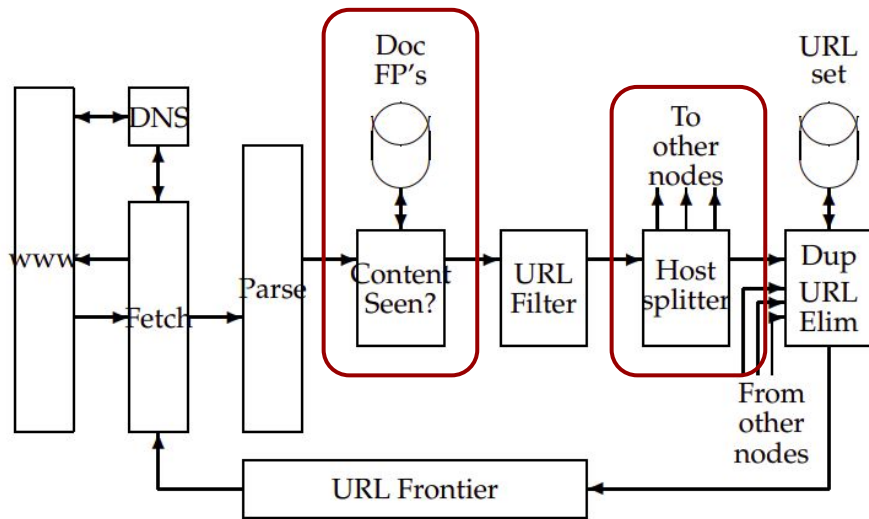
Communication between nodes happens usually using **JSON** and **REST WS**



Distributing the crawler

Introducing the Host Splitter module makes the module “Content Seen” more complex

The same content might appear **on different web servers managed by different remote threads**



URL Frontier

URL Frontier contains all the URL that should be still processed

The next URL to be processed **depends on the priority** index assigned to each URL

The priority index is computed as a combination of two factors:

- pages that change frequently should be prioritized for frequent crawling
- high quality pages

URL Frontier

Building a URL frontier just as a priority queue does not work.

Let's think about a web page having several URL links to the same host. All of them will have the same priority.

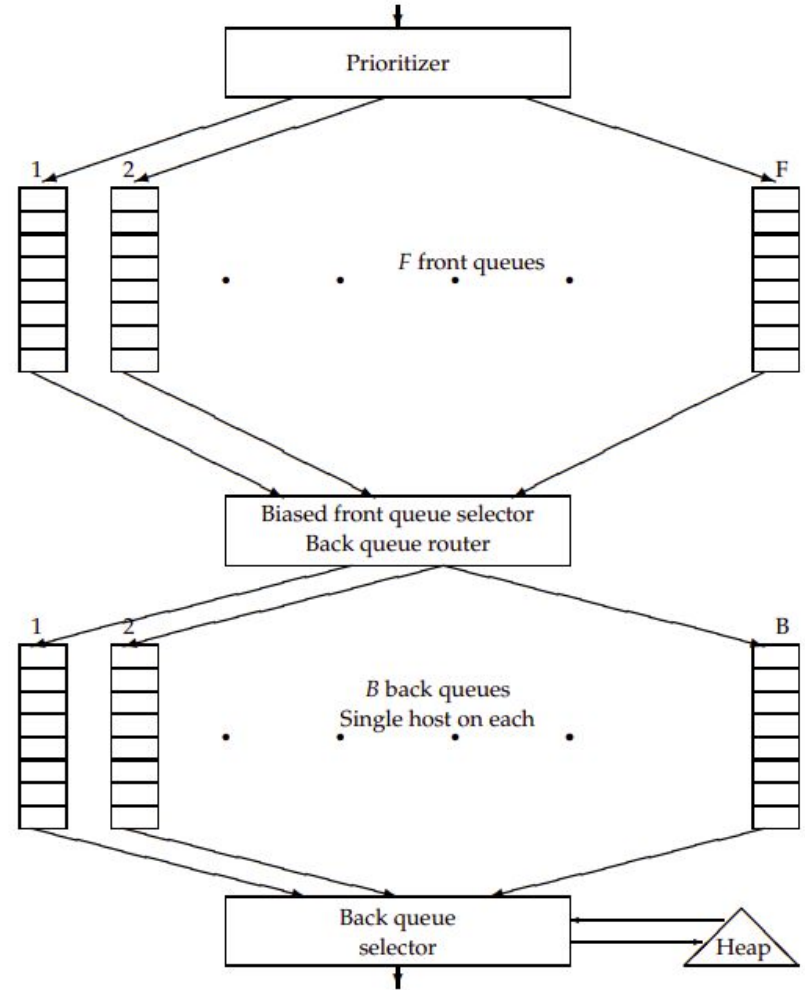
This might result in a **burst of fetch requests** to the same host. This violates the **politeness principle**

Solution: insert a **gap between successive fetch** requests to a host that is larger than the time taken for the most recent fetch

URL Frontier

Aims of this implementation:

1. only one connection is open at a time to any host
2. a waiting time of a few seconds occurs between successive requests to a host
3. high-priority pages are crawled preferentially



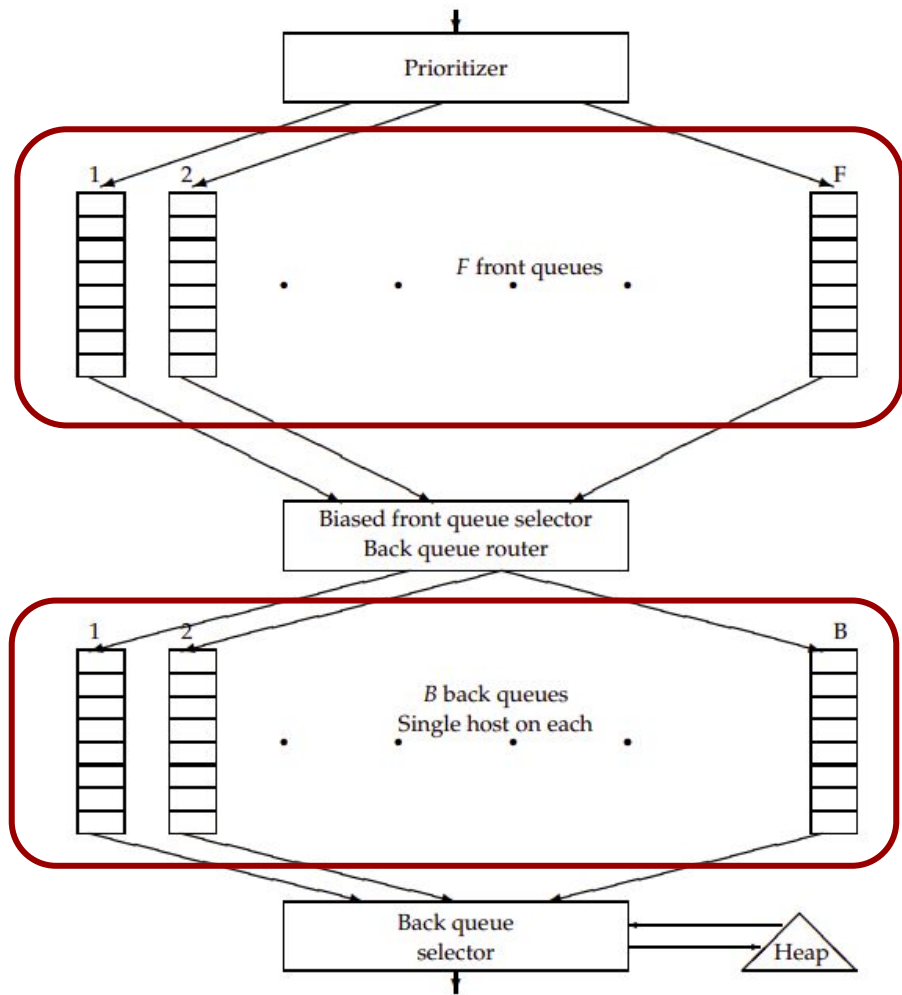
URL Frontier

Two Modules

1. F Front Queues (for prioritization)
2. B Back Queues (for politeness)

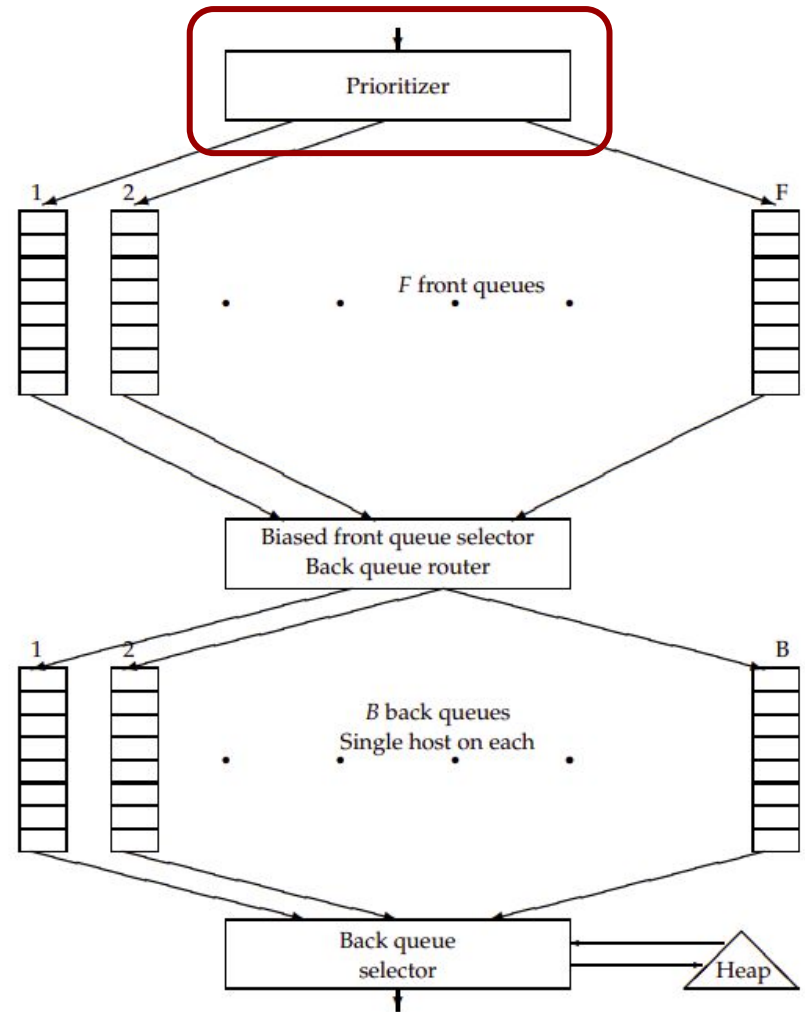
The **value of B and F** depends on the implementation

Both are **FIFO queues**



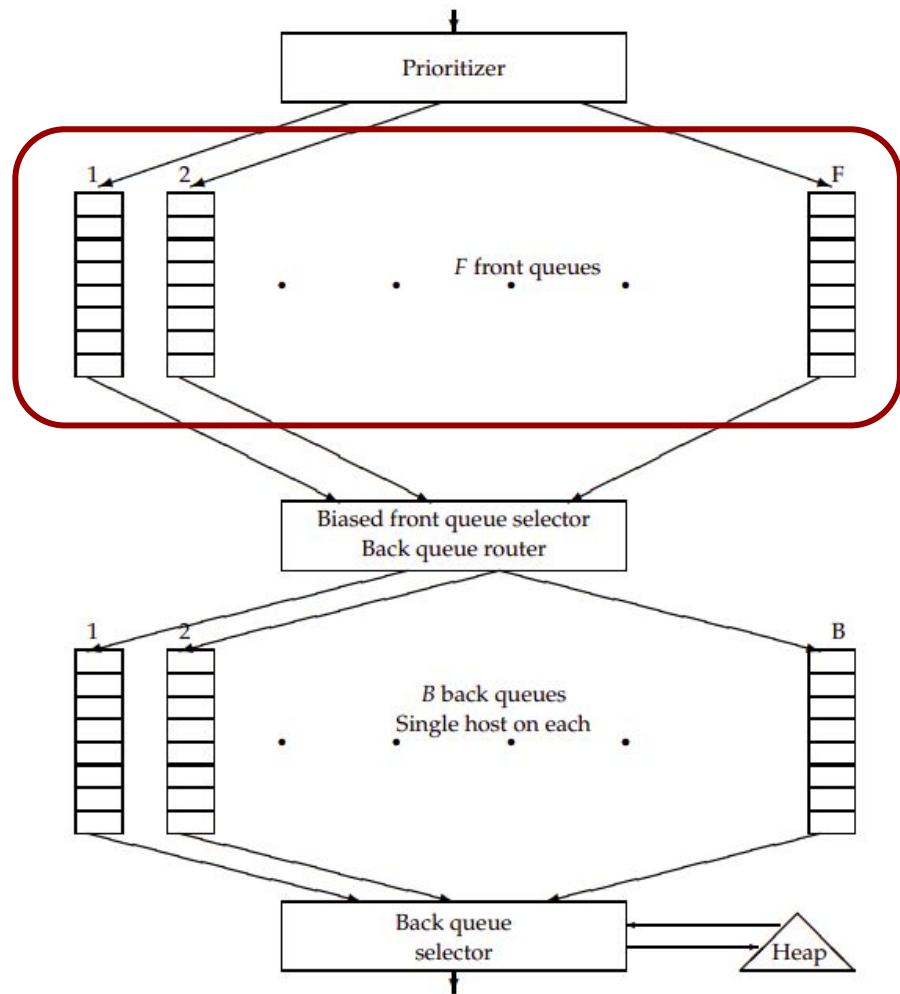
URL Frontier

- Each URL is assigned to an integer
- The priority index ranges between 1 and F
- The priority depends mainly from the **frequency of updates**
- Other heuristics can be applied (for example, if the URL refers to a **news** web site, the priority is always high)



URL Frontier

- When a URL is assigned to the index i , it is queued in the Front Queue i
- A queue can contains URL referring to **different hosts** (but all of them having the **same priority**)

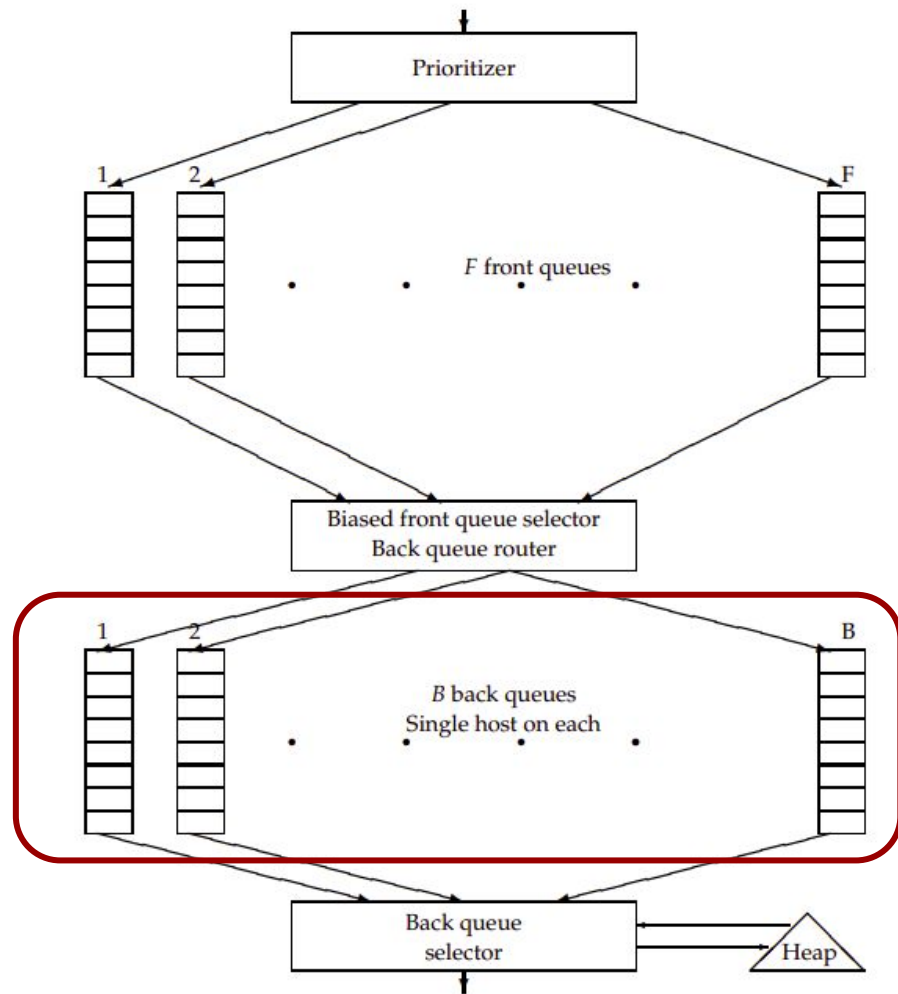


URL Frontier

The B queues maintain **two invariants**:

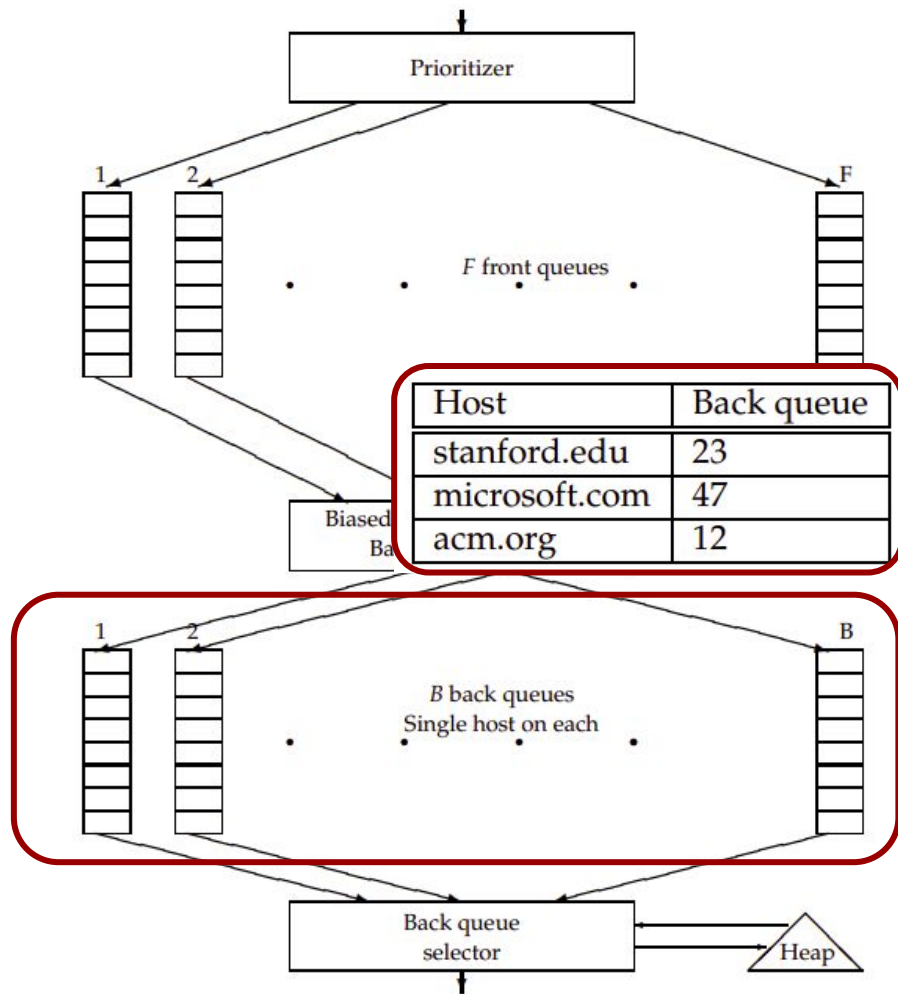
- all B queues are always non empty
- all B queues contain only links from the same host

Note that the value of B is much **smaller** than the **number of hosts**, but this is not a problem.



URL Frontier

An **auxiliary table** for mapping hosts to the Back Queues must be managed. By looking up the host into the mapping table, it would be possible to identify the queue containing all URLs referring to that host



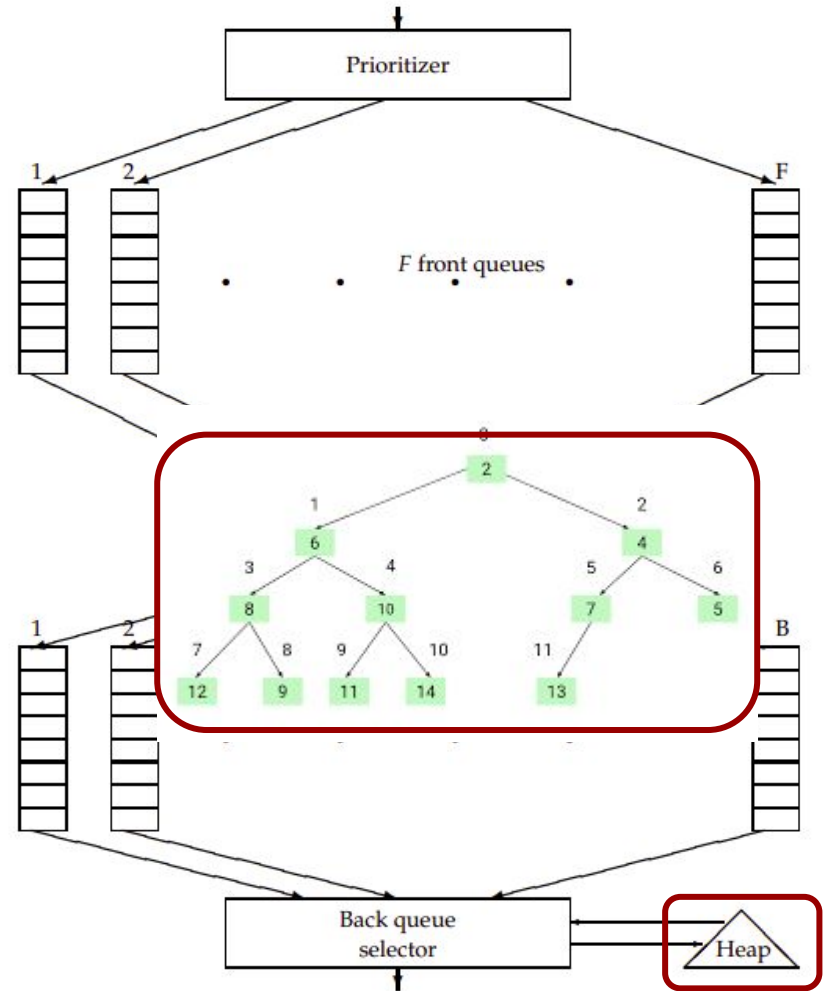
URL Frontier

There is also a **min-heap** with an entry for each back queue.

It is implemented like a binary tree

Each node contains at least two values:

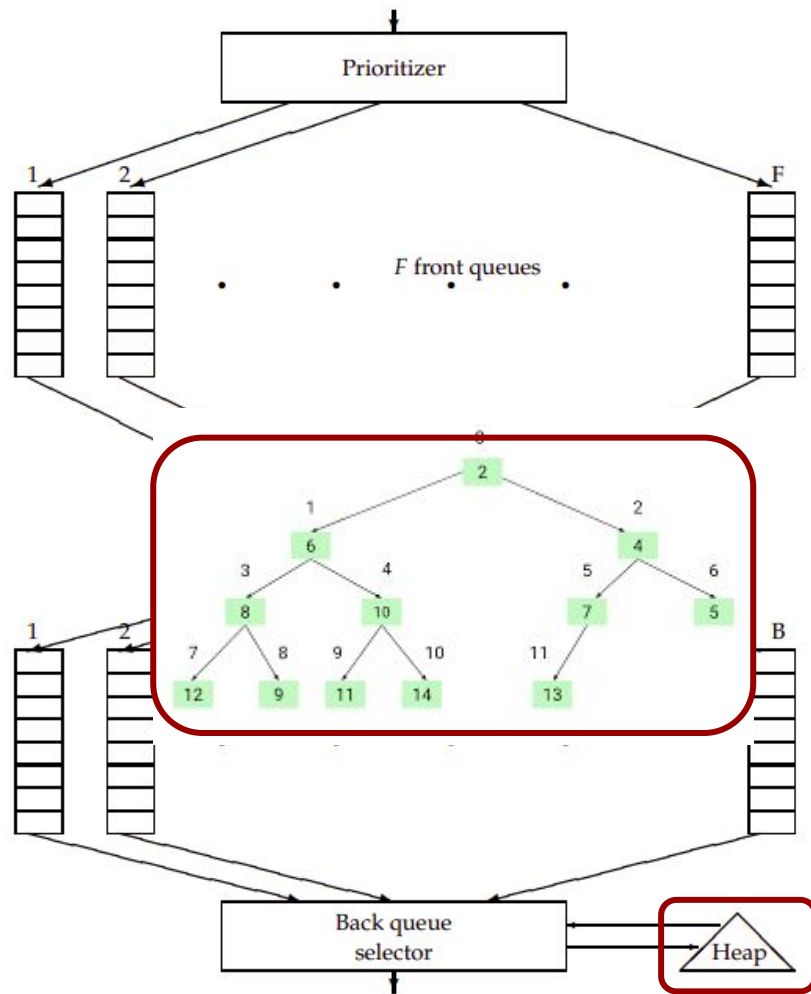
1. the corresponding queue represented by the node
2. the time when the host should be contacted back again



URL Frontier

The **heap is sorted** depending on the time the host can be contacted again.

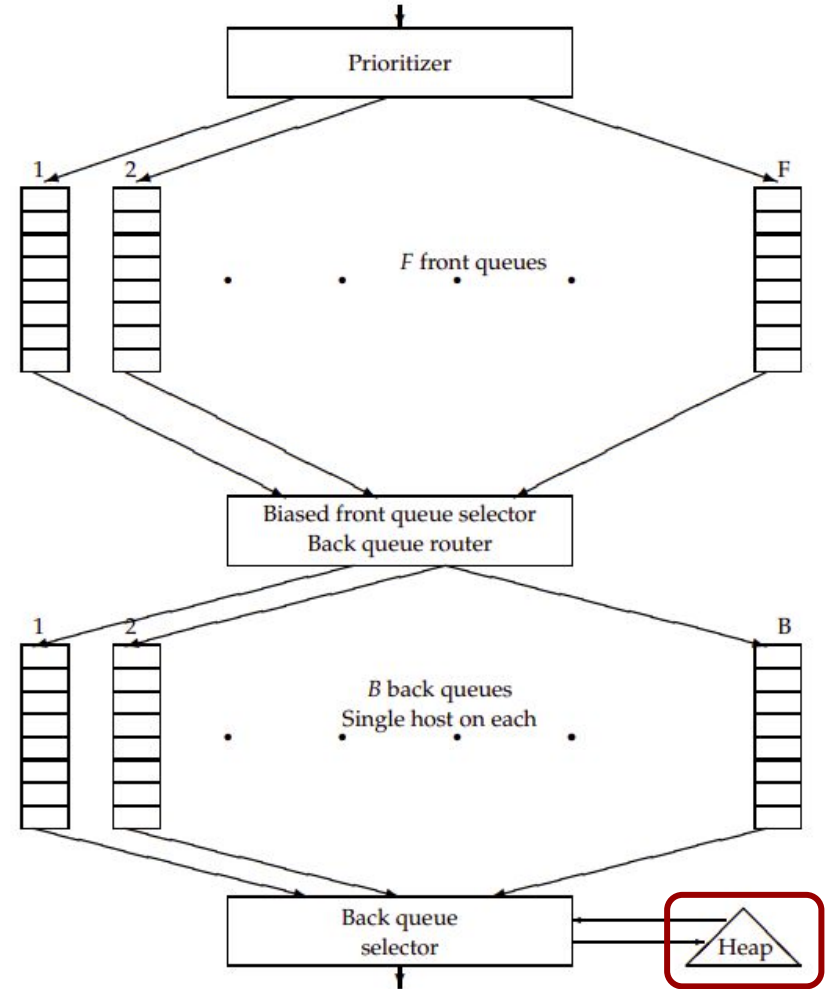
Being a min-heap, the root of the heap refers to **the host to be contacted the first**.



URL Frontier

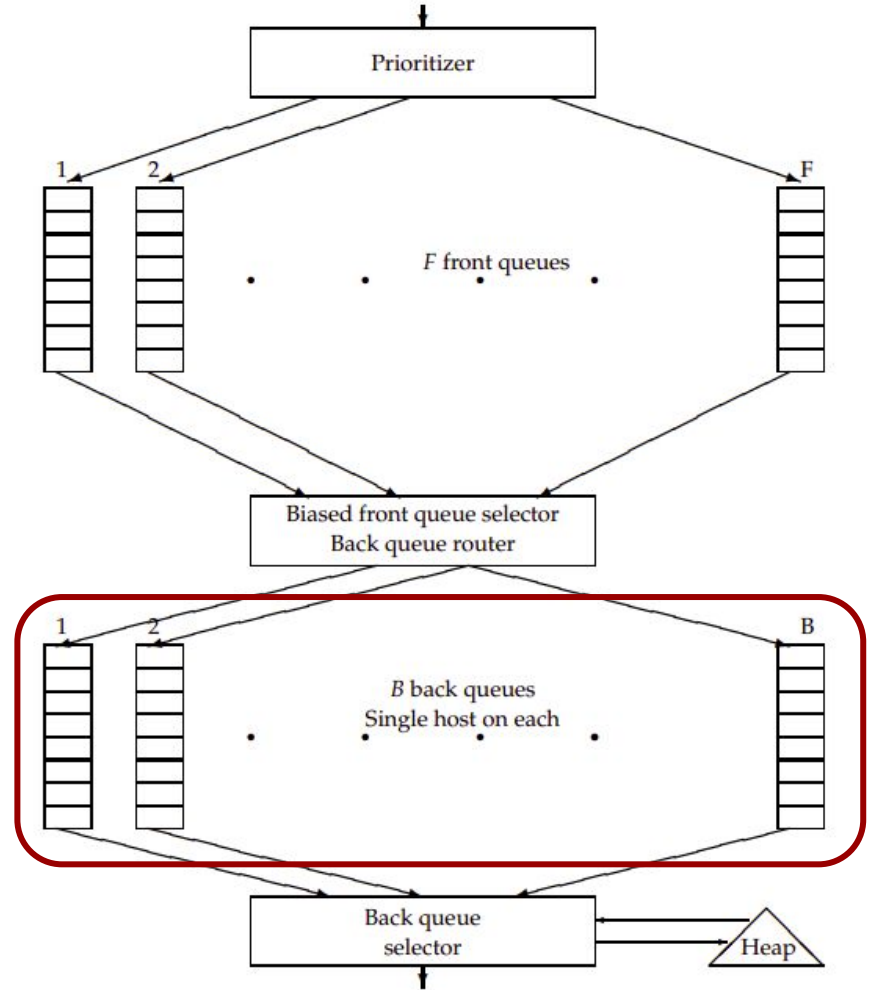
Now, let's see how URL are moved within the URL Frontier:

1. Moving URL is in charge of crawler threads
2. A crawler thread needs the next URL to be crawled, so it extracts the root of the heap
3. The heap is rearranged such that the heap root will be the next host to be contacted



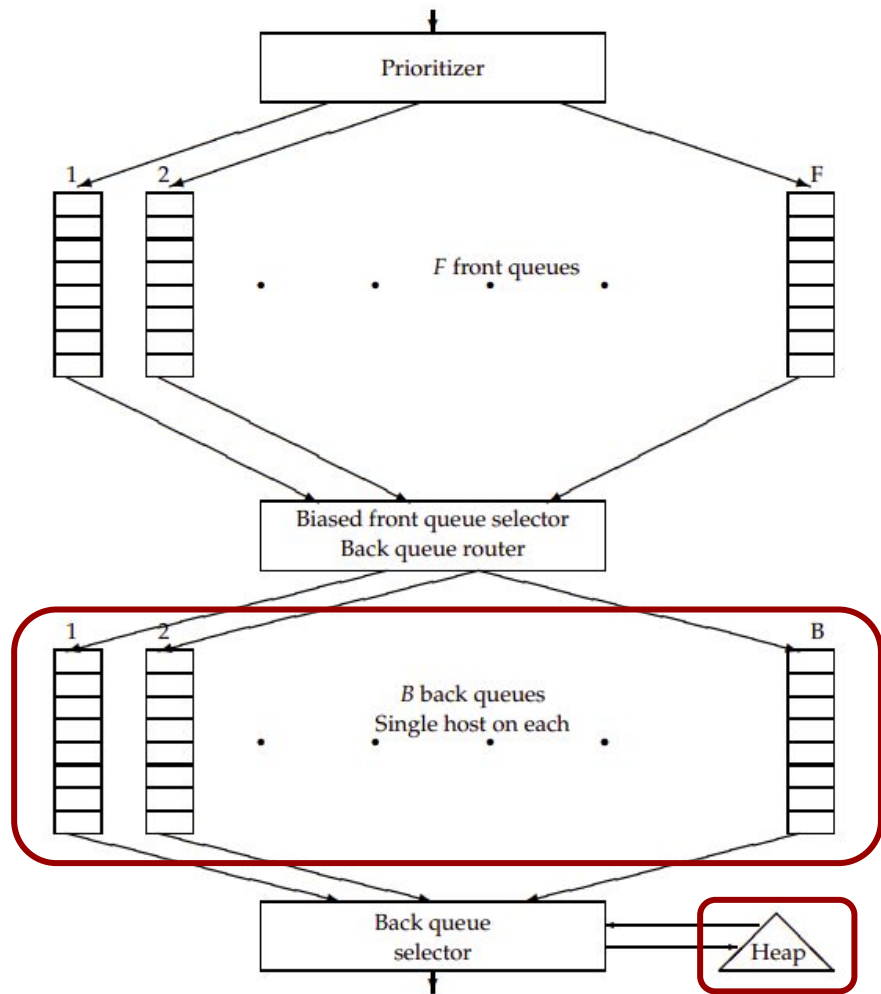
URL Frontier

4. If the time to contact the host is in the future, the crawler thread waits
5. When the time is ready, the crawler thread retrieves from the back queue b represented by the heap root the next URL u to fetch
6. The URL u in the head of the back queue b is then fetched



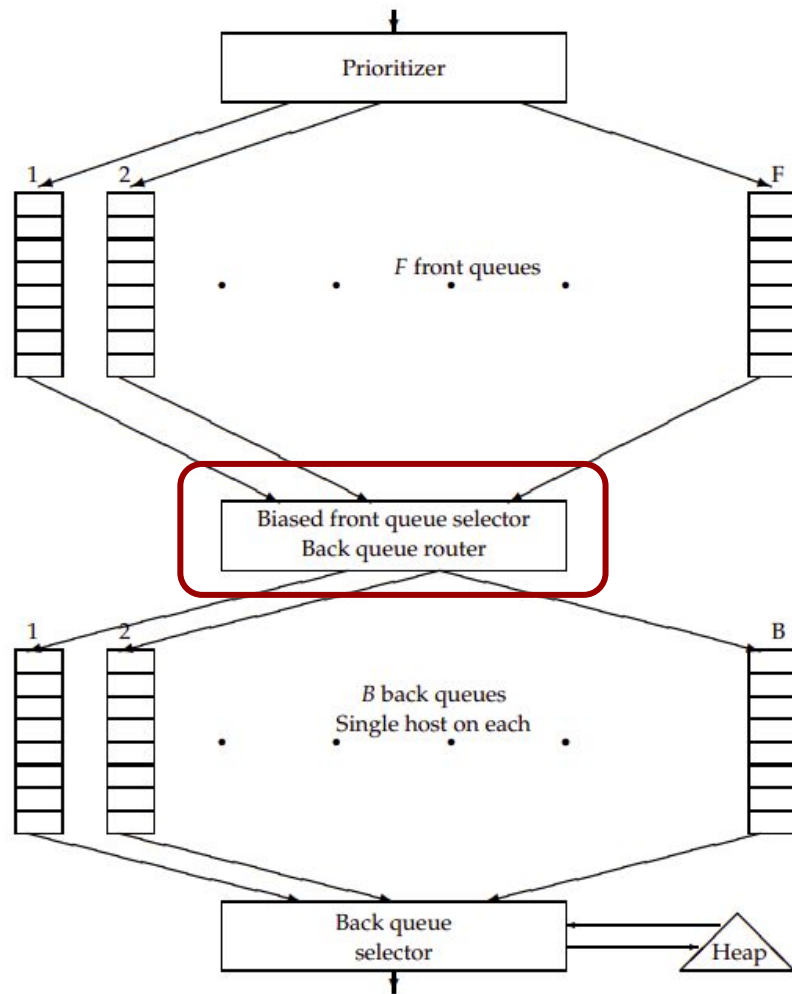
URL Frontier

7. After fetching u , the crawler thread verifies if the back queue b is empty.
 - a. If not empty: the back queue b is again inserted in the heap with a new time to visit
 - b. if empty:
 - i. the auxiliary table is updated (by removing the correspondence of the URL with the Back Queue ID)
 - ii. the Crawler Thread selects the index of the Front Queue f from which to retrieve the URL.



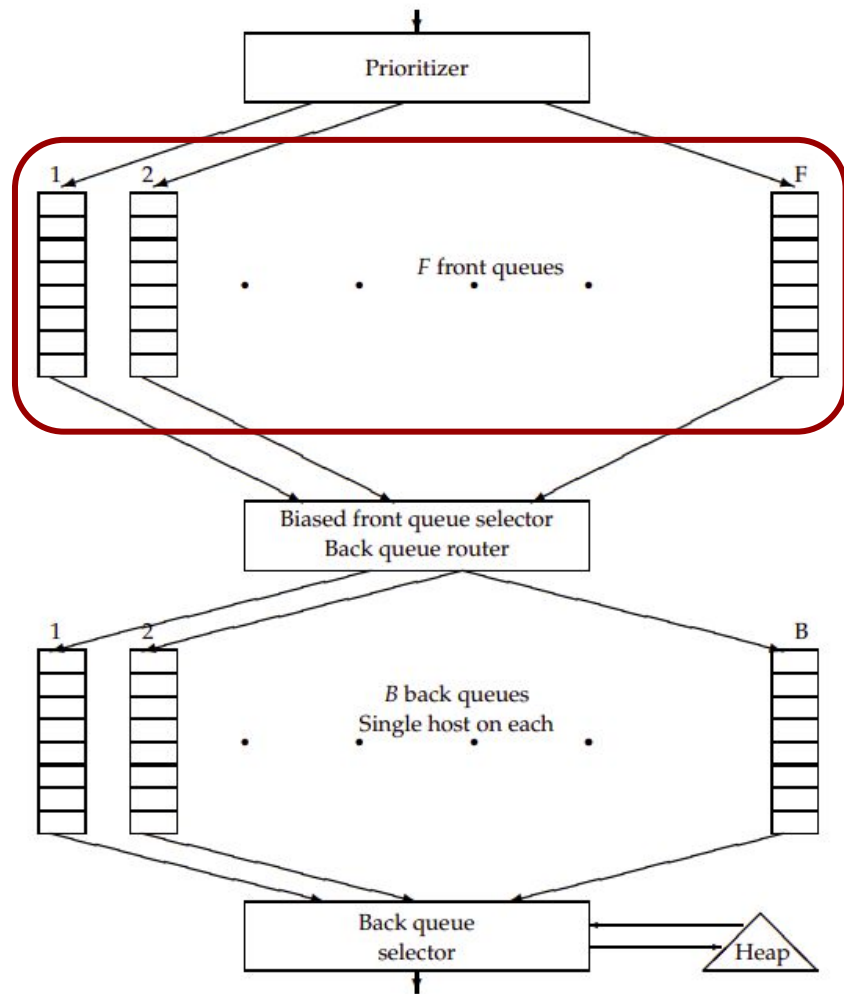
URL Frontier

The selection of the Front Queue should ensure that URLs of high priority are **selected more often** than those having low priority.



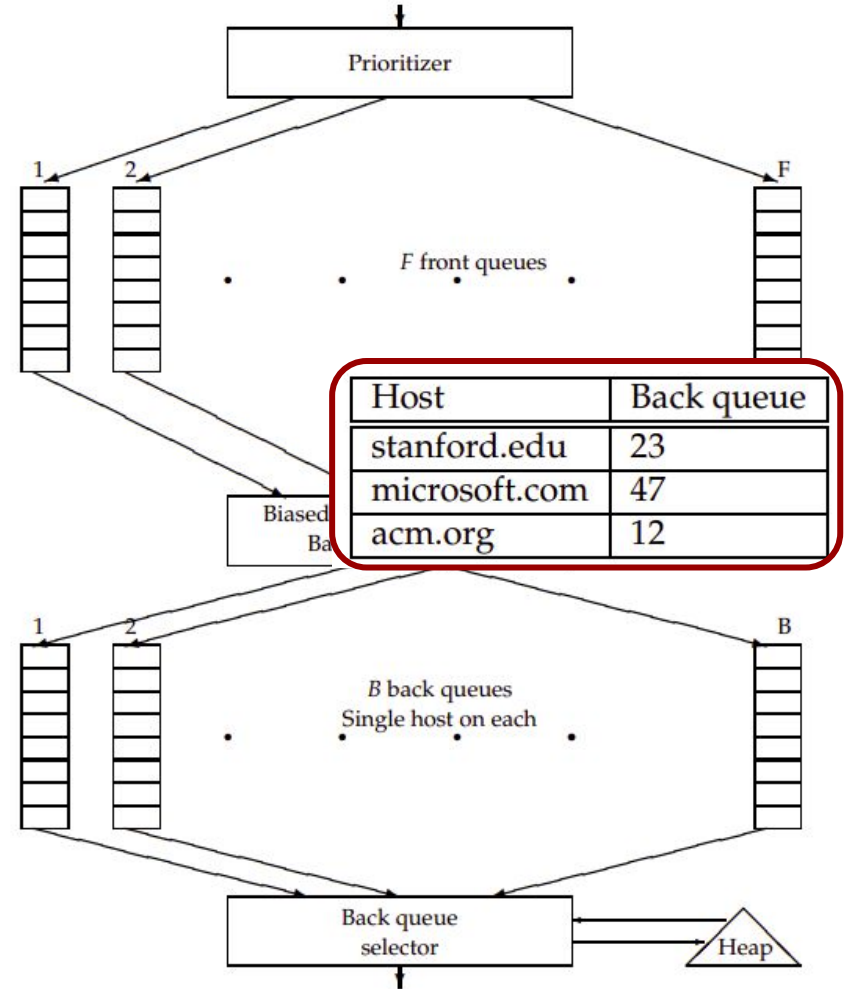
URL Frontier

8. Once the Front Queue f has been selected, the crawler process extract the URL v in the head of Front Queue f



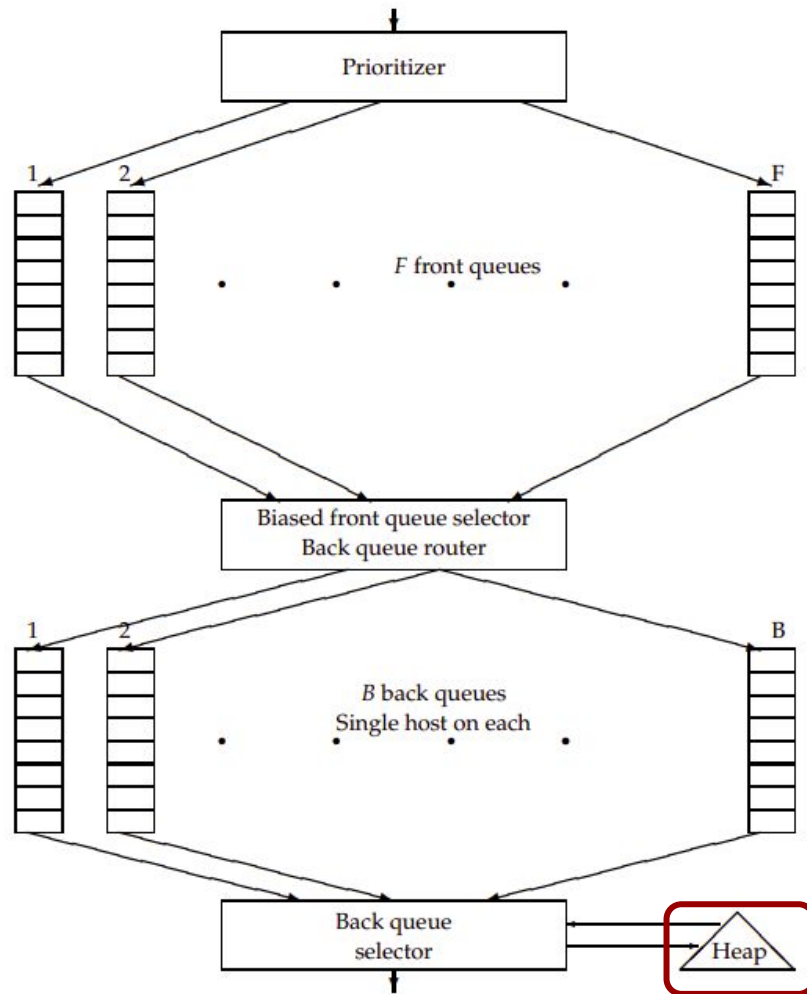
URL Frontier

9. By looking at the mapping table, the Crawler Thread verifies whether the URL v is already in a Back Queue.
 - a. if yes: v is added to the back queue and a new Front End Queue is selected for the next candidate URL to be inserted into the empty back queue b .
 - b. if not: v is added to the empty Back Queue b and the mapping table is updated



URL Frontier

10. If v is added to the Empty Back Queue j , the Crawler Thread inserts a new entry on the Heap for j with a new earliest time.
11. The Crawler Thread continues with its processing



URL Frontier

Some details:

- the number of Front Queues, the heuristics of assigning priorities and the policy for selecting the next Front Queue determine the **priority properties** of our implementation
- Recommendation suggests to use a number of Back Queues **three times higher** the the number of Crawler Threads
- URL frontier may grow and require more memory than that available on the node. The solution is to let most of the **URL frontier reside on disk** and moving into the memory just a small portion

Neural Network, Embeddings and Word2Vec

Academic Year 2025/2026

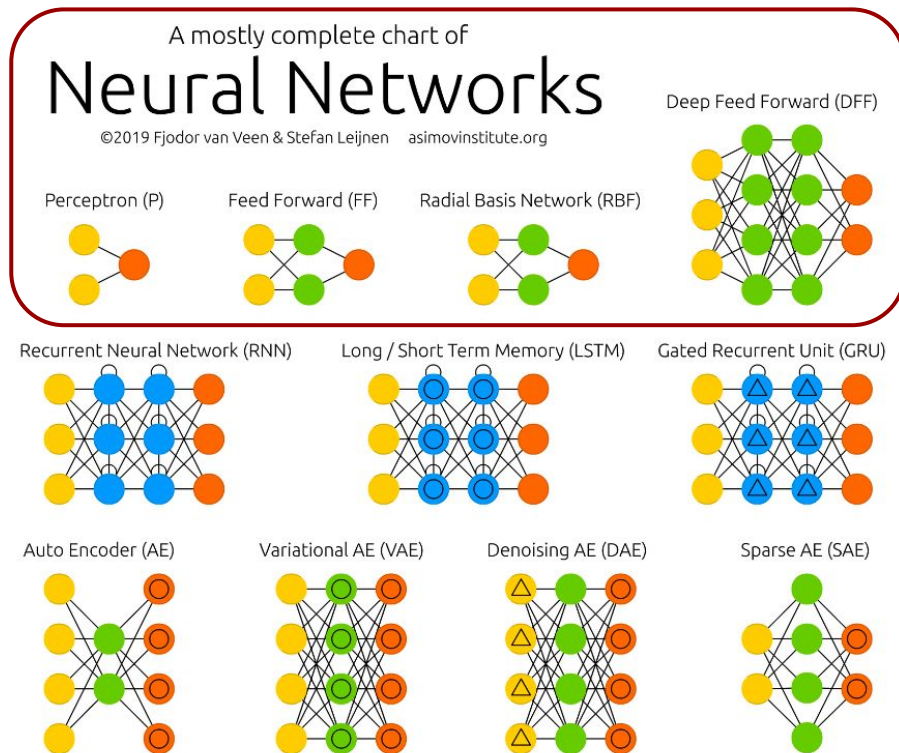
Luigi Santangelo
luigi.santangelo@unipv.it

University of Pavia
**Master Degree in
Computer Engineering**

Neural Network

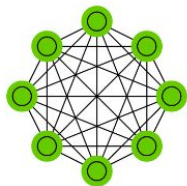
Neural Network Topology Overview

- Input Cell
- Backfed Input Cell
- △ Noisy Input Cell
- Hidden Cell
- Probabilistic Hidden Cell
- △ Spiking Hidden Cell
- Capsule Cell
- Output Cell
- Match Input Output Cell
- Recurrent Cell
- Memory Cell
- △ Gated Memory Cell
- Kernel
- Convolution or Pool

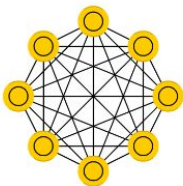


Neural Network Topology Overview

Markov Chain (MC)



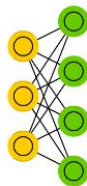
Hopfield Network (HN)



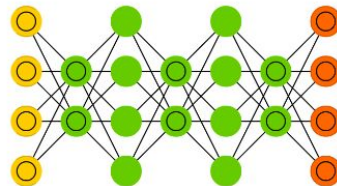
Boltzmann Machine (BM)



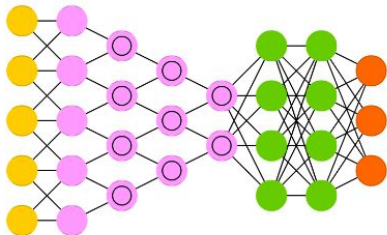
Restricted BM (RBM)



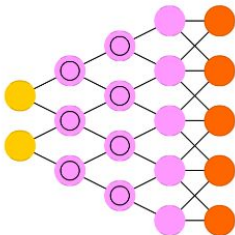
Deep Belief Network (DBN)



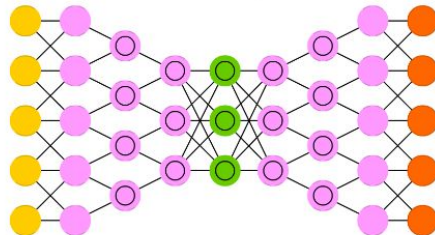
Deep Convolutional Network (DCN)



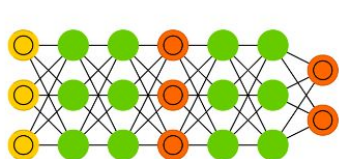
Deconvolutional Network (DN)



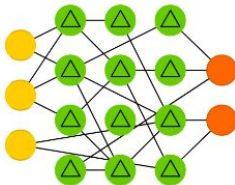
Deep Convolutional Inverse Graphics Network (DCIGN)



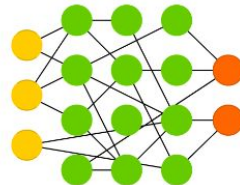
Generative Adversarial Network (GAN)



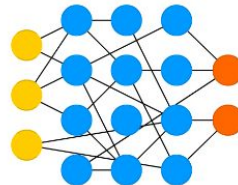
Liquid State Machine (LSM)



Extreme Learning Machine (ELM)

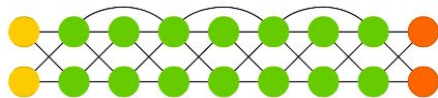


Echo State Network (ESN)

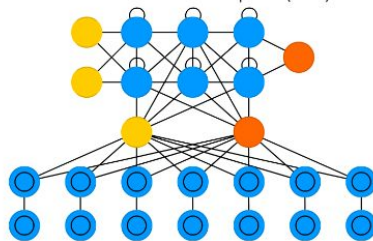


Neural Network Topology Overview

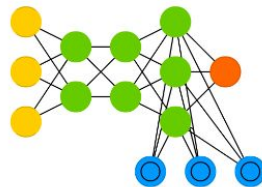
Deep Residual Network (DRN)



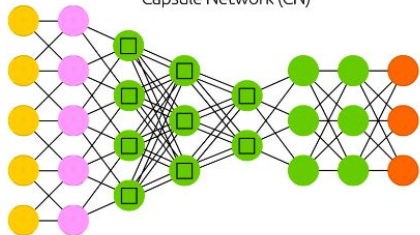
Differentiable Neural Computer (DNC)



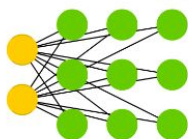
Neural Turing Machine (NTM)



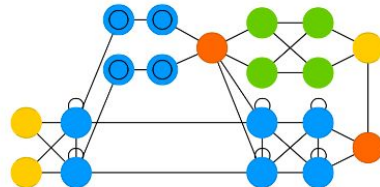
Capsule Network (CN)



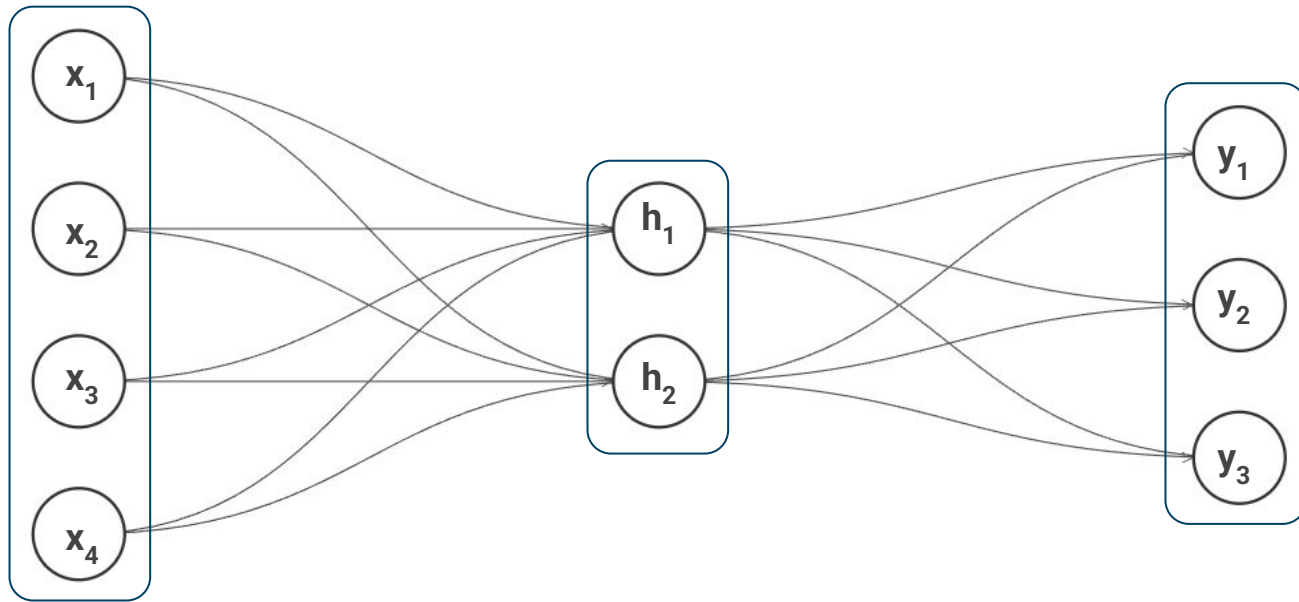
Kohonen Network (KN)



Attention Network (AN)



The structure of a simple Neural Network

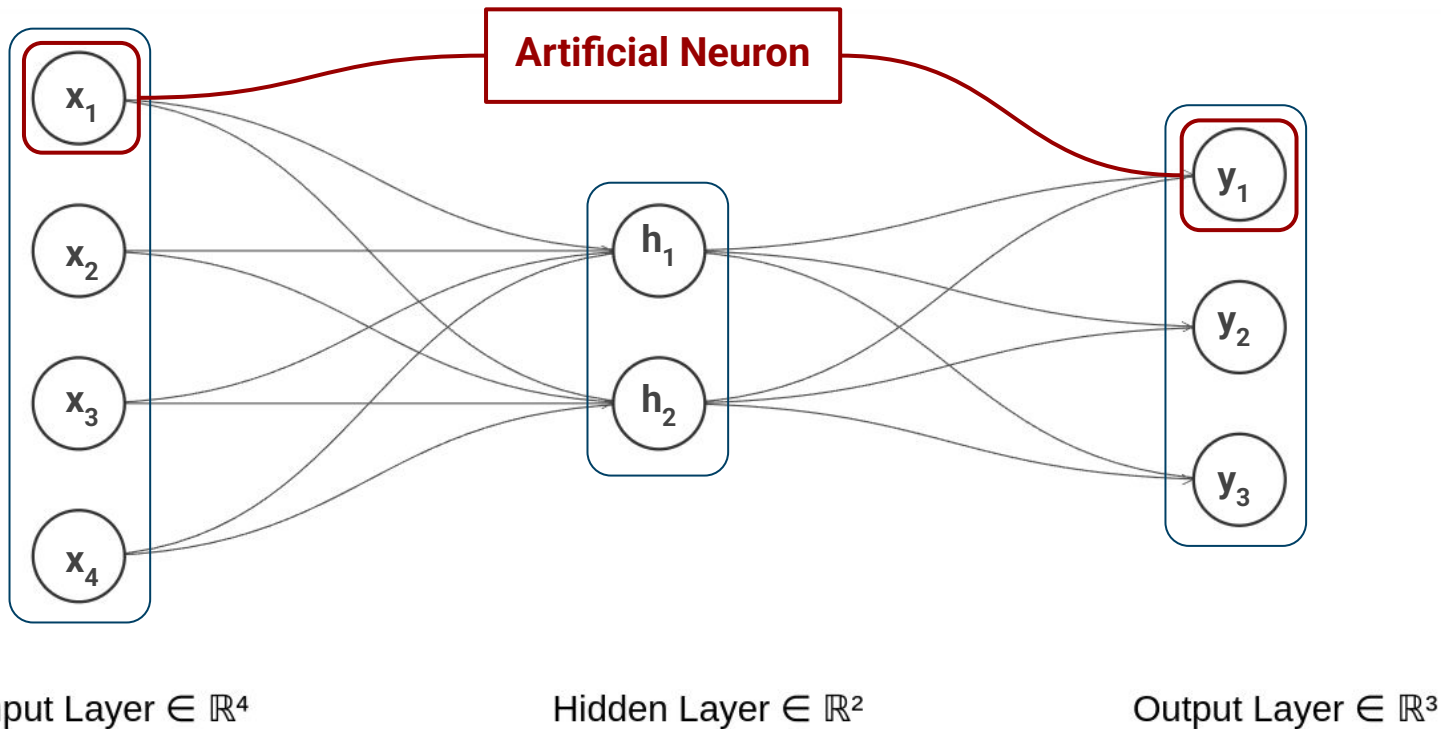


Input Layer $\in \mathbb{R}^4$

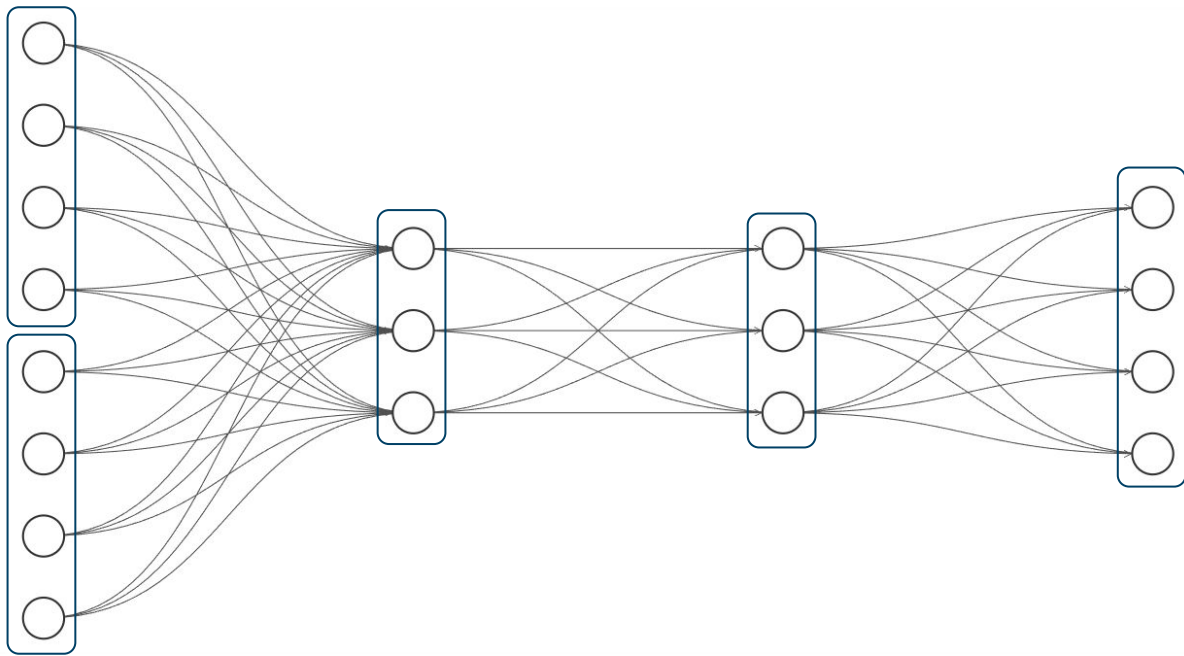
Hidden Layer $\in \mathbb{R}^2$

Output Layer $\in \mathbb{R}^3$

The structure of a simple Neural Network



A more complex Neural Network

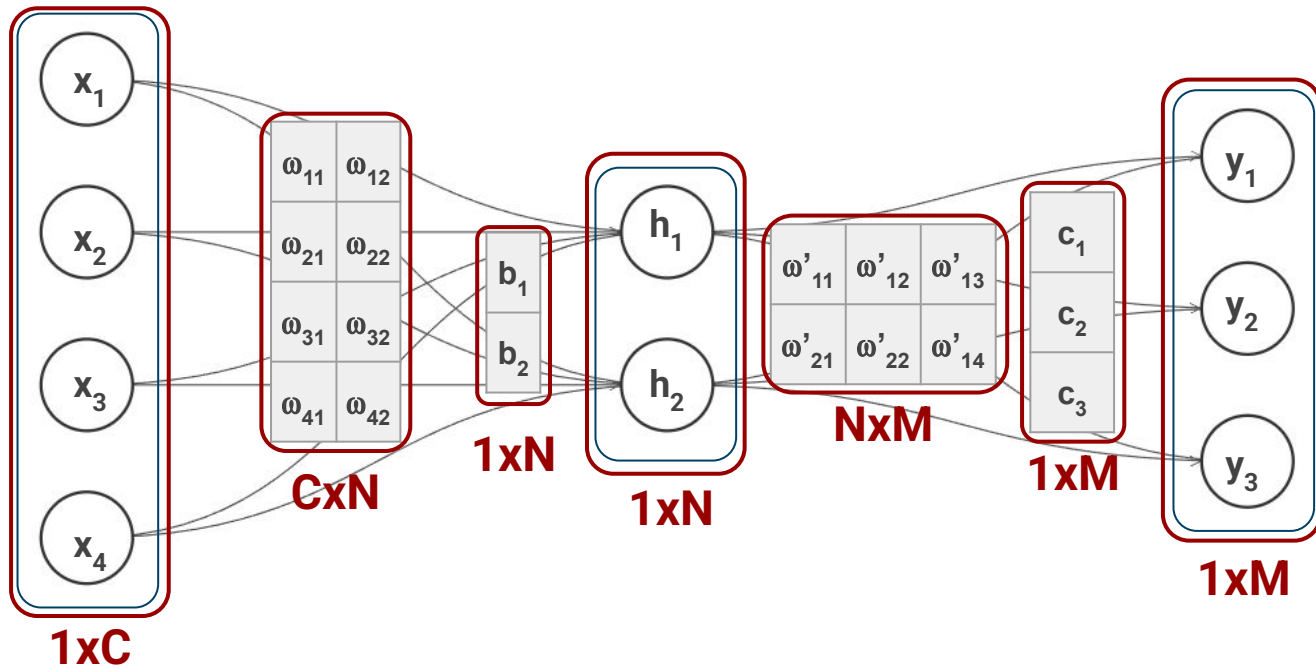


How many hidden layers?

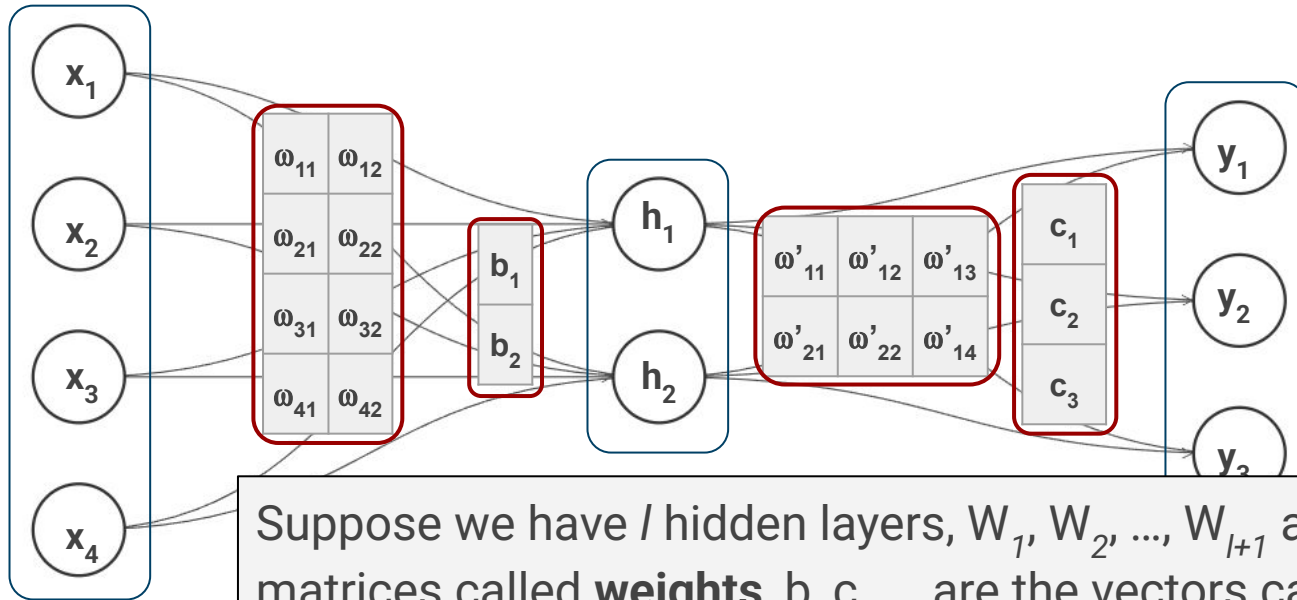
How many nodes in each hidden layers?

How to interconnect nodes from one layer to the next?

The structure of a simple Neural Network

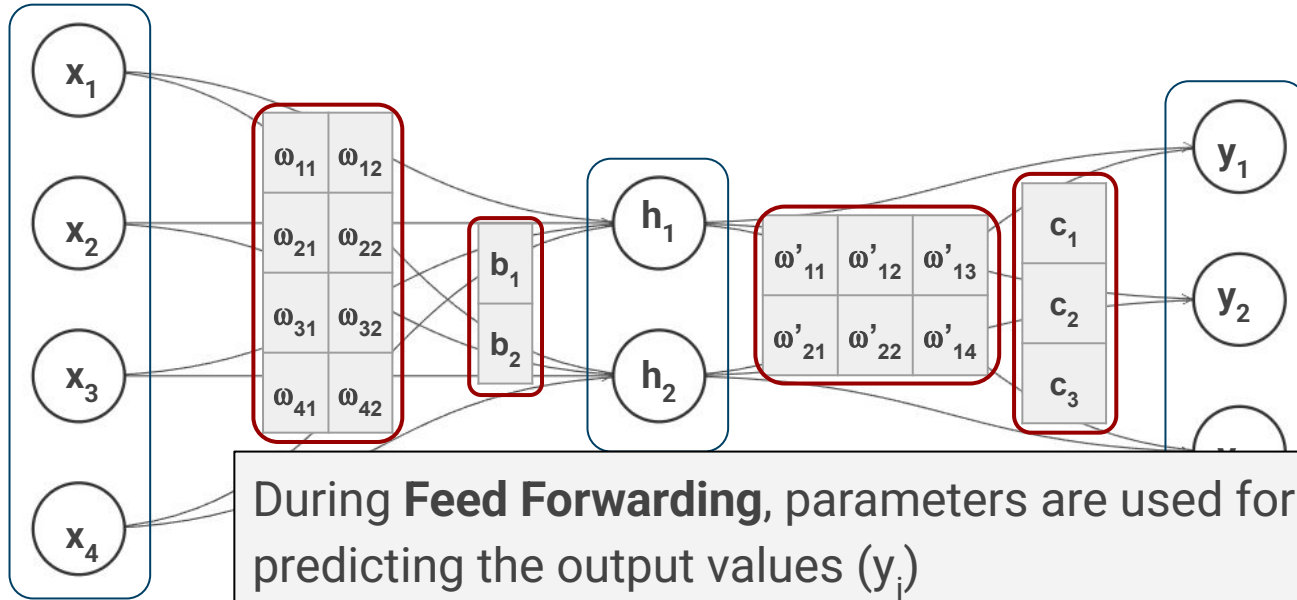


The structure of a simple Neural Network

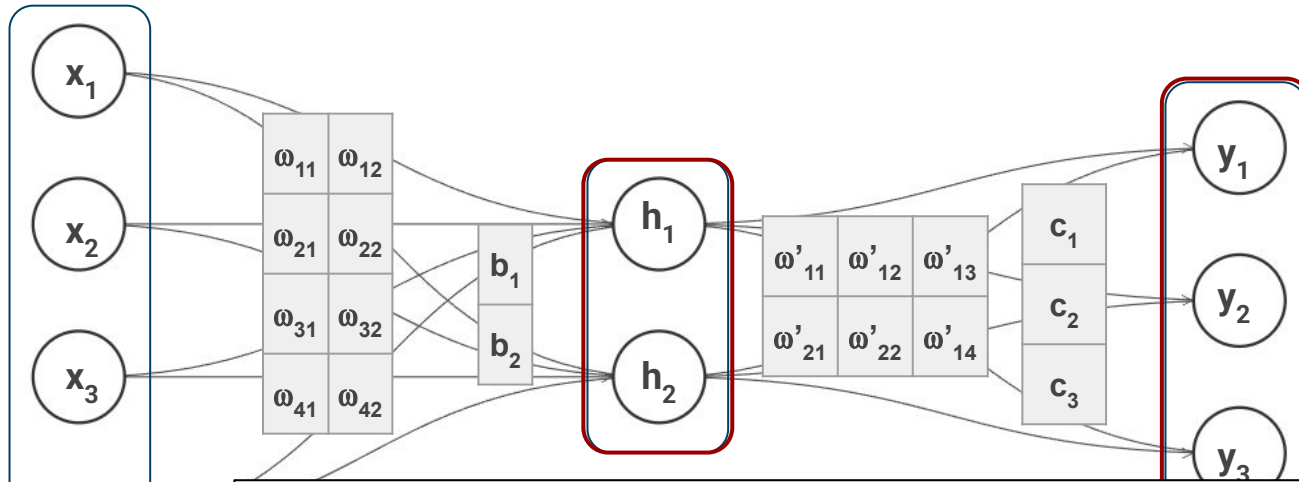


Suppose we have l hidden layers, $W_1, W_2, \dots, W_{l+1}$ are the matrices called **weights**, $b, c, \dots$, are the vectors called **bias**. These are **parameters**. Our objective is to learn the **best values** for these parameters.

Neural Network: two phases



Neural Network: activation functions



The values in the hidden layer and in the output layer are usually computed applying an **activation function**, that calculates the output of the node based on its individual inputs and their weights

Typical Activation functions for classification

Name	Function	Derivative
Sigmoid	$\sigma(x) = \frac{1}{1 + e^{-x}}$	$\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$
Softmax	$\mu(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$	$\frac{d\mu(x_i)}{dx_j} = \begin{cases} \mu(x_i)(1 - \mu(x_i)) & \text{if } i = j \\ -\mu(x_i)\mu(x_j) & \text{if } i \neq j \end{cases}$

Considerations on Sigmoid

1. The function is continuous
2. The function is differentiable
3. For large positive x, the function approaches 1
4. For large negative x, the function approaches 0
5. So it ranges between 0 and 1
6. The output can be interpreted as a probability

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

x	$\sigma(x)$
10	0,99995
8	0,99966
-2	0,11920
4	0,98201
-3	0,04743
9	0,99988
2	0,88080
1	0,73106
-5	0,00669

Considerations on Softmax

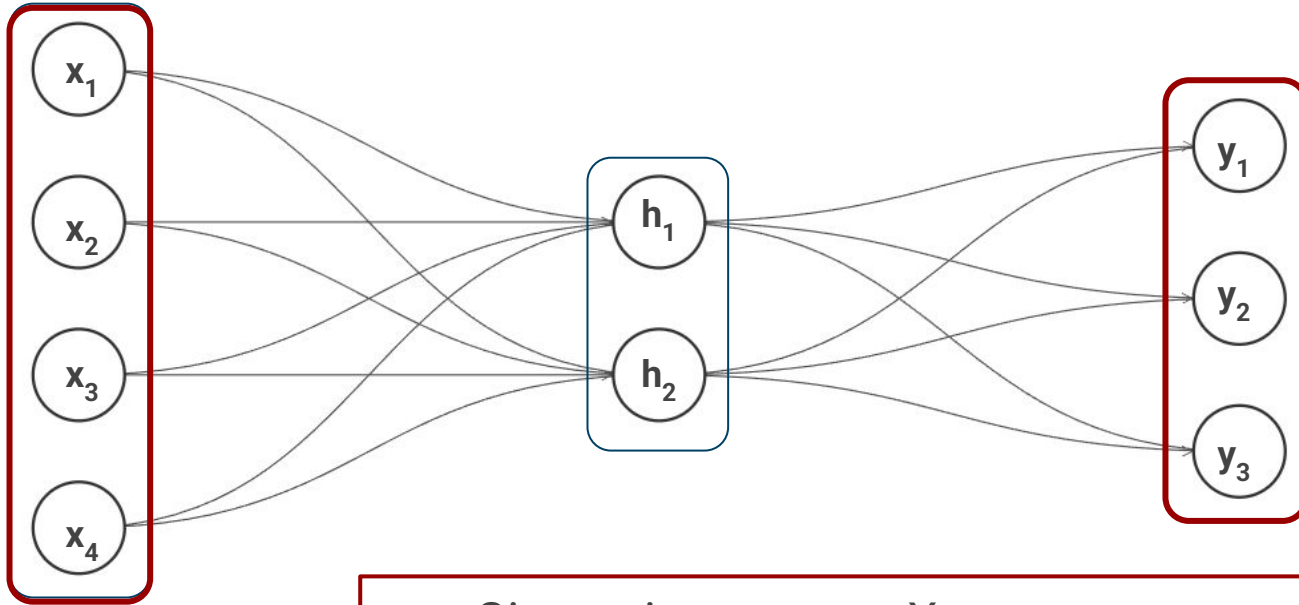
1. The function is continuous
2. The function is differentiable
3. For large positive x , the function approaches 1
4. For large negative x , the functions approaches 0
5. Summing up all softmax values is equal to 1
6. So it range between 0 and 1
7. The output can be interpreted as a probability

$$\mu(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

x	$\mu(x)$
10	0,66394
8	0,08985
-2	0,00000
4	0,00165
-3	0,00000
9	0,24425
2	0,00022
1	0,00008
-5	0,00000
	1

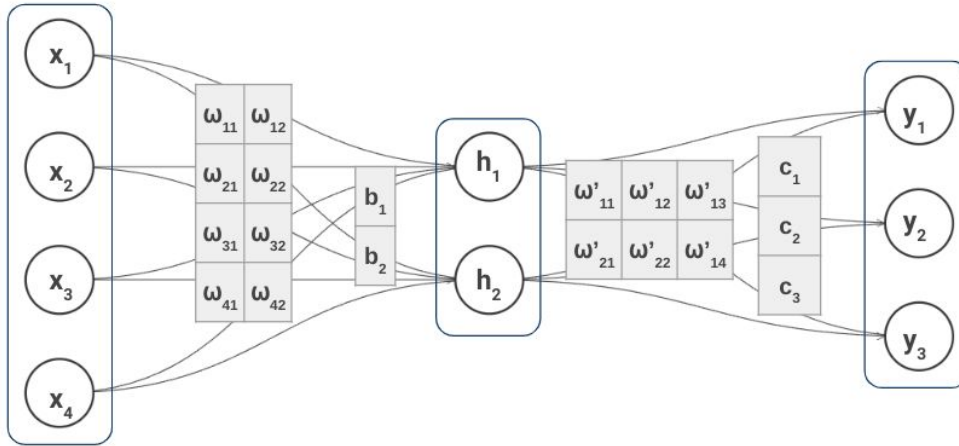
Neural Network Feed Forward

Neural Network: Feed Forwarding



Given a input vector X , we want to determine the output vector Y

Neural Network: Feed Forwarding



$$v[j] = \sum_{i=1}^C x[i]w[i, j] + b[j] \quad \forall j = 1..N$$

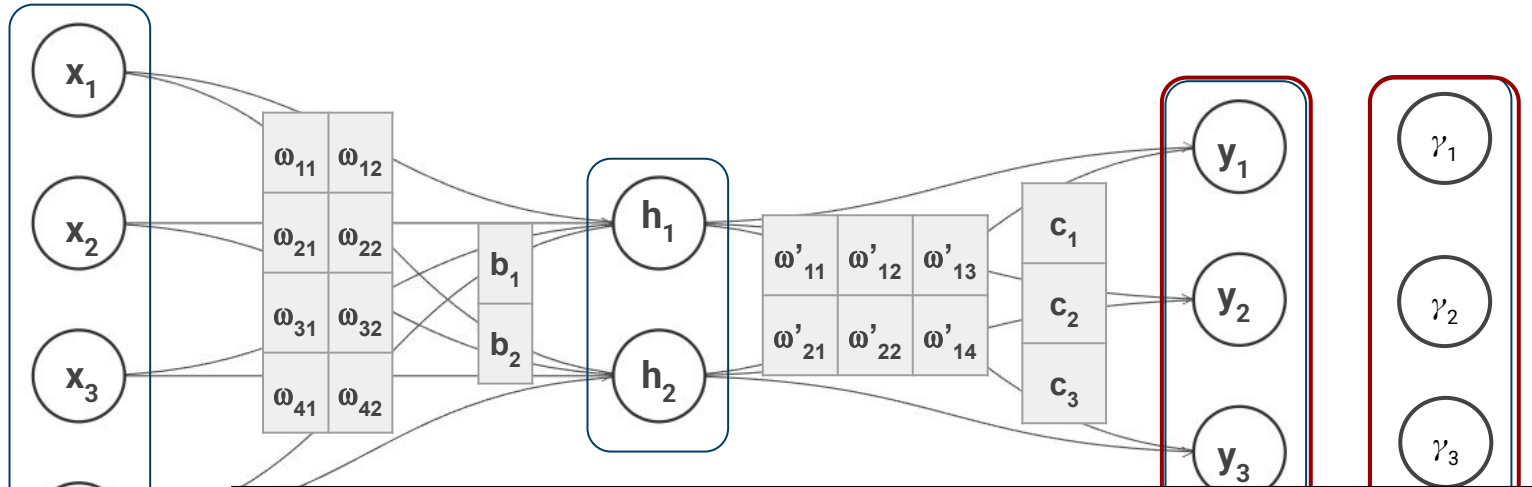
$$h[j] = \sigma(v[j]) = \frac{1}{1 + e^{-v[j]}} \quad \forall j = 1..N$$

$$u[k] = \sum_{j=1}^N h[j]w'[j, k] + c[k] \quad \forall k = 1..M$$

$$y[k] = \sigma(u[k]) = \frac{1}{1 + e^{-u[k]}} \quad \forall k = 1..M$$

Sigmoid function

Neural Network: predicted and target values



We aim to define all parameters and bias such that the **distance** between the **predicted values** and the **target value** is minimal. A **Loss Function** quantifies the error margin between a model's prediction and the actual target value

Neural Network: Loss Functions for regression

Mean Squared Error

$$L(y, \gamma) = \frac{1}{M} \sum_{k=1}^M (y[k] - \gamma[k])^2$$

This value quantifies the prediction error for a **single input**.

Neural Network: Loss Functions for classificat.

If $\mathbf{y}$ and $\boldsymbol{\gamma}$ are two **distributions**, we can use the **Cross Entropy**.

$$0 \leq \gamma[k] \leq 1 \quad \forall k = 1..M \qquad 0 \leq y[k] \leq 1 \quad \forall k = 1..M$$

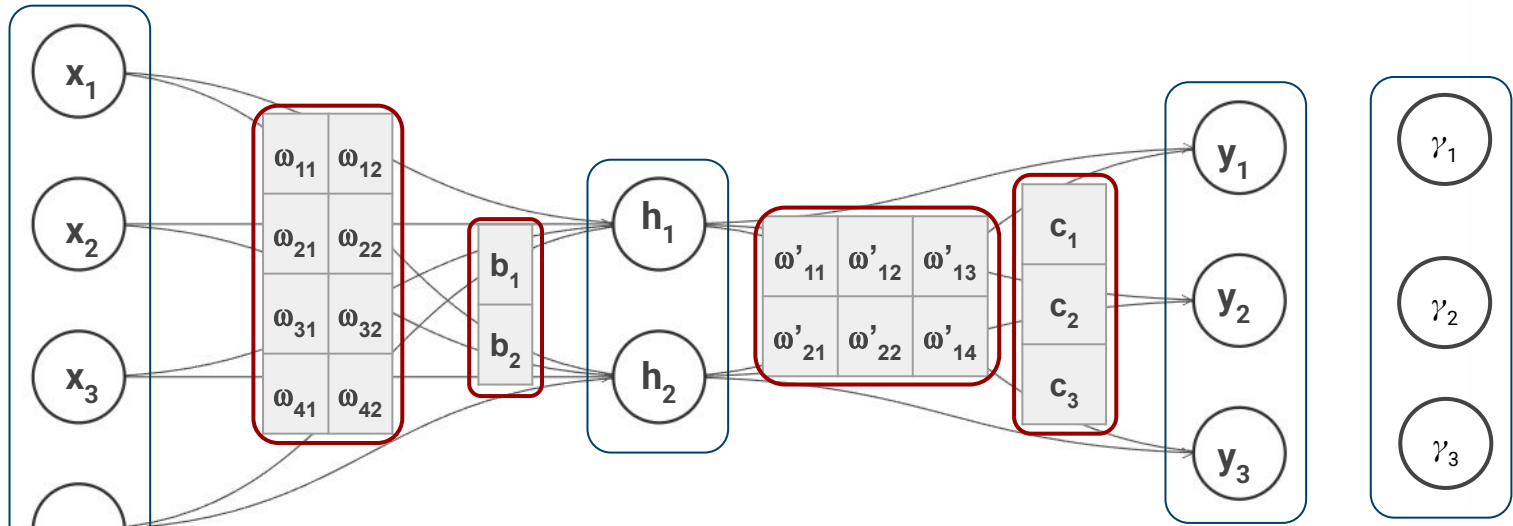
$$\sum_{k=1}^M \gamma[k] = 1$$

$$\sum_{k=1}^M y[k] = 1$$

$$L(y, \gamma) = - \sum_{k=1}^M (\gamma[k] \log_e(y[k]))$$

This is the prediction error for a **single input**

Neural Network: Stochastic Gradient Descent



Our aim is to **minimize the Error** by updating parameters. Parameters are adjusted running the **Backpropagation** phase and the **Gradient Descent**

Gradient Descent

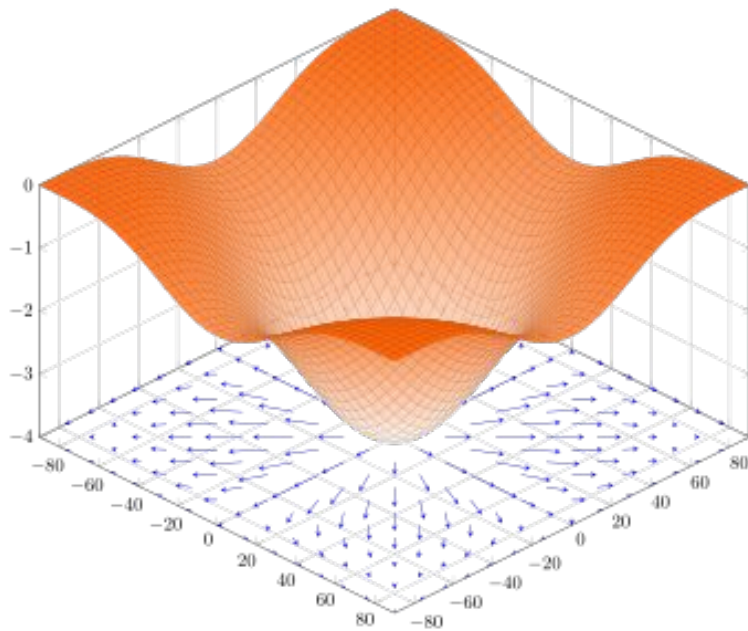
The idea behind Gradient Descent is to **compute the gradient of the loss function** with respect to the parameters of the network.

Then, we can adjust the parameters slightly **in the directions that will reduce the loss**, and **repeat the process** until we reach a selection of parameter values for which little improvement in the loss is possible.

The Gradient: from math point of view

The gradient of a function is a vector representing the direction of greatest change of the function.

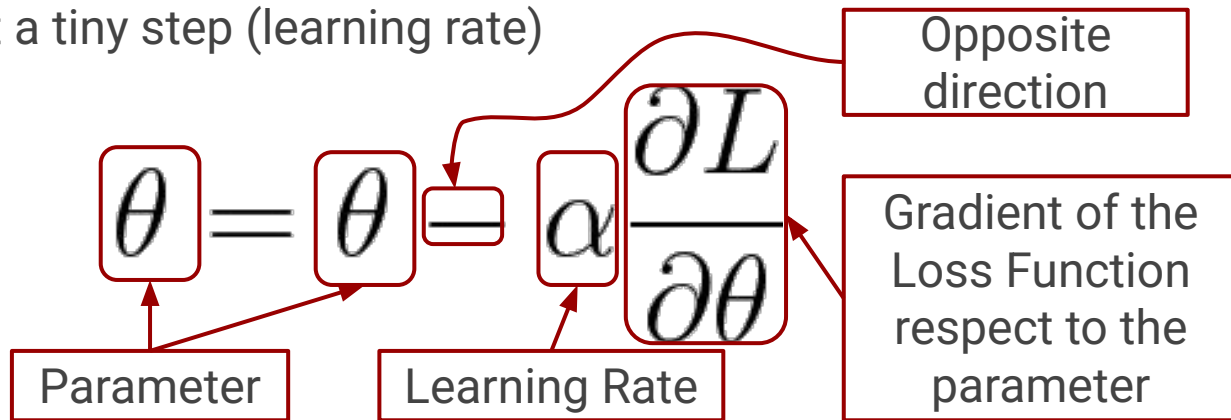
Gradients point in the direction of the steepest ascent



The Gradient: updating parameters

Once we have found the gradient of the Loss Function, we can adjust our parameters following this two rules:

- moving iteratively on the opposite direction of the gradient (toward the minimum value)
- moving for just a tiny step (learning rate)



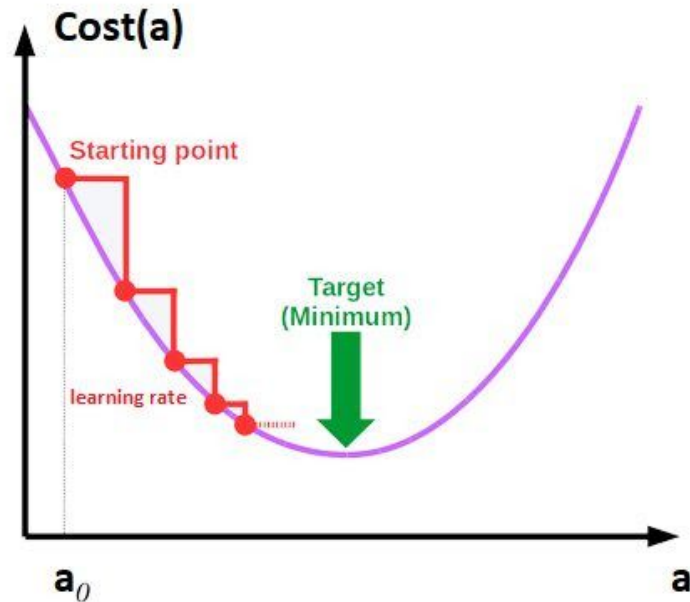
The Gradient: the learning rate

The **learning rate** is a constant that controls the step size in the parameter space. Usually is a value very little such as 0.01

By iteratively updating the model's parameters in the negative gradient direction, gradient descent gradually converges towards the optimal set of parameters that yields the lowest cost.

The Gradient: toward the minimum

After several iterations we have computed a parameter which makes our Loss Function very close to the minimum



Neural Network Back Propagation

Neural Network: BackPropagation

We need to understand how the Loss Function changes when parameters b , c , w and w' change as well.

We need to compute the following partial derivative:

$$\begin{array}{cccc}\frac{\partial L}{\partial y[k]} & \frac{\partial L}{\partial u[k]} & \frac{\partial L}{\partial c[k]} & \frac{\partial L}{\partial w'[j, k]} \\ \frac{\partial L}{\partial h[j]} & \frac{\partial L}{\partial v[j]} & \frac{\partial L}{\partial b[j]} & \frac{\partial L}{\partial w[i, j]}\end{array}$$

$$v[j] = \sum_{i=1}^C x[i]w[i, j] + b[j] \quad \forall j = 1..N$$

$$h[j] = \sigma(v[j]) = \frac{1}{1 + e^{-v[j]}} \quad \forall j = 1..N$$

$$u[k] = \sum_{j=1}^N h[j]w'[j, k] + c[k] \quad \forall k = 1..M$$

$$y[k] = \sigma(u[k]) = \frac{1}{1 + e^{-u[k]}} \quad \forall k = 1..M$$

$$L(y, \gamma) = \frac{1}{M} \sum_{k=1}^M (y[k] - \gamma[k])^2$$

Neural Network: BackPropagation

$$\frac{\partial L}{\partial y[k]} = \frac{2}{M} (y[k] - \gamma[k])$$

$$v[j] = \sum_{i=1}^C x[i]w[i, j] + b[j] \quad \forall j = 1..N$$

$$h[j] = \sigma(v[j]) = \frac{1}{1 + e^{-v[j]}} \quad \forall j = 1..N$$

$$u[k] = \sum_{j=1}^N h[j]w'[j, k] + c[k] \quad \forall k = 1..M$$

$$y[k] = \sigma(u[k]) = \frac{1}{1 + e^{-u[k]}} \quad \forall k = 1..M$$

$$L(y, \gamma) = \frac{1}{M} \sum_{k=1}^M (y[k] - \gamma[k])^2$$

Neural Network: BackPropagation

$$\frac{\partial L}{\partial u[k]} = \frac{\partial L}{\partial y[k]} \frac{\partial y[k]}{\partial u[k]}$$

Chain Rule

$$= \frac{2}{M} (y[k] - \gamma[k]) y[k] (1 - y[k])$$

Remind that: $\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$

and

$$\mathbf{y}[\mathbf{k}] = \sigma(\mathbf{u}[\mathbf{k}])$$

$$v[j] = \sum_{i=1}^C x[i] w[i, j] + b[j] \quad \forall j = 1..N$$

$$h[j] = \sigma(v[j]) = \frac{1}{1 + e^{-v[j]}} \quad \forall j = 1..N$$

$$u[k] = \sum_{j=1}^N h[j] w'[j, k] + c[k] \quad \forall k = 1..M$$

$$y[k] = \sigma(u[k]) = \frac{1}{1 + e^{-u[k]}} \quad \forall k = 1..M$$

$$\frac{\partial L}{\partial y[k]} = \frac{2}{M} (y[k] - \gamma[k])$$

Neural Network: BackPropagation

When should the BackPropagation stop?

1. Reaching a maximum number of iterations
2. After a predetermined number of training epochs (an epoch is a complete pass through the training dataset)
3. Convergence of the Loss: when the change in the loss between consecutive epochs becomes smaller than a predefined threshold: $|L^{(t)} - L^{(t-1)}| < \epsilon$

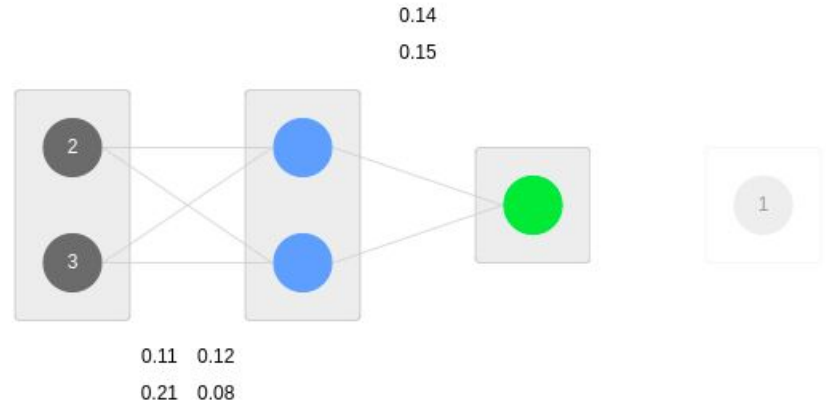
Neural Network Playground

Neural Network Online Simulator

NetFlowJS:

<https://hmkcode.com/netflow/>

You can see visualization of the forward pass and backpropagation



Word Embedding and Word2Vec

Word Embedding

If I say “Ruler” or “Worker”, which one of these words is more similar to “King”?

Perhaps many people say “Ruler”, because it makes more sense.

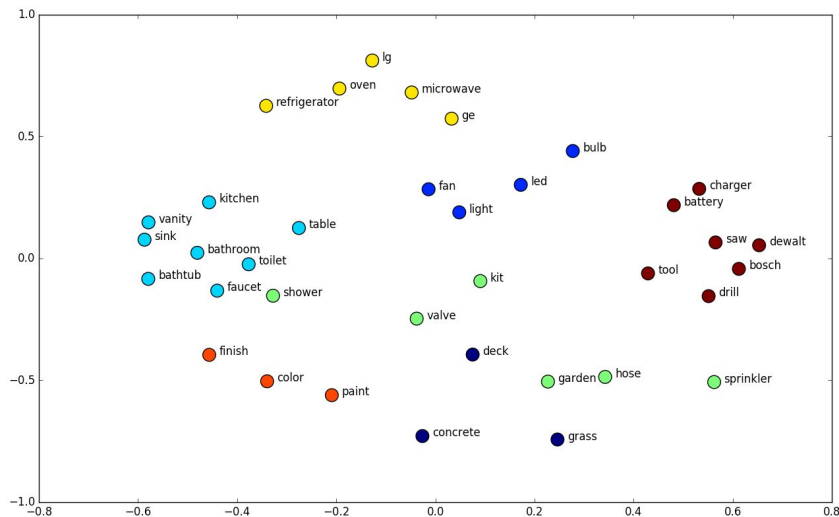
But how can we teach this intuition to a computer?

That’s where word embeddings come in handy

Word Embedding

A word embedding is a **representation of a word**

A word is represented by **real-valued vector**, which encodes the meaning of the word in such a way that the **words that are closer in the vector space are expected to be similar in meaning**.



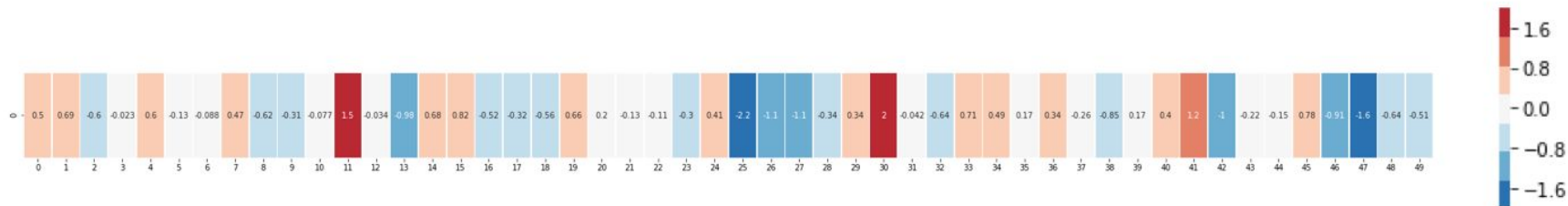
Word Embedding

Word Embedding is used very often:

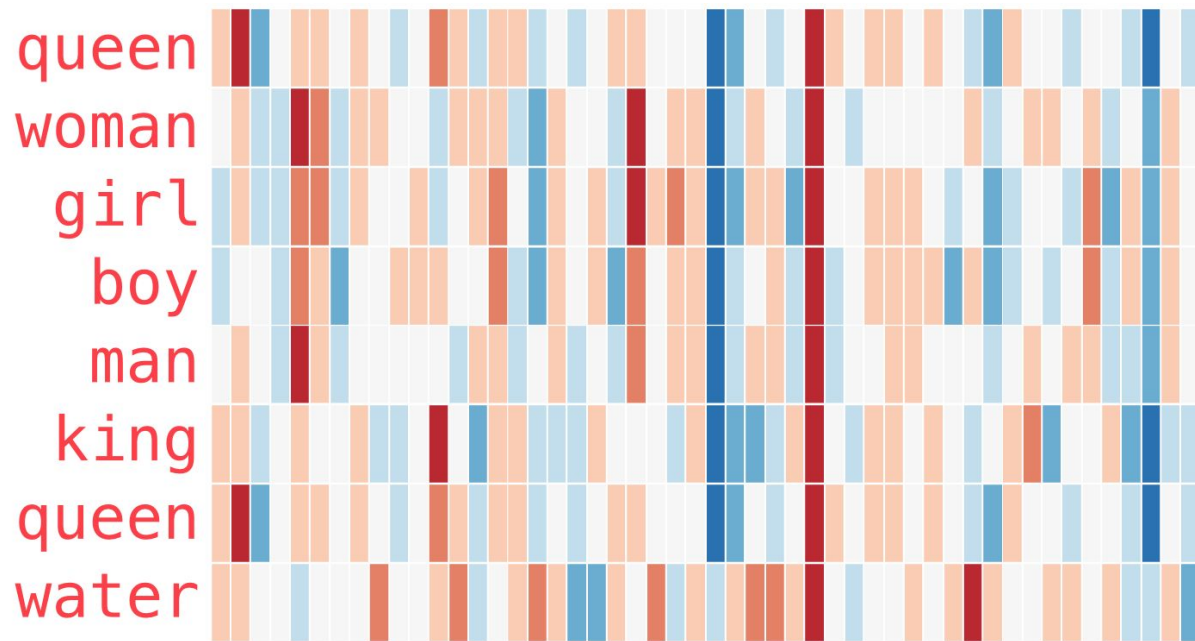
- Syntactic Parsing
- Sentiment Analysis
- Document Clustering
- Text Classification
- Next Word Prediction

The vector for the word “King”

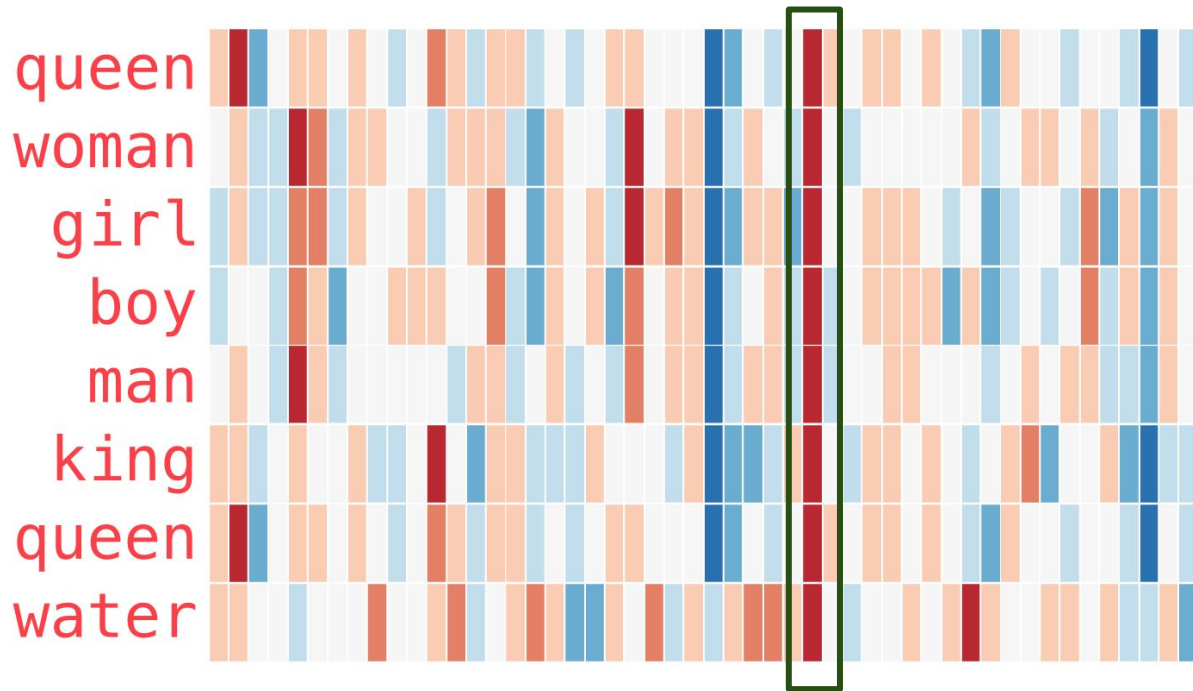
```
[ 0.50451 , 0.68607 , -0.59517 , -0.022801, 0.60046 , -0.13498 , -0.08813 , 0.47377 , -0.61798 , -0.31012 ,  
-0.076666, 1.493 , -0.034189, -0.98173 , 0.68229 , 0.81722 , -0.51874 , -0.31503 , -0.55809 , 0.66421 , 0.1961  
, -0.13495 , -0.11476 , -0.30344 , 0.41177 , -2.223 , -1.0756 , -1.0783 , -0.34354 , 0.33505 , 1.9927 ,  
-0.04234 , -0.64319 , 0.71125 , 0.49159 , 0.16754 , 0.34344 , -0.25663 , -0.8523 , 0.1661 , 0.40102 , 1.1685 ,  
-1.0137 , -0.21585 , -0.15155 , 0.78321 , -0.91241 , -1.6106 , -0.64426 , -0.51042 ]
```



Vectors for eight words

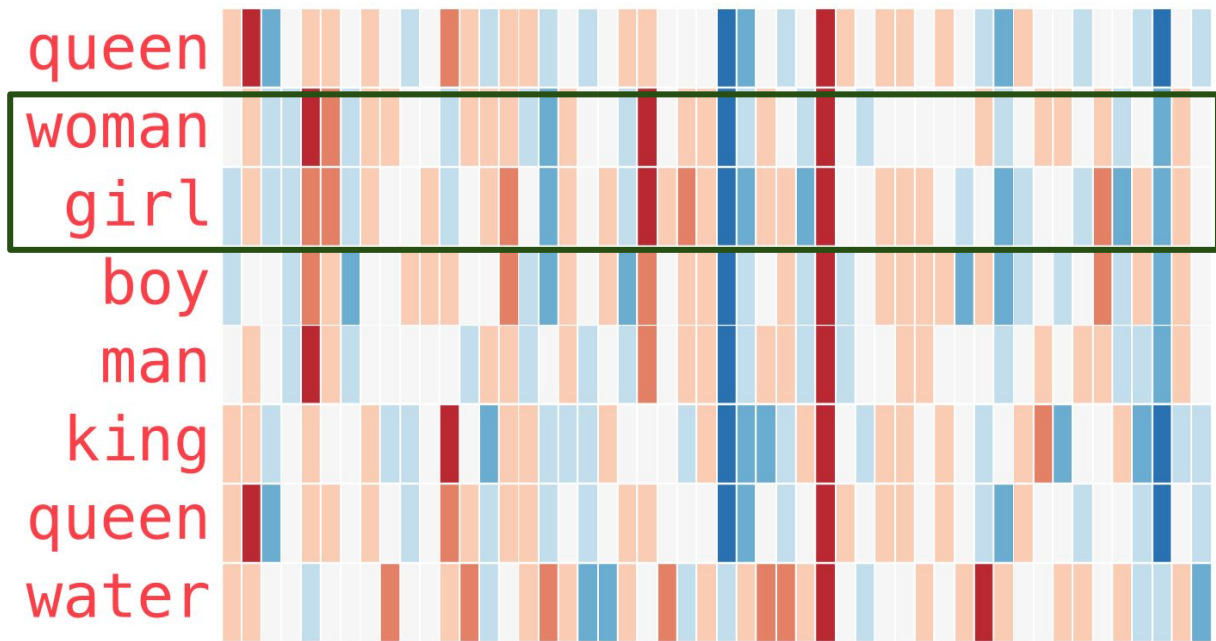


Vectors for eight words



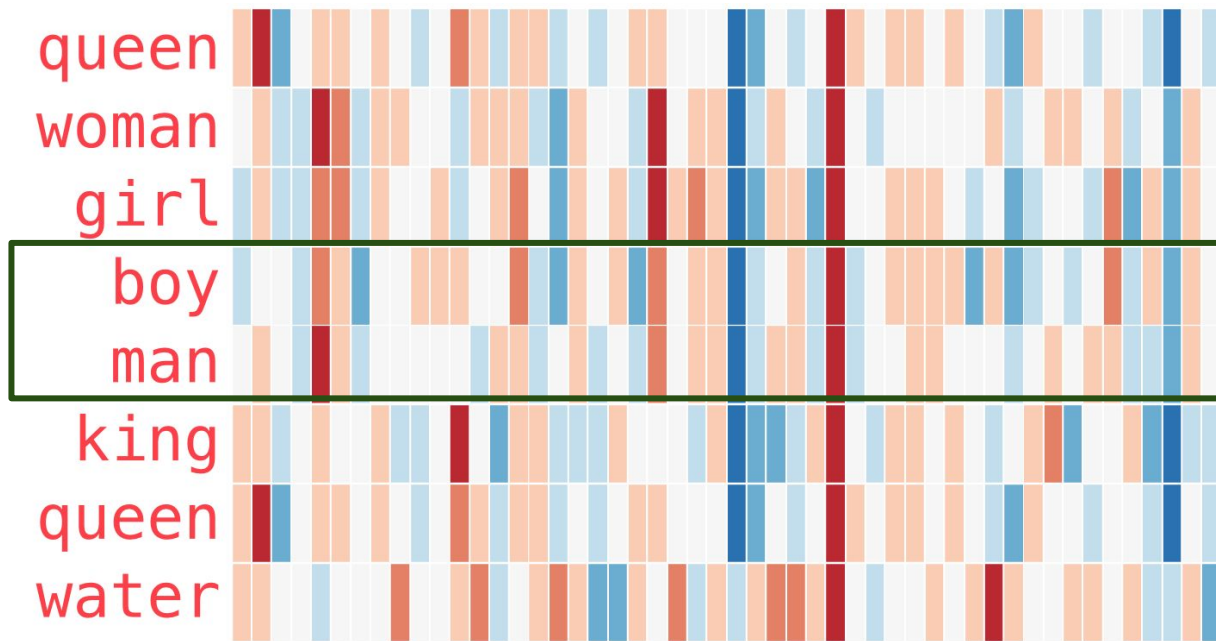
All words are similar along this dimension (we don't know what each dimension codes for)

Vectors for eight words



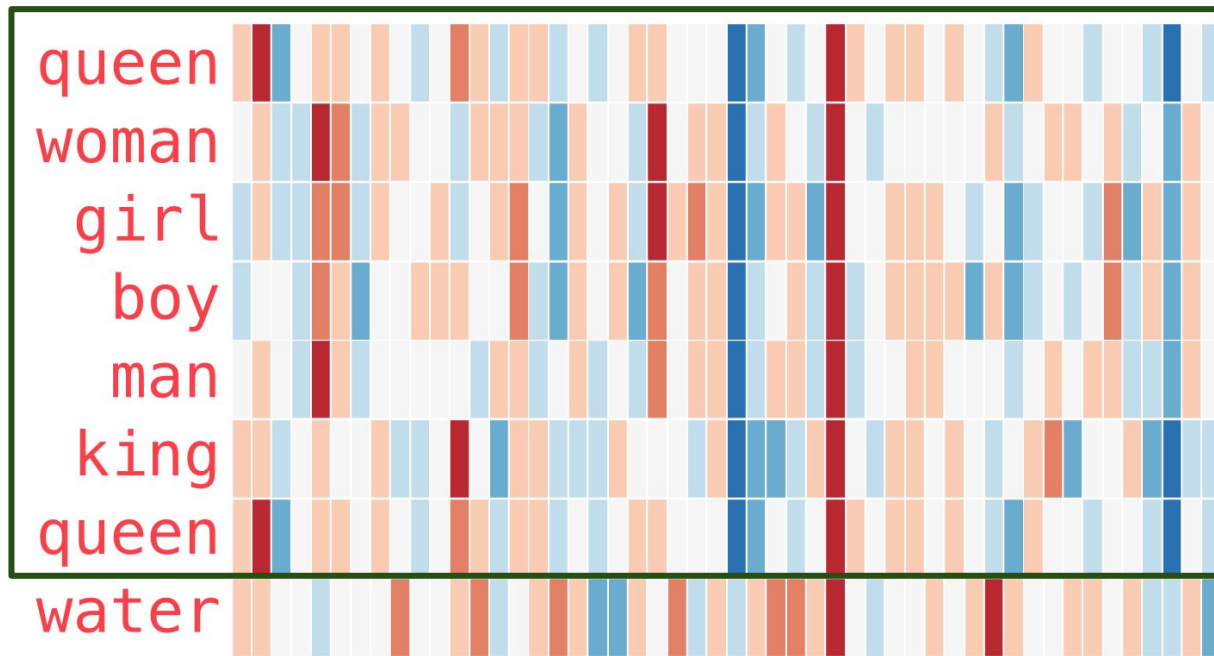
Woman and Girl are very similar to each other in a lot of dimensions

Vectors for eight words



Even Boy and Man are very similar to each other in a lot of dimensions

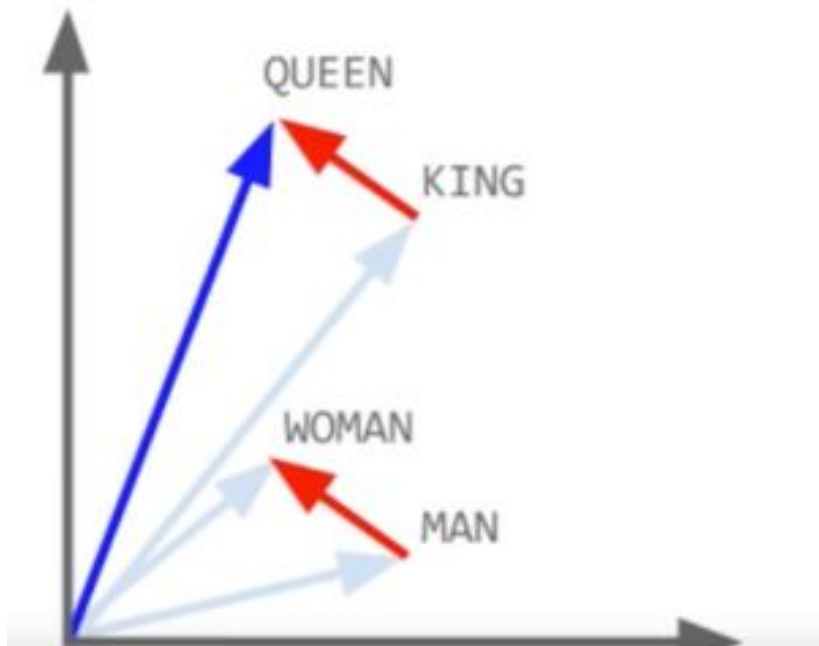
Vectors for eight words



All vectors representing people share many dimensions. They seem to be belonging to the same category.

Water instead seems to be very different to the others

What can we use vectors for?



Using vectors we can use classical vector operation such as sum and difference.

For example we can see that the difference between the vectors representing KING and QUEEN is very similar to the difference between MAN and WOMAN

Adding and Subtracting Vectors

king - man + woman $\approx$ queen



Adding and Subtracting Vectors

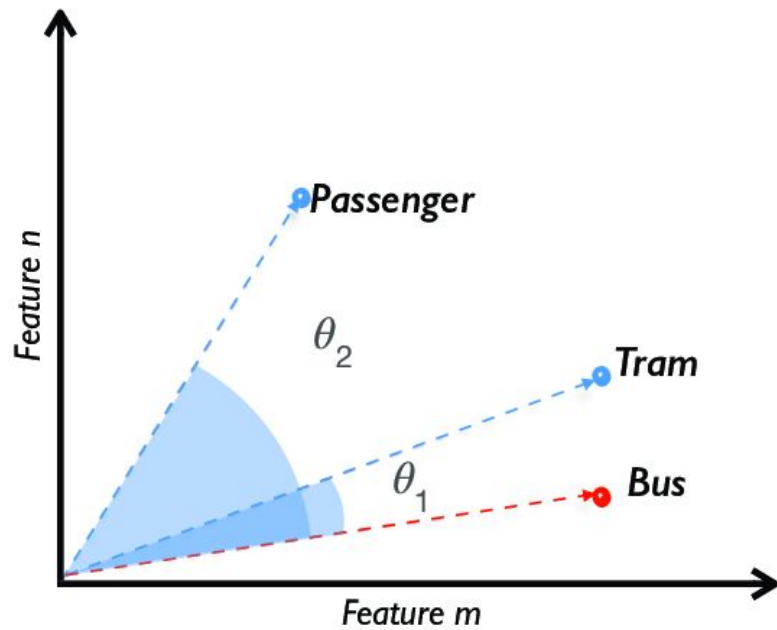
We can also use arithmetic on these embeddings to **derive meaning**.

For instance: “Berlin-Germany+France=Paris”

Similarity between words

Representing words by vectors, we can also compute similarity between words.

Usually **Cosine Similarity** is used



Two different paradigms for embeddings

1. Count-based paradigm

- a. Up until 2013, the traditional models for NLP tasks were count-based models. They mainly involve computing a **co-occurrence matrix** to capture meaningful relationships among words.

2. Prediction-based paradigm

- a. this model uses **neural networks** to learn representation of words in a vector space

Count-Based Paradigm

Doc 1: *all that glitters is not gold*

Doc 2: *all is well that ends well*

*	START	all	that	glitters	is	not	gold	well	ends	END
START	0	2	0	0	0	0	0	0	0	0
all	2	0	1	0	1	0	0	0	0	0
that	0	1	0	1	0	0	0	1	1	0
glitters	0	0	1	0	1	0	0	0	0	0
is	0	1	0	1	0	1	0	1	0	0
not	0	0	0	0	1	0	1	0	0	0
gold	0	0	0	0	0	1	0	0	0	1
well	0	0	1	0	1	0	0	0	1	1
ends	0	0	1	0	0	0	0	1	0	0
END	0	0	0	0	0	0	1	1	0	0

Count-Based Paradigm

- Square Matrix $V \times V$
- V is the size of the vocabulary
- 2 special tokens are added (START and END) to represent the beginning and end of sentences, paragraphs or documents
- Symmetric
- Sparse (too many zeros)

*	START	all	that	glitters	is	not	gold	well	ends	END
START	0	2	0	0	0	0	0	0	0	0
all	2	0	1	0	1	0	0	0	0	0
that	0	1	0	1	0	0	0	1	1	0
glitters	0	0	1	0	1	0	0	0	0	0
is	0	1	0	1	0	1	0	1	0	0
not	0	0	0	0	1	0	1	0	0	0
gold	0	0	0	0	0	1	0	0	0	1
well	0	0	1	0	1	0	0	0	1	1
ends	0	0	1	0	0	0	0	1	0	0
END	0	0	0	0	0	0	1	1	0	0

Count-Based Paradigm

- The word *is* appears one time next to the word *all*, *glitters*, *not*, *well*
- The word *gold* never appears neither next to the word *glitters* nor to the word *ends*
- Related words are observed (counted) together more often than unrelated words

*	START	all	that	glitters	is	not	gold	well	ends	END
START	0	2	0	0	0	0	0	0	0	0
all	2	0	1	0	1	0	0	0	0	0
that	0	1	0	1	0	0	0	1	1	0
glitters	0	0	1	0	1	0	0	0	0	0
is	0	1	0	1	0	1	0	1	0	0
not	0	0	0	0	1	0	1	0	0	0
gold	0	0	0	0	0	1	0	0	0	1
well	0	0	1	0	1	0	0	0	1	1
ends	0	0	1	0	0	0	0	1	0	0
END	0	0	0	0	0	0	1	1	0	0

Count-Based Paradigm

- This is a 10D vector representing the word *glitters*
- The vector represents the position of the word in ten-dimensional space

*	START	all	that	glitters	is	not	gold	well	ends	END
START	0	2	0	0	0	0	0	0	0	0
all	2	0	1	0	1	0	0	0	0	0
that	0	1	0	1	0	0	0	1	1	0
glitters	0	0	1	0	1	0	0	0	0	0
is	0	1	0	1	0	1	0	1	0	0
not	0	0	0	0	1	0	1	0	0	0
gold	0	0	0	0	0	1	0	0	0	1
well	0	0	1	0	1	0	0	0	1	1
ends	0	0	1	0	0	0	0	1	0	0
END	0	0	0	0	0	0	1	1	0	0

Count-Based Paradigm

When V is too large, the matrix become unmanageable.

In most NLP tasks, the co-occurrence matrix goes through PCA or SVD for **dimensionality reduction** and decomposed into a new k -dimensional matrix

*	k_1	k_2
START	0.705	0.484
all	0.705	-0.484
that	0.654	-0.783
glitters	0.52	0
is	1.027	0
not	0.654	0.783
gold	0.382	0.656
well	0.382	-0.656
ends	1.394	-1.061
END	1.394	1.061

Toward Prediction-based Paradigm

Many attempts were made to improve the performance of this model, but the model did not do well in capturing complex relationships among words.

In 2013, Thomas Mikolov proposed the prediction-based modeling technique, called Word2Vec.

The model uses **neural networks** to learn intelligent representations of words in a vector space.

Word2Vec

Word2Vec is a method to efficiently create word embeddings and has been founded by Google in 2013.

Word2Vec is a technique used to convert words into **multidimensional vectors**

Word2Vec takes a large **corpus of text as input** and generates a vector space with **hundreds of dimensions**. Each unique word in the corpus is assigned a vector in this space.

What is behind Word2Vec?

Word2Vec is trained using a neural network.

How is Word2Vec trained?

The tool defines two training models:

1. Continuous Bag Of Words (CBow)
2. Skip-Gram

One-hot encoding vector

	1	2	3	4	5	6	7	8
I	1	0	0	0	0	0	0	0
ate	0	1	0	0	0	0	0	0
an	0	0	1	0	0	0	0	0
apple	0	0	0	1	0	0	0	0
and	0	0	0	0	1	0	0	0
played	0	0	0	0	0	1	0	0
the	0	0	0	0	0	0	1	0
piano	0	0	0	0	0	0	0	1

Both CBoW and Skip Gram takes into account words represented by **one-hot encoding vectors**.

A one-hot encoding vector is a vector having size V (the number of words in the vocabulary). There are just a single 1 and $V-1$ zeros.

In the example, $[0, 0, 0, 1, 0, 0, 0, 0]$ is the word **apple**

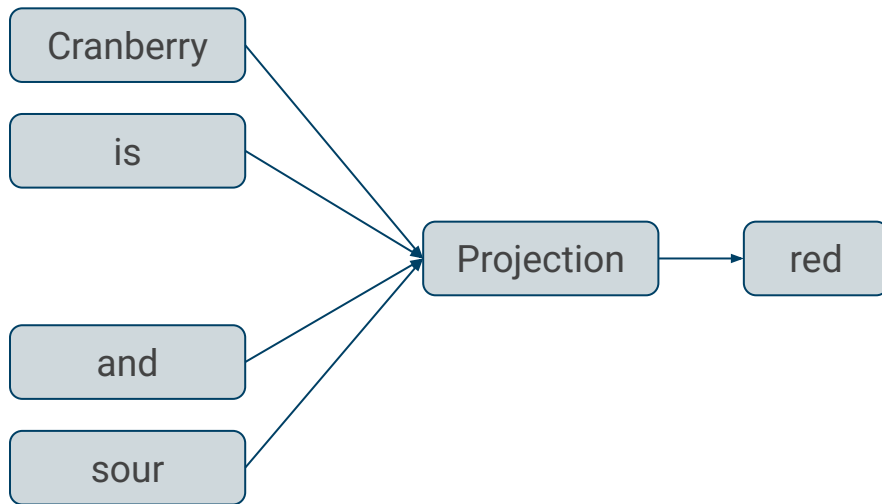
Continuous Bag Of Words

Continuous Bag Of Words

This model predicts the **center word** using the surrounding **context words**, which comprises a few words before and after the current word

You can think of CBOW like the “fill the gap” game.

Cranberry is ____ and sour



Continuous Bag Of Words: Sliding Window

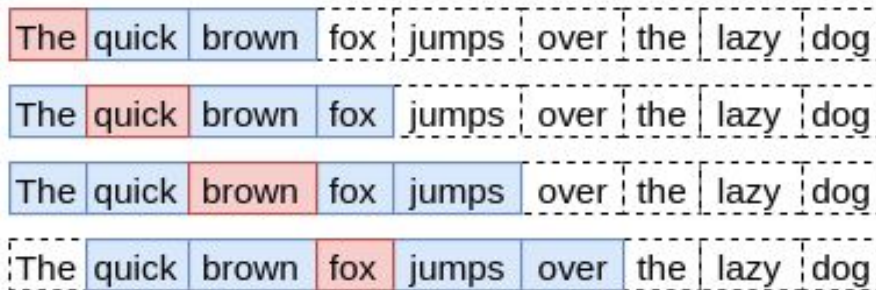


Text is processed using a **Sliding Window**.

In this example, the **window size** is 2 (two words before the center word and two after)

Red word is the **center word**, whilst blue words are the **context words**.

Continuous Bag Of Words: Sliding Window



CBOW predicts the red word given the context (blue words)

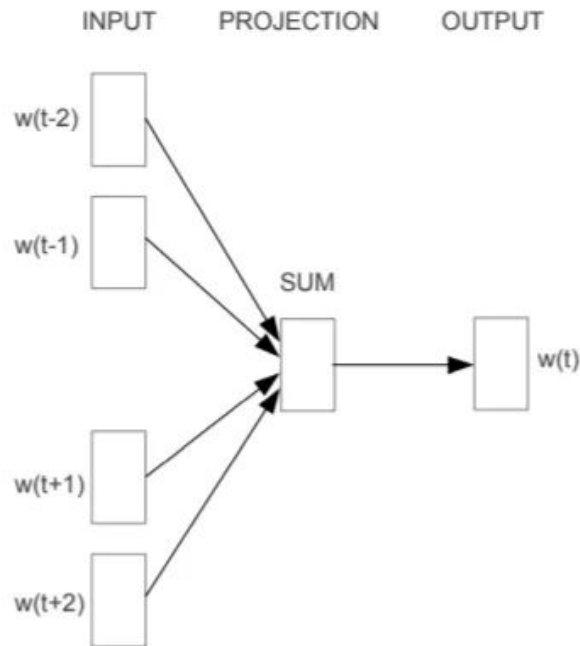
The size of the sliding window is a **hyper-parameter** and usually ranges between 1 and 10

Continuous Bag Of Words

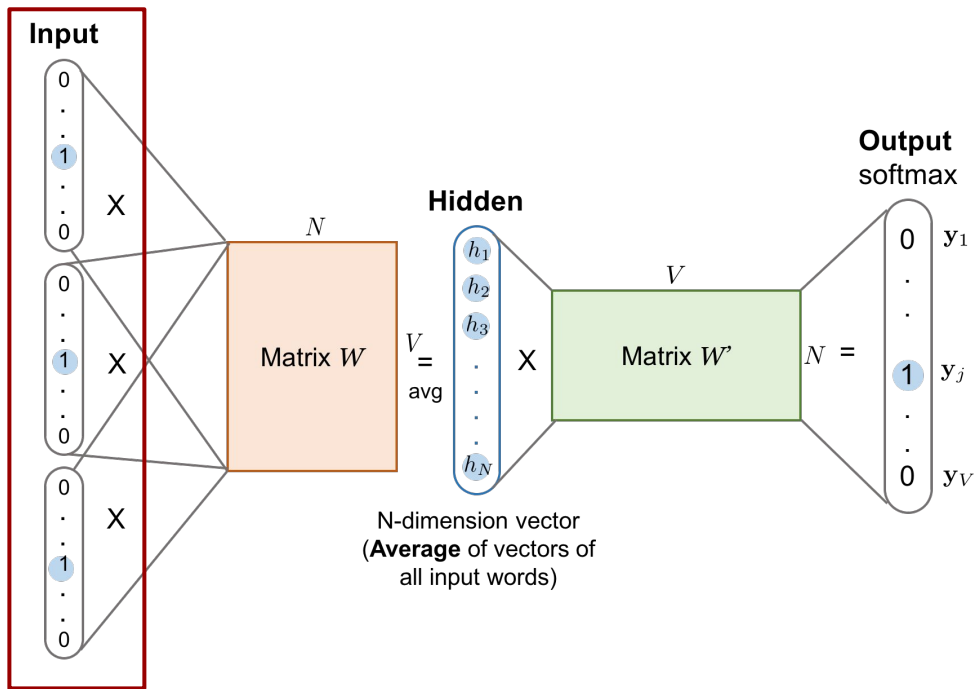
The CBOW model is based on a **Neural Network** consisting of an input layer, a hidden layer, and an output layer.

The input layer is used to represent the **context words**.

The output layer is used to predict the **center word**



Continuous Bag Of Words



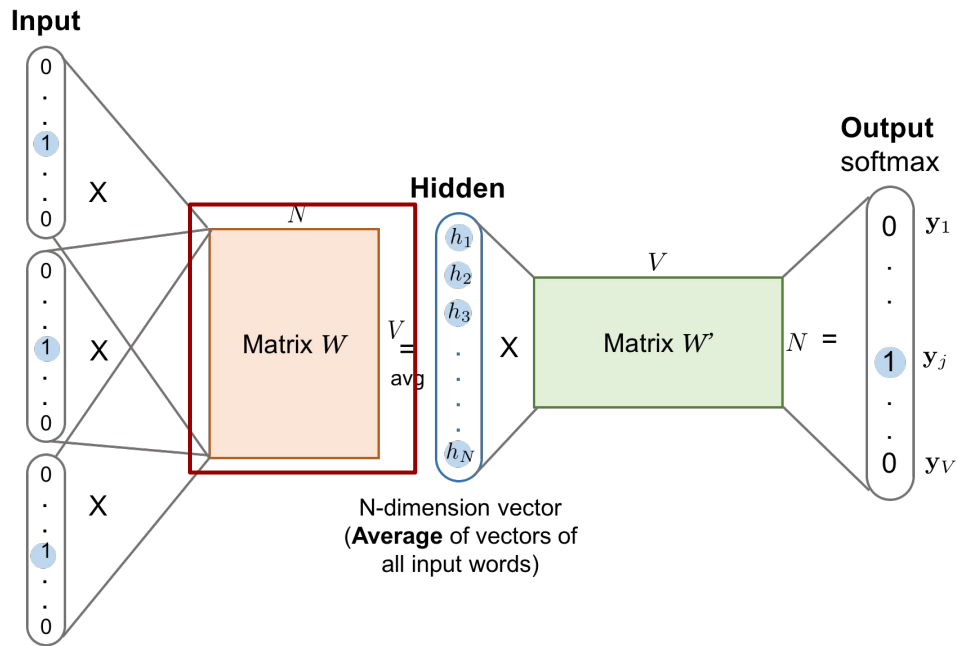
All the m surrounding context words are represented by V -dimension vectors.

V is the size of the vocabulary

M is the number of words in the context

Input vectors are **one-hot encoded vectors** ($V-1$ zeros, only one 1)

Continuous Bag Of Words

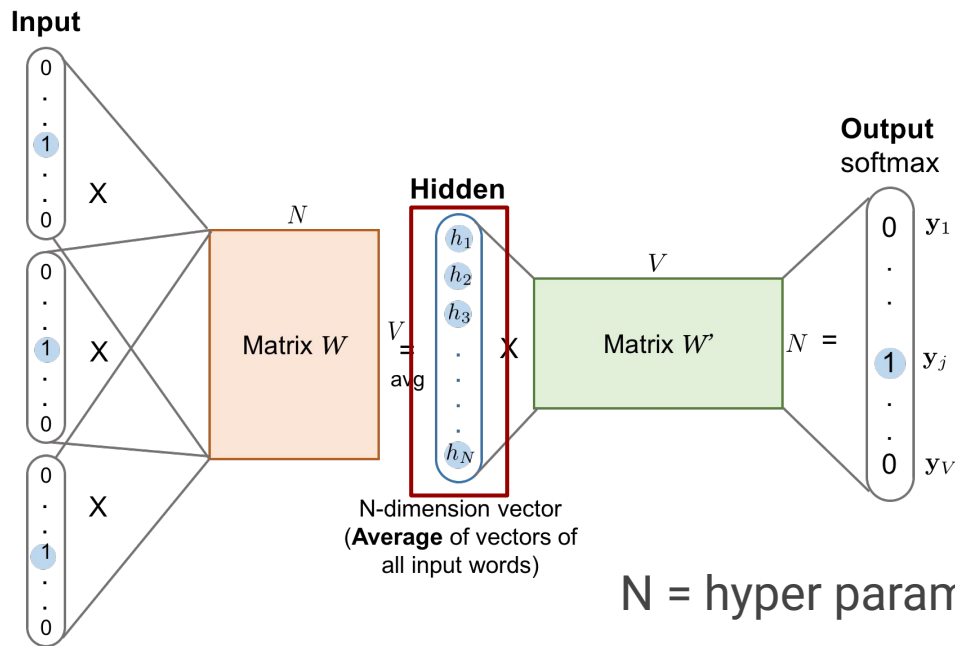


Matrix W is a $V \times N$ matrix representing the **context embedding matrix**

All the elements in W matrix are the first set of parameters that our Neural Network must learn, Usually it is initialized **randomly**

N is an hyper-parameter

Continuous Bag Of Words



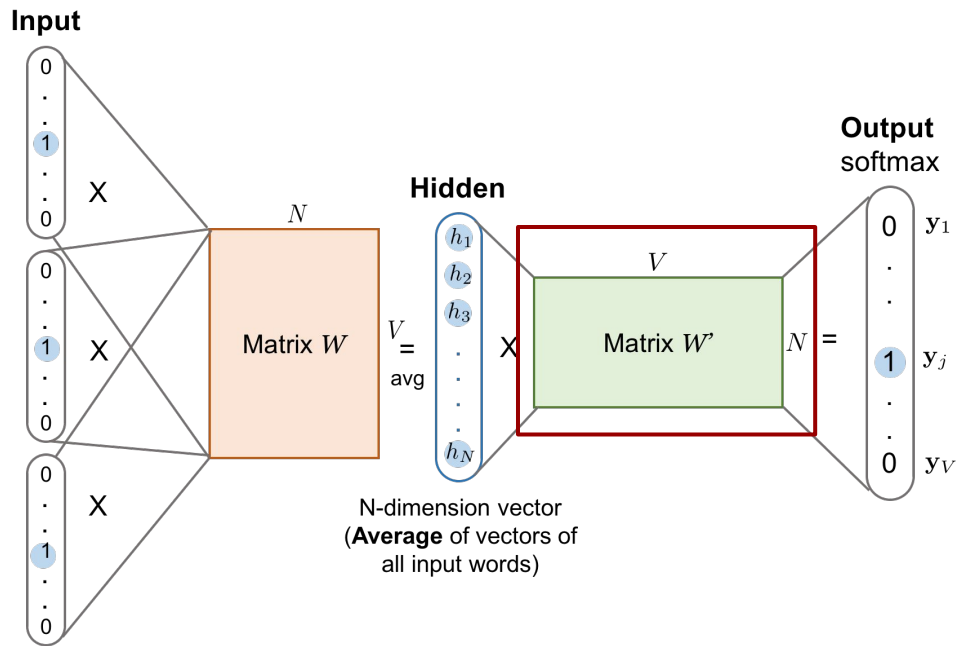
The hidden layer is a N -dimension vector which is computed by multiplying all input vectors by the matrix W , then computing the average.

$$h[j] = \frac{1}{M} \sum_{m=1}^M \sum_{i=1}^V x_m[i] W[i, j]$$

N = hyper parameter; V = Vocabulary size;

M = number of word i in the context

Continuous Bag Of Words

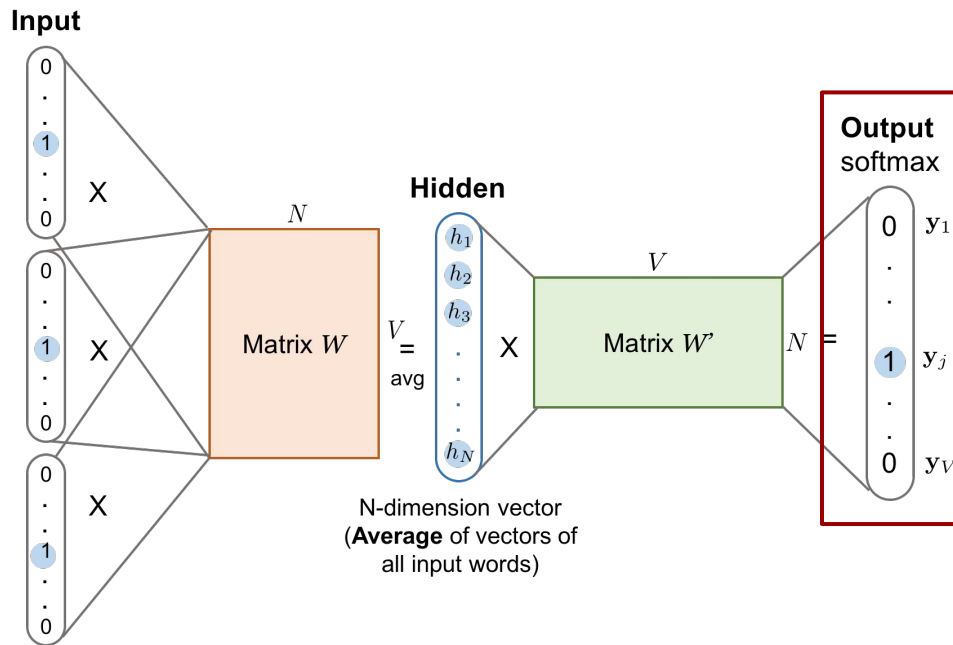


Matrix W' is a $N \times V$ matrix representing the **word embedding matrix**.

All the elements in W' matrix are the second set of parameters that our Neural Network must learn. Usually it is initialized **randomly**

N is an hyper-parameter

Continuous Bag Of Words

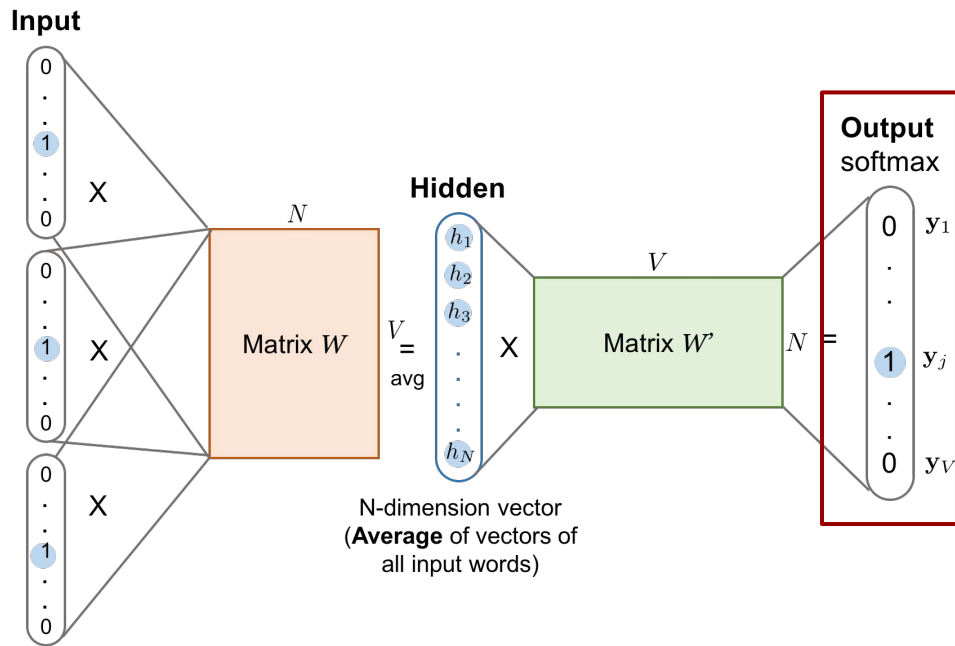


Predicted center word is represented by V -dimension vectors.

V is the size of the vocabulary

This vector is computed by multiplying the hidden vector by the matrix W' and applying the **Softmax Activation Function**

Continuous Bag Of Words

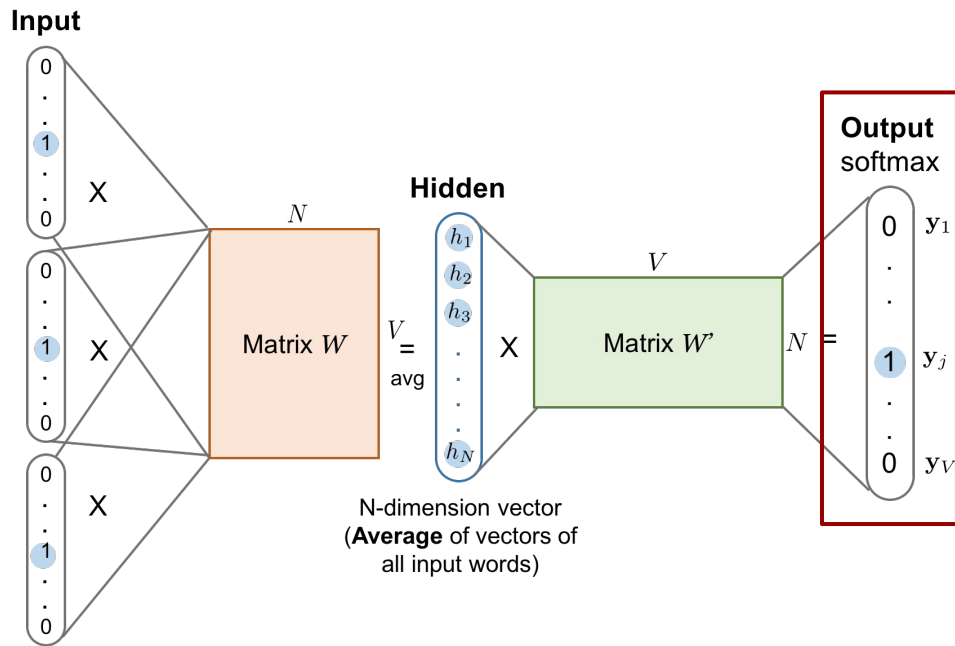


$$u[k] = \sum_{j=1}^N h[j]W'[j, k]$$

Softmax Activation Function

$$y[k] = \mu(u[k]) = \frac{e^{u[k]}}{\sum_{l=1}^V e^{u[l]}}$$

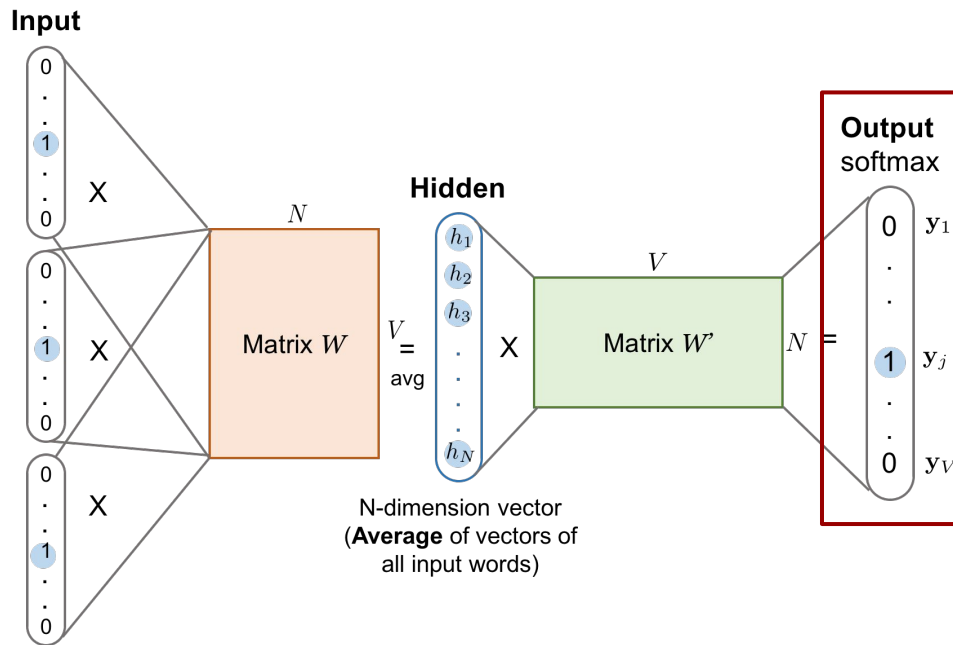
Continuous Bag Of Words



Ideally, we would like to have an output vectors like **one-hot encoded vector** ($V-1$ zeros, only one 1)

In reality, each element is a real value belonging to $[0, 1]$ representing the **conditional probability** for each word of being the center word.

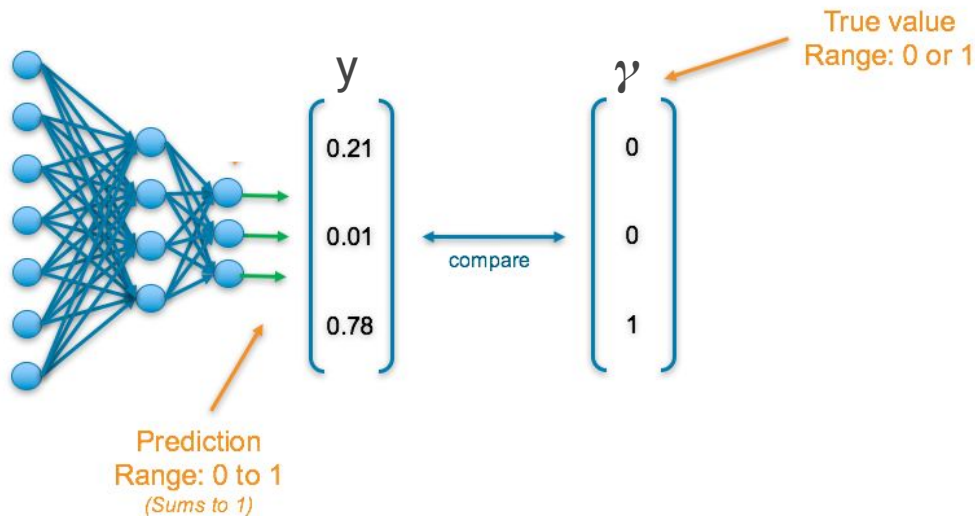
Continuous Bag Of Words



The sum of all elements belonging to the predicted vector must be **equal to 1**

The target word is selected as the word with the **highest probability** in y .

Continuous Bag Of Words: Error

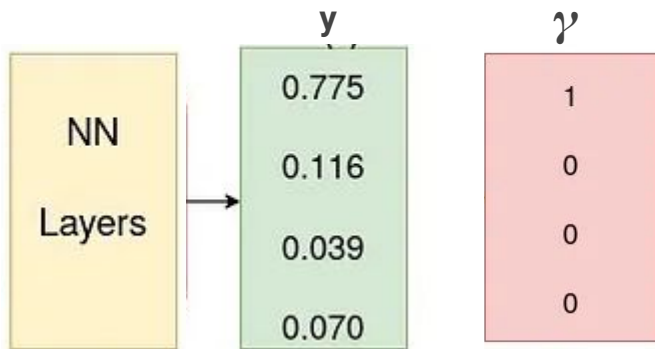


The prediction is not always the same as the the actual value.

The distance between the predicted value and the actual value is the **error**.

We would like to have a minimum error. So we need to define a **Loss Function**

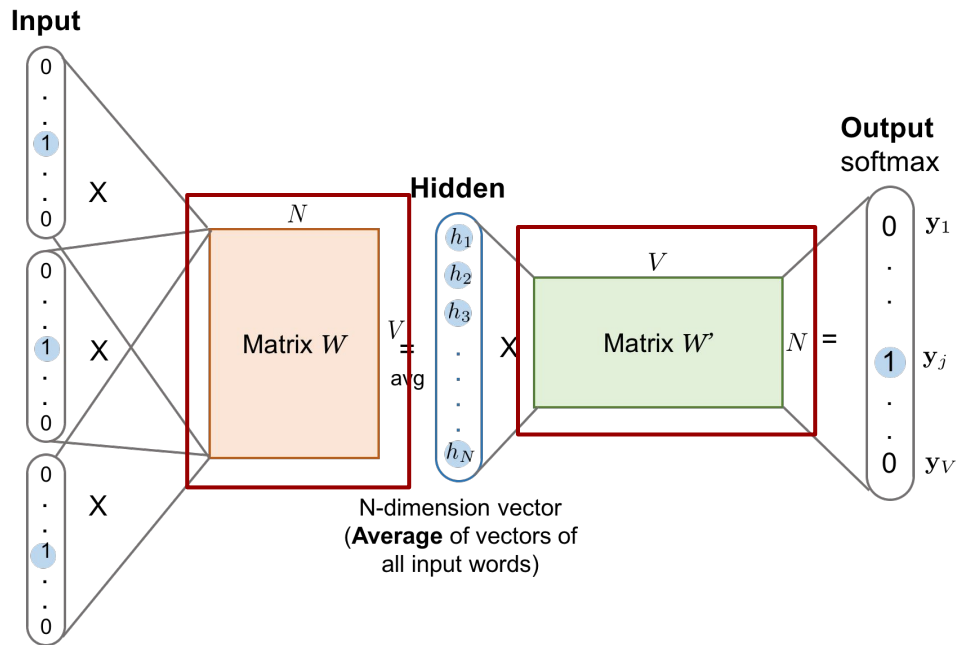
Continuous Bag Of Words: Cross Entropy



Both y and γ can be considered as **distributions**, so we can use the **Cross Entropy**.

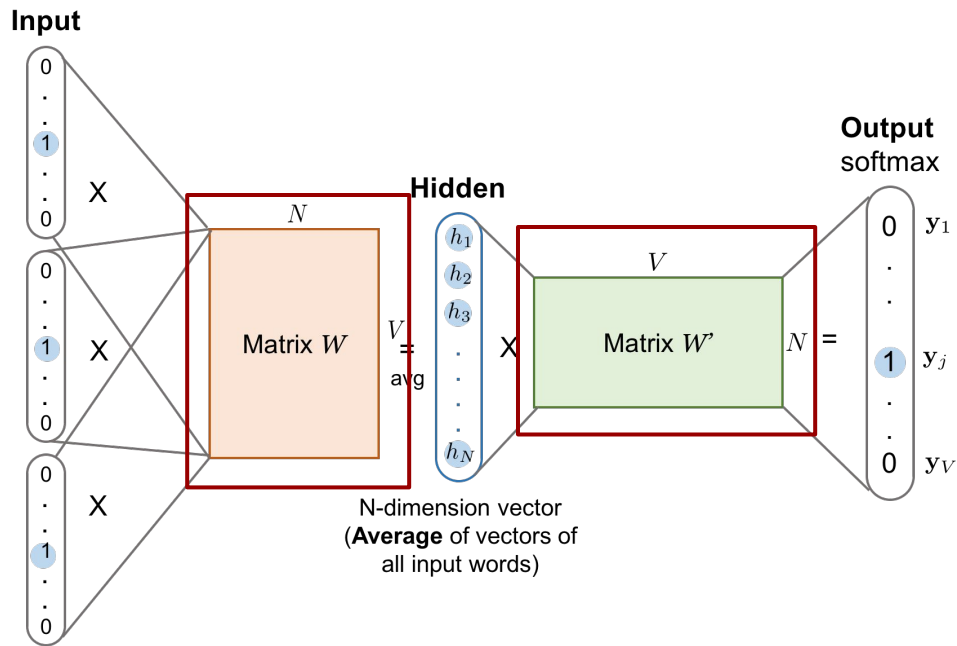
$$L(y, \gamma) = - \sum_{k=1}^V \gamma[k] \log_e(y[k])$$

Continuous Bag Of Words



To **reduce the error** between the predicted center word and the real center word, Matrices W and W' are optimized through **backward** propagations

Continuous Bag Of Words



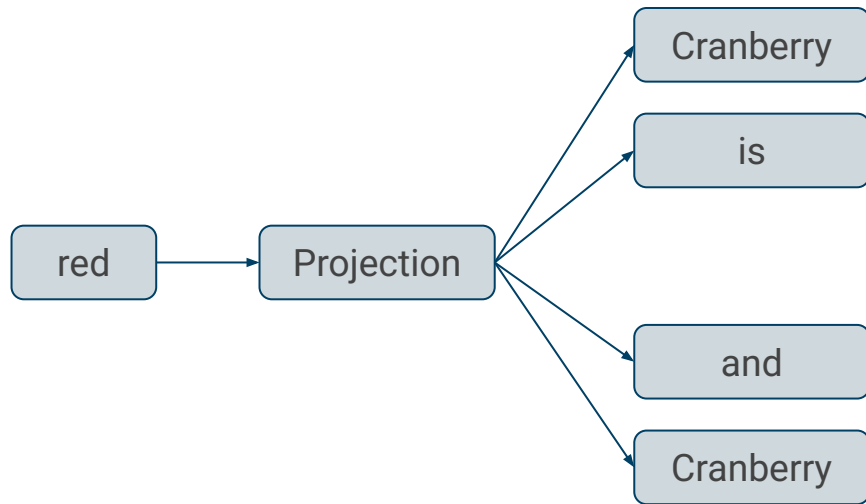
Parameters are adjusted running the Backpropagation phase and the **Stochastic Gradient Descent**

Skip-Gram

Skip-Gram

This model is the opposite of CBOW.

It aims to predict the surrounding **context words** given the **center word**.



Skip-Gram: Sliding Window



Even in Skip-Gram, text is processed using a **Sliding Window**.

In this example, the window size is 2 (two words before the center word and two after)

Red word is the **center word**, whilst blue words are the **context words**.

Skip-Gram: Sliding Window

The	quick	brown	fox	jumps	over	the	lazy	dog
The	quick	brown	fox	jumps	over	the	lazy	dog
The	quick	brown	fox	jumps	over	the	lazy	dog
The	quick	brown	fox	jumps	over	the	lazy	dog

Skip-Gram predicts the blue words given the center word (red word)

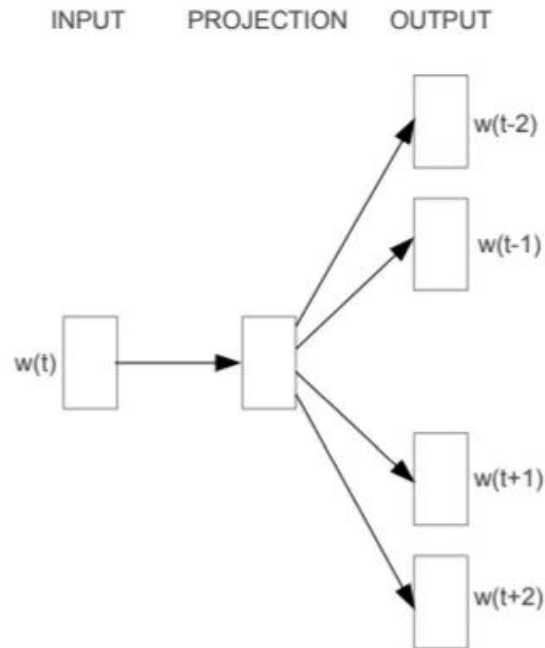
The size of the sliding window is a hyper-parameter and usually ranges between 1 and 10

Skip-Gram

Even the Skip-Gram model is based on a **Neural Network** consisting of an input layer, a hidden layer, and an output layer.

The input layer is used to represent the **center word**.

The output layer is used to predict the **surrounding context words**

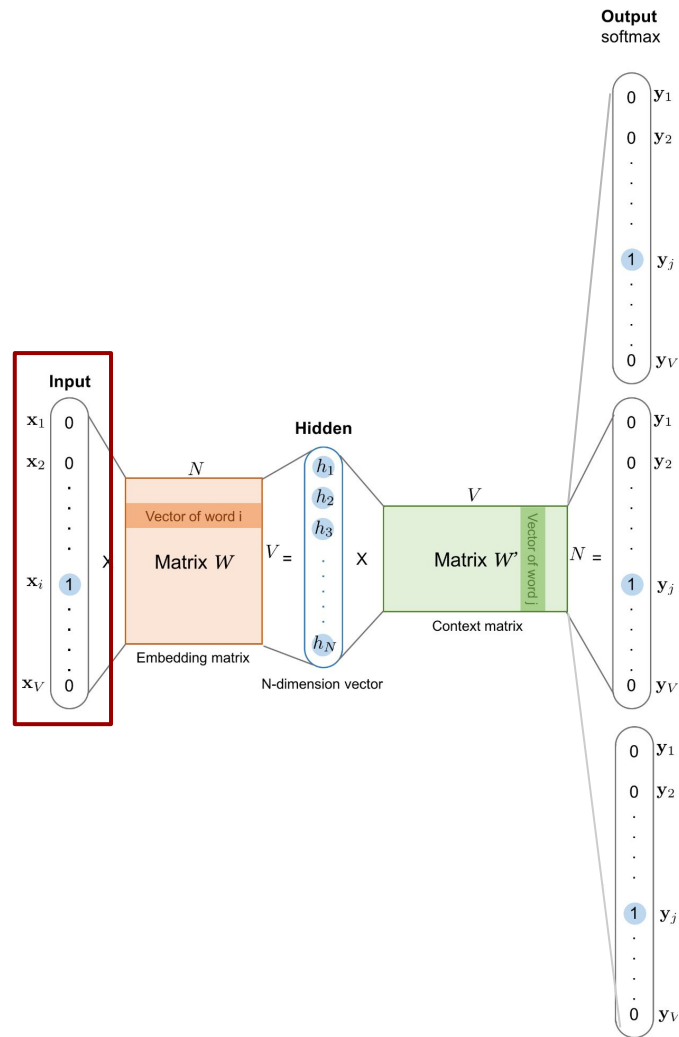


Skip-Gram

Opposite of CBOW:

One **one-hot encoded** V -dimension vector for the center word ($V-1$ zeros, only one 1)

V is the size of the vocabulary

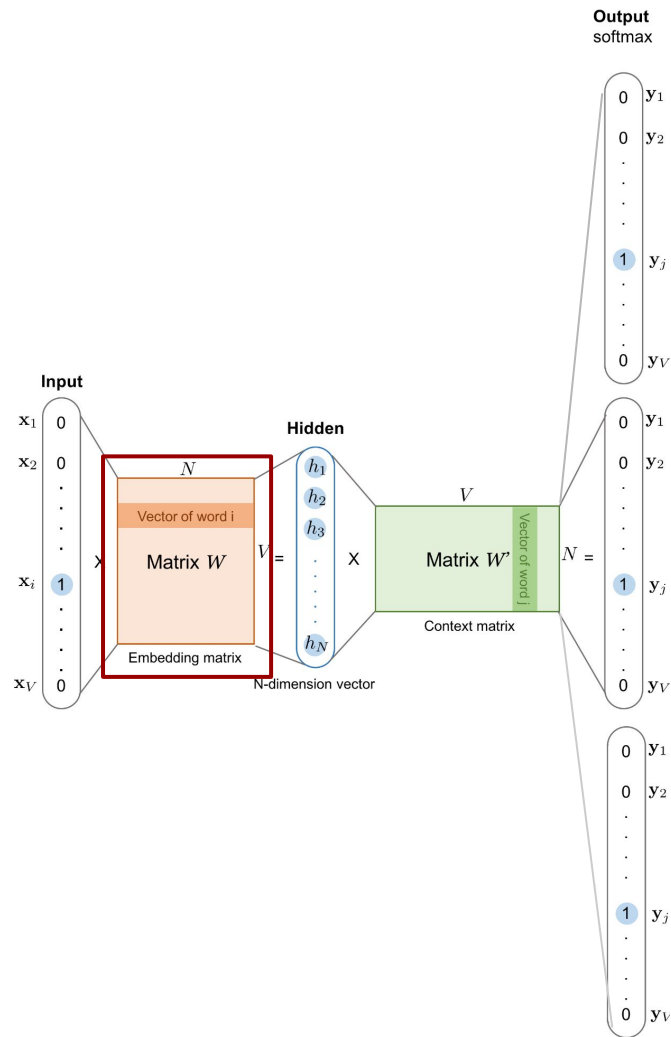


Skip-Gram

Matrix W is a $V \times N$ matrix representing the **word embedding matrix**

All the elements in W matrix are the first set of parameters that our Neural Network must learn. Usually it is initialized **randomly**

N is an hyper-parameter.



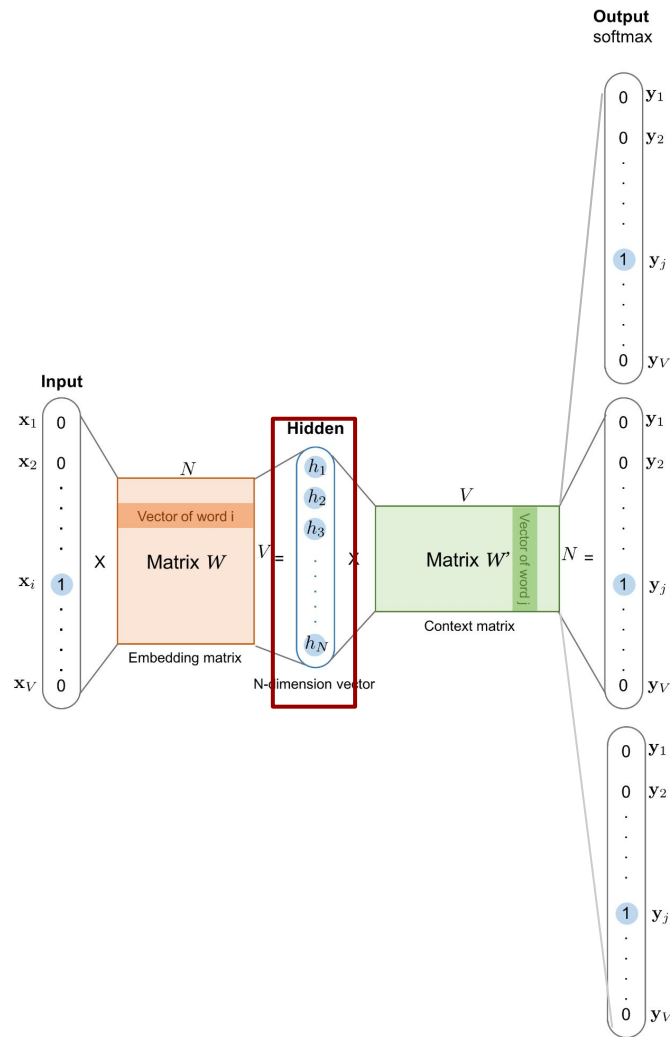
Skip-Gram

The hidden layer is a N-dimension vector which is computed by multiplying the input vector by the matrix W.

$$h[j] = \sum_{i=1}^V x[i]W[i, j]$$

V = Vocabulary Size

N = Hyper Parameter

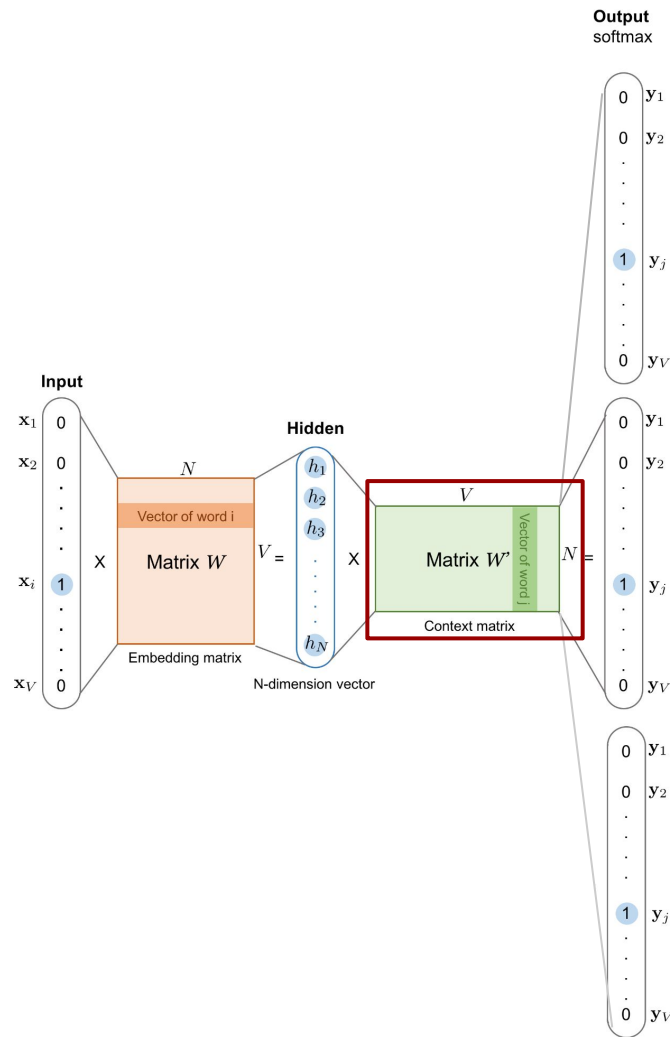


Skip-Gram

Matrix W' is a $N \times V$ matrix representing the **Context Matrix**.

All the elements in W' matrix are the second set of parameters that our Neural Network must learn. Usually it is initialized **randomly**

N is an hyper-parameter

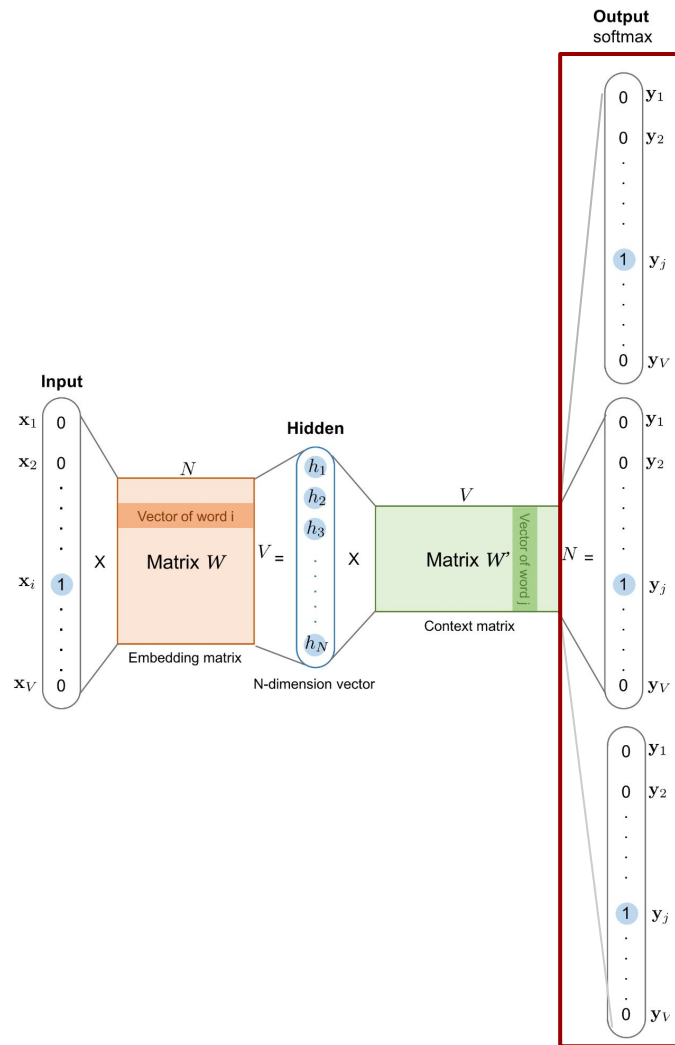


Skip-Gram

The output are m N -dimension vectors for the output representing all the predicted words in the context

V is the size of the vocabulary

These vectors are computed by multiplying the hidden vector by the matrix W' and applying the **Softmax Activation Function**

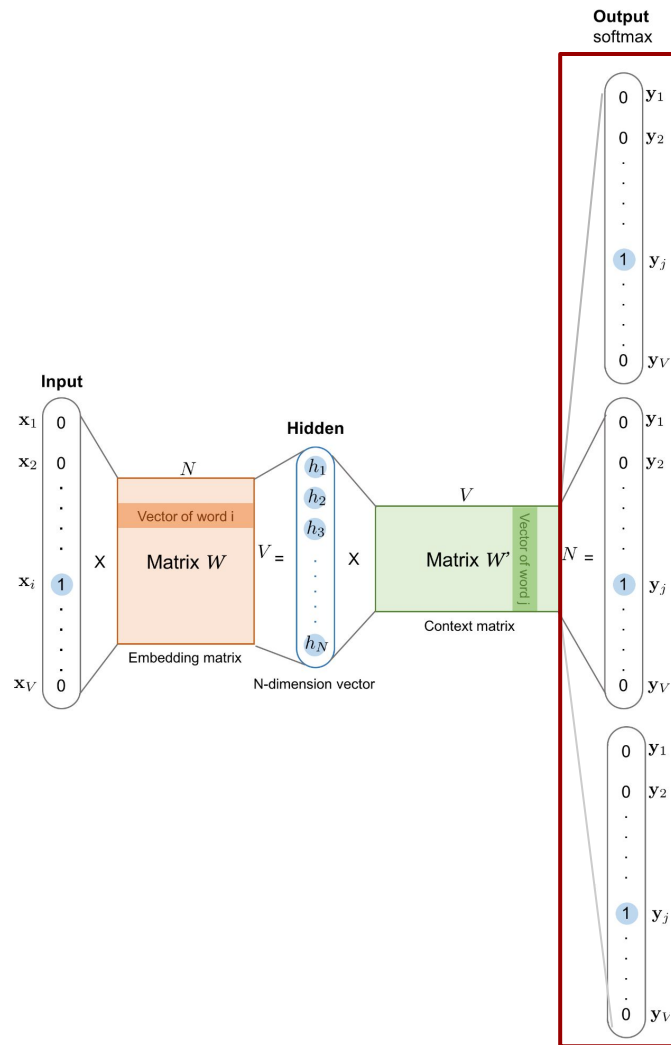


Skip-Gram

$$u[k] = \sum_{j=1}^N h[j] W'[j, k]$$

Softmax Activation Function

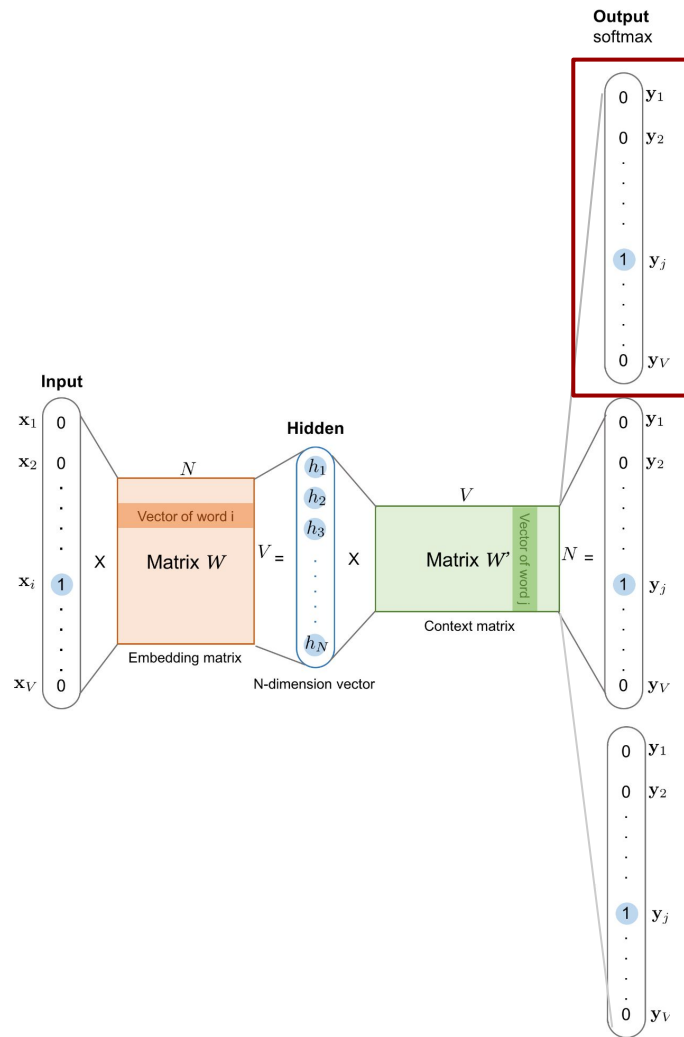
$$y_m[k] = \mu(u[k]) = \frac{e^{u[k]}}{\sum_{l=1}^V e^{u[l]}}$$



Skip-Gram

Ideally, we would like to have an output vectors like **one-hot encoded vector** ($V-1$ zeros, only one 1)

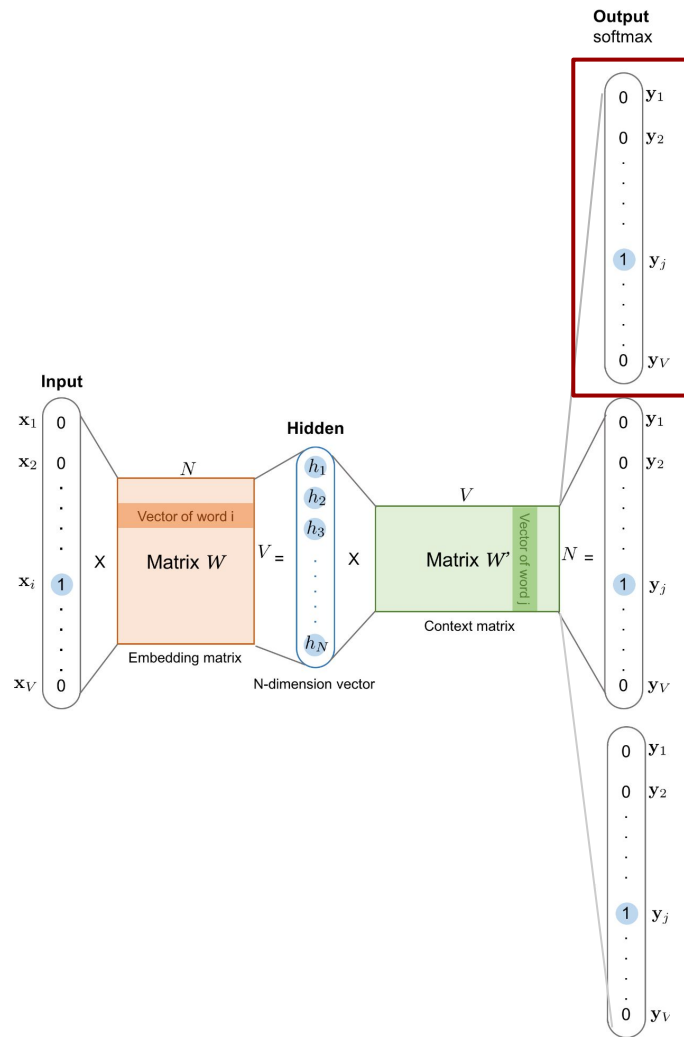
In reality, each element is a real value belonging to $[0, 1]$ representing the **conditional probability** for each word of being the context word.



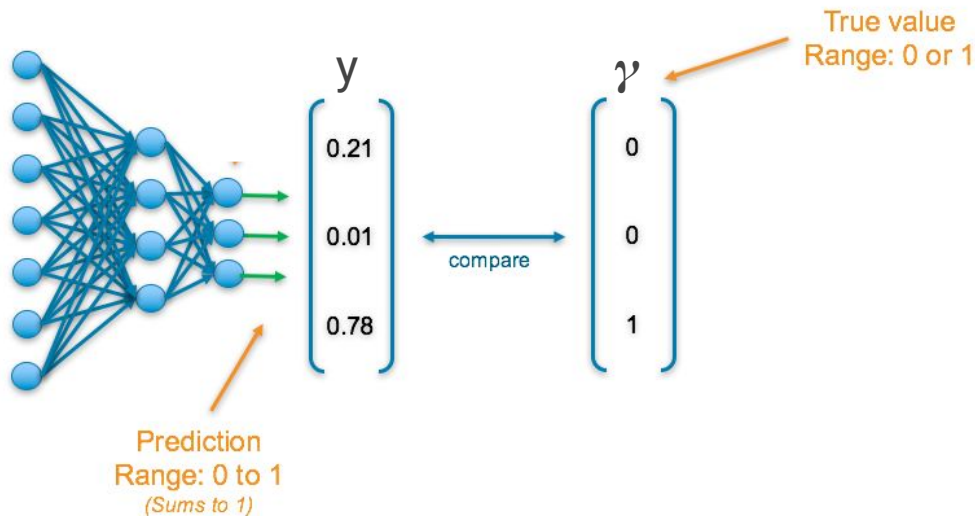
Skip-Gram

The sum of all elements belonging to each predicted vector must be **equal to 1**

The target word is selected as the word with the **highest probability** in y .



Skip-Gram: Error

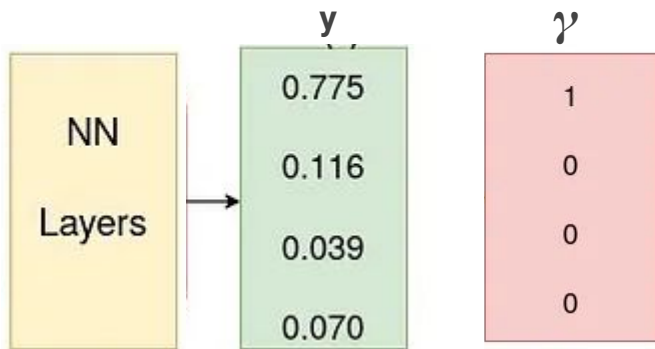


The prediction is not always the same as the the actual value.

The distance between the predicted value and the actual value is the **error**.

We would like to have a minimum error. So we need to define a **Loss Function**

Skip-Gram: Cross Entropy

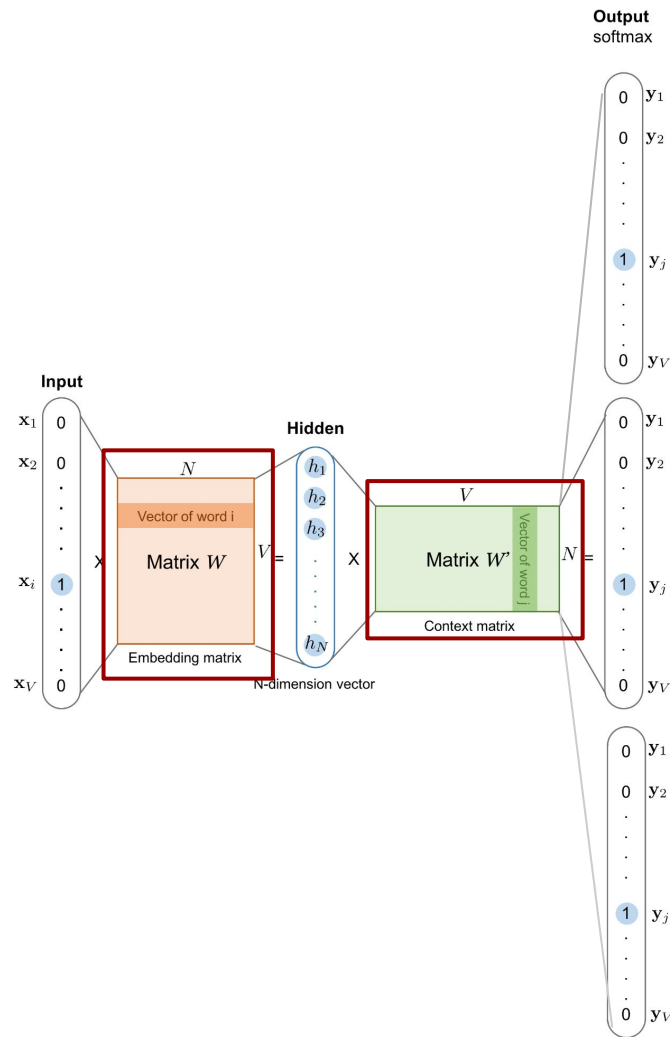


Both y and γ can be considered as **distributions**, so we can use the **Cross Entropy**.

$$L(y, \gamma) = - \sum_{m=1}^M \sum_{k=1}^V \gamma[k] \log_e(y_m[k])$$

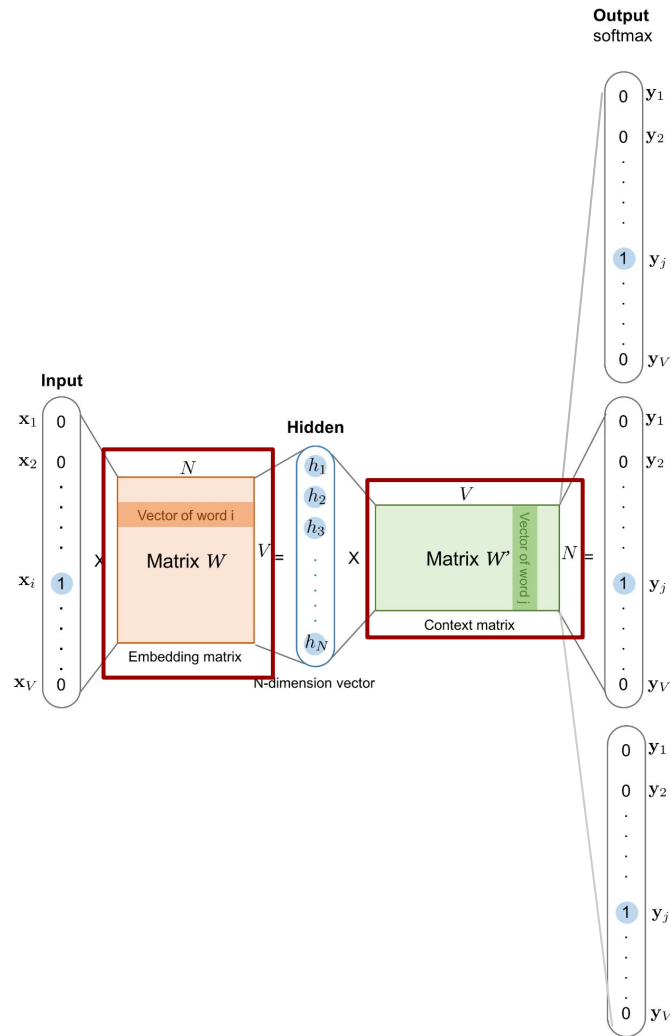
Skip-Gram

To reduce the error between the predicted context words and the real context words, Matrices W and W' are optimized through forward and **backward** propagations.



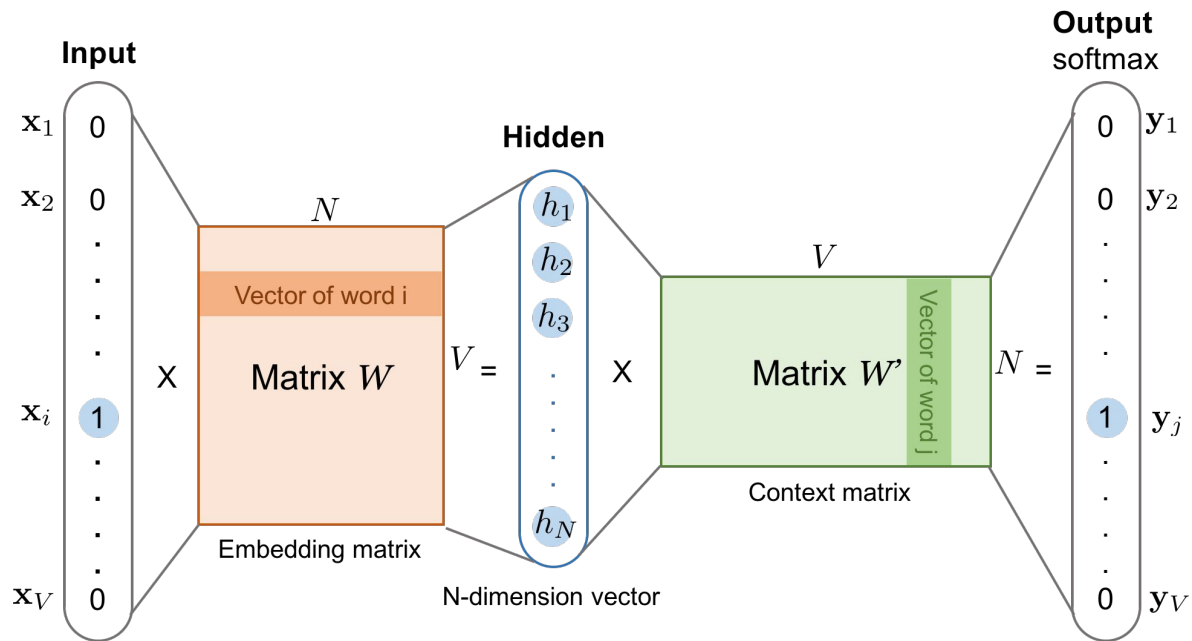
Skip-Gram

Parameters are adjusted running the Backpropagation phase and the **Stochastic Gradient Descent**



Vectorization

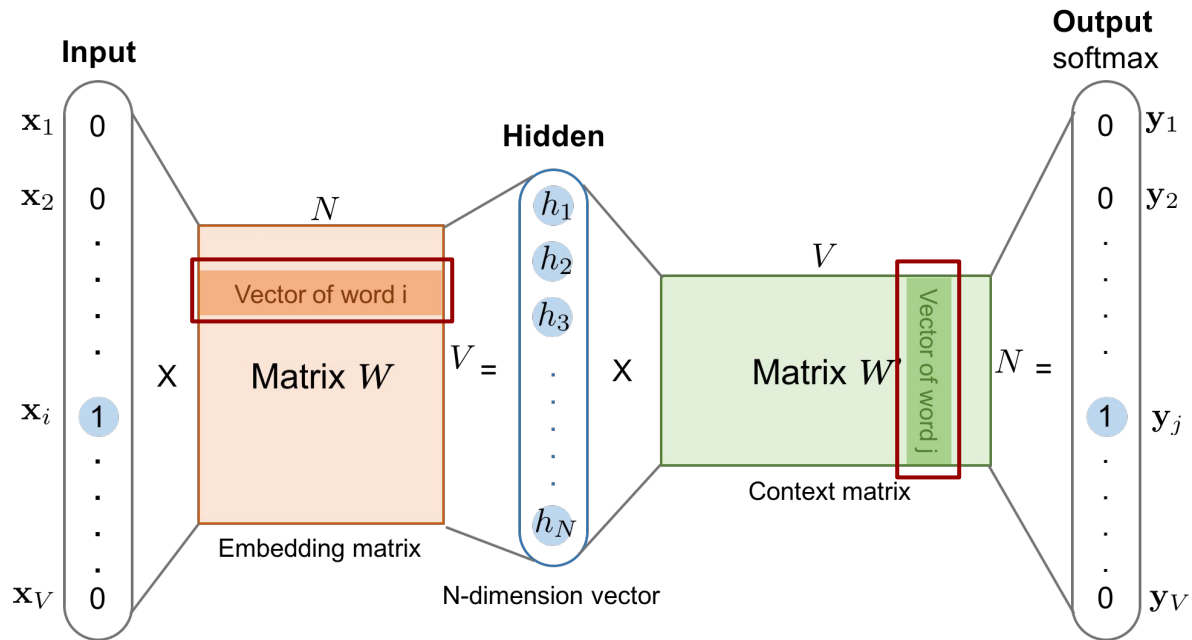
How vectors are really computed?



Supposing to have only one word in the surrounding context, the shape of the neural network will be like this.

This NN tries to predict the following word given a center word

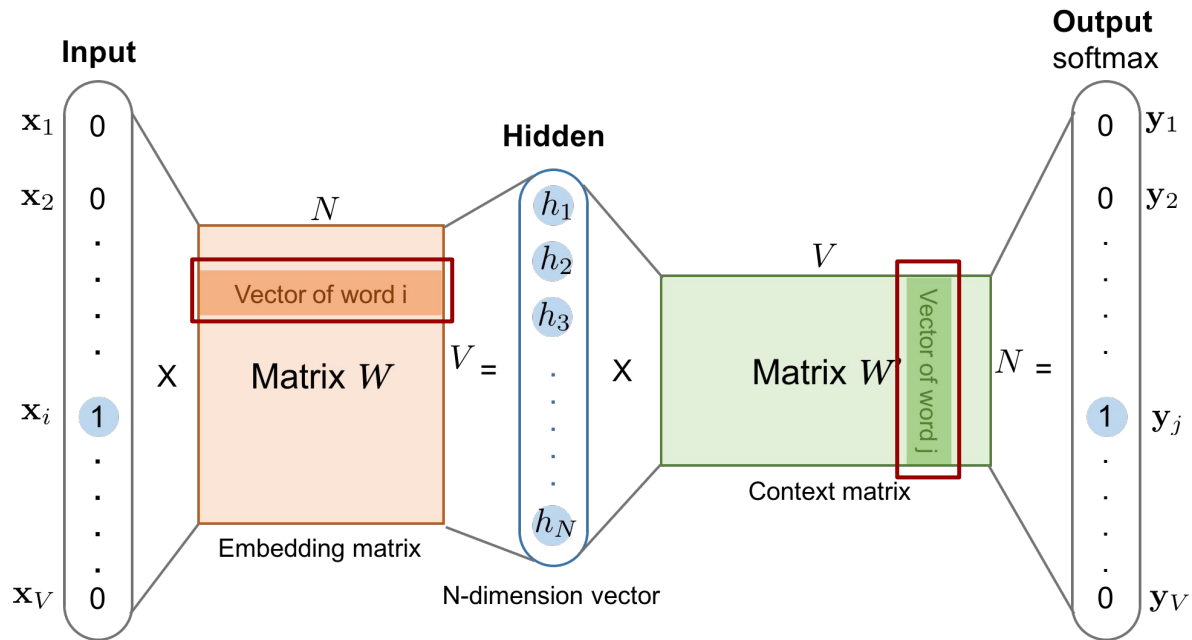
How vectors are really computed?



A row in the matrix W represents the **word emembedding** for a word in the vocabulary

A column in matrix W' encodes the **meaning of words as context**

How vectors are really computed?



Despite the name, W' is independent of W , not a transpose or inverse or whatsoever.

N is the size of the embedded word

Word2Vec Tool

Download and compile Word2Vec

Open <https://code.google.com/archive/p/word2vec/>

Select Source > Download > source-archive.zip

```
unzip source-archive.zip
```

```
cd word2vec/trunk
```

```
make
```

Download the corpus text

wget <http://mattmahoney.net/dc/text8.zip>

unzip text8.zip

text8 is a text corpus containing 16,718,843 words

text8 contains 71,291 unique words (to be inserted into the vocabulary)

Word2Vec

The word2vec tool takes a text corpus as input and produces the word vectors as output. It first constructs a **vocabulary** from the training text data and then learns **vector representation** of words. The resulting word vector file can be used as features in many natural language processing and machine learning applications.

Run Word2Vec

```
./word2vec
-train text8          // text corpus
-output vectors.bin   // word vector file
-cbow 1              // 1 = Continuous Bag-of-Words (CBOW) 0 = Skip-gram model (SG)
-size 200             // vector dimensionality
-window 8            // size of the context window
-threads 20          // number of threads to use
-binary 1            // the format of the output word vector file (0 = text, 1 = binary)
-iter 15             // Number of iterations (epochs) over the corpus
```

Running Similarities

Suppose we want to compute similarities between the word *learning* and all the words in the dataset:

```
$ ./distance vectors.bin learning
```

Word	Cosine distance
educational	0.539611
learners	0.520408
teaching	0.514668
learn	0.490442
knowledge	0.487726
cognitive	0.477808
education	0.476150

Running Linguistic Regularities

paris - france + berlin = germany

```
$ ./word-analogy vectors.bin Paris France Berlin
```

germany	0.630140
reunified	0.590625
gdr	0.503846
austria	0.486030
russia	0.482404
rhineland	0.477394
germanies	0.461999
dortmund	0.444944
reunification	0.439780

Where to obtain training data

The quality of the word vectors increases significantly with amount of the training data. For research purposes, you can consider using data sets that are available on-line:

- First Billion Characters from wikipedia: <http://mattmahoney.net/dc/enwik9.zip>
- Latest Wikipedia dump:
<http://dumps.wikimedia.org/enwiki/latest/enwiki-latest-pages-articles.xml.bz2>
- ...

Pretrained Word and Phrase Vectors

Google News: [GoogleNews-vectors-negative300.bin.gz](#) (1.5GB)

The dataset contains vectors trained on part of Google News dataset (about 100 billion words). The model contains 300-dimensional vectors for 3 million words and phrases.

Pretrained Italian Word Vectors

[Italian Word Embeddings | Italian Natural Language Processing Lab](#)

two dataset:

1. itWaC: billion word corpus constructed from the Web limiting the crawl to the .it domain and using medium-frequency words from the Repubblica corpus and basic Italian vocabulary lists as seeds
2. Twitter: 46.935.207 tweets

Document Embedding

Doc2Vec

Tool for transforming a document into a vector

A vector represents the theme or the overall meaning of a document

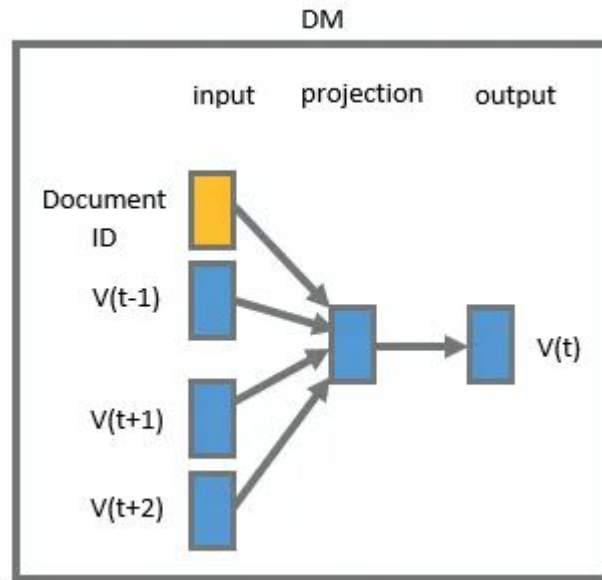
Two training model:

- Distributed Memory Model Of Paragraph Vectors (PV-DM), which is similar to the Continuous Bag of Words Model (CBOW)
- Paragraph Vector With A Distributed Bag Of Words (PV-DBOW), which is similar to the Skip-Gram Model

Distributed Memory Model

Similar to CBOW model

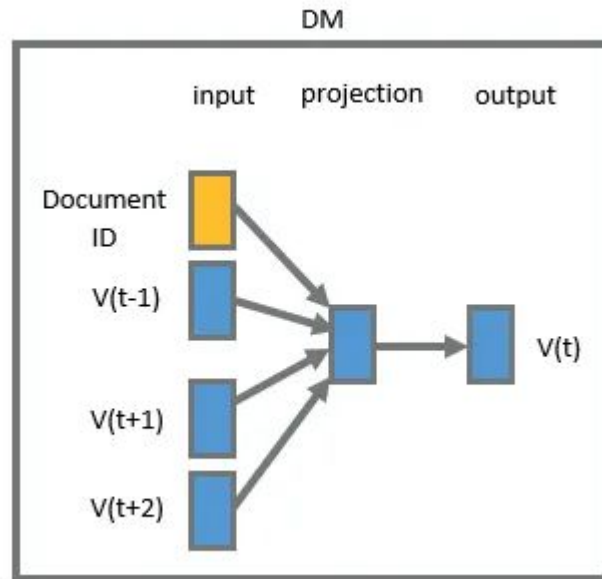
It tries to predict target word given its surrounding context words with the **addition of a Document ID**



Distributed Memory Model

The context words are used to predict a target word, and the document ID is used to capture the **overall meaning** of the document.

The projection layer is responsible for creating the **word vectors** and **document vectors**



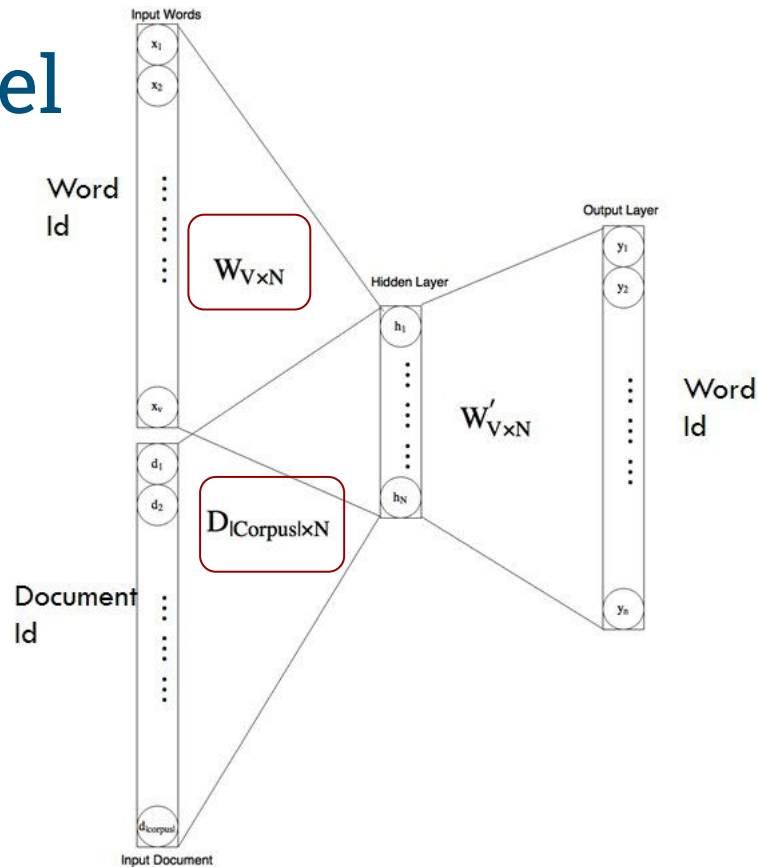
Distributed Memory Model

Two matrices:

W: $V \times N$ matrix for word embedding

D: $C \times N$ matrix for document embedding

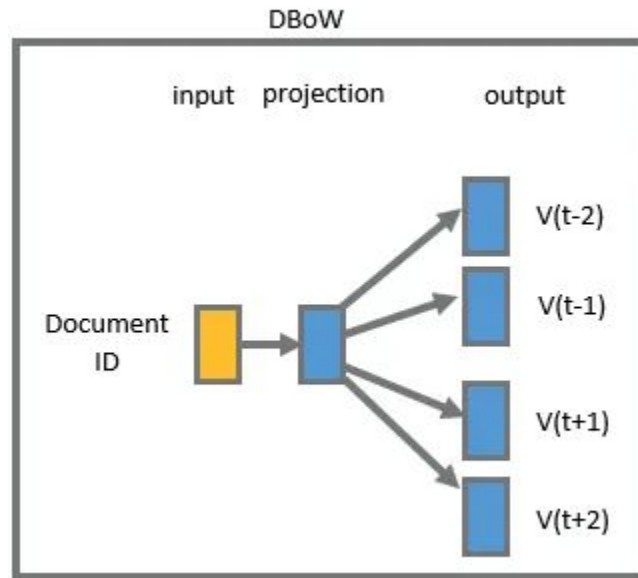
C is the number of documents in the collection



Distributed Bag-Of-Words Model

Similar to Skip-Gram model

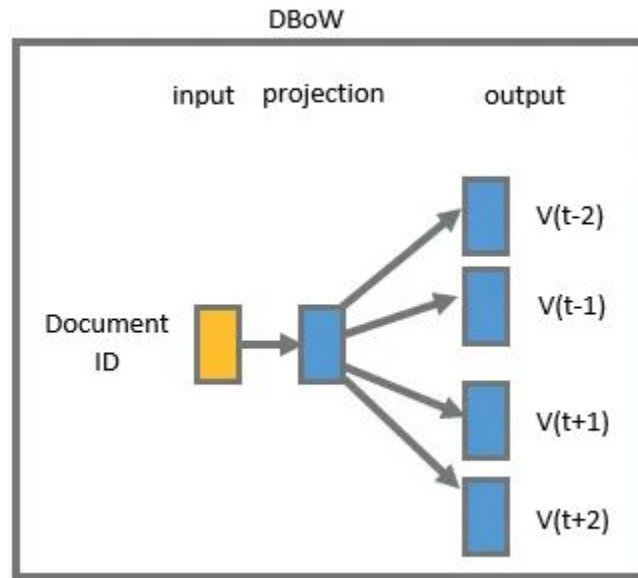
Instead of using the target word as the input, It takes the document ID as the input and tries to **predict the probability of each word** in the document given only the document vector



Distributed Bag-Of-Words Model

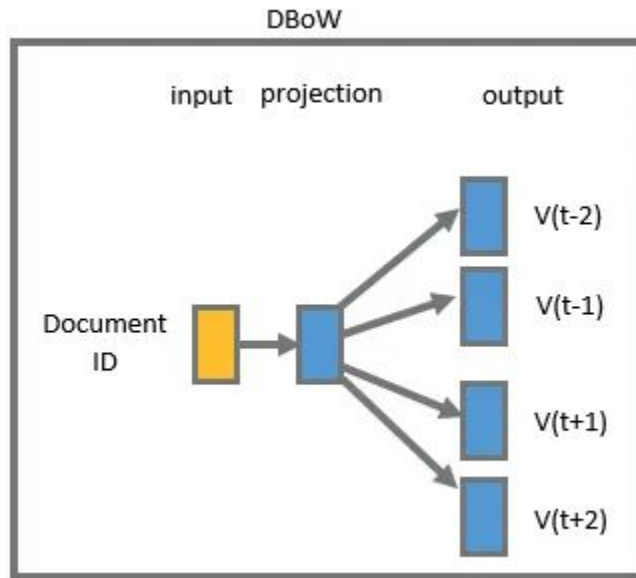
Similar to Skip-Gram model

It focuses on understanding how **words are distributed** in a text, rather than their meaning



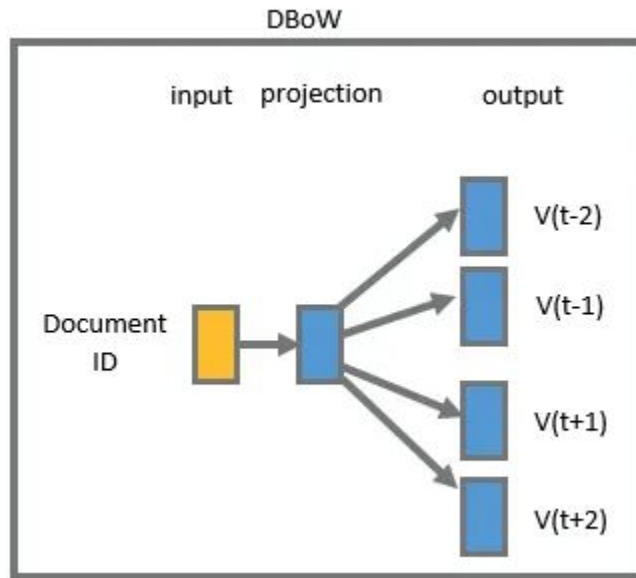
Distributed Bag-Of-Words Model

The model does not take into account the order of the words in the document, treating the document as a collection or “bag ” of words.



Distributed Bag-Of-Words Model

This makes the DBOW architecture faster to train than DM, but potentially **less powerful in capturing the meaning** of the documents



FastText



FastText

It is an extension of the popular Word2Vec model.

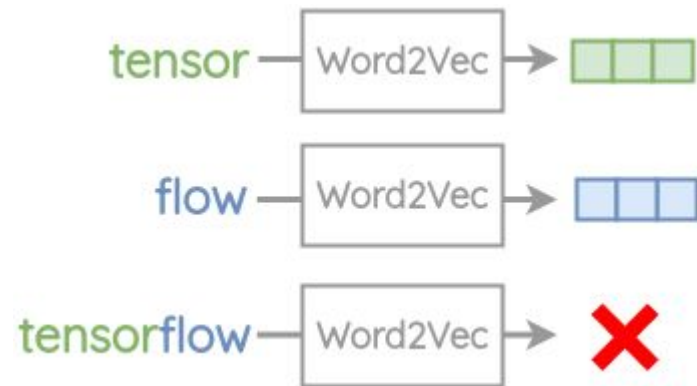
It has been developed by Facebook in 2017

Tools, libraries and already-available vector representation for 157 languages are available of **<https://fasttext.cc/>**

FastText: limits #1 of Word2Vec

In Word2Vec, an embedding is created for each word.

For missing words, there is no way to get the vector representation, not even if the missing word is made by the concatenation of two available words



FastText: limits #2 of Word2Vec

In Word2Vec, each word is learned uniquely **based on the context** it appears in.

Vectors are created without utilizing the **internal structure** of the word to make the process more efficient.

Shared radical

eat eats eaten eater eating

FastText: how it works

The idea behind FastText: instead of creating embedding of just words, it creates embeddings of subword (called **n-grams**).

n is an integer ranging between 3 and 6

To capture meaning for suffixes/prefixes, **angular brackets** are added at the beginning and at the end of each word.

Embeddings can be created using either CBoW or SkipGram.

FastText: how it works

eating → <eating>

We add the brackets

<ea, eat, ati, tin, ing, ng>

n = 3 - 6 three-grams

<eat, eati, atin, ting, ing>

n = 4 - 5 four-grams

<eati, eatin, ating, ting>

n = 5 - 4 five-grams

<eatin, eating, ating>

n = 6 - 3 six-grams

FastText: how it works

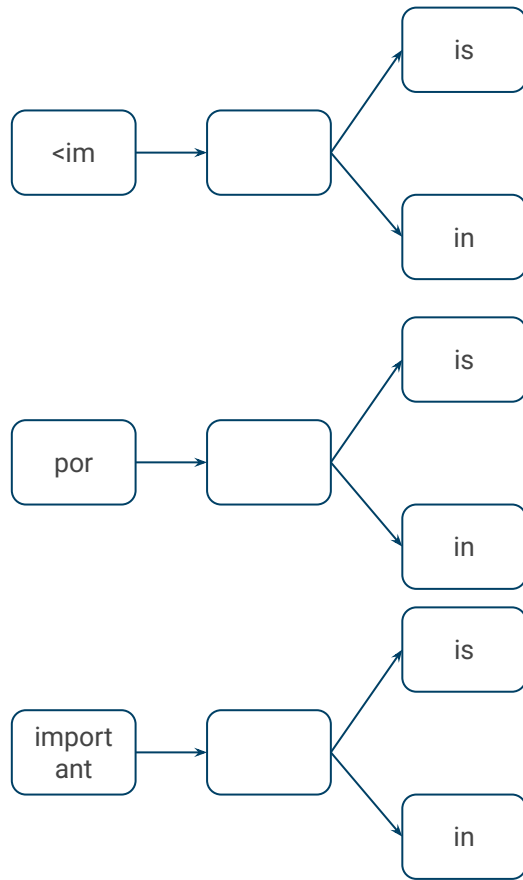
- During training, FastText learns embeddings **for each n-gram** and, if the word itself is common enough, it also learns an embedding for the **whole word**
 - FastText considers a word as common if it appears at least **5 times** into the corpus
- Embedding can make use of CBoW or SkipGram
- Each n-gram and the whole word is considered as **center word**
- The context instead is the **context** of the whole word

FastText: how it works

For example, the sentence is

strategy is important in playing chess

- center word: important
- n-grams: <im, imp, mpo, por, ...
- size of window: 1
- model: skip-gram



FastText: how it works

After completing the learning process, when a word vector is needed, FastText computes the word vector dynamically by **averaging the embeddings of all relevant n-grams** and the whole-word vector if it exists

$$\text{vector}_{\text{word}} = \frac{1}{N+1} \left(\sum_{i=1}^N \text{vector}_{\text{n-gram}_i} + \text{vector}_{\text{whole-word}} \right)$$

N is the total number of n-grams (3-grams, 4-grams, 5-grams, and 6-grams) for the word

Mini Boolean Retrieval Information System

Create a small-scale search engine for a specific collection of documents (e.g., a set of Wikipedia articles, a university's course catalog). This project would involve at least building a crawler to gather data, one or more persistent inverted indexes (Standard Inverted Index, BiWord Index, Phrase Index, Positional Index, Permuterm Index, K-Gram Index, Parametric Index, Zone Index, Tiered Index) for efficient searching, and a ranking algorithm like TF-IDF or BM25. The project must make use of the data structures described during lectures (Linked List, B-Tree, Binary Heap, Trie, Binary Tree) and should implement one or more further functionalities:

- Conjunctive query, Optimized Conjunctive Query, Disjunctive query, Phrase query
- WildCard Query
- Proximity Operators
- Linguistic Preprocessing: Normalization, Stop List, Stop Word, Stemming
- Augmentation
- Skip List
- Spelling Correction (Edit Distance, K-gram overlap, Soundex Algorithm)
- Distributed Indexing
- Posting List Compression and Dictionary Compression
- Page Similarity (Fingerprint, Signature)

The paper must describe at least:

- the document collection
- the data structures used in the project to implement Dictionary and Posting Lists
- all the implemented functionality used during indexing and searching
- strengths and weakness of the implementation

Mini Semantic Retrieval Information System

Build a search engine that retrieves documents based on their semantic relevance to a query, not just on exact keyword matches. You must represent both the query and documents as vectors and then rank documents by their cosine similarity to the query vector. Vector components can be computed using the Term Count (tc), the Term Frequency (tf), the TF_IDF model or by averaging the Word2Vec or FastText vectors of the words they contain, or Doc2Vec. The project can also implement one or more functionalities listed in the previous section.

The paper must describe at least:

- the document collection
- how vectors have been computed
- all the implemented functionality used during indexing and searching

- strengths and weakness of the implementation

Experimental Projects

- **Image Search Engine:** Develop a search engine that retrieves images based on text queries. This would require using a content-based image retrieval (CBIR), which involves extracting features like color, texture, and shape.
- **Sentiment Analysis:** Use a pre-trained Word2Vec model to get word embeddings for movie reviews or customer feedback. You can then average the word vectors in a sentence to get a document vector, which can be used as input for a machine learning model (like a support vector machine or a simple neural network) to classify the text as positive, negative, or neutral.
- **Document Clustering:** Group a collection of documents (e.g., news articles, scientific papers) into clusters based on their content. You would represent each document as a vector using Word2Vec and then apply a clustering algorithm like K-Means to group the documents with similar semantic meanings
- **Topic Classification:** Train a FastText or a Word2Vec model on a specialized corpus, like medical journals, legal documents, or financial reports. These domain-specific embeddings will be more effective for tasks within that field than generic, pre-trained models.
- **Spam Detection:** Create a classifier to distinguish between spam and non-spam emails or messages. FastText's speed and efficiency make it suitable for real-time applications like this.
- **Next Word Prediction:** By using Word Embedding, build a Next Word Predictor which is able to predict the next word of a sentence given the context of the sentence